

Lecția 1

Introducere în programarea orientată pe obiecte

1.1 Primii pași în programarea orientată pe obiecte

Totul a început datorită necesității disperate de a organiza mai bine programele. Într-o viziune primitivă programele sunt văzute ca fiind “făcute” din proceduri și funcții, fiecare implementând o bucățică din funcționalitatea programului (altfel spus, un algoritm) și făcând uz de un fond global comun de date. Prin intermediul apelurilor de proceduri și funcții, acești algoritmi sunt combinați în vederea obținerii funcționalității dorite din partea programului.

O lungă perioadă de timp programatorii au scris programe având în minte această viziune de organizare. Ei bine, odată ce programele au început să devină tot mai mari și mai complexe, această modalitate de organizare a început să-și arate deficiențele. Astfel, în loc să se ocupe de implementarea propriu-zisă a unor noi bucăți de funcționalitate necesare extinderii unui program, programatorii au început să petreacă tot mai mult timp încercând să înțeleagă diverse proceduri și funcții pentru a putea combina noile bucăți de funcționalitate cu cele deja existente. Cum înțelegerea subprogramelor devenea din ce în ce mai grea datorită modificărilor succesive aduse lor, adesea de programatori diferiți, aceste programe au început să arate în scurt timp ca niște “saci cu foarte multe petice”. Cel mai grav, s-a ajuns în multe situații ca nimeni să nu mai știe rolul unui “petec” iar ușoare încercări de modificare a lor ducea la “spargerea sacului”, adică la un comportament anormal al programului. Încet încet, programul scăpa de sub control, nimeni nu îl mai putea extinde și în scurt timp nici un client nu mai era interesat în a-l achiziționa din moment ce nu mai corespundea noilor sale necesități.

În vederea limitării acestor situații neplăcute s-au dezvoltat diferite modalități de organizare a programelor cunoscute sub numele de stiluri arhitecturale. Stilul arhitectural descris anterior a fost numit “Main program and subroutines”. Există descrise multe

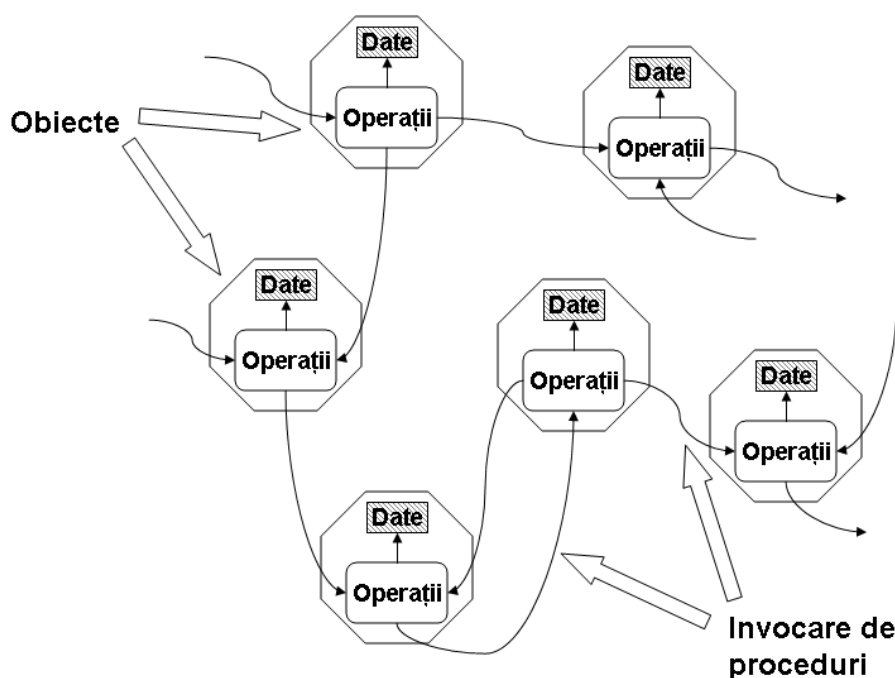


Figura 1.1: ORGANIZAREA ORIENTATĂ PE OBIECTE A UNUI SISTEM SOFTWARE.

alte stiluri în literatura de specialitate. Pe parcursul acestei cărți noi ne vom concentra asupra stilului arhitectural denumit *object-oriented*. În viziunea acestui stil de organizare, un program este “făcut” din obiecte, la fel ca lumea reală. **Un obiect trebuie să “grupeze” împreună un set de date și un set de operații primitive singurele care știu manipula respectivele date.** Obiectele cooperează între ele în vederea obținerii funcționalității dorite din partea programului prin intermediul apelurilor la operațiile primitive corespunzătoare fiecărui obiect. Figura 1.1 surprinde aspectele legate de organizarea orientată pe obiecte a unui program.

Atenție

Este total împotriva spiritului de organizare orientat pe obiecte ca un obiect să facă uz de datele unui alt obiect. Singura formă de cooperare permisă între obiecte trebuie realizată prin intermediul apelului la operații primitive. Și apropo, o operație a cărei singur rol e de a returna o dată a obiectului asociat NU e în general o operație primitivă.

În acest context putem da o definiție parțială a programării orientate pe obiecte.

Definiție 1 *Programarea orientată pe obiecte este o metodă de implementare a programelor în care acestea sunt organizate ca și colecții de obiecte care cooperează între ele.*

12LECȚIA 1. INTRODUCERE ÎN PROGRAMAREA ORIENTATĂ PE OBIECTE

Ei bine, acum știm parțial ce înseamnă programarea orientată pe obiecte. Dar putem deja să scriem programe conform programării orientate pe obiecte? Răspunsul este categoric NU!!! Cum “descompunem” un program în obiectele sale componente? Cum stabilim operațiile primitive asociate unui obiect? Cum implementăm operațiile primitive asociate unui obiect? Înainte de toate, pentru a putea răspunde la aceste întrebări, trebuie să înțelegem anumite concepte care stau la baza organizării orientate pe obiecte. Fără o bună înțelegere a lor nu vom ști niciodată să programăm corect orientat pe obiecte. Nici măcar dacă știm la perfecție limbajul de programare Java!

Important

În urma discuției de mai sus anumite persoane ar putea ajunge la anumite concluzii pripite. *“Tot ce am învățat despre programare până acum este inutil.”* Este total fals. Tendința aproape instinctuală a unei persoane, când are de scris un program care să rezolve o anumită problemă, este de a identifica o serie de algoritmi necesari rezolvării unor părți din problema inițială, de a implementa acești algoritmi ca subprograme și de a combina aceste subprograme în vederea rezolvării întregii probleme. Prin urmare, pare natural să organizezi un program așa cum am amintit la începutul acestui capitol, dar în același timp nu este bine să procedăm așa. Ei bine, nu este într-un tot adevărat. Acest mod de organizare este bun dacă e folosit unde trebuie. Nu e bine să organizăm astfel întregul program, dar, după cum vom vedea mai târziu, obiectele sunt organizate astfel în intimitatea lor.

“Limbajul de programare C este depășit și nu are sens să-l mai studiez și nici să-l mai utilizez.” Este mai mult decât fals. Utilizarea filosofiei organizării orientate pe obiecte a unui program nu depinde absolut de utilizarea unui limbaj de programare orientat pe obiecte cum sunt, de exemplu, Java și C++. Este adevărat că limbajul C nu deține anumite mecanisme absolut necesare programării orientate pe obiecte, dar nu este imposibil să implementăm un program în C și în același timp să-l organizăm orientat pe obiecte. În astfel de situații se vorbește despre programare cu tipuri de date abstracte sau despre programare bazată pe obiecte. În capitolele următoare vom arăta care sunt diferențele.

1.1.1 Abstractizarea

Lumea care ne înconjoară este făcută din obiecte. Zi de zi interacționăm cu obiecte pentru a duce la îndeplinire anumite activități. Unele obiecte sunt mai simple, altele sunt mai complexe. Dar poate cineva să ne spună în cele mai mici detalii cum funcționează fiecare obiect cu care interacționează el sau ea zilnic? Răspunsul e simplu: nu! Motivul ar putea fi șocant pentru unii: pentru că în general ne interesează ceea ce fac obiectele și nu cum fac. Cu alte cuvinte, simplificăm sau abstractizăm obiectele reținând doar caracteristicile lor esențiale. În cazul programării orientate pe obiecte aceste caracteristici se referă exclusiv la comportamentul observabil al unui obiect.



Mulți dintre noi interacționează zilnic cu obiectul televizor pentru a urmări diferite programe Tv. Dar oare câți dintre noi pot să ne spună în cele mai mici detalii cum funcționează televizorul de acasă? Evident foarte puțini. Motivul este că pe majoritatea dintre noi ne interesează doar să utilizăm televizorul pentru a urmări programe Tv. Cum anume ajunge imaginea din studioul unei televiziuni pe ecranul televizorului îi interesează doar pe cei care fac televizoare.

Dar sunt aceste caracteristici esențiale aceleași pentru oricare persoană care interacționează cu un obiect particular? Evident nu. Depinde de la persoană la persoană.



Pentru a achita valoarea unei călătorii cu autobusul, un pasager utilizează un compostor de bilete pentru a-și perfora biletul de călătorie. Din perspectiva firmei care pune la dispoziție serviciul de transport, obiectul compostor poate fi utilizat și pentru a determina numărul de călători dintr-o zi și/sau intervalul orar cel mai aglomerat, lucruri care nu sunt importante din perspectiva pasagerului.

Definiție 2 *O abstracțiune (rezultatul procesului de abstractizare) denotă caracteristicile esențiale ale unui obiect care-l disting de orice alt obiect definind astfel granițele conceptuale ale obiectului relativ la punctul de vedere din care este văzut.*



O aceeași abstracțiune poate corespunde mai multor obiecte din lumea care ne înconjoară dacă ele sunt identice din perspectiva din care sunt privite. De exemplu, dacă pe o persoană o interesează cât e ora la un moment dat poate folosi la fel de bine un ceas mecanic sau un telefon mobil cu ceas. Dacă altcineva vrea însă să dea un telefon evident că acestor obiecte nu le poate corespunde aceeași abstracțiune. O aceeași abstracțiune poate corespunde în acest caz mai multor feluri de telefoane (mobile sau fixe).

1.1.2 Interfața

După cum reiese din secțiunea anterioară abstracțiunea scoate în evidență comportamentul observabil al unui obiect din punctul de vedere al unui utilizator. Cu alte cuvinte, abstracțiunea focalizează atenția asupra serviciilor pe care le pune la dispoziție obiectul spre a fi utilizate de alte obiecte. Aceste servicii nu sunt altceva decât operațiile primitive ale obiectului iar ansamblul tuturor operațiilor primitive se numește *interfața* obiectului respectiv. Un nume utilizat pentru identificarea unei anumite interfețe se numește *tip* (mai general tip de date abstract).

14LECȚIA 1. INTRODUCERE ÎN PROGRAMAREA ORIENTATĂ PE OBIECTE



Să revenim la exemplul cu ceasul mecanic și telefonul mobil. Din perspectiva unei persoane care vrea să știe cât e ceasul ambele obiecte au aceeași interfață care pune la dispoziție operația prin care omul sau, în general, un alt obiect poate afla ora exactă. Din punctul lui de vedere obiectele sunt de același tip. Este clar că pentru o persoană care vrea să dea un telefon obiectele au interfețe diferite și deci tipuri diferite. Să ne gândim acum la compostorul de bilete. Pare ciudat, dar același obiect compostor are două tipuri diferite. Aceasta pentru că diferă perspectiva din care este privit de clienți, adică de către călător, respectiv de către managerul firmei de transport.

1.1.3 Ascunderea informației

Abstractizarea este utilă pentru că focalizează atenția doar asupra caracteristicilor comportamentale esențiale ale unui obiect din perspectiva unui utilizator, permițând astfel identificarea interfeței obiectului. Dar, după cum am spus la începutul acestei expunerii, un obiect grupează un set de date și un set de operații primitive, singurele care știu manipula aceste date. Îl interesează pe utilizator aceste date în mod direct? Mai mult, operațiile primitive trebuie să aibă o implementare. Interfața ne spune doar care sunt aceste operații și nimic altceva. Îl interesează pe utilizator modul lor de implementare în mod direct?

Aici intervine așa numitul *principiu al ascunderii informației*. O discuție completă despre acest principiu este după părerea noastră cu mult dincolo de scopul acestei cărți. Drept urmare, ne vom rezuma a spune că acest principiu este utilizat când se decide să se ascundă ceva neesențial despre un obiect oarecare de orice utilizator potențial. Ei bine, în contextul programării orientate pe obiecte acest principiu este utilizat când spunem că reprezentarea datelor unui obiect și implementarea operațiilor sale primitive trebuie ascunse de orice client al obiectului. Cum anume realizăm această ascundere vom vedea în următorul paragraf.



În urma abstractizării televizorului de acasă, un utilizator ar putea ajunge la concluzia că interfața unui astfel de obiect conține următoarele operații: `comutăPornitOprit`, `măreșteVolumul`, `micșoreazăVolumul`, `comutăPeProgramulAnterior`, `comutăPeProgramulUrmător`. Ei bine, toate aceste operații primitive au o implementare undeva înăuntrul cutiei televizorului. Folosind principiul ascunderii informației constructorul televizorului a hotărât ca toate detaliile de implementare să fie ascunse utilizatorului televizorului. Atenție, doar a hotărât că trebuie ascunse. Faptul că ele au fost ascunse folosind o cutie nu are importanță din punctul de vedere al principiului ascunderii informației.

1.1.4 Încapsularea

În programarea orientată pe obiecte, abstractizarea ajută la determinarea serviciilor furnizate de un obiect. Prin utilizarea principiului ascunderii informației se spune că datele și implementarea operațiilor unui obiect trebuie ascunse de orice client potențial al obiectului. Cu alte cuvinte, obiectul nu trebuie să spună nimic despre datele și implementarea operațiilor sale. Încapsularea vine să completeze cele două noțiuni, reprezentând mecanismul necesar punerii împreună a interfeței unui obiect cu o anumită implementare a acestei interfețe. Mecanismul permite în același timp ascunderea implementării de orice posibil client al respectivei interfețe, făcând astfel posibilă aplicarea principiului ascunderii informației.

Definiție 3 *Încapsularea este procesul de compartimentare a elementelor unei abstracțiuni care constituie structura și comportamentul său. Încapsularea servește la separarea interfeței unei abstracțiuni și a implementării sale.*

Atenție

Este total împotriva noțiunii de programare orientată pe obiecte ca un client să poată accesa datele unui obiect sau să știe detaliile de implementare ale unei operații primitive. Aceste informații trebuie ascunse. Încapsularea permite să se ascundă implementarea unui obiect, dar nu de la sine. De obicei, programatorul este cel care trebuie să spună explicit atunci când definește un obiect ce este ascuns și ce nu. Cu excepția motivelor bine întemeiate, toate datele unui obiect trebuie ascunse explicit de către programator. Promisiuni de genul “promit să nu mă uit niciodată la datele tale” NU se acceptă deoarece mai devreme sau mai târziu cineva tot le va încălca.



Să revenim la exemplul anterior despre televizor. Clientul este interesat doar de interfața televizorului prin intermediul căreia poate controla acest obiect. Folosind principiul ascunderii informației, constructorul televizorului decide să ascundă detaliile de implementare ale acestei interfețe. Pentru a realiza acest lucru el folosește de obicei o cutie. Încapsularea este procesul prin care televizorul în ansamblul său este introdus în respectiva cutie. Ea va ascunde tot ce ține de detaliile de funcționare ale televizorului și lasă accesibil doar ce ține de interfața televizorului.

1.2 Primii pași în Java

Dacă în prima parte a acestei lecții am introdus primele elemente legate de programarea orientată pe obiecte, în această secțiune vom prezenta pe scurt câteva noțiuni de bază necesare scrierii unui program în limbajul Java.

1.2.1 Comentariile

Comentariile reprezintă porțiuni de cod sursă care sunt ignorate de către compilator și sunt utilizate cu precădere pentru documentarea codului sursă. Modul de marcare a comentariilor este prezentat în exemplul de mai jos.

```
/*Exemplu de comentariu pe o linie*/  
  
/*Exemplu de comentariu pe  
    mai multe linii*/  
  
//Alt exemplu de comentariu.
```

În cazul primului tip de comentariu, tot ce este cuprins între `/*` și `*/` este ignorat de către compilator. În cazul celui de-al doilea tip, tot ce urmează după `//` până la terminarea liniei este ignorat.

1.2.2 Identificatori, tipuri primitive și declararea de variabile

Identificatorul este un nume asociat unei entități dintr-un program. Numele unei variabile, a unui parametru sau a unei funcții (în Java metode) reprezintă identificatori. Principial, în Java un identificator poate începe cu `_` sau cu o literă și poate conține litere, cifre și caracterul `_`¹. Există un număr de identificatori care sunt rezervați deoarece ei reprezintă cuvinte cheie ale limbajului Java, de exemplu cuvintele cheie *main* sau *class*. Nu vom furniza aici o listă a tuturor acestor cuvinte cheie deoarece ele vor fi învățate pe parcurs. Momentan ne vom opri doar asupra cuvintelor cheie corespunzătoare tipurilor primitive. În tabelele următoare sunt prezentate tipurile primitive definite în limbajul de programare Java.

Tip	Număr de biți utilizați	Domeniul valoric
byte	8	-128, 127
short	16	-32768, 32767
int	32	-2147483648, 2147483647
long	64	-9223372036854775808, 9223372036854775807

Tabelul 1.1: TIPURI NUMERICE ÎNTREGI.

Este important de amintit aici și tipul *String* asociat șirurilor de caractere. După cum vom vedea ceva mai târziu, acest tip nu este unul primitiv. Limbajul Java tratează într-un mod special acest tip pentru a face mai ușoară munca programatorilor (șirurile de caractere se utilizează des în programe). Din acest motiv el se aseamănă cu tipurile primitive și poate fi amintit aici.

¹În Java, un identificator poate începe și conține caractere Unicode corespunzătoare unui simbol de monedă.

Tip	Număr de biți utilizați	Domeniul valoric
float	32	$\pm 1.4E - 45, \pm 3.4028235E + 38$
double	64	$\pm 4.9E - 324, \pm 1.7976931348623157E + 308$

Tabelul 1.2: TIPURI NUMERICE ÎN VIRGULĂ FLOTANTĂ.

Tip	Număr de biți utilizați	Exemple de valori
char	16	• 'a' - caracterul a • '\n' - caracterul linie nouă • '\u3C00' - caracterul π specificat în hexazecimal
boolean	nespecificat	Sunt posibile doar valorile logice date de cuvintele cheie <i>true</i> și <i>false</i>

Tabelul 1.3: ALTE TIPURI PRIMITIVE.

În continuare, vom arăta prin intermediul unui exemplu modul în care se declară variabile în Java, mod ce nu diferă fundamental de declararea variabilelor în limbajul C.

```
//Declaraarea unei variabile are forma: Tip NumeVariabila = Initializare
char unCaracter, altCaracter = 'A';
int i;
String sir = "Acesta e un sir de caractere";
```

1.2.3 Expresii și operatori

În paragraful anterior am văzut care sunt tipurile primitive ale limbajului Java și modul în care putem declara variabile de diferite tipuri. Variabilele împreună cu literalii (constante de genul 'A' sau "Un sir de caractere") sunt cele mai simple expresii posibile, denumite și expresii primare. Expresii mai complicate pot fi create combinând expresiile primare cu ajutorul *operatorilor*. Câțiva dintre operatorii Java sunt prezentați în Tabelul 1.4.

Există două noțiuni importante care trebuie amintite în contextul operatorilor. Prima este *precedența* (P) operatorilor care dictează ordinea în care se vor efectua operațiile. Ca și exemplu, operatorul de înmulțire are o precedență mai ridicată decât operatorul de adunare și prin urmare înmulțirea se realizează înainte de adunare. Precedența

18LECȚIA 1. INTRODUCERE ÎN PROGRAMAREA ORIENTATĂ PE OBIECTE

implicită a operatorilor poate fi modificată prin utilizarea parantezelor.

```
int a,b,c;

//Operatorul * are o precedenta mai ridicata decat + deci se executa primul
a * b + c

//Operatorul * are o precedenta mai ridicata decat + dar din cauza
//parantezelor care prezinta explicit ordinea operatiilor adunarea
//se va executa prima
a * (b + c)
```

A doua noțiune importantă legată de operatori constă în *asociativitatea* (A) lor. Ea specifică ordinea în care se execută operațiile atunci când o expresie utilizează mai mulți operatori de aceeași precedență. Un operator poate fi asociativ la stanga sau asociativ la dreapta. În primul caz expresia se evaluează de la stânga la dreapta, iar în al doilea caz de la dreapta la stânga. Evident, utilizarea parantezelor poate modifica ordinea de evaluare implicită.

```
int a,b,c;

//Operatorul - este asociativ la stanga, deci prima data se executa
//scaderea iar apoi adunarea
a - b + c

//Operatorul - este asociativ la stanga, dar parantezele spun ca prima data
//se executa adunarea apoi scaderea
a - (b + c)

//Operatorii += si = sunt asociativi la dreapta, deci prima data se da
//valoarea 5 lui b dupa care se da valoarea a + 5 lui a
a+= b = 5
```

1.2.4 Tipărirea la ieșirea standard

După cum am mai spus anterior, un program organizat în spiritul stilului orientat pe obiecte este alcătuit din obiecte. Prin urmare, pentru a afișa o valoare pe ecranul calculatorului, avem nevoie de un obiect care știe face acest lucru. Limbajul Java pune automat la dispoziție un astfel de obiect, accesibil prin “variabila” *out*. El are o operație primitivă ² (mai exact metodă) denumită *println* care știe să afișeze parametrul dat, indiferent de tipul său, la ieșirea standard. În exemplul următor se arată modul în care se realizează afișarea pe ecran dintr-un program.

²De fapt are mai multe dar pentru moment nu ne interesează acest lucru.

P	A	Operator	Tipul operanzilor	Efectul execuției
15	S	++, --	variabilă de tip numeric /char	Post incrementare/decrementare
14	D	++, --	variabilă de tip numeric /char	Pre incrementare/decrementare
14	D	+, -	numeric/char	Plus/Minus unar
14	D	~	întreg/char	Negare pe biți
14	D	!	boolean	Negare logică
12	S	*, /, %	numeric/char, numeric /char	Înmulțire/împărțire/rest
11	S	+,-	numeric/char, numeric /char	Adunare/scădere
11	S	+	String, orice tip	Concatenare de șiruri de caractere
10	S	<<	întreg/char, întreg/char	Deplasare stânga pe biți
10	S	>>	întreg/char, întreg/char	Deplasare dreapta pe biți cu inserție de semn
10	S	>>>	întreg/char, întreg/char	Deplasare dreapta pe biți cu inserție de 0
9	S	<, <=	numeric/char, numeric /char	Mai mic/mai mic sau egal
9	S	>, >=	numeric/char, numeric /char	Mai mare/mai mare sau egal
8	S	==	orice tip, orice tip	Egalitate valorică
8	S	!=	orice tip, orice tip	Inegalitate valorică
7	S	&	întreg/char, întreg/char	Și pe biți
7	S	&	boolean, boolean	Și logic
6	S	^	întreg/char, întreg/char	Sau exclusiv pe biți
6	S	^	boolean, boolean	Sau exclusiv logic
5	S		întreg/char, întreg/char	Sau pe biți
5	S		boolean, boolean	Sau logic
4	S	&&	boolean, boolean	Și logic
3	S		boolean, boolean	Sau logic
1	D	=	variabilă de orice tip, tipul variabilei	Atribuire

Tabelul 1.4: UN SUBSET DE OPERATORI JAVA.

20LECȚIA 1. INTRODUCERE ÎN PROGRAMAREA ORIENTATĂ PE OBIECTE

```
//Afisarea unui sir de caractere
System.out.println("Acest mesaj va fi afisat pe ecranul calculatorului!");

//Afisarea valorii unei variabile
System.out.println(i);
```

Atenție

Prezența lui *System* înaintea obiectului *out* este obligatorie. Motivul îl vom vedea mai târziu.

1.2.5 Instrucțiuni de control

Instrucțiunile de control sunt necesare pentru a permite unui program să ia anumite decizii la execuția sa. Principalele instrucțiuni de decizie sunt *if* și *switch*. Buclele de program se realizează prin instrucțiunile *while*, *do* și *for*. Deoarece toate aceste instrucțiuni sunt asemănătoare celor din C nu vom insista asupra lor, rezumându-ne în a prezenta mai jos forma lor generală.

```
if (ExpresieLogica)
    //Instrucțiunea se executa daca ExpresieLogica este adevarata
    //Daca sunt mai multe instructiuni ele trebuie cuprinse intre { si }
else {
    //Instrucțiuni
    //Ramura else poate sa lipseasca daca nu e necesara
}

switch (Expresie) {
    //Expresie trebuie sa fie de tip char, byte, short sau int
    case ExpresieConstanta1:
        //Instrucțiuni ce se executa cand Expresie ia valoarea lui
        //ExpresieConstanta1
        break;
    case ExpresieConstanta2:
        //Instrucțiuni ce se executa cand Expresie ia valoarea lui
        //ExpresieConstanta2
        break;
    case ExpresieConstanta3:
        //Instrucțiuni ce se executa cand Expresie ia valoarea lui
        //ExpresieConstanta3
        break;
    default:
        //Instrucțiuni ce se executa cand Expresie ia o valoare diferita
        //de oricare ExpresieConstanta. Ramura default poate lipsi
}
```

```
while(ExpresieLogica) {
    //Instructiuni ce se executa atata timp cat ExpresieLogica este
    //adevarata
}

do {
    //Instructiuni ce se repeta atata timp cat ExpresieLogica e adevarata
    //Ele se executa cel putin o data pentru ca ExpresieLogica este testata
    //la sfarsitul buclei
} while(ExpresieLogica);

for (Initializare;ExpresieLogica;Incrementare) {
    //Instructiuni ce se repeta atata timp cat ExpresieLogica e adevarata
    //Inaintea primei iteratii se executa Initializare
    //Dupa fiecare iteratie se executa Incrementare
}
```

La fel ca în limbajul C, există de asemenea instrucțiunile *continue* și *break*. Instrucțiunea *continue* trebuie să apară în interiorul unui ciclu, iar efectul său constă în trecerea imediată la execuția următoarei iterații din bucla imediat înconjurătoare instrucțiunii *continue*. Tot în interiorul ciclurilor poate apare și instrucțiunea *break*. Efectul ei constă în terminarea imediată a ciclului imediat înconjurător instrucțiunii *break*. În plus, instrucțiunea *break* poate apare și în corpul unei instrucțiuni *switch*, mai exact pe una dintre posibilele ramuri *case* ale instrucțiunii. Execuția unui astfel de *break* conduce la terminarea execuției instrucțiunii *switch*. Dacă pe o ramură *case* nu apare instrucțiunea *break* atunci la terminarea execuției instrucțiunilor respectivei ramuri se va continua cu execuția instrucțiunilor ramurii următoare (evident dacă există una). Situația este exemplificată mai jos.

```
char c;
//Instructiuni
switch(c) {
    case '1': System.out.println("Unu ");break;
    case '2': System.out.println("Doi ");
    case '3': System.out.println("Trei");
}
//Daca c este caracterul '1' pe ecran se va tipari
//    Unu
//Daca c este caracterul '2' pe ecran se va tipari
//    Doi
//    Trei
//Daca c este caracterul '3' pe ecran se va tipari
//    Trei
```

1.2.6 Primul program

În acest paragraf vom vedea un prim program Java și modul în care îl lansăm în execuție.

```
class PrimulProgram {  
  
    public static void main(String argv[]) {  
        System.out.println("Hello world!");  
    }  
}
```

La fel ca și în limbajul de programare C, execuția unui program Java începe în funcția (în Java metoda) *main*. Singurul parametru al acestei metode este un tablou de șiruri de caractere prin intermediul căruia se transmit parametri din linia de comandă. Metoda nu returnează nici o valoare, motiv pentru care se specifică tipul *void* ca tip returnat.

După cum vom vedea în lucrarea următoare, în Java orice metodă trebuie să fie inclusă într-o *clasă*, în acest caz clasa *PrimulProgram*. Tot acolo vom înțelege și rolul cuvintelor cheie *public* și *static*. Deocamdată nu vom vorbi despre ele.

Pentru a rula programul, acesta trebuie mai întâi compilat. Să presupunem că programul prezentat mai sus se află într-un fișier denumit *PrimulProgram.java*. Pentru a-l compila se folosește comanda:

```
javac PrimulProgram.java
```

Rezultatul va fi un fișier cu numele *PrimulProgram.class* care conține, printre altele, codul executabil al metodei *main*.

Important

Să presupunem că programul de mai sus s-ar fi aflat în fișierul *Program.java*. În acest caz comanda de compilare ar fi fost *javac Program.java*, dar rezultatul compilării ar fi fost tot *PrimulProgram.class* deoarece numele fișierului rezultat se obține din numele clasei compilate și nu din numele fișierului ce conține codul sursă.

Pentru a se lansa în execuție programul se utilizează comanda:

```
java PrimulProgram
```

Ca urmare a acestei comenzi, mașina virtuală Java va căuta în fișierul *PrimulProgram.class* codul metodei *main* după care va trece la execuția sa. În acest caz se va afișa pe ecran mesajul "Hello world!".

Atenție

Dacă clasa dată ca parametru mașinii virtuale Java nu conține metoda *main* se va semnala o eroare.

1.3 Exerciții

1. Compilați și lansați în execuție programul “Hello World!” dat ca exemplu în Secțiunea 1.2.6.
2. Scrieți un program Java care inițializează două variabile întregi cu două valori constante oarecare. În continuare, programul va determina variabila ce conține valoarea maximă și va tipări conținutul ei pe ecran.
3. Scrieți un program Java care afișează pe ecran numerele impare și suma numerelor pare cuprinse în intervalul 1-100 inclusiv.

Sfat

Pentru a rezolva exercițiile 2 și 3 folosiți doar variabile locale metodei *main*. În Java nu există variabile globale ca în Pascal sau C.

Bibliografie

1. Edward V. Berard, *Abstraction, Encapsulation, and Information Hiding*, <http://www.itmweb.com/essay550.htm>, 2002.
2. Grady Booch, *Object-Oriented Analysis And Design With Applications*, Second Edition, Addison Wesley, 1997.
3. David Flanagan, *Java In A Nutshell. A Desktop Quick Reference*, Third Edition, O'Reilly, 1999.
4. Mary Shaw, David Garlan, *Software Architecture. Perspectives On An Emerging Discipline*, Prentice Hall, 1996.

Lecția 2

Clase și Obiecte

Presupunem că dorim să *descriem* într-un limbaj de programare un obiect ceas. În general, unui ceas îi putem seta ora, minutul și secunda și în orice moment putem ști ora exactă. Cum am putea realiza acest lucru?

Dacă *descriem* acest obiect (tip de date abstract) într-un limbaj de programare structural, spre exemplu C, atunci vom crea, ca mai jos, o structură *Ceas* împreună cu niște funcții cuplate de această structură. Cuplajul e realizat prin faptul că orice funcție care operează asupra unui ceas conține în lista sa de parametri un parametru de tip *Ceas*.

```
typedef struct _Ceas {
    int ora, minut, secunda;
} Ceas;

void setareTimp(Ceas *this, int ora, int minut, int secunda) {
    this->ora = ((ora >= 0 && ora < 24) ? ora : 0);
    this->minut = ((minut >= 0 && minut < 60) ? minut : 0);
    this->secunda = ((secunda >= 0 && secunda < 60) ? secunda : 0);
}

void afiseaza(Ceas *this) {
    printf("Ora setata %d:%d:%d", this->ora, this->minut, this->secunda);
}
```

Dacă modelăm acest obiect într-un limbaj orientat pe obiecte (în acest caz, Java), atunci vom crea o clasă *Ceas* ca mai jos.

```
class Ceas {

    private int ora, minut, secunda;
```

```
public void setareTimp(int o, int m, int s) {
    ora = ((o >= 0 && o < 24) ? o : 0);
    minut = ((m >= 0 && m < 60) ? m : 0);
    secunda = ((s >= 0 && s < 60) ? s : 0);
}

public void afiseaza() {
    System.out.println("Ora setata "+ora+": "+minut+": "+secunda);
}
}
```

Se poate ușor observa din cadrul exemplului de mai sus că atât datele cât și funcțiile care operează asupra acestora se găsesc în interiorul aceleiași entități numită *clasă*. Desigur, conceptele folosite în codul de mai sus sunt încă necunoscute dar cunoașterea lor este scopul principal al acestei lecții.

2.1 Clase și obiecte

2.1.1 Ce este un obiect? Ce este o clasă?

În cadrul definiției parțiale a programării orientate pe obiecte pe care am dat-o în lecția anterioară se spune că ”programele orientate pe obiecte sunt organizate ca și colecții de obiecte”. Dar până acum nu s-a spus ce este, de fapt, un obiect. În acest context, este momentul să completăm Definiția 1:

Definiție 4 *Programarea orientată pe obiecte este o metodă de implementare a programelor în care acestea sunt organizate ca și colecții de obiecte care cooperează între ele, fiecare obiect reprezentând instanța unei clase.*

Definiția de mai sus este încă incompletă, forma ei completă fiind Definiția 7 din Lecția 5.



Atunci când un producător crează un produs, mai întâi acesta specifică toate caracteristicile produsului într-un document de specificații iar pe baza celui document se crează fizic produsul. De exemplu, televizorul este un produs creat pe baza unui astfel de document de specificații. La fel stau lucrurile și într-un program orientat pe obiecte: mai întâi se crează clasa obiectului (documentul de specificații) care înglobează toate caracteristicile unui obiect (instanță a clasei) după care, pe baza acesteia, se crează obiectul în memorie.

În general, putem spune că o clasă furnizează un șablon ce specifică datele și operațiile ce aparțin obiectelor create pe baza șablonului – în documentul de specificații pentru un televizor se menționează că acesta are un tub catodic precum și niște butoane care pot fi apăstate.

2.1.2 Definirea unei clase

Din cele de mai sus deducem că o clasă descrie un obiect, în general, un nou tip de dată. Într-o clasă găsim date și metode ce operează asupra datelor respective.

Pentru a defini o clasă trebuie folosit cuvântul cheie *class* urmat de numele clasei:

```
class NumeClasa {  
    //Date si metode membru  
}
```

Atenție

O metodă nu poate fi definită în afara unei clase.

Acum haideți să analizăm puțin exemplul dat la începutul capitolului. Clasa *Ceas* modelează tipul de date abstract *Ceas*. Un ceas are trei date de tip *int* reprezentând ora, minutul și secunda precum și două operații: una pentru setarea acestor atribute iar alta pentru afișarea lor. Cuvintele *public* și *private* sunt cuvinte cheie ale căror roluri sunt explicate în Secțiunea 2.3.1.

Important

Datele *ora*, *minut*, *secunda* definite în clasa *Ceas* se numesc *atribute*, *date-membru*, *variabile-membru* sau *câmpuri* iar operațiile *setareTimp*, *afiseaza* se numesc *metode* sau *funcții-membru*.

2.1.3 Crearea unui obiect

Spuneam mai sus că un obiect este o instanță a unei clase. În Java instanțierea sau crearea unui obiect se face dinamic, folosind cuvântul cheie *new*:

```
new Ceas();
```



Așa cum fiecare televizor construit pe baza documentului de specificații are propriul său tub catodic, fiecare obiect de tip *Ceas* are propriile sale atribute.

2.2 Referințe la obiecte

În secțiunile anterioare am văzut cum se definește o clasă și cum se crează un obiect. În această secțiune vom vedea cum putem executa operațiile furnizate de obiecte.

Pentru a putea avea acces la operațiile furnizate de către un obiect, trebuie să deținem o *referință* spre acel obiect.



Telecomanda asociată unei lămpi electrice poate fi văzută ca o referință la aceasta. Atâta timp cât avem telecomanda, putem oricând prin intermediul ei să accesăm un serviciu furnizat de către lampă (spre exemplu, pornirea/oprirea sau mărirea/micșorarea intensității luminoase). Dacă părăsim încăperea și dorim în continuare să controlăm lampa, luăm telecomanda și nu lampa cu noi!

Declararea unei referințe numite *ceas* spre un obiect de tip *Ceas* se face în felul următor:

```
Ceas ceas;
```

Atenție

Limbajul Java este *case-sensitive* și din această cauză putem avea referința numită *ceas* spre un obiect de tip *Ceas*.

Atenție

Faptul că avem la un moment dat o referință nu implică și existența unui obiect indicat de acea referință. Până în momentul în care referinței nu i se atașează un obiect, aceasta nu poate fi folosită.

Mai sus a fost creată o referință spre un obiect de tip *Ceas* dar aceștia încă nu i s-a atașat vreun obiect și, prin urmare, referința încă nu poate fi utilizată. Atașarea unui obiect la referința *ceas* se face printr-o operație de atribuire, ca în exemplul următor:

```
ceas = new Ceas();
```



Putem avea doar telecomanda unui lămpi fără a deține însă și lampa. În acest caz telecomanda nu referă nimica. Pentru ca o referință să nu indice *vreun obiect* aceștia trebuie să i se atribuie valoarea *null*.



Valoarea *null*, ce înseamnă *nici un obiect referit*, nu este atribuită automat tuturor variabilelor referință la declararea lor. Regula este următoarea: dacă referința este un membru al unei clase și ea nu este inițializată în nici un fel, la instanțierea unui obiect al clasei respective referința va primi implicit valoarea *null*. Dacă însă referința este o variabilă locală ce aparține unei metode, inițializarea implicită nu mai funcționează. De aceea se recomandă ca programatorul să realizeze *întotdeauna* o inițializare explicită a variabilelor.

Acum, fiindcă avem o referință spre un obiect de tip *Ceas* ar fi cazul să setăm obiectului referit ora exactă. Apelul către metodele unui obiect se face prin intermediul referinței spre acel obiect ca în exemplul de mai jos:

```
class ClientCear {  
  
    public static void main(String[] args) {  
        Cear ceas = new Cear();  
        int o = 15;  
        ceas.setareTimp(o, 12, 20);  
        ceas.afiseaza();  
    }  
}
```

Atenție

Apelul metodei *afiseaza* nu este *afiseaza(ceas)* ci *ceas.afiseaza()* întrucât metoda *afiseaza* aparține obiectului referit de *ceas* – se apelează metoda *afiseaza* pentru obiectul referit de variabila *ceas* din fața lui.

Haideți să considerăm exemplul de mai jos în care creăm două obiecte de tip *Cear* precum și trei referințe spre acest tip de obiecte. Fiecare dintre obiectele *Cear* create are alocată o zonă proprie de memorie în care sunt stocate valorile câmpurilor *ora*, *minut*, *secunda*. Ultima referință definită în exemplul de mai jos, prin atribuirea *c3* = *c2* va referi și ea exact același obiect ca și *c2*, adică al doilea obiect creat. Vizual, referințele și obiectele create după execuția primelor cinci linii de cod de mai jos sunt reprezentate în Figura 2.1.

```
class AltClientCear {  
  
    public static void main(String[] args) {  
        Cear c1 = new Cear();  
        c1.setareTimp(15, 10, 0);  
        Cear c2 = new Cear();  
        c2.setareTimp(15, 10, 0);  
        Cear c3 = c2;  
        c3.setareTimp(8, 12, 20);  
        c2.afiseaza();  
    }  
}
```



La un moment dat, putem avea două telecomenzi asociate unei lămpi electrice. Exact la fel se poate întâmpla și în cazul unui program – putem avea acces la serviciile puse la dispoziție de un obiect prin intermediul mai multor referințe.

În contextul exemplului anterior, se pune problema ce se va afișa în urma execuției ultimei linii? Deoarece atât *c3* cât și *c2* referă același obiect, se va afișa:

```
//Ora setata 8:12:20
//NU !! Ora setata 15:10:0
```

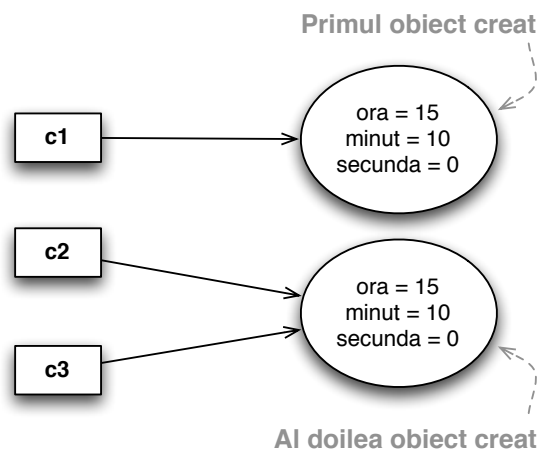


Figura 2.1: OBIECTE. REFERINȚE LA OBIECTE.

Dar ce se va afișa în urma execuției *c1.afiseaza()*? Deoarece *c1* referă un alt obiect ceas decât *c2*, respectiv *c3*, se va afișa:

```
c1.afiseaza();
//Ora setata 15:10:0
```



Să presupunem că am achiziționat două lămpi de același tip. Putem oare să spunem că cele două lămpi sunt identice? Hmmmm,...la prima vedere am putea spune: sigur, ele ne oferă aceleași servicii deci *categoric* sunt identice. Numai că atunci când vom cere cuiva să realizeze operația ”Aprinde lampa” va trebui să precizăm și care dintre cele două lămpi dorim să fie aprinsă. Vom preciza care lampă dorim să fie aprinsă în funcție de *identitatea* sa – proprietatea unui obiect care îl distinge de restul obiectelor – spre exemplu, locul ocupat în spațiu e o proprietate care le poate distinge. La fel ca cele două lămpi, și cele două obiecte *ceas* create de noi au propria identitate – în acest caz identitatea poate fi dată de locul fizic pe care îl ocupă în memorie obiectele. Atribuirea *c3 = c2* nu a făcut altceva decât să atașeze referinței *c3* obiectul având aceeași identitate ca și cel referit de *c2*, adică obiectul secund creat.

2.3 Componenta unei clase

Clasele, așa cum am văzut deja, sunt definite folosind cuvântul cheie *class*. În următoarele secțiuni vom vorbi despre diferite categorii de membri care pot apărea în interiorul unei clase.

2.3.1 Specificatori de acces

În componența clasei *Ceas* se observă prezența cuvintelor cheie *private* și *public*. Aceste cuvinte se numesc *specificatori de acces* și rolul lor este de a stabili drepturile de acces asupra membrilor unei clase (atribute și metode). În afară de cei doi specificatori de acces prezenți în clasa *Ceas* mai există și alții dar despre ei se va vorbi în alte lecții.

Când discutăm despre drepturile de acces la membrii unei clase trebuie să abordăm acest subiect din două perspective: *interiorul* respectiv *exteriorul* clasei.

```
class Specificator {  
  
    public int atributPublic;  
    private int atributPrivat;  
  
    public void metodaPublica() {  
        atributPublic = 20;  
        atributPrivat = metodaPrivata();  
    }  
  
    private int metodaPrivata() {  
        return 10;  
    }  
  
    public void altaMetoda(Specificator s) {  
        atributPublic = s.atributPrivat;  
        //Corect, deoarece altaMetoda se afla in clasa Specificator  
        atributPrivat = s.metodaPrivata();  
        s.metodaPublica();  
    }  
}
```

În cadrul metodelor unei clase există acces nerestricțiv la toți membrii clasei, atribute sau metode. Exemplul de mai sus ilustrează acest lucru.

În legătură cu accesul din interiorul unei clase trebuie spus că absența restricțiilor se aplică și dacă este vorba despre membrii altui obiect, instanță a aceleiași clase.

```
class ClientSpecificator {  
  
    public static void main(String[] args) {  
        Specificator spec = new Specificator();  
        spec.metodaPublica();  
        spec.metodaPrivata(); //Eroare  
        System.out.println(spec.atributPublic);  
        System.out.println(spec.atributPrivat); //Eroare  
    }  
}
```

Membrii declarați cu specificatorul *private* NU sunt vizibili în afara clasei, ei fiind ascunși. Clienții unei clase pot accesa doar acei membri care au ca modifier de acces cuvântul *public*. Dacă încercăm să accesăm, așa cum am făcut în metoda *main* de mai sus, membrii *private* ai clasei *Specificator* prin intermediul referinței *spec* compilatorul va semnala o eroare.

De ce e nevoie de specificatorul de acces *private*?

Clasa *Ceas* descrie obiecte de tip ceas. Atributele *ora*, *minut* și *secunda* pentru orice ceas trebuie să aibă valori cuprinse între 0 și 23, respectiv 0 și 59. Pentru ca un ceas să indice o oră validă metoda pentru setarea atributelor de mai sus trebuie scrisă astfel încât să nu permită setarea acestora cu o valoare eronată.

Dacă cele trei atribute ale unui obiect de tip *Ceas* ar putea fi accesate din exterior, atunci valorile lor ar putea fi setate în mod eronat.



În exemplul anterior scris în C nu putem ascunde cele trei câmpuri ale structurii *Ceas* și în consecință acestea vor putea primi oricând alte valori decât cele permise. În general, când scrieți o clasă accesul la atributele sale trebuie să fie limitat la metodele sale membru pentru a putea controla valorile atribuite acestora.

În clasa *Ceas* cele trei atribute existente sunt de tip *int*. Dar oare nu există și altă metodă prin care pot fi stocate cele trei caracteristici ale unui ceas? Cele trei caracteristici ale unui ceas ar putea fi stocate, de exemplu, în loc de trei atribute întregi într-un tablou cu trei elemente întregi – dar un utilizator de ceasuri nu are voie să știe *detaliile de implementare* ale unui obiect de tip *Ceas*!!!



Și totuși, ce e rău în faptul ca un utilizator de obiecte *Ceas* să știe exact cum e implementat ceasul pe care tocmai îl folosește? Probabil că lucrurile ar fi *bune și frumoase* până când proiectantul de ceasuri ajunge la concluzia că trebuie neapărat să modifice implementarea ceasurilor...și atunci

toți cei care folosesc ceasuri și sunt dependenți de implementarea acestora vor trebui să se *modifice!!!* În schimb, dacă clienții unei clase depind de *serviciul oferit de implementator și nu de implementarea acestuia* atunci ei nu vor fi afectați de modificările ulterioare aduse!!!

2.3.2 Constructori

În multe cazuri, atunci când instanțiem un obiect, ar fi folositor ca obiectul să aibă anumite atribute inițializate.



Cum ar fi ca în momentul la care un producător a terminat de construit un ceas, ceasul să arate deja ora exactă? Beneficiul pe care l-am avea datorită acestui fapt este că am putea folosi imediat ceasul, nemaifiind nevoie să efectuăm inițial operația de setare a timpului curent.

Inițializarea atributelor unui obiect se poate face în mod automat, la crearea obiectului, prin intermediul unui *constructor*. Principalele caracteristici ale unui constructor sunt:

- un constructor are același nume ca și clasa în care este declarat.
- un constructor nu are tip returnat.
- un constructor se apelează automat la crearea unui obiect.
- un constructor se execută la crearea obiectului și numai atunci.

```
class Ceas {  
    ...  
    public Ceas(int o, int m, int s) {  
        ora = ((o >= 0 && o < 24) ? o : 0);  
        minut = ((m >= 0 && m < 60) ? m : 0);  
        secunda = ((s >= 0 && s < 60) ? s : 0);  
    }  
}
```

Astfel, atunci când creăm un obiect, putem specifica și felul în care vrem ca acel obiect să arate inițial.

```
class ClientCeas {  
  
    public static void main(String[] args) {  
        Ceas ceas;  
        ceas = new Ceas(12, 15, 15);  
        ceas.afiseaza();  
    }  
}
```

Dacă programatorul nu prevede într-o clasă nici un constructor, atunci compilatorul va genera pentru clasa respectivă un constructor implicit fără nici un argument și al cărui corp de instrucțiuni este vid. Datorită acestui fapt am putut scrie înaintea definirii propriului constructor pentru clasa *Ceas*

```
Ceas ceas;  
ceas = new Ceas();
```

Dacă programatorul include într-o clasă cel puțin un constructor, compilatorul nu va mai genera constructorul implicit. Astfel, după introducerea constructorului anterior, codul de mai sus va genera erori la compilare. Să considerăm acum un alt exemplu:

```
class Constructor {  
  
    private int camp = 17;  
  
    public Constructor(int c) {  
        camp = c;  
    }  
  
    public void afiseaza() {  
        System.out.println("Camp " + camp);  
    }  
}
```

În clasa *Constructor* de mai sus am inițializat atributul *camp* în momentul declarării sale cu *17*. Dar, după cum se poate observa, la instanțierea unui obiect se modifică valoarea atributului *camp*. În acest context se pune problema cunoașterii valorii atributului *camp* al obiectului instanțiat mai jos. Va fi aceasta *17* sau *30*?

```
class ClientConstructor {  
  
    public static void main(String[] args) {  
        Constructor c;  
        c = new Constructor(30);  
        //Va afisa "Camp 30" datorita modului in care  
        //se initializeaza campurile unui obiect  
        c.afiseaza();  
    }  
}
```

Pentru o clasă în care apar mai multe mecanisme de inițializare, așa cum este cazul clasei *Constructor*, ordinea lor de execuție este următoarea:

- Inițializările specificate la declararea atributelor.

- Constructorii.

Atenție

Un constructor poate fi *private*!

2.3.3 Membri statici

Atunci când definim o clasă, specificăm felul în care obiectele de tipul acelei clase arată și se comportă. Dar până la crearea efectivă a unui obiect folosind *new* nu se alocă nici o zonă de memorie pentru atributele definite în cadrul clasei iar la crearea unui obiect se alocă acestuia memoria necesară pentru fiecare atribut existent în clasa instanțiată. Tot până la crearea efectivă a unui obiect nu putem beneficia de serviciile definite în cadrul unei clase. Ei bine, există și o excepție de la regula prezentată anterior – membrii statici (atribute și metode) ai unei clase. Acești membri ai unei clase pot fi folosiți direct prin intermediul numelui clasei, fără a deține instanțe a respectivei clase.

Spre exemplu, o informație de genul *câte obiecte de tipul Ceas s-au creat?* nu caracterizează o instanță a clasei *Ceas*, ci caracterizează însăși clasa *Ceas*. Ar fi nepotrivit ca atunci când vrem să aflăm numărul de instanțe ale clasei *Ceas* să trebuiască să creăm un obiect care va fi folosit doar pentru aflarea acestui număr – în loc de a crea un obiect, am putea să aflăm acest lucru direct de la clasa *Ceas*.

Important

Un membru static al unei clase caracterizează clasa în interiorul căreia este definit precum și toate obiectele clasei respective.

Un membru al unei clase (atribut sau metodă) este static dacă el este precedat de cuvântul cheie *static*. După cum reiese din Figura 2.2, un atribut static este un câmp comun ce are aceeași valoare pentru fiecare obiect instanță a unei clase.

În continuare este prezentat un exemplu în care se numără câte instanțe ale clasei *Ceas* s-au creat.

```
class Ceas {  
  
    private static int numarObiecte = 0;  
  
    ...  
    public Ceas(int o, int m, int s) {  
        ...  
        numarObiecte++;  
    }  
    ...  
    public static int getNumarDeObiecte() {  
        return numarObiecte;  
    }  
}
```

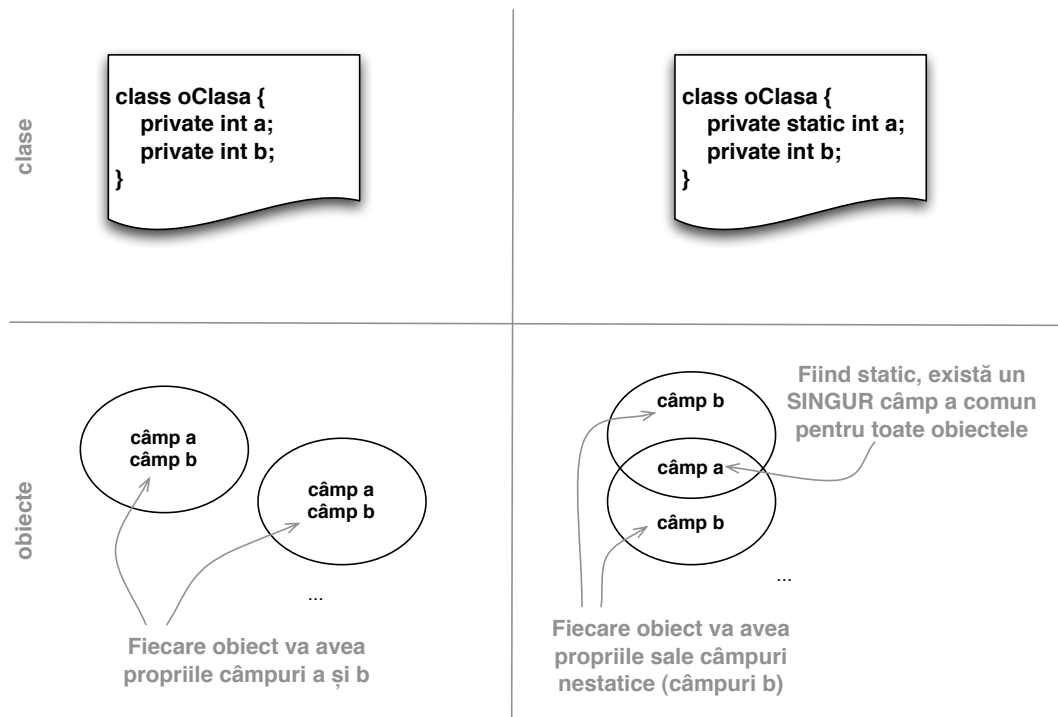


Figura 2.2: MEMBRI STATICI VERSUS MEMBRI NESTATICI.

Accesarea unui membru static al unei clase se poate face în cele două moduri prezentate mai jos. Evident, dacă metoda *getNumarDeObiecte* nu ar fi fost statică, ea nu ar fi putut fi apelată decât prin intermediul unei referințe la un obiect instanță a clasei *Ceas*.

```
//prin intermediul numelui clasei
Ceas.getNumarDeObiecte()
//prin intermediul unei referințe
Ceac ceas = new Ceas(12:12:50);
ceas.getNumarDeObiecte();
```

E bine ca referirea unui membru static să se facă prin intermediul numelui clasei și nu prin intermediul unei referințe; dacă un membru static e accesat prin intermediul numelui clasei atunci un cititor al programului va ști imediat că acel membru este static, în caz contrar această informație fiindu-i ascunsă.

Atenție

Din interiorul unei metode statice pot fi accesați doar alți membri statici ai clasei în care este definită metoda, accesarea membrilor nestatici ai clasei producând o eroare de compilare.

Spuneam anterior că un atribut static este un câmp comun ce are aceeași valoare pentru fiecare obiect instanță a unei clase. Datorită acestui fapt, dacă noi setăm valoarea atributului *atributStatic* obiectului referit de *e1* din exemplul de mai jos, și obiectul referit de *e2* va avea aceeași valoare corespunzătoare atributului static, adică 25.

```
class ExempluStatic {
    public static int atributStatic = 15;
    public int atributNeStatic = 15;

    public static void main(String[] argv) {
        ExempluStatic e1 = new ExempluStatic();
        ExempluStatic e2 = new ExempluStatic();

        e1.atributStatic = 25;
        e1.atributNeStatic = 25;
        System.out.println("Valoare S" + e2.atributStatic);
        //Se va afișa 25 deoarece atributStatic este stocat într-o
        //zona de memorie comună celor două obiecte
        System.out.println("Valoare NeS" + e2.atributNeStatic);
        //Se va afișa 15 deoarece fiecare obiect are propria zona
        //de memorie aferentă atributului nestatic
    }
}
```

2.3.4 Constante

În Java o constantă se declară folosind cuvintele cheie *static final* care preced tipul constantei.

```
class Ceas{
    public static final int MARCA = 323;
    ...
}
```

O constantă, fiind un atribut static, este accesată ca orice atribut static: *Ceas.MARCA*. Datorită faptului că variabila *MARCA* este precedată de cuvântul cheie *final*, acestea i se poate atribui doar o singură valoare pe tot parcursul programului.



Un exemplu de membru *static final* este atributul *out* al clasei *System* pe care îl folosim atunci când tipărim pe ecran un mesaj.

2.4 Convenții pentru scrierea codului sursă

Convențiile pentru scrierea codului sursă îmbunătățesc lizibilitatea codului, permițându-ne să-l înțelegem mai repede și mai bine. În acest context ÎNTOTDEAUNA:

- numele unei clase este format dintr-unul sau mai multe substantive, prima literă a fiecărui substantiv fiind o literă mare (exemplu: *Ceas*, *CeasMana*).
- numele unei metode începe cu un verb scris cu litere mici iar dacă numele metodei este format din mai multe cuvinte, prima literă a fiecărui cuvânt este mare (exemplu: *seteazaTimp*, *afiseaza*).
- numele unei constante este format dintr-unul sau mai multe cuvinte scrise cu litere mari separate, dacă este cazul, prin `_` (exemplu: `LATIME`, `LATIME_MAXIMA`).

2.5 Notății UML pentru descrierea programelor

Atunci când scriem un program este bine ca etapa de scriere a codului să fie precedată de o etapă de proiectare, etapă în care se stabilesc principalele componente ale programului precum și legăturile dintre ele. Componentele identificate în faza de proiectare vor fi apoi implementate.

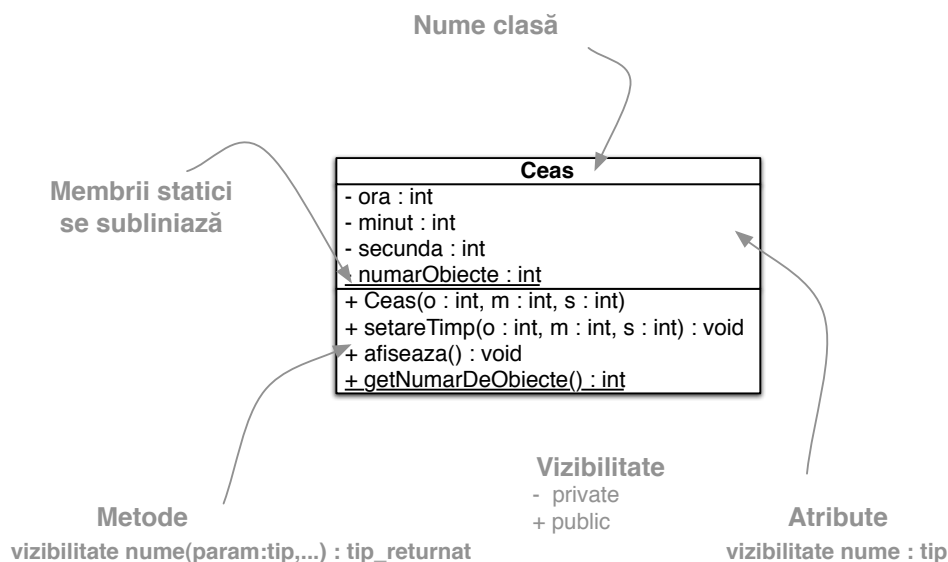


Figura 2.3: REPREZENTAREA UNEI CLASE ÎN UML.

Una dintre cele mai utile notații pentru reprezentarea componentelor este UML (Unified Modeling Language). O reprezentare grafică în UML se numește diagramă. O diagramă arată principial ca un graf în care nodurile pot fi clase, obiecte, stări ale unui obiect iar arcele reprezintă relațiile dintre nodurile existente.

Pentru a reprezenta clasele dintr-un program vom crea diagrame de clase, o clasă

reprezentându-se ca în Figura 2.3. Ca exemplu se prezintă clasa *Ceas* cu toate elementele discutate pe parcursul acestei lecții. Diferitele tipuri de relații ce pot exista între două sau mai multe clase vor fi prezentate în cadrul altor lecții.

2.6 Exerciții

1. Creați o clasă cu un constructor privat. Vedeți ce se întâmplă la compilare dacă creați o instanță a clasei într-o metodă *main*.
2. Creați o clasă ce conține două atribute nestatice private, un *int* și un *char* care nu sunt inițializate și tipăriți valorile acestora pentru a verifica dacă Java realizează inițializarea implicită.

```
class Motor {  
  
    private int capacitate;  
  
    public Motor(int c) {  
        capacitate = c;  
    }  
  
    public void setCapacitate(int c) {  
        capacitate = c;  
    }  
  
    public void tipareste() {  
        System.out.println("Motor de capacitate " + capacitate);  
    }  
}
```

Fiind dată implementarea clasei *Motor*, se cere să se precizeze ce se va afișa în urma rulării secvenței:

```
Motor m1, m2;  
m1 = new Motor(5);  
m2 = m1;  
m2.setCapacitate(10);  
m1.tipareste();
```

4. Un sertar este caracterizat de o lățime, lungime și înălțime. Un birou are două sertare și, evident, o lățime, lungime și înălțime. Creați clasele *Sertar* și *Birou* corespunzătoare specificațiilor de mai sus. Creați pentru fiecare clasă constructorul potrivit astfel încât caracteristicile instanțelor să fie setate la crearea acestora. Clasa *Sertar* conține o metodă *tipareste* al cărei apel va produce tipărirea pe ecran a sertarului sub forma "*Sertar* " + *l* + *L* + *H*, unde *l*, *L*, *H* sunt valorile corespunzătoare lățimii, lungimii și înălțimii sertarului. Clasa *Birou* conține o metodă

tipareste cu ajutorul căreia se vor tipări toate componentele biroului. Creați într-o metodă *main* două sertare, un birou și tipăriți componentele biroului.

5. Definiți o clasă *Complex* care modelează lucrul cu numere complexe. Membrii acestei clase sunt:

- două atribute de tip *double* pentru părțile reală, respectiv imaginară ale numărului complex
- un constructor cu doi parametri de tip *double*, pentru setarea celor două părți ale numărului (reală și imaginară)
- o metodă de calcul a modulului numărului complex. Se precizează că modulul unui număr complex este egal cu radical din $(re*re+img*img)$ unde *re* este partea reală, iar *img* este partea imaginară. Pentru calculul radicalului se va folosi metoda statică predefinită *Math.sqrt* care necesită un parametru de tip *double* și returnează tot un *double*
- o metodă de afișare pe ecran a valorii numărului complex, sub forma $re + i * im$
- o metodă care returnează suma dintre două obiecte complexe. Această metodă are un parametru de tip *Complex* și returnează suma dintre obiectul curent (obiectul care oferă serviciul de adunare) și cel primit ca parametru. Tipul returnat de această metodă este *Complex*.
- o metodă care returnează de câte ori s-au afișat pe ecran numere complexe.

Pe lângă clasa *Complex* se va defini o clasă *ClientComplex* care va conține într-o metodă *main* exemple de utilizare ale metodelor clasei *Complex*.

Bibliografie

1. Grady Booch, James Rumbaugh, Ivar Jacobson. *The Unified Modeling Language User Guide*. Addison-Wesley, 1999.
2. Harvey Deitel & Paul Deitel. *Java - How to program*. Prentice Hall, 1999, Capitolul 8, Object-Based Programming.
3. Bruce Eckel. *Thinking in Java, 4th Edition*. Prentice-Hall, 2006. Capitolul Everything is an object.
4. Martin Fowler. *UML Distilled, 3rd Edition*. Addison-Wesley, 2003.
5. Kris Jamsa. *Succes cu C++*. Editura All, 1997, Capitolul 2, Acomodarea cu clase și obiecte.
6. *Java Code Conventions*. <http://java.sun.com/docs/codeconv>.

Lecția 3

Transmiterea mesajelor

Când apelăm o operație pusă la dispoziție de un obiect prin intermediul interfeței sale spunem că transmitem obiectului un mesaj. Obiectul care primește mesajul se numește obiect *apelat* sau *receptor*. Această lecție prezintă câteva aspecte legate de transmiterea mesajelor.

3.1 Supraîncărcarea metodelor

În Java o metodă poate fi declarată doar în interiorul unei clase. Ea este alcătuită din două componente numite *prototipul* metodei, respectiv *corpul* metodei. Prototipul unei metode este alcătuit din specificatori, tip returnat, nume, lista parametrilor formali și clauza opțională *throws* despre care vom vorbi mai târziu. Corpul unei metode este format dintr-un bloc de instrucțiuni iar în anumite condiții, după cum vom vedea mai târziu, acesta poate lipsi.

Fiecare metodă are o *semnătură*.

Definiție 5 *Semnătura unei metode este alcătuită din numele metodei împreună cu numărul și tipurile parametrilor formali din prototipul metodei.*

Atunci când într-o clasă definim două sau mai multe metode cu același nume spunem că le supraîncărcăm. Pentru a face posibilă distincția la nivelul compilatorului între metodele supraîncărcate trebuie ca semnăturile respectivelor metode să difere fie prin numărul de parametri formali ai metodei fie prin tipurile acestora.



Atunci când cineva spune că tipărește ceva, putem trage concluzia că o anumită informație va fi tipărită pe un suport fizic. Suportul fizic pe care va fi tipărită informația, dacă acesta prezintă interes, poate fi dedus din contextul în care se află persoana care realizează tipărirea: un programator va tipări pe ecran, un ziarist va tipări într-un ziar, etc. Așa stau lu-

crurile și cu o metodă supraîncărcată. Spre exemplu, e suficient să știm că metoda *System.out.println(...)* tipărește pe ecran informația pe care o transmitem acesteia prin intermediul parametrilor. Dintr-un apel concret al acesteia putem afla exact și ce tip de informație se va tipări (un întreg, un boolean).

Atenție

Metodele de mai jos sunt supraîncărcate, deși la prima vedere au aceeași listă de parametri. În acest caz distincția dintre ele se face în funcție de ordinea tipurilor parametrilor actuali din cadrul apelului metodei.

```
void tipareste(String sir, boolean b) {  
    if(b) System.out.println("String: " + sir);  
}  
  
void tipareste(boolean b, String sir) {  
    if(b) System.out.println("String: " + sir);  
}
```

Atenție

În literatură *supraîncărcarea* metodelor este cunoscută sub numele de *overloading*.

3.1.1 Supraîncărcarea constructorilor

Constructorii sunt în cele din urmă un fel de metode și, prin urmare, pot fi supraîncărcați. Supraîncărcarea constructorilor permite instanțierea unui obiect în mai multe moduri, pentru fiecare mod existând câte un constructor. Spre exemplu, clasa *Valoare* de mai jos are doi constructori și această fapt permite instanțierea obiectelor de acest tip în două moduri.

```
class Valoare {  
  
    private int valoare;  
  
    public Valoare() {  
        valoare = 0;  
    }  
  
    public Valoare(int v) {  
        valoare = v;  
    }  
  
    public void seteazaValoare(int v) {  
        valoare = v;  
    }  
}
```

```
public void tipareste() {  
    System.out.println("Valoarea mea: " + valoare);  
}  
}
```



Metoda *seteazaValoare* din interiorul clasei *Valoare* se numește *metodă accesor* pentru că singurul ei scop e setarea unei valori.

Datorită existenței celor doi constructori, ambele moduri de instanțiere a obiectelor de mai jos sunt corecte.

```
Valoare o1 = new Valoare();  
Valoare o2 = new Valoare(2);
```

3.1.2 Supraîncărcarea și tipurile de date primitive

O variabilă de tip primitiv poate fi automat convertită spre un tip primitiv mai larg. Această conversie poate produce confuzii atunci când e vorba despre identificarea exactă a unei metode supraîncărcate care se apelează.

```
class ParametriPrimitivi {  
  
    public static void care(float f) {  
        System.out.println("FLOAT " + f);  
    }  
  
    public static void care(double f) {  
        System.out.println("DOUBLE " + f);  
    }  
  
    public static void main(String[] args) {  
        care(3.14);  
        care(5);  
    }  
}
```

Spre exemplu, rezultatele produse de execuția funcțiilor de mai sus vor fi

```
DOUBLE 3.14 //3.14 este implicit un double iar pentru a fi tratat  
            //ca un float el trebuie succedat de sufixul f  
FLOAT 5.0
```

Dacă există o metodă supraîncărcată al cărei parametru formal are exact același tip ca și parametrul actual, se apelează acea metodă. Dacă, în schimb, nu există nici o metodă cu proprietatea de mai sus, atunci se va încerca o conversie automată de *lărgire* a tipului primitiv și se va apela metoda având parametrul formal cel mai apropiat ca tip, dar mai mare decât cel al parametrului actual.

Având în vedere că un *int* poate fi convertit automat la oricare din tipurile de mai jos

- *int* - *long*, *float* sau *double*

și că avem două metode *care*, una cu un parametru formal de tip *float*, alta cu un parametru formal de tip *double* și că tipul *float* e *cel mai apropiat* de tipul *int*, se va apela metoda *care* cu parametrul formal de tip *float*.

Important

Tipul cel mai apropiat de tipul *char* este tipul *int*.

O variabilă de tip primitiv NU poate fi automat convertită spre un tip primitiv mai mic, conversia spre un tip primitiv mai mic fiind posibilă doar prin intermediul conversiilor explicite.

3.2 Transmiterea parametrilor

Atunci când definim o metodă ce are parametri, parametrii acesteia pot fi văzuți ca niște variabile locale ce vor primi valori concrete doar atunci când se apelează metoda. Parametrii din cadrul prototipului unei metode se numesc *parametri formali*. Parametrii ce se pun în punctul de apel al unei metode se numesc *parametri actuali*.

Există două moduri de transmitere a parametrilor:

- **prin valoare** – parametri formali vor fi înlocuiți cu valorile parametrilor actuali; după execuția metodei valorile parametrilor actuali nu vor fi schimbate.
- **prin referință(adresă)** – parametri formali vor referi exact aceeași zonă de memorie ca și cei actuali; după execuția metodei valorile parametrilor actuali pot fi schimbate.

În Java parametrii se transmit prin **valoare!!!**

În metoda *main* a clasei *TipPrimitiv* de mai jos se ilustrează efectul transmiterii prin *valoare* a unei variabile de tip primitiv. Evident, valoarea parametrului actual va fi *15* și după execuția metodei apelate.

```
class TipPrimitiv {  
  
    public static void trimitValoarePrimitiva(int p) {  
        p = 10;  
    }  
  
    public static void main(String[] arg) {  
        int v = 15;  
        trimitValoarePrimitiva(v);  
        System.out.println("Valoarea lui v: " + v); //v va fi 15  
    }  
}
```

Dacă efectul transmiterii unui parametru de tip primitiv este evident, efectul transmiterii unui parametru de tip referință poate să nu fie așa de evident. În acest caz, la apel, metoda va primi referința unui obiect, ceea ce se transmite *prin valoare* fiind o referință și nu obiectul indicat de referință!!!

Aceasta înseamnă că modificările aduse asupra obiectului referit de parametrul *o* din exemplul de mai jos vor fi vizibile și în metoda *main*. Pe de altă parte, modificarea referinței din cadrul metodei *trimitValoareReferinta2* nu este vizibilă în metoda *main* datorită trimiterii prin *valoare* a referinței.

```
class TipReferinta {  
  
    public static void trimitValoareReferinta1(Valoare o) {  
        o.seteazaValoare(10);  
    }  
  
    public static void trimitValoareReferinta2(Valoare q) {  
        q = new Valoare(50);  
    }  
  
    public static void main(String[] arg) {  
        Valoare v = new Valoare(5);  
        v.tiparire(); //Va afisa Valoarea mea: 5  
  
        trimitValoareReferinta1(v);  
        v.tiparire(); //Va afisa Valoarea mea: 10 deoarece  
                       //obiectul indicat de s si-a modificat continutul  
  
        trimitValoareReferinta2(v);  
        v.tiparire(); //Va afisa Valoarea mea: 10  
                       //din cauza transmiterii referintei prin valoare  
    }  
}
```



Dacă dorim ca unui parametru de tip primitiv din cadrul prototipului unei metode să nu i se poată atribui în interiorul metodei o altă valoare decât cea primită la apel, în cadrul prototipului acesteia trebuie să apară corespunzător modificatorul *final*. Dacă parametrul este o referință, chiar dacă este specificat ca fiind *final*, prin intermediul său se pot aduce modificări obiectului referit dar nu și referinței însăși.

3.3 Cuvântul cheie this

Cuvântul cheie *this* este o referință spre obiectul receptor al unui mesaj. *this* poate să apară doar în interiorul metodelor nestatice ale unei clase precum și în interiorul constructorilor.

```
public void seteazaValoare(int v) {  
    valoare = v;  
    //echivalent cu  
    //this.valoare = v;  
}
```

Există mai multe situații care impun folosirea lui *this*. În continuare se vor exemplifica câteva dintre ele.

- Conflicte de nume – cu ajutorul lui *this* se poate face distincție între un atribut aferent obiectului receptor al unui mesaj și un parametru formal al mesajului, atunci când atributul, respectiv parametrul formal au același nume.

```
public void seteazaValoare(int valoare) {  
    this.valoare = valoare;  
}
```

- O metodă trebuie să returneze ca rezultat o referință la obiectul ei receptor.
- Referința la obiectul receptor trebuie transmisă ca parametru la apelul unei metode.
- Apelul unui constructor din alt constructor – în clasa *Valoare* vrem să adăugăm un constructor care primește ca parametru o referință la un obiect de tip *Valoare* și vrem ca obiectul instanțiat în acest mod să aibă atributul său *valoare* egal cu cel al obiectului referit de *v*.

```
public Valoare(Valoare v) {  
    this(v.valoare);  
    //se apeleaza constructorul Valoare(int v)  
}
```

Atenție

Apelul unui constructor din interiorul altui constructor trebuie să fie prima instrucțiune din cadrul constructorului apelant, în caz contrar compilatorul va semnaliza o eroare. Apelul unui constructor din interiorul altui constructor nu înseamnă crearea unui alt obiect ci doar apelarea codului celui alt constructor (pentru inițializări).

Atenție

Un constructor nu poate fi apelat cu ajutorul lui *this* din interiorul altor metode, ci doar din interiorul unui constructor.

Atenție

Referința *this* nu poate fi folosită în interiorul unei metode statice deoarece metoda statică aparține unei clase și, de obicei, aceasta nu se execută pentru un obiect.

Atenție

Referința *this* la obiectul receptor al unui mesaj nu poate fi modificată, adică acesteia nu i se poate atribui o altă valoare. Deci obiectul receptor al unui mesaj nu poate fi modificat pe parcursul tratării mesajului.

3.4 Metodele clasei Object

În Java orice clasă pe care o definim se află într-o relație specială cu o clasă numită *Object*. Ca urmare a acestei relații, orice clasă definită are și ea toate metodele clasei *Object*. În Tabelul 3.1 sunt prezentate câteva metode ale clasei *Object*.

Prototipul	Descriere
<code>Object clone()</code>	Crează și returnează o referință la o clonă a obiectului receptor
<code>boolean equals(Object obj)</code>	Testează dacă obiectul referit de <code>obj</code> este egal cu cel receptor
<code>void finalize()</code>	Se apelează la distrugerea obiectului de către Colectorul de reziduuri (Garbage Collector)
<code>int hashCode()</code>	Returnează codul hash al obiectului receptor
<code>String toString()</code>	Returnează o reprezentare sub formă de <code>String</code> a obiectului receptor

Tabelul 3.1: CÂTEVA METODELE ALE CLASEI OBJECT.

3.4.1 Compararea obiectelor

După cum am văzut în Tabelul 1.4, atunci când comparăm valorile a două variabile de tip primitiv, folosim operatorul relațional `==`.

```
class TestareIdentitate {  
  
    public static void main(String[] arg) {  
  
        Valoare v1 = new Valoare(10);  
        Valoare v2 = new Valoare(10);  
  
        if(v1 == v2)  
            System.out.println("Obiectele sunt egale");  
        else  
            System.out.println("Obiectele NU sunt egale");  
    }  
}
```

În Java operatorul relațional `==` poate fi aplicat și variabilelor de tip referință, ca în exemplul anterior. Poate surprinzător, rezultatul comparației de mai sus este fals și în consecință pe ecran se va afișa:

```
Obiectele NU sunt egale
```

De fapt, atunci când aplicăm operatorul `==` la două referințe ceea ce comparăm este *identitatea fizică* a obiectelor referite, adică se verifică dacă cele două referințe indică același obiect.

Dacă dorim să testăm *echivalența* dintre două obiecte, adică dacă două obiecte au conținut identic, este bine să folosim metoda *equals* pe care orice clasă o are datorită relației speciale dintre ea și clasa *Object*. Dar această metodă *equals* trebuie *modificată* pentru fiecare clasă în parte astfel încât apelul acesteia să compare două obiecte din punct de vedere al *echivalenței* (vom vedea mai târziu, de fapt, cum se numește această *modificare*). Fără *modificarea* de mai sus, metoda *equals* va testa *identitatea fizică* dintre obiectul receptor al metodei și cel referit de parametrul metodei și nu *echivalența* din punct de vedere al conținutului acestora.

```
class Valoare {  
  
    ...  
    public boolean equals(Object o) {  
        if(o instanceof Valoare)  
            return (((Valoare)o).valoare == valoare);  
        else  
            return false;  
    }  
}
```

În interiorul metodei *equals* din cadrul clasei *Valoare* a trebuit să testăm dacă într-

adevăr s-a trimis o instanță a clasei *Valoare* folosind operatorul *instanceof*. Dacă acest lucru s-a întâmplat, atunci rezultatul testării *echivalenței* dintre cele două obiecte este dat de aplicarea operatorului `==` celor două atribute primitive.



clase.

Majoritatea claselor predefinite din Java oferă o implementare adecvată a metodei *equals* astfel încât aceasta să compare obiectele implicate din punct de vedere al *echivalenței* și nu al *identității*. Trebuie însă să consultați paginile de manual pentru a vedea ce înseamnă echivalența pentru acele

Testarea *echivalenței* dintre obiectele referite de *v1* și *v2* se face în felul următor:

```
if(v1.equals(v2)) {  
    System.out.println("Obiectele sunt egale dpdv al echivalentei");  
} else {  
    System.out.println("Obiectele NU sunt egale dpdv al echivalentei");  
}
```



vorbi mai târziu.

Se poate testa echivalența dintre două obiecte și prin intermediul unei metode care nu se numește *equals* dar existența unei alte metode pentru efectuarea comparației va reduce drastic înțelegerea programelor și va duce la pierderea unor facilități oferite de Java, facilități despre care vom

3.4.2 Metoda *toString*

Metoda *toString* a clasei *Object*, ca și celelalte metode ale acestei clase, este și ea existentă în fiecare clasă pe care o definim. În consecință, fiecare obiect creat de noi are o metodă *toString* pe care o putem oricând apela în program.

```
//1 se apeleaza metoda supraincercata println cu un parametru de tip String  
System.out.println(v1.toString());  
//2 se apeleaza metoda supraincercata println cu un parametru de tip Object  
System.out.println(v1);
```

După cum prezintă Tabelul 3.1, metoda returnează o reprezentare sub formă de *String* a obiectului receptor. Concret, efectul execuției primului apel de mai sus poate fi afișarea pe ecran a textului

```
Valoare@fd13b5  
//pentru o alta executie, codul hash fd13b5 al obiectului v1 va fi altul
```

Reprezentarea implicită sub formă de șir de caractere este formată din numele clasei pe care o instanțiază obiectul receptor al metodei *toString* urmat de @ și de codul hash

al obiectului. În continuare vom vedea care este rostul acestei metode.

Dacă ne uităm în documentația Java disponibilă la adresa <http://java.sun.com/j2se/1.5.0/docs/api/java/io/PrintStream.html> vom vedea că metoda *println* este supraîncărcată, existând printre prototipurile ei următoarele:

```
public void println(String x)
public void println(Object x)
```

Faptul că există o metodă *println* având un parametru de tip *String* va produce apelarea acesteia în primul caz. În al doilea caz, se va apela cea care are parametrul de tip *Object* datorită faptului că orice obiect instanțiat este într-o relație specială cu clasa *Object*. Ceea ce face a doua metodă *println* nu este altceva decât afișarea pe ecran a șirului de caractere returnat de apelul metodei *x.toString()* și, deci, se va tipări același șir de caractere ca și în cazul primului apel.

Reprezentarea sub formă de *String* pe care o oferă implicit clasa *Object* nu este satisfăcătoare. De exemplu, pentru o instanță a clasei *Valoare* reprezentarea sub formă de *String* ar putea fi

```
Valoarea mea este 10
```

Pentru ca reprezentarea sub formă de *String* a unei instanțe a clasei *Valoare* să fie cea de mai sus, metoda *toString* moștenită de la clasa *Object* trebuie *modificată* ca mai jos.

```
class Valoare {
    ...
    public String toString() {
        return "Valoarea mea este " + valoare;
    }
}
```

Dar oare care va fi efectul execuției codului de mai jos?

```
System.out.println("***" + v1);
```

În Java operatorul *+* este folosit și pentru concatenări de șiruri de caractere. Pentru cazul de mai sus al doilea operand nu este altcineva decât reprezentarea sub formă de *String* a lui *v1* iar, în consecință, pe ecran se va afișa

```
*** Valoarea mea este 10
```

Atenție

Nu este bine să avem în clasa *Valoare* o metodă de genul:

```
public void afiseaza() {  
    System.out.println("Valoarea mea este " + valoare);  
}
```

3.4.3 Eliberarea memoriei ocupate de obiecte

Până acum am văzut cum se crează dinamic un obiect dar nu am spus nimic despre eliberarea memoriei ocupate de către acesta.

În Java, programatorul nu trebuie să elibereze explicit zone de memorie ocupate de obiectele instanțiate, acest lucru fiind realizat automat de către suportul de execuție. Unul din cazurile în care un obiect devine automat candidat la ștergere (prin ștergere înțelegându-se *eliberarea zonei de memorie alocate pentru el la crearea sa*) este atunci când nu mai există nici o referință spre el. În exemplul de mai jos, primul obiect instanțiat va deveni candidat la ștergere deoarece *v* referă după a doua instrucțiune un alt obiect și nu există o altă referință spre primul obiect creat. Este sarcina colectorului de reziduuri (garbage collector - GC) să elibereze memoria ocupată de primul obiect.

```
Valoare v;  
v = new Valoare(5);  
v = new Valoare(10);
```

Atenție

Nu este obligatoriu ca un obiect să fie șters din memorie imediat ce dispare ultima referință la el. Cert este că el devine candidat la ștergere iar momentul în care se întâmplă ștergerea depinde de gradul de ocupare al memoriei heap.

Atenție

Putem forța ștergerea din memorie a tuturor obiectelor candidate pentru această operație invocând explicit colectorul de reziduuri *System.gc()*.

Atunci când colectorul de reziduuri eliberează zona de memorie ocupată de un obiect, el invocă metoda *finalize*:

```
protected void finalize() throws Throwable
```

Evident, această metodă trebuie *modificată* atunci când e nevoie să se efectueze anumite operații speciale la ștergerea unui obiect din memorie.

Atenție

E posibil ca un obiect să nu fie colectat de către GC pe întreg parcursul execuției unui program, caz în care metoda *finalize* nu va fi deloc apelată.

3.5 Exerciții

1. Metodele de mai jos sunt supraîncărcate?

```
public void faCeva(int x) {...}
public int  faCeva(int x) {...}
```

2. O carte este caracterizată printr-un număr de pagini. Spunem că două cărți sunt identice dacă acestea au același număr de pagini. Creați clasa *Carte* și atașați-i o metodă *potrivită* pentru compararea a două cărți. Apelați metoda care realizează compararea a două cărți într-o metodă *main*.
3. Un pătrat este caracterizat de latura sa. Scrieți o clasă *Patrat* ce are doi constructori, un constructor fără nici un parametru care setează latura pătratului ca fiind 10 iar altul care setează latura cu o valoare egală cu cea a unui parametru transmis constructorului. Atașați clasei o metodă *potrivită* pentru tipărirea unui pătrat sub forma "*Patrat*" l "*Aria*" a, unde l este valoarea laturii iar a este valoarea ariei pătratului. Creați într-o metodă *main* diferite obiecte de tip *Patrat* și tipăriți-le.
4. Creați o clasă *Piramida* ce are un atribut întreg n. Atașați clasei o metodă *potrivită* pentru tipărirea unei piramide ca mai jos:

```
1 1 1 1
2 2 2
3 3
4 --> n
```

Creați într-o metodă *main* diferite obiecte de tip *Piramida* și tipăriți-le.

5. Definiți o clasă *Suma* cu metodele statice de mai jos:

```
// returneaza suma dintre a si b
a) public static int suma(int a, int b) ...
//returneaza suma dintre a, b si c
b) public static int suma(int a, int b, int c) ...
// returneaza suma dintre a, b, c si d
c) public static int suma(int a, int b, int c, int d) ...
```

Implementați metodele astfel încât fiecare metodă să efectueze o singură adunare. Apelați-le dintr-o metodă *main*.

Bibliografie

1. Bruce Eckel. *Thinking in Java, 4th Edition*. Prentice-Hall, 2006. Capitolul Initialization & Cleanup.
2. *Java Language Specification*. <http://java.sun.com/docs/books/jls/>, Capitolul 8.4, Method Declarations.

Lecția 4

Câteva clase Java predefinite

În Lecția 2 am văzut cum definim și creăm un obiect într-un limbaj de programare obiectual, în particular Java. Pentru a fixa mai bine noțiunile independente de limbaj clasă și obiect dar și pentru a învăța anumite particularități ale limbajului Java vom trece în revistă câteva clase predefinite.

4.1 Clasa String

4.1.1 Șirul de caractere în Java

În Java nu există tipul primitiv șir de caractere. Orice șir de caractere este un obiect instanță a *String*, iar variabilele de tip *String* nu sunt altceva decât referințe la obiecte șir de caractere. Conform cu modul de creare a obiectelor studiat în Lecția 2, pentru a crea un șir de caractere putem proceda după cum urmează:

```
String sir = new String("Un sir de caractere");
```

Datorită faptului că șirurile de caractere sunt frecvent utilizate în cadrul programelor, compilatorul de Java vine în ajutorul programatorilor simplificând lucrul cu șirurile de caractere. Ca urmare, orice constantă șir de caractere poate fi folosită ca o referință la un obiect *String*, compilatorul fiind cel care se ocupă de crearea obiectului propriu-zis. Prin urmare, putem declara și inițializa o variabilă șir de caractere și în modul prezentat mai jos.

```
String sir = "Un sir de caractere";
```

Trebuie amintit aici că cele două secvențe de cod nu sunt perfect echivalente deoarece constanta "Un sir de caractere" este deja un obiect. Prin urmare, în al doilea exemplu variabila *sir* va referi obiectul creat implicit de compilator, pe când în primul exemplu variabila va referi un obiect *String* ce copiază conținutul obiectului creat implicit

de compilator. De cele mai multe ori însă, acest comportament diferit nu e foarte important.

4.1.2 Operații cu șiruri de caractere

În Tabelul 4.1 prezentăm un subset de constructori și metode definite de clasa *String* pentru lucrul cu șiruri de caractere. Mult mai multe pot fi consultate în documentația Java, la adresa web <http://java.sun.com/j2se/1.5.0/docs/api/java/lang/String.html>.

Una dintre metodele prezentate este *concat(String sir)* care crează un nou obiect *String* (adică un șir de caractere) obținut prin concatenarea șirului apelat (adică a secvenței de caractere corespunzătoare obiectului apelat) cu șirul dat ca parametru. Același efect se obține și prin utilizarea operatorului '+' cu operanzi șiruri de caractere. Prin urmare, următoarele porțiuni de cod conduc la crearea unui obiect *String* ce va fi referit de variabila *sir2*, obiect ce conține secvența de caractere "Un sir de caractere".

```
String sir1 = "Un sir ";  
String sir2 = sir1.concat("de caractere");
```

```
String sir1 = "Un sir ";  
String sir2 = sir1 + "de caractere";
```

Atenție

Operația de concatenare produce un alt șir de caractere (alt obiect) și nu-l modifică pe cel asupra căruia se aplică metoda *concat*. La fel se întâmplă și în cazul altor metode care realizează anumite transformări ale șirurilor de caractere. Cu alte cuvinte, un șir de caractere odată creat nu mai poate fi modificat.

Atenție

Este corect să folosim operatorul + pentru a concatena două șiruri de caractere, dar nu este corect să utilizăm operatorul == pentru a vedea dacă două șiruri conțin aceeași secvență de caractere.

O altă metodă importantă e *equals(Object o)*. Ea testează dacă șirul de caractere apelat și cel dat ca parametru conțin aceeași secvență de caractere. În acest context este foarte important să înțelegem rolul operatorului ==. El testează dacă valoarea celor doi operanzi este identică. Variabilele de tip referință (cum sunt și variabilele de tip *String*) au ca valoare o referință la un obiect, iar egalitatea a două astfel de variabile denotă că ele referă același obiect! Prin urmare, dacă două variabile *String* sunt egale înseamnă că ele referă același obiect și implicit aceeași secvență de caractere. Dar două obiecte diferite pot și ele conține aceeași secvență de caractere, lucru ce nu poate fi testat prin ==. Pentru a testa acest aspect trebuie folosită metoda *equals* prezentată în Tabelul 4.1.

Prototipul	Descriere
<code>String()</code>	Crează un obiect <i>String</i> corespunzător șirului de caractere vid
<code>String(String str)</code>	Crează un obiect <i>String</i> corespunzător aceluiași șir de caractere ca și șirul corespunzător obiectului argument
<code>char charAt(int index)</code>	Returnează caracterul aflat pe poziția <i>index</i> în șirul de caractere apelat
<code>String concat(String str)</code>	Returnează un nou obiect <i>String</i> reprezentând șirul de caractere obținut prin concatenarea șirului dat ca argument la sfârșitul șirului apelat. Dacă <i>str</i> este șirul vid atunci se returnează obiectul apelat
<code>boolean endsWith(String suffix)</code>	Testează dacă șirul apelat se termină cu șirul <i>suffix</i>
<code>boolean equals(Object str)</code>	Testează dacă șirul apelat reprezintă aceeași secvență de caractere ca și șirul dat ca parametru (nu același obiect șir de caractere !!!)
<code>int indexOf(int ch)</code>	Returnează cea mai din stânga poziție din șirul apelat în care apare caracterul <i>ch</i> . Dacă <i>ch</i> nu apare în șir atunci se returnează -1
<code>int indexOf(String str)</code>	Returnează cea mai din stânga poziție din șirul apelat de la care apare subșirul dat ca argument. Dacă <i>str</i> nu apare ca subșir se returnează -1
<code>String intern()</code>	Returnează referința la șirul de caractere care conține aceeași secvență de caractere ca și obiectul apelat (sunt egale din punctul de vedere a lui <code>equals</code>) și care a fost primul astfel de șir de caractere pentru care s-a apelat metoda <i>intern</i>
<code>int lastIndexOf(int ch)</code>	Returnează cea mai din dreapta poziție din șirul apelat în care apare caracterul <i>ch</i> . Dacă <i>ch</i> nu apare în șir atunci se returnează -1
<code>int lastIndexOf(String str)</code>	Returnează cea mai din dreapta poziție din șirul apelat de la care apare subșirul dat ca argument. Dacă <i>str</i> nu apare ca subșir se returnează -1
<code>int length()</code>	Returnează lungimea șirului de caractere apelat
<code>boolean startsWith(String prefix)</code>	Testează dacă șirul apelat începe cu șirul <i>prefix</i>
<code>String substring(int beginIndex)</code>	Returnează un nou șir de caractere reprezentând subșirul de caractere din șirul apelat care începe la poziția dată ca argument
<code>String toUpperCase()</code>	Returnează un nou șir de caractere conținând același șir de caractere ca și cel apelat dar în care toate literele mici sunt convertite în litere mari

Tabelul 4.1: CONSTRUCTORI ȘI METODE PENTRU OBIECTE DE TIP STRING.

```
String sir1 = "Un sir";
String sir2 = "Un sir";

if(sir1 == sir2) {
    System.out.println("sir1 este egal cu sir2");
}
```

Explicația de mai sus ar putea fi suspectă ca eronată la rularea porțiunii de cod prezentate mai sus. La rulare, mesajul “sir1 este egal cu sir2” este tipărit pe ecran, deși la prima vedere *sir1* și *sir2* nu referă același obiect. Prin urmare s-ar putea crede că operatorul `==` testează egalitatea secvenței de caractere dintr-un obiect șir de caractere. Ei bine, este total eronat. Mesajul este afișat pentru că *sir1* și *sir2* referă același obiect. În general, orice expresie constantă care are ca valoare un șir de caractere este “internalizată” automat de compilator. Mai exact, pe obiectul *String* ce reprezintă valoarea sa este apelată metoda *intern* iar rezultatul dat de ea este returnat ca valoare a expresiei. Urmărind descrierea metodei *intern* din Tabelul 4.1 este clar de ce *sir1* și *sir2* referă același obiect.



Ca să ne convingem că orice șir de caractere dintr-un program Java este un obiect încercați următorul cod. Surpriză! Este chiar corect.

```
class Surpriza {

    public static void main(String argv[]) {
        System.out.println("Acest sir are lungimea 25".length());
    }
}
```

4.1.3 Alte metode definite de clasa String

În plus față de metodele prezentate în Tabelul 4.1, clasa *String* mai definește un set de metode statice. Reamintim că metodele statice nu sunt executate pe un obiect, ele reprezentând operații ce caracterizează clasa căreia aparțin și nu obiectele definite de respectiva clasă. În principiu, după cum se poate observa din Tabelul 4.2, aceste metode sunt utile pentru obținerea reprezentării sub formă de șir de caractere a valorilor corespunzătoare tipurilor primitive.

Prototipul	Descriere
String valueOf(boolean b)	Returnează un șir de caractere corespunzător valorii logice date de argumentul <i>b</i>
String valueOf(char c)	Returnează un șir de caractere format numai din caracterul <i>c</i>
String valueOf(double d)	Returnează un șir de caractere corespunzător valorii argumentului <i>d</i>
String valueOf(float f)	Returnează un șir de caractere corespunzător valorii argumentului <i>f</i>
String valueOf(int i)	Returnează un șir de caractere corespunzător valorii argumentului <i>i</i>
String valueOf(long l)	Returnează un șir de caractere corespunzător valorii argumentului <i>l</i>

Tabelul 4.2: UN SUBSET DE METODE STATICE DEFINITE DE CLASA STRING.

4.2 Clase înfășurătoare

4.2.1 Orice ar putea fi obiect

Într-un limbaj de programare pur obiectual, totul e considerat a fi obiect și tratat ca atare. Ca urmare, inclusiv valorile numerice sau logice corespunzătoare tipurilor primitive sunt considerate a fi obiecte. Java nu merge chiar atât de departe, dar predefinește câte o clasă pentru fiecare tip primitiv, denumită clasă înfășurătoare pentru acel tip primitiv. O clasă înfășurătoare poate fi folosită de programatori dacă se dorește tratarea unei valori corespunzătoare tipului primitiv asociat ca obiect. Fiecare clasă înfășurătoare are o denumire sugestivă care spune aproape totul despre ea: *Boolean*, *Character*, *Byte*, *Short*, *Integer*, *Long*, *Float*, *Double*.

În general, toate clasele înfășurătoare prezintă constructori și metode similare. Din acest motiv noi ne vom rezuma aici doar asupra constructorilor și metodelor definite de clasa *Integer* amintind punctual eventuale particularități ale celorlalte clase acolo unde este necesar (vezi Tabelul 4.3). Detalii despre metodele definite de clasele înfășurătoare pot fi găsite în documentația Java la adresa web <http://java.sun.com/j2se/1.5.0/docs/api/>. Pe lângă aceste metode, clasa *Integer* definește și metode statice pe care le prezentăm în Tabelul 4.4.

4.2.2 Mecanismul de autoboxing și unboxing

După cum am văzut în secțiunea anterioară, utilizarea claselor înfășurătoare permite tratarea valorilor tipurilor primitive ca și obiecte. Spre deosebire însă de șirurile de caractere care sunt întotdeauna obiecte, tratarea valorilor tipurilor primitive ca și obiecte trebuie realizată explicit de către programator. Să considerăm un exemplu concret.

Prototipul	Descriere	În alte clase înfășurătoare
Integer(int value)	Crează un obiect de tip <i>Integer</i> pentru valoarea <i>int</i> dată ca parametru	Fiecare clasă are un constructor asemănător ce ia ca parametru o valoare de tipul asociat clasei respective (de exemplu, <i>Double</i> are un constructor <i>Double(double value)</i>)
Integer(String s)	Crează un obiect de tip <i>Integer</i> pentru valoarea <i>int</i> obținută prin conversia șirului de caractere dat ca parametru la <i>int</i>	Cu excepția clasei <i>Character</i> , toate celelalte au un constructor asemănător convertind șirul de caractere la o valoare de tipul asociat clasei respective
String toString()	Returnează un șir de caractere reprezentând valoarea <i>int</i> conținută de obiect	Toate clasele definesc o astfel de metodă returnând șirul de caractere ce reprezintă valoarea înfășurată
int intValue()	Returnează valoarea de tip <i>int</i> înfășurată	Toate clasele definesc o metodă ce returnează valoarea înfășurată (de exemplu, <i>Character</i> definește <i>char charValue()</i>). Pentru tipurile numerice se definește câte o metodă pentru a returna aceeași valoare convertită la un alt tip numeric (<i>Double</i> definește și <i>intValue()</i>)
boolean equals(Object o)	Testează dacă obiectul apelat și cel dat ca parametru înfășoară aceeași valoare <i>int</i>	Toate clasele definesc această metodă
int compareTo(Integer i)	Returnează o valoare negativă dacă obiectul apelat înfășoară o valoare <i>int</i> mai mică decât cel dat ca parametru, 0 dacă sunt egale, și o valoare pozitivă în caz contrar	Toate clasele definesc o astfel de metodă care primește ca parametru o instanță a aceleiași clase. Citiți documentația Java pentru a vedea ce se întâmplă în cazul clasei <i>Boolean</i>

Tabelul 4.3: CONSTRUCTORI ȘI METODE PENTRU OBIECTE DE TIP INTEGER.

Prototipul	Descriere	În alte clase înfășurătoare
<code>Integer.valueOf(int value)</code>	Construiește un obiect <i>Integer</i> ce înfășoară valoarea dată ca parametru. Este bine să utilizăm această metodă atunci când nu trebuie să creăm efectiv obiectul deoarece metoda folosește un cache pentru a nu mai crea noi obiecte dacă o aceeași valoare <i>int</i> este înfășurată de mai multe ori	Cu excepția clasei <i>Character</i> , toate clasele înfășurătoare definesc o astfel de metodă statică, având ca parametru o valoare corespunzătoare tipului înfășurat (de exemplu, <i>Double</i> definește <i>Double.valueOf(double value)</i>)
<code>int parseInt(String s)</code>	Returnează valoarea întreagă reprezentată în șirul de caractere dat ca parametru	Cu excepția clasei <i>Character</i> , toate clasele înfășurătoare definesc o astfel de metodă statică ce convertește șirul de caractere într-o valoare corespunzătoare tipului asociat clasei respective (de exemplu <i>Boolean</i> definește <i>Boolean.parseBoolean(String s)</i>)

Tabelul 4.4: METODE STATICE DEFINITE DE CLASA INTEGER.

```
class Autoboxing {

    public static void tiparesteIntreg(Integer x) {
        System.out.println("Intregul este:" + x);
    }

    public static void main(String argv[]) {
        tiparesteIntreg(5);
    }
}
```

În secvența de cod de mai sus există o problemă. Parametrul metodei *tiparesteIntreg* trebuie să fie o referință la un obiect de tip *Integer* și nu o valoare de tip *int*! Prin urmare, compilarea acestui cod va genera o eroare ¹. Pentru a rezolva problema fără a modifica metoda *tiparesteIntreg*, va trebui să creăm un obiect de tip *Integer* care să înfășoare valoarea 5 și pe care să-l trimitem ca parametru metodei. Prin urmare, apelul corect al metodei este:

```
tiparesteIntreg(new Integer(5));
```

¹Dacă se utilizează un compilator anterior versiunii 1.5.

Generalizând, este posibil ca la un moment dat să deținem o referință la un obiect ce înfășoară o valoare de tip primitiv iar noi să avem nevoie în acel moment de valoarea propriu-zisă. Ca urmare, va trebui să apelăm explicit o metodă corespunzătoare pentru a determina această valoare (de exemplu pentru un obiect *Integer* valoarea înfășurată e dată de metoda *intValue()*). Pe de altă parte este posibil ca la un moment dat să deținem o valoare de un anumit tip primitiv, iar noi să avem nevoie de tratarea respectivei valori ca un obiect, adică avem nevoie de o referință la un obiect ce înfășoară respectiva valoare. Ca urmare, va trebui să creăm explicit obiectul ca în exemplul de mai sus.

Toate aceste operații care trebuie realizate explicit de către programator *aglomerează* artificial codul sursă al programului. Pentru a veni în ajutorul programatorilor, începând cu versiunea 1.5 a limbajului Java, compilatorul de Java furnizează așa numitele mecanisme de *autoboxing* și *unboxing*. Aceste mecanisme vin să rezolve problema amintită mai sus, permițându-ne să ignorăm diferențele dintre un tip primitiv și tipul definit de clasa înfășurătoare asociată respectivului tip primitiv. Astfel, începând cu versiunea Java 1.5, exemplul dat la începutul acestei secțiuni compilează fără eroare! Acest lucru se întâmplă pentru că mecanismul de *autoboxing* crează automat un obiect înfășurător pentru valoarea 5 care va fi dat ca argument metodei *tiparesteIntreg*. Cu alte cuvinte compilatorul Java 1.5 crează implicit obiectul pe care programatorul era obligat să-l creeze explicit dacă folosea un compilator anterior versiunii 1.5.

Mecanismul de *unboxing* este inversul mecanismului de *autoboxing*. Să considerăm secvența de mai jos.

```
class Unboxing {  
  
    public static void tiparesteIntreg(Integer x) {  
        System.out.println(5 + x);  
    }  
  
    public static void main(String argv[]) {  
        tiparesteIntreg(new Integer(5));  
    }  
}
```

Pentru un compilator de Java anterior versiunii 1.5 exemplul de mai sus conține o eroare. Nu se poate aduna o referință la un obiect, în acest caz de tip *Integer*, cu valoarea 5 pentru că pur și simplu nu are sens. Pentru compilatorul de Java 1.5 codul este corect datorită mecanismului de *unboxing*. Astfel, compilatorul își dă seama că trebuie să adune o valoare întreagă cu valoarea înfășurată de obiectul referit de parametrul *x* pentru că obiectul respectiv este instanță a clasei *Integer*. Ca urmare, compilatorul apelează în mod implicit metoda *intValue()* pentru obiectul referit de *x*, iar valoarea returnată este apoi adunată la 5. La execuție, acest program va afișa valoarea 10.

Concluzionând, mecanismele de autoboxing și unboxing introduse în Java 1.5 simplifică sursa programelor permițând tratarea valorilor corespunzătoare tipurilor primitive ca și instanțe ale claselor înfășurătoare corespunzătoare, respectiv tratarea instanțelor claselor înfășurătoare ca și valori primitive corespunzătoare.

4.3 Clase destinate operațiilor de intrare-ieșire

Există un număr mare de clase predefinite în Java destinate realizării operațiilor de intrare-ieșire. Explicarea exactă a rolului fiecărei clase și a posibilităților de combinare a instanțelor lor este destul de dificilă în acest moment, când încă nu cunoaștem toate mecanismele specifice programării orientate pe obiecte. Din acest motiv ne vom limita la exemple de citire a șirurilor de caractere (conversia lor la valori corespunzătoare tipurilor primitive a fost descrisă mai devreme în această lucrare) și la exemple de realizare a operațiilor de ieșire.

Abstracțiunea de bază utilizată în cadrul operațiilor de intrare și ieșire este fluxul de intrare, respectiv fluxul de ieșire. Un flux de intrare poate fi văzut ca o secvență de “entități” care “vin” sau care “curg” către un program din exteriorul programului respectiv. Analog, un flux de ieșire poate fi văzut ca o secvență de “entități” care “pleacă” sau care “curg” dinspre un program spre exteriorul său. La un moment dat, pot exista mai multe fluxuri de intrare și de ieșire pentru un program.

La cel mai primitiv nivel o astfel de “entitate” este octetul, vorbindu-se astfel de flux de intrare de octeți respectiv de flux de ieșire de octeți. În Java aceste abstracțiuni se numesc *InputStream* respectiv *OutputStream*. În general, un obiect corespunzător acestor abstracțiuni știe să citească următorul octet din flux (metoda *read()*), respectiv știe să scrie următorul octet în flux (metoda *write(int b)*). În ambele cazuri obiectul știe să închidă fluxul prin metoda *close()*.

4.3.1 Citirea liniilor de text

La inițializarea unui program Java, se crează implicit un obiect corespunzător abstracțiunii *InputStream* care știe să ne furnizeze prin metoda *read()* un octet provenit de la intrarea standard a programului reprezentată de cele mai multe ori de tastatură. Acest obiect poate fi accesat prin referința *System.in*.

Dacă se dorește citirea dintr-un fișier vom crea un obiect corespunzător abstracțiunii *InputStream* prin instanțierea clasei *FileInputStream*. Acest obiect știe să ne furnizeze următorul octet disponibil din fișier prin metoda *read()*.

```
FileInputStream file_stream = new FileInputStream("fisierul_meu.txt");
```



Pentru unii ar putea fi greu de înțeles cum atât obiectul *System.in* cât și un obiect instanță a clasei *FileInputStream* poate corespunde abstracțiunii *InputStream*. Imaginați-vă că pe cineva îl interesează cât este ora la un moment dat. Pentru a rezolva această problemă are nevoie de un obiect corespunzător abstracțiunii Ceas care știe să-i spună cât e ora. Pentru a afla efectiv ora persoana respectivă poate să se uite la un ceas de mână dar poate să se uite și la telefonul său mobil. Prin urmare, ambelor obiecte le corespunde aceeași abstracțiune din perspectiva întrebării “Cât este ora?”. Asemănător, din perspectiva întrebării “Care e următorul octet din fluxul de intrare?” atât obiectului *System.in* cât și oricărei instanțe a clasei *FileInputStream* le poate corespunde aceeași abstracțiune, în cazul nostru *InputStream*.

Utilizând obiectele descrise mai sus, am putea să citim informații de la tastatură sau dintr-un fișier octet cu octet. Totuși, acest lucru nu ne prea ajută dacă vrem să citim un caracter sau, mai rău, un șir de caractere.

Atenție

În Java un caracter nu e reprezentat printr-un octet care conține codul ASCII al caracterului respectiv !!!

Prin urmare, ar fi foarte util un alt obiect care să convertească un flux de intrare de octeți într-un flux de intrare de caractere. Un astfel de obiect se poate obține prin instanțierea clasei *InputStreamReader*. Constructorul acestei clase primește ca parametru fluxul de intrare de octeți care va fi convertit.

```
InputStreamReader keyboard_char_stream = new InputStreamReader(System.in);

InputStreamReader file_char_stream =
    new InputStreamReader(new FileInputStream("fisierul_meu.txt"));
```

Interfața acestui obiect declară metoda *read()* prin intermediul căreia se poate citi următorul caracter disponibil din fluxul de intrare. Mai rămâne de rezolvat o singură problemă: eficiența citirilor. Citirea unui caracter din fluxul de intrare de caractere implică citirea unui sau mai multor octeți din fluxul de intrare de octeți. Din motive de organizare a discului, citirea octet cu octet a unui fișier este inefficientă ca timp. Pentru a crește eficiența citirilor este de dorit ca la un moment dat să se citească mai mulți octeți într-un singur acces la disc. În cazul nostru acest lucru implică citirea mai multor caractere și păstrarea lor în memorie. Acest lucru se poate realiza prin crearea unui obiect al clasei *BufferedReader* așa cum se arată mai jos.

```
BufferedReader keyboard_char_stream =  
    new BufferedReader(new InputStreamReader(System.in));  
  
BufferedReader file_char_stream =  
    new BufferedReader(new InputStreamReader(  
        new FileInputStream("fisierul_meu.txt")));
```

Sfat

Documentația Java sfătuiește programatorii să utilizeze un obiect *BufferedReader* atunci când se realizează citiri din fișiere dar și pentru a evita lucrul direct cu obiecte *InputStreamReader*, care prezintă o implementare destul de ineficientă a metodei *read()*.



Gândiți-vă că o persoană trebuie să prelucreze într-un anumit fel boabele de grâu dintr-un hambar situat la 1 kilometru distanță de locul unde trebuie realizată prelucrarea. Ar fi cam ineficient ca pentru fiecare bob în parte persoana să se deplaseze la hambar pentru a lua bobul după care să-l prelucreze. Pentru a crește eficiența activității sale persoana va aduce la locul prelucrării un sac de boabe de grâu. Ei bine, clasa *BufferedReader* definește un obiect care conține un astfel de “sac” de caractere. Când sacul se golește obiectul îl umple la loc, eliberându-l pe clientul său de sarcina reîncărcării “sacului” și lăsându-l să se ocupe doar de prelucrarea efectivă a caracterelor.

Odată ce am creat un obiect *BufferedReader* așa cum am arătat mai sus, putem utiliza metodele *read()* și *readLine()* pentru a citi eficient următorul caracter din flux-ul de intrare respectiv pentru a citi o linie de text din același flux. Aceste metode returnează *0* respectiv *null* când nu mai sunt caractere disponibile în fluxul de intrare (a apărut sfârșitul de fișier).

4.3.2 Scrierea liniilor de text

Discuția din secțiunea anterioară este în esență aplicabilă și în cazul operațiilor de ieșire. Singura diferență este că lucrurile trebuie privite în sens invers, dinspre program spre exteriorul său. Vorbim astfel de fluxuri de ieșire de caractere și fluxuri de ieșire de octeți. Clasa *OutputStreamWriter* definește un obiect care transformă un flux de caractere într-un flux de octeți. Acești octeți sunt trimiși apoi spre un obiect *OutputStream* dat ca parametru constructorului clasei *OutputStreamWriter*. După cum se poate observa, lucrurile sunt foarte asemănătoare cu cele prezentate în secțiunea anterioară.

La inițializarea unui program Java se crează implicit un obiect corespunzător abstracțiunii *OutputStream* care știe să scrie prin metoda *write(int b)* un octet la ieșirea standard a programului, reprezentată de cele mai multe ori de monitor. Acest obiect poate fi accesat prin referința *System.out*.

Dacă se dorește scrierea într-un fișier vom crea un obiect corespunzător abstracțiunii *OutputStream* prin instanțierea clasei *FileOutputStream*. În continuare putem folosi un obiect *OutputStreamWriter* care ne permite să lucrăm cu caractere și nu cu octeți. Metoda *write(int b)* a acestui obiect permite scrierea unui caracter în fluxul de ieșire.

```
FileOutputStream file_stream = new FileOutputStream("fisierul_meu.txt");
OutputStreamWriter file_char_stream = new OutputStreamWriter(file_stream);
```

Totuși, utilizând un astfel de obiect, tipărirea de rezultate de către program ar fi cam dificilă (totul ar trebui convertit în caractere). Pentru a rezolva această situație, este mult mai bine să se utilizeze un obiect instanță a clasei *PrintStream* (în realitate *System.out* este un obiect de acest tip).

```
PrintStream file_complex_stream =
    new PrintStream(new FileOutputStream("fisierul_meu.txt"));
```

Un astfel de obiect are în principiu două metode: *print* și *println*. Efectul lor e asemănător. Fiecare metodă convertește valoarea unicului său parametru într-un șir de caractere care va fi convertit apoi mai departe într-o secvență de octeți corespunzătoare. Acești octeți sunt apoi scriși în fluxul de ieșire conținut de obiect. Singura deosebire este că a doua metodă trece la linie nouă după scriere. Este important de știut că cele două metode sunt supraîncărcate, ele putând să ia ca parametru o valoare de tip primitiv, un obiect *String* sau un orice alt obiect.

Important

Nu uitați să închideți un flux în momentul în care nu mai este nevoie de el. Acest lucru se realizează prin apelarea metodei *close()* a obiectului flux. Atenție însă la închiderea fluxurilor reprezentate de obiectele *System.in* și *System.out*.

Sfat

Consultați pagile de manual ale claselor discutate în acest paragraf. Ele pot fi găsite la adresa web <http://java.sun.com/j2se/1.5.0/docs/api/java/io/package-summary.html>. Acolo veți putea vedea că începând cu versiunea 1.5 a limbajului Java, clasa *PrintStream* definește și metode destinate scrierii formate (asemănătoare funcției *printf* din limbajul C).

4.3.3 Exemplu

În continuare vom vedea un exemplu de utilizare a claselor descrise în această secțiune. Programul următor citește un număr de întregi de la tastatură și calculează suma lor. Numerele citite vor fi memorate într-un fișier. În final, suma lor este scrisă la rândul ei în același fișier dar va fi și afișată pe ecran.

```
import java.io.*;

class ExempluIO {

    public static void main(String argv[]) {
        int n,i,suma,temporar;
        try {
            BufferedReader in_stream_char =
                new BufferedReader(new InputStreamReader(System.in));
            PrintStream out_stream = new PrintStream(
                new FileOutputStream("out.txt"));

            System.out.print("Dati numarul de intregi:");
            n = Integer.parseInt(in_stream_char.readLine());

            suma = 0;
            for(i = 1; i <= n; i++) {
                System.out.print("Dati numarul " + i + ":");
                temporar = Integer.parseInt(in_stream_char.readLine());
                suma+= temporar;
                out_stream.println(temporar);
            }

            out_stream.println(suma);
            System.out.println("Suma este:" + suma);
            out_stream.close();

        } catch(IOException e) {
            System.out.println("Eroare la operatiile de intrare-iesire!");
            System.exit(1);
        }
    }
}
```

O particularitate a acestui program este utilizarea blocurilor *try-catch*. Orice operație de intrare-ieșire poate produce erori. De exemplu, accesul la fișierul în care dorim să scriem este restricționat. În astfel de cazuri se generează un “mesaj” de eroare de tip *IOException*. În principiu este obligatoriu ca aceste “mesaje” să fie “prinse” utilizând blocuri *try-catch*. Blocul *try* cuprinde codul asociat execuției normale a programului. Dacă apare o eroare *IOException* se trece în mod automat la execuția instrucțiunilor din blocul *catch* asociat erorii *IOException*.

4.4 Tablouri

În Java tablourile sunt obiecte. În general, declararea unei referințe la un tablou se face în felul următor.

```
tip_elemente[] nume_referinta_tablou;
```

În exemplul de mai sus doar am declarat o referință la un obiect tablou. Ca și în cazul obiectelor obișnuite, tablourile trebuie create explicit folosind operatorul *new*. Mai mult, trebuie specificată și dimensiunea tabloului care nu se mai poate schimba odată ce tabloul a fost creat. În exemplul de mai jos se declară și se inițializează o referință către un tablou care conține zece valori de tip *int* și o referință către un tablou care conține zece referințe la obiecte *Integer*.

```
//Declaratia si crearea unui tablou de zece elemente intregi
int[] tablou_intregi = new int[10];

//Declaratia si crearea unui tablou de zece referinte la obiecte Integer
Integer[] tablou_obiecte_intregi = new Integer[10];
```

Atenție

La crearea celui de-al doilea tablou din exemplu, se alocă memorie pentru zece referințe la obiecte *Integer*. Cu alte cuvinte NU se crează zece obiecte *Integer*. Sarcina creării lor revine programatorului.

Accesul la elementele unui tablou se realizează prin indexare. Cum numele tabloului este o referință la un obiect, indexarea poate fi privită ca apelarea unei metode speciale a obiectului referit. Mai mult, deoarece tabloul este un obiect, el ar putea avea și câmpuri. Există un astfel de câmp special denumit *length* care conține dimensiunea tabloului.

Atenție

Numerotarea elementelor unui tablou începe de la 0.

```
//Initializarea tabloului de intregi
int i;
for(i = 0; i < tablou_intregi.length; i++) {
    tablou_intregi[i] = 0;
}

//Initializarea tabloului de referinte la obiecte Integer
int j;
for(j = 0; j < tablou_intregi.length; j++) {
    tablou_obiecte_intregi[j] = new Integer(0);
}
```

Atenție

Încercarea de a accesa un tablou printr-o referință neinițializată (care nu referă nici un tablou) constituie o eroare.



O metodă poate avea ca și parametri referințe la tablouri. În același timp o metodă poate să aibă ca valoare returnată o referință la un tablou. Un exemplu este prezentat mai jos: metoda primește o referință la un tablou de referință *Integer*, inițializează toate elementele tabloului și întoarce spre apelant referința primită prin parametru.

```
class Utilitare {  
  
    public static Integer[] init(Integer[] tab) {  
        int i;  
        for(i = 0; i < tab.length; i++) {  
            tab[i] = new Integer(0);  
        }  
        return tab;  
    }  
}
```

Discuția de până acum a prezentat toate elementele necesare utilizării tablourilor în programe Java. Mai rămâne de discutat o singură problemă: cum anume putem lucra cu tablouri multi-dimensionale (de exemplu cu matrice). Răspunsul este foarte simplu. După cum am spus un tablou este un obiect. În particular putem avea un tablou de referințe la un anumit tip de obiecte (în exemplele anterioare am avut un tablou de referințe la obiecte *Integer*). Dar cum tablourile sunt obiecte, putem avea tablouri de tablouri (mai exact, tablouri de referințe la tablouri). În exemplul următor arătăm modul de declarare și creare a unei matrice.

```
//Declararea si crearea unei matrice de 10 X 5 elemente  
int[] [] matrice_intregi = new int[10][5];  
  
//Instrucțiunea de mai sus e echivalenta cu urmatoarea  
int i;  
int[] [] matrice_intregi = new int[10][];  
for(i = 0; i < 10; i++) {  
    matrice_intregi[i] = new int[5];  
}
```



Este interesant de observat că în ultimul exemplu nimic nu ne-ar fi oprit să atribuim elementelor tabloului de tablouri referințe spre tablouri de dimensiuni diferite. În exemplul următor vom crea o matrice triunghiulară.

```
//0 matrice triunghiulara
int i;
int[] [] matrice_speciala = new int[10] [];
for(i = 0; i < 10; i++) {
    matrice_speciala[i] = new int[i + 1];
}

//Tiparirea elementelor ei
int a,b;
for(a = 0; a < matrice_speciala.length; a++) {
    for(b = 0; b < matrice_speciala[a].length; b++) {
        System.out.print(matrice_speciala[a][b] + " ");
    }
    System.out.println();
}
```

4.5 Exerciții

1. Rulați și studiați programul dat ca exemplu în Secțiunea 4.3.3.
2. Cum determinați dacă două obiecte *Boolean* înfășoară aceeași valoare logică, fără a utiliza metoda *booleanValue()*? Verificați răspunsul printr-un program Java.
3. Scrieți un program Java care citește de la tastatură o linie de text și numele unui fișier. Programul trebuie să determine și să afișeze pe ecran numărul de linii de text din fișierul indicat care sunt egale cu linia de text citită de la tastatură.
4. Să se scrie un program Java care citește de la tastatură două matrice de numere reale de dimensiune $N \times M$, respectiv $M \times P$, înmulțește cele două matrice și scrie într-un fișier matricea rezultată. Toate matricele trebuie să conțină ca elemente obiecte *Double*.
5. Se dă un fișier “intervale.dat” care conține perechi de întregi pozitivi (câte un întreg pe linie) reprezentând intervale numerice și un număr oarecare de fișiere care conțin numere reale. Să se scrie un program Java care calculează pentru fiecare interval dat procentul de numere reale (din fișierele menționate mai sus) conținute.

Programul trebuie să respecte următoarele cerințe:

- toate numerele reale citite din fișiere trebuie utilizate ca obiecte *Double* sau *Float* și nu ca variabile de tip primitiv *double* sau *float*.
- numele fișierelor ce conțin numerele reale se citesc de la tastatură unul câte unul.
- numerele reale dintr-un fișier nu se prelucrează de mai multe ori; dacă utilizatorul furnizează același fișier de mai multe ori, programul atrage atenția utilizatorului asupra erorii.

- după prelucrarea tuturor fișierelor cu numere reale, statistica trebuie scrisă într-un fișier al cărui nume este specificat ca argument al programului; dacă nu se specifică nici un argument, statistica va fi tipărită în fișierul standard de ieșire (ecran).
- fiecare interval va fi reprezentat printr-un obiect; clasa acestuia va conține câmpuri private pentru limitele intervalului, pentru numărul de numere testate și pentru numărul de numere testate conținute de intervalul respectiv; clasa mai conține: un constructor ce permite inițializarea corespunzătoare a câmpurilor (cele care trebuie inițializate), o metodă de testare ce preia ca parametru un obiect *Double/Float* și care verifică apartenența parametrului la intervalul respectiv actualizând corespunzător câmpurile mai sus amintite și o metodă care ia ca parametru un flux de ieșire în care scrie rezultatele obținute (intervalul respectiv și procentul obținut).

Sfat

Argumentele date unui program sunt disponibile prin parametrul metodei *main*. Acest parametru este un tabloul de șiruri de caractere.

Bibliografie

1. David Flanagan, *Java In A Nutshell. A Desktop Quick Reference*, Third Edition, O'Reilly, 1999.
2. Sun Microsystems Inc., *Online Java 1.5 Documentation*, <http://java.sun.com/j2se/1.5.0/docs/api/>, 2005.

Lecția 5

Relația de moștenire

Între obiectele lumii care ne înconjoară există de multe ori anumite relații. Spre exemplu, putem spune despre un obiect autovehicul că are ca și parte componentă un obiect motor. Pe de altă parte, putem spune că motoarele diesel sunt un fel mai special de motoare. Din exemplul secund derivă cea mai importantă relație ce poate exista între două clase de obiecte: *relația de moștenire*. Practic, relația de moștenire reprezintă *inima* programării orientate pe obiecte.

5.1 Ierarhizarea

La fel ca și noțiunile de abstractizare și încapsulare, ierarhizarea este un concept fundamental în programarea orientată pe obiecte. După cum am învățat în prima lecție, rolul procesului de abstractizare (cel care conduce la obținerea unei abstracțiuni) este de a identifica și separa, dintr-un punct de vedere dat, ceea ce este important de știut despre un obiect de ceea ce nu este important. Tot în prima lecție am văzut că rolul mecanismului de încapsulare este de a permite ascunderea a ceea ce nu este important de știut despre un obiect. După cum se poate observa, abstractizarea și încapsularea tind să micșoreze cantitatea de informație disponibilă utilizatorului unei abstracțiuni. O cantitate mai mică de informație conduce la o înțelegere mai ușoară a respectivei abstracțiuni. Dar ce se întâmplă dacă există un număr foarte mare de abstracțiuni?

Într-o astfel de situație, des întâlnită în cadrul dezvoltării sistemelor software de mari dimensiuni, simplificarea înțelegerii problemei de rezolvat se poate realiza prin ordonarea acestor abstracțiuni formându-se astfel *ierarhii* de abstracțiuni.

Definiție 6 *O ierarhie este o clasificare sau o ordonare a abstracțiunilor.*

Este important de menționat că ordonarea abstracțiunilor nu este una artificială. Între abstracțiuni există de multe ori implicit anumite relații. Spre exemplu, un motor este parte componentă a unei mașini. Într-o astfel de situație vorbim de o relație de tip *part*

of. Ca un alt exemplu, medicii cardiologi sunt un fel mai special de medici. Într-o astfel de situație vorbim de o relație de tip *is a* între clase de obiecte. În cadrul programării orientate pe obiecte, aceste două tipuri de relații stau la baza așa numitelor *ierarhii de obiecte*, respectiv *ierarhii de clase*. În continuare vom discuta despre aceste două tipuri de ierarhii insistând asupra ierarhiilor de clase.



Imaginați-vă că sunteți într-un hipermarket și vreți să cumpărați un anumit tip de anvelopă de mașină. Este absolut logic să vă îndreptați spre raionul denumit “Autovehicule”. Motivul? Anvelopa este *parte componentă* a unei mașini și implicit trebuie să fie parte a raionului asociat acestora. Ar fi destul de greu să găsiți o anvelopă dacă aceasta ar fi plasată pe un raft cu produse lactate din cadrul raionului “Produse alimentare”. Odată ajunși la raionul “Autovehicule” veți căuta raftul cu anvelope. Acolo veți găsi o sumedenie de tipuri de anvelope de mașină, printre care și tipul dorit de voi. Toate au fost puse pe același raft pentru că fiecare este în cele din urmă *un fel de anvelopă*. Dacă ele ar fi fost împrăștiate prin tot raionul “Autovehicule” ar fi fost mult mai complicat să găsiți exact tipul de anvelopă dorit de voi. Acesta este numai un exemplu în care se arată cum relațiile de tip *part of* și *is a* pot conduce la o înțelegere mai ușoară a unei probleme, în acest caz organizarea produselor într-un hipermarket.

5.1.1 Ierarhia de obiecte. Relația de agregare

Ierarhia de obiecte este o ierarhie de tip întreg/parte. Să considerăm un obiect dintr-o astfel de ierarhie. Pe nivelul ierarhic imediat superior acestui obiect se găsește obiectul din care el face parte. Pe nivelul ierarhic imediat inferior se găsesc obiectele ce sunt părți ale sale.

Este simplu de observat că o astfel de ierarhie descrie relațiile de tip *part of* dintre obiecte. În termeni aferenți programării orientate pe obiecte o astfel de relație se numește *relație de agregare*.

În porțiunea de cod de mai jos se poate vedea cum este transpusă o astfel de relație în cod sursă Java. În acest exemplu un obiect mașină agregă un obiect motor. Figura 5.1 descrie modul de reprezentare UML a relației de agregare dată ca exemplu, într-o diagramă de clase. Este interesant de observat că, deși relația se reprezintă ca o relație între clase, agregarea se referă la obiecte (adică, fiecare *obiect Masina* are un *obiect Motor*).

```
class Masina {

    //Aceasta variabila contine o referinta la un obiect motor
    private Motor m;
```

```

//Orice masina va trebui sa aiba un motor; semantic, n
//nu ar trebui sa fie niciodata null
public Masina(Motor n) {
    ...
    this.m = n;
    ...
}

//Elemente specifice unui obiect masina
}

class Motor {

    //Elemente specifice unui obiect motor
}

```

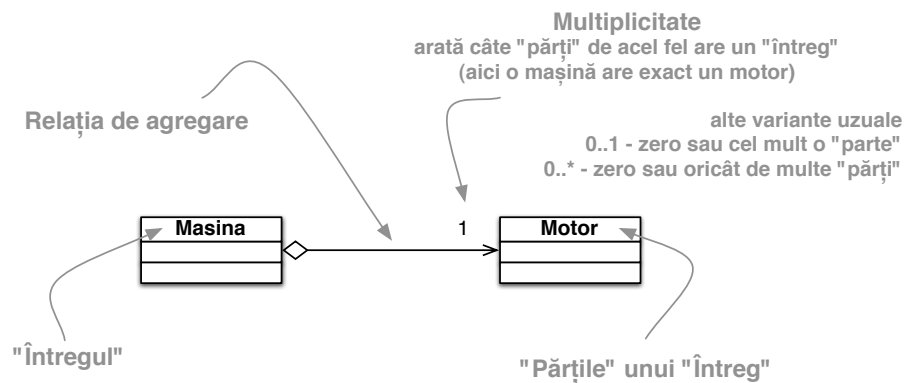


Figura 5.1: REPREZENTAREA UML A RELAȚIEI DE AGREGARE.

Rezoluți

Cum ați implementa în cod sursă Java o relație de agregare în care un întreg poate avea 0 sau oricât de multe părți (multiplicitate 0..*) ?



Să considerăm exemplul de mai jos. Reflectă această porțiune de cod o relație de agregare între un obiect mașină și un obiect motor? Răspunsul corect este nu, pentru că din cod nu reiese că fiecare instanță a clasei *Masina* are ca și parte a sa o instanță a clasei *Motor* (variabila *m* nu este câmp al clasei *Masina*). Este drept că în momentul execuției constructorului clasei

Masina se crează o instanță a clasei *Motor* dar acest lucru denotă o altfel de relație între clase denumită *dependență*. Despre aceasta relație nu vom vorbi însă acum.

```
class Masina {  
  
    public Masina() {  
        Motor m;  
        //Avem nevoie de un obiect Motor pentru a efectua anumite operatii  
        //de initializare a unui obiect Masina. Dupa terminarea  
        //constructorului nu mai e nevoie de acest obiect.  
        m = new Motor();  
        ...  
    }  
    //Elemente specifice unui obiect masina  
}
```

5.1.2 Ierarhia de clase. Relația de moștenire

Ierarhia de clase este o ierarhie de tip generalizare/specializare. Să considerăm o clasă B care face parte dintr-o astfel de ierarhie. Pe nivelul ierarhic imediat superior se găsește o clasă A care definește o abstracțiune mai generală decât abstracțiunea definită de clasa B. Cu alte cuvinte, clasa B definește un set de obiecte mai speciale inclus în setul de obiecte definite de clasa A. Prin urmare, putem spune că *B este un fel de A*.

După cum se poate observa, ierarhia de clase este generată de relațiile de tip *is a* dintre clasele de obiecte, această relație numindu-se relație de moștenire. Într-o astfel de relație clasa A se numește *superclasă* a clasei B, iar B se numește *subclasă* a clasei A.



Toată lumea știe că “pisica este un fel de felină”. Trebuie să observăm că afirmația este una generală în sensul că “toate pisicile sunt feline”. Ca urmare, afirmația se referă la clase de obiecte și nu la un anumit obiect (nu se referă doar la o pisică particulară). Rezultatul este că între clasa pisicilor și cea a felinei există o relație de moștenire în care *Pisica* este subclasă a clasei *Felina* iar *Felina* este superclasă a clasei *Pisica*. În Figura 5.2 se exemplifică modul de reprezentare UML a relației de moștenire între două clase.

După cum am spus încă de la începutul acestei lecții, relația de moștenire este inima programării orientate pe obiecte. Este normal să apară întrebarea: de ce? Ei bine, limbajele de programare orientate pe obiecte, pe lângă faptul că permit programatorului să marcheze explicit relația de moștenire dintre două clase, mai oferă următoarele facilități:

- o subclasă preia (moștenește) reprezentarea internă (datele) și comportamentul (metodele) de la superclasa sa.

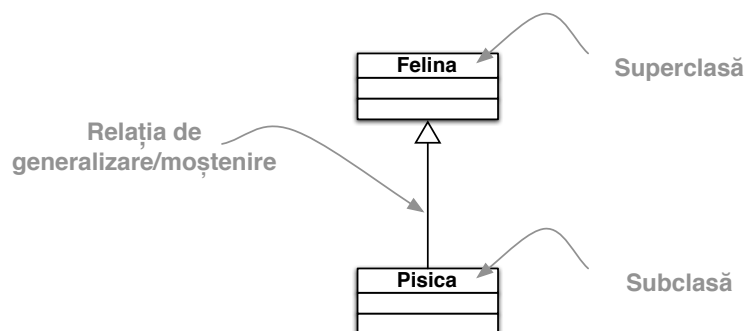


Figura 5.2: REPREZENTAREA UML A RELAȚIEI DE MOȘTENIRE.

- un obiect instanță a unei subclase poate fi utilizat în locul unei instanțe a superclasei sale.
- legarea dinamică a apelurilor metodelor.

În această lecție ne vom rezuma exclusiv la prezentarea primelor două facilități, cunoscute și sub numele de moștenire de clasă, respectiv moștenire de tip. Legarea dinamică va fi tratată în lecția următoare.

5.2 Definiția programării orientate pe obiecte

Relația de moștenire reprezintă elementul fundamental care distinge programarea orientată pe obiecte de programarea structurată. Acum că am descris relația de moștenire putem da o definiție completă a programării orientată pe obiecte.

Definiție 7 *Programarea orientată pe obiecte este o metodă de implementare a programelor în care acestea sunt organizate ca și colecții de obiecte care cooperează între ele, fiecare obiect reprezentând instanța unei clase, fiecare clasă fiind membra unei ierarhii de clase ce sunt unite prin relații de moștenire.*

5.3 Declararea relației de moștenire în Java

Exprimarea relației de moștenire dintre o subclasă și superclasa sa se realizează în Java utilizând cuvântul cheie *extends*.

```
class nume_subclasa extends nume_superclasa {  
    // definirea elementelor specifice subclasei  
}
```

Deși această construcție Java exprimă atât moștenirea de clasă cât și moștenirea de tip între cele două clase, vom trata separat cele două noțiuni pentru a înțelege mai bine distincția dintre ele.

5.4 Moștenirea de clasă în Java

Moștenirea de clasă este o facilitată a limbajelor de programare orientate pe obiecte care permite să definim implementarea unui obiect în termenii implementării altui obiect. Mai exact, o subclasă preia sau moștenește reprezentarea internă (datele) și comportamentul (metodele) de la superclasa sa.

După cum se poate observa, această facilitată permite reutilizarea de cod. În contextul relației de moștenire, dacă spunem că “o clasă B este un fel de clasă A” atunci se înțelege că orice “știe să facă A știe să facă și B”. Ca urmare, întregul cod sursă al clasei A ar trebui copiat în codul sursă al clasei B, lucru ce ar conduce la o creștere artificială a dimensiunii programului. Ei bine, prin moștenirea de clasă, această problemă e eliminată, subclasa moștenind implicit codul de la superclasa ei. Acest lucru permite programatorului care scrie clasa B să se concentreze exclusiv asupra elementelor specifice clasei B, asupra a ceea ce “știe să facă clasa B în plus față de A”.

5.4.1 Vizibilitatea membrilor moșteniți. Specificatorul de acces *protected*

Într-o lucrare anterioară am văzut că drepturile de acces la membrii unei clase pot fi menționate explicit prin specificatori de acces. Tot acolo am văzut care sunt regulile de vizibilitate impuse de specificatorii *public* și *private*. În continuare vom extinde aceste reguli în contextul moștenirii de clasă. Reamintim că drepturile de acces trebuie discutate atât din perspectiva interiorului unei clase cât și din perspectiva exteriorului (clienților) ei.

- În interiorul unei subclase pot fi referiți doar acei membri moșteniți de la superclasă a căror declarație a fost precedată de specificatorii de acces *public* sau *protected*. Accesul la membrii declarați *private* nu este permis deși ei fac parte din instanțele subclasei.
- În general, clienții unei subclase pot referi doar acei membri moșteniți de la superclasă a căror declarație a fost precedată de specificatorii de acces *public*.
- În general, clienții unei clase nu pot accesa membrii clasei ce sunt declarați ca fiind *protected*.
- Dacă o subclasă este client pentru o instanță a superclasei sale (de exemplu o metodă specifică subclasei primește ca argument o instanță a superclasei sale) drepturile la membrii acelei instanțe sunt aceleași ca pentru un client obișnuit.

Sfat

În anumite condiții, Java permite unui client al unei subclase să acceseze și membrii moșteniți declarați *protected*. Recomandăm evitarea acestei practici deoarece ea contravine definirii teoretice a specificatorului *protected*. În alte limbaje de programare obiectuale (de exemplu C++), accesul la membrii *protected* e permis doar în condițiile menționate mai sus.

Aceste reguli de vizibilitate sunt exemplificate în porțiunea de cod de mai jos. Se poate observa că din perspectiva unui client nu se face distincție între membrii moșteniți de o clasă și cei specifici ei.

```
class SuperClasa {
    public int super_a;
    private int super_b;
    protected int super_c;
}

class SubClasa extends SuperClasa {

    public void metoda(SuperClasa x) {
        super_a = 1; //Corect
        super_b = 2; //Eroare de compilare
        super_c = 3; //Corect

        x.super_a = 1; //Corect
        x.super_b = 2; //Eroare de compilare
        x.super_c = 3; //Corect in anumite conditii(clasele sunt in acelasi
                        //pachet). Incercati sa evitati.
    }
}

class Client {

    public void metoda() {

        SuperClasa sp = new SuperClasa();
        SubClasa sb = new SubClasa();

        sp.super_a = 1; //Corect
        sp.super_b = 2; //Eroare de compilare
        sp.super_c = 3; //Corect in anumite conditii

        sb.super_a = 1; //Corect
        sb.super_b = 2; //Eroare de compilare
        sb.super_c = 3; //Corect in anumite conditii
    }
}
```



În UML, vizibilitatea membrilor protected se marchează cu simbolul #. Figura 5.3 exemplifică modul de reprezentarea a clasei *SuperClasa* din exemplul anterior.

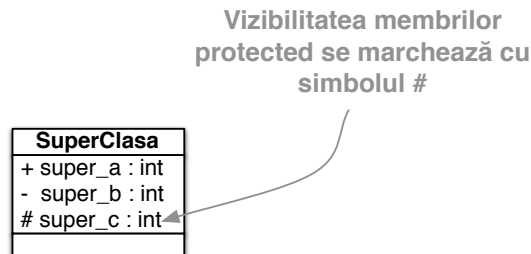


Figura 5.3: VIZIBILITATEA MEMBRILOR PROTECTED ÎN UML.

5.4.2 Cuvântul cheie super

Să considerăm exemplul de mai jos. Care câmp denumit *a* va fi inițializat cu valoarea 1: cel moștenit sau cel privat?

```

class SuperClasa {
    protected int a;
}

class SubClasa extends SuperClasa {

    private int a;

    public void metoda() {
        this.a = 1;
    }
}
  
```

Standardul Java prevede ca în astfel de situații să se acceseze câmpul *a* local clasei *SubClasa*. Dacă dorim să accesăm câmpul *a* moștenit vom proceda ca mai jos, făcând uz de cuvântul cheie *super*. Acesta trebuie văzut ca o referință la “bucata” moștenită a obiectului apelat.

```

class SuperClasa {
    protected int a;
}
  
```

```
class SubClasa extends SuperClasa {  
  
    private int a;  
  
    public void metoda() {  
        super.a = 1;  
    }  
}
```

5.4.3 Constructorii în contextul moștenirii de clasă

Într-o lucrare anterioară ați învățat că la crearea unui obiect trebuie specificat un constructor al clasei care se instanțiază, având rolul de a inițializa într-un anumit mod obiectul creat. Prin urmare, la instanțierea unei subclase, trebuie să specificăm un constructor al respectivei subclase.

Pe de altă parte, în lucrarea de față am văzut că o subclasă moștenește câmpurile definite în superclasa sa. Mai mult, câmpurile moștenite ar putea fi private și deci nu pot fi accesate din subclasă. Apare natural întrebarea: cum anume se inițializează câmpurile moștenite de superclasă? Răspunsul vine la fel de natural: trebuie să apelăm undeva constructorul superclasei. Și unde s-ar preta cel mai bine să apară acest apel? Evident, în interiorul constructorilor subclasei.

Standardul Java spune că prima instrucțiune din orice constructor al unei subclase trebuie să fie un apel la un constructor al superclasei sale.



Există o excepție de la această regulă. Tot prima instrucțiune dintr-un constructor poate fi un apel la un alt constructor al aceleiași clase (e posibilă supraîncărcarea constructorilor). Într-o astfel de situație constructorul în cauză nu va mai apela deloc constructorul superclasei. Motivul e simplu: constructorul apelat va apela un constructor din superclasă.

Totuși, sarcina introducerii acestui apel nu cade totdeauna în sarcina programatorului. Dacă superclasa are un constructor fără argumente (denumit și constructor no-arg), compilatorul introduce singur un apel la acest constructor în toți constructorii subclasei, cu excepția cazului în care un constructor apelează alt constructor al subclasei. Acest lucru se întâmplă, chiar dacă subclasa nu are nici un constructor. După cum am învățat într-o lecție anterioară, dacă o clasă nu conține nici un constructor compilatorul generează implicit un constructor no-arg pentru respectiva clasă. În cazul unei astfel de subclase, constructorul generat va conține și un apel la constructorul no-arg al superclasei sale.

În schimb, dacă superclasa are doar constructori cu argumente, programatorul trebuie să introducă explicit, în constructorii subclasei, un apel la unul din constructorii superclasei. În caz contrar se va genera o eroare la compilare deoarece compilatorul nu știe care și/sau cu ce parametri trebuie apelat constructorul superclasei. Acest lucru implică existența a cel puțin unui constructor în subclasă.

În continuare exemplificăm modul de apelare al unui constructor din superclasă. Se observă utilizarea cuvântului cheie *super* discutat în secțiunea anterioară.

```
class SuperClasa {  
    private int x;  
  
    public SuperClasa(int x) {  
        this.x = x;  
    }  
}  
  
class SubClasa extends SuperClasa {  
  
    private int a;  
  
    public SubClasa(int a,int x) {  
        super(x);        //Apel la constructorul superclasei  
        this.a = a;  
    }  
  
    public SubClasa(int a) {  
        this(a,0);        //Apel la primul constructor.  
                           //In acest constructor nu se mai poate apela  
                           //constructorul superclasei.  
    }  
}
```

5.4.4 Exemplu de moștenire de clasă

Să considerăm o aplicație în care lucrăm cu numere complexe și cu numere reale: trebuie să putem calcula modulul unui număr complex, trebuie să putem calcula modulul unui număr real, trebuie să putem afișa numerele reale și complexe în forma *real + imaginar * i*, trebuie să putem compara două numere reale.

Pentru lucrul cu numerele complexe vom defini clasa de mai jos.

```
class NumarComplex {
```

```
protected double re,im;

public NumarComplex(double re, double im) {
    this.re = re;
    this.im = im;
}

public double modul() {
    return Math.sqrt( re * re + im * im );
}

public String toString() {
    return re + " + " + im + " * i";
}
}
```

Pentru lucrul cu numere reale, trebuie să observăm că un număr real este un fel de număr complex. Mai exact, este un număr complex cu partea imaginară zero. Prin urmare, clasa *NumarReal* se definește astfel:

```
class NumarReal extends NumarComplex {

    public NumarReal(double re) {
        super(re,0);
    }

    public boolean maiMare(NumarReal a) {
        return re > a.re;
    }
}
```

Utilizând aceste două clase putem efectua operațiile cerute în cerințe.

```
class Client {

    public static void main(String argv[]) {

        NumarComplex a = new NumarComplex(1,1);
        System.out.println("Numarul este: " + a);
        System.out.println("Modulul sau este: " + a.modul());

        NumarReal c = new NumarReal(5);
        NumarReal d = new NumarReal(-6);
```

```

        System.out.println("Primul numar este: " + c);
        System.out.println("Modulul sau este: " + c.modul());
        System.out.println("Al doilea numar este: " + d);
        System.out.println("Modulul sau este: " + d.modul());
        System.out.println("E primul numar mai mare ca al doilea? - " +
            c.maiMare(d));
    }
}

```

Din acest exemplu se poate vedea cum *NumarReal* moștenește reprezentarea și comportamentul clasei *NumarComplex*. Astfel, un număr real știe să se tipărească și să-și calculeze modulul la fel ca un număr complex. În plus, un număr real știe să se compare cu alt număr real.

Atenție

Codul de mai jos va genera o eroare de compilare. Acest lucru se întâmplă pentru că *a* este o referință la un obiect *NumarComplex* și nu la *NumarReal*. Operația *maiMare* e specifică doar numerelor reale.

```

NumarComplex a = new NumarComplex(1, 0);
NumarReal b = new NumarReal(2);
a.maiMare(b);

```

5.5 Moștenirea de tip în Java

Moștenirea de tip este o facilitare a limbajelor de programare orientate pe obiecte care permite să utilizăm (substituim) o instanță a unei subclase în locul unei instanțe a superclasei sale. Să considerăm un exemplu.

```

class SuperClasa {...}

class SubClasa extends SuperClasa {...}

class Client {

    public void oMetoda() {
        ...
        SuperClasa a;
        SubClasa b = new SubClasa();
        a = b; //!!!!//
        ...
    }
}

```

Datorită moștenirii de tip acest cod este corect, deoarece este permis să utilizăm un obiect *SubClasa* ca și cum el ar fi o instanță de tip *SuperClasa*. Este logic de ce e permis acest lucru: subclasa moștenește reprezentarea și comportamentul de la superclasă. Prin urmare, tot ce știe să facă superclasa știe să facă și subclasa. Așadar, nu ne interesează dacă variabila *a* din exemplu referă un obiect *SubClasa*, pentru că sigur el va ști să se comporte și ca o instanță din *SuperClasa*.



De ce se numește această facilitate moștenire de tip? Totalitatea metodelor publice ale unei clase reprezintă interfața obiectului definit iar din prima lecție știm că interfața denotă tipul obiectului. În contextul relației de moștenire, o subclasă moștenește totul de la superclasă, deci și metodele publice sau altfel spus tipul (interfața) ei. Din acest motiv se vorbește de moștenire de tip. Pe de altă parte subclasa ar putea defini noi metode publice, extinzând astfel tipul moștenit. Astfel, se spune că subclasa definește un subtip al tipului superclasei. Tipul superclasei se mai numește și supertip pentru tipul subclasei.

5.5.1 Exemplu de utilizare a moștenirii de tip

Să considerăm același exemplu ca la moștenirea de clasă și să presupunem că dorim să putem aduna două numere complexe, un număr real cu un număr complex sau două numere reale. Datorită moștenirii de tip, această problemă se poate rezolva adăugând clasei *NumarComplex* o metodă.

```
class NumarComplex {  
  
    protected double re,im;  
  
    public NumarComplex(double re, double im) {  
        this.re = re;  
        this.im = im;  
    }  
  
    public NumarComplex adunare(NumarComplex a) {  
        return new NumarComplex(re + a.re, im + a.im);  
    }  
  
    public double modul() {  
        return Math.sqrt( re * re + im * im );  
    }  
  
    public String toString() {  
        return re + " + " + im + " * i";  
    }  
}
```

Un obiect *NumarComplex* poate fi adunat cu un obiect *NumarComplex* folosind metoda *adunare*. Un *NumarComplex* poate fi adunat cu un *NumarReal* deoarece metoda *adunare* poate primi ca parametru și o instanță *NumarReal* datorită moștenirii de tip. Clasa *NumarReal* moștenește metoda *adunare* deci se poate aduna cu un *NumarComplex* sau cu un *NumarReal*. Prin urmare cerințele problemei au fost satisfăcute. Mai jos dăm un exemplu de utilizare a operației de adunare.

```
class Client {  
  
    public static void main(String argv[]) {  
  
        NumarComplex a = new NumarComplex(1,1);  
        NumarReal b = new NumarReal(5);  
  
        System.out.println("Suma este:" + a.adunare(b));  
        //Se obtine aceeași suma și astfel  
        System.out.println("Suma este:" + b.adunare(a));  
    }  
}
```

5.5.2 Operatorii instanceof și cast

Datorită moștenirii de tip, o referință declarată de un anumit tip poate referi obiecte de orice subtip al tipului respectiv. În anumite situații este necesar să știm tipul concret al obiectului indicat de o referință. Acest lucru se poate realiza prin operatorul *instanceof*.

```
referinta_obiect instanceof nume_clasa
```

O astfel de expresie are valoarea *true* dacă *referinta_obiect* indică un obiect instanță a clasei *nume_clasa* sau a unei clase ce moștenește *nume_clasa*. Altfel valoarea expresiei este *false*. Mai jos dăm un exemplu de utilizare a operatorului, folosind clasele definite în secțiunea anterioară.

```
class Client {  
  
    public static void test(NumarComplex x) {  
  
        if (x instanceof NumarReal)  
            System.out.println("NumarReal");  
        else  
            System.out.println("NumarComplex");  
    }  
}
```

```
public static void main(String argv[]) {  
  
    NumarComplex a = new NumarComplex(1,1);  
    NumarReal b = new NumarReal(5);  
    test(a); //Se va tipari NumarComplex  
    test(b); //Se va tipari NumarReal  
}  
}
```

În acest exemplu, metoda *test* își dă seama dacă parametrul său indică un obiect *NumarReal* sau nu. Să presupunem acum că aceeși metode trebuie să afișeze "NumarReal mai mare ca 0" sau "NumarReal mai mic sau egal cu 0" dacă parametrul său referă un obiect *NumarReal*.

```
class Client {  
  
    public static void test(NumarComplex x) {  
        if (x instanceof NumarReal) {  
            NumarReal tmp = new NumarReal(0);  
            if(x.maiMare(tmp)) //EROARE!!!  
                System.out.println("NumarReal mai mare ca 0");  
            else  
                System.out.println("NumarReal mai mic sau egal cu 0");  
        } else  
            System.out.println("NumarComplex");  
    }  
}
```

Exemplul de mai sus va produce o eroare de compilare, datorită faptului că parametrul *x* este de tip *NumarComplex* iar un număr complex nu definește operația *maiMare*, ea fiind specifică obiectelor *NumarReal*. Soluția constă în utilizarea operatorului *cast*, înlocuind linia marcată cu eroare cu linia de mai jos.

```
if (((NumarReal)x).maiMare(tmp))
```

Atenție

Dacă o instrucțiune de genul *((NumarReal)x).maiMare(tmp)* ajunge să se execute într-o situație în care *x* nu referă o instanță *NumarReal*, se va semnaliza o eroare și programul se va opri. Evident, în exemplul dat nu se întâmplă acest lucru pentru că am utilizat operatorul *instanceof* pentru a fi siguri că *x* referă o instanță *NumarReal*.



Nu abuzați de operatorii *instanceof* și *cast* pentru că sunt periculoși. Dacă e posibil nici nu-i utilizați. Motivul e destul de greu de înțeles acum așa că va trebui să ne credeți pe cuvânt. În esență, moștenirea de tip ne permite să tratăm UNIFORM toate obiectele ale căror clase au o superclasă comună. De exemplu, instanțele claselor *NumarReal* și *NumarComplex* pot fi toate privite ca instanțe ale clasei *NumarComplex*. Tratarea lor uniformă în majoritatea codului sursă face programul mai simplu de înțeles. În momentul în care se folosesc abuziv operatorii *instanceof* și *cast* pentru a determina tipul real al obiectului, nu mai poate fi vorba de o tratare UNIFORMĂ. Programul devine mai greu de înțeles pentru că fiecare tip de obiect e tratat într-un mod particular lui (deci neuniform). Gândiți-vă ce se întâmplă când avem 10, 20 sau 100 de tipuri de obiecte diferite (nu doar două). Complexitatea va deveni uriașă și din păcate va fi doar vina programatorului care a refuzat utilizarea facilității de moștenire de tip a limbajului. Un astfel de program NU mai este orientat pe obiecte, chiar dacă e scris în Java!!!

5.6 Exerciții

1. Rulați și studiați programele date ca exemplu în Secțiunile 5.4.4, 5.5 și 5.5.2.
2. Fie o clasă *Punct* care are două câmpuri private *x* și *y* reprezentând coordonatele sale în plan. Clasa are un singur constructor cu doi parametri care permite inițializarea coordonatelor unui obiect *Punct* la crearea sa. Clasa *PunctColorat* extinde (moștenește) clasa *Punct* și mai conține un câmp *c* reprezentând codul unei culori. Argumentați dacă este sau nu necesară existența unui constructor în clasa *PunctColorat* pentru ca să putem crea obiecte *PunctColorat* și, dacă da, dați un exemplu de posibil constructor pentru această clasă.
3. Adăugați clasei *NumarComplex* dată ca exemplu în Secțiunea 5.5 o metodă pentru înmulțirea a două numere *NumarComplex*. Apoi scrieți un program care citește de la tastatură o matrice de dimensiuni $N \times M$ și o matrice de dimensiuni $M \times P$, ambele putând conține atât numere reale cât și numere complexe (la citirea fiecărui număr utilizatorul specifică dacă introduce un numar complex sau unul real). În continuare, programul înmulțește cele două matrice (făcând uz de metodele de adunare și înmulțire care sunt deja disponibile) și afișează rezultatul pe ecran. Înmulțirea trebuie realizată într-o metodă statică ce primește ca parametri matricele de înmulțit.
4. Dorim să modelăm printr-un program Java mai multe feluri de avioane care formează flota aeriană a unei țări. Știm că această țară dispune de avioane de călători și de avioane de luptă. Avioanele de călători sunt de mai multe feluri, și anume Boeing și Concorde. De asemenea, avioanele de luptă pot fi Mig-uri sau TomCat-uri (F14). Fiecare tip de avion va fi modelat printr-o clasă iar avioanele propriu-zise vor fi instanțe ale claselor respective.

Fiecare avion poate să execute o anumită gamă de operații și proceduri, după cum se specifică în continuare. Astfel, orice avion trebuie să conțină un membru *planeID* de

tip *String* și o metodă *public String getPlaneID()* care să returneze valoarea acestui membru. Mai mult, orice avion trebuie să conțină un membru *totalEnginePower* de tip întreg și o metodă *public int getTotalEnginePower()* care să returneze valoarea acestui membru. Deoarece fiecare avion trebuie să poată decola, zbura și ateriza, este normal ca pentru fiecare avion să putem apela metodele *public void takeOff()*, *public void land()* și *public void fly()*. Metoda *takeOff()* va produce pe ecran textul "PlaneID_Value - Initiating takeoff procedure - Starting engines - Accelerating down the runway - Taking off - Retracting gear - Takeoff complete". Metoda *fly()* va produce pe ecran textul "PlaneID_Value - Flying". Metoda *land()* va produce pe ecran textul "PlaneID_Value - Initiating landing procedure - Enabling airbrakes - Lowering gear - Contacting runway - Decelerating - Stopping engines - Landing complete".

Avioanele de călători și numai acestea trebuie să conțină un membru *maxPassengers* de tip întreg și o metodă *public int getMaxPassengers()* care să returneze valoarea acestui membru. Avioanele de călători de tip Concorde sunt supersonice, deci are sens să apelăm pentru un obiect de acest tip metodele *public void goSuperSonic()* și *public void goSubSonic()* care vor produce pe ecran "PlaneID_Value - Supersonic mode activated", respectiv "PlaneID_Value - Supersonic mode deactivated".

Avioanele de luptă și numai acestea au posibilitatea de a lansa rachete asupra diferitelor ținte, de aceea pentru orice avion de luptă trebuie să putem apela metoda *public void launchMissile()* care va produce pe ecran urmatorul text "PlaneID_Value - Initiating missile launch procedure - Acquiring target - Launching missile - Breaking away - Missile launch complete". Avioanele Mig și numai acestea au geometrie variabilă pentru zbor de mare viteză, respectiv pentru zbor normal. Clasa corespunzătoare trebuie să conțină metodele *public void highSpeedGeometry()* și *public void normalGeometry()* care vor produce pe ecran "PlaneID_Value - High speed geometry selected", respectiv "PlaneID_Value - Normal geometry selected". Avioanele TomCat și numai acestea au posibilitatea de realimentare în zbor, deci pentru astfel de avioane are sens să apelăm o metodă *public void refuel()* care va produce pe ecran "PlaneID_Value - Initiating refueling procedure - Locating refueller - Catching up - Refueling - Refueling complete".

Se cere:

- Implementați corespunzător clasele diferitelor feluri de avioane. Din cerințe rezultă că o parte din funcționalitate/date este comună tuturor sau mai multor feluri de avioane în timp ce o altă parte este specifică doar avioanelor de un anumit tip. Prin urmare, părțile comune vor trebui factorizate făcând uz de moștenirea de clasă.
- Într-o metodă *main*, declarați mai multe variabile referință. Obligatoriu, toate variabilele vor avea același tip declarat. Creați apoi mai multe avioane (cel

puțin unul de fiecare fel). Pentru a referi aceste obiecte folosiți doar variabilele amintite anterior bazându-vă pe moștenirea de tip. În continuare apălați diferitele operații disponibile fiecărui avion/fel de avion.

- Desenați diagrama UML de clase pentru ierarhia de clase obținută.

Bibliografie

1. Grady Booch, *Object-Oriented Analysis And Design With Applications*, Second Edition, Addison Wesley, 1997.
2. Martin Fowler. *UML Distilled, 3rd Edition*. Addison-Wesley, 2003.
3. Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides, *Design Patterns. Elements of Reusable Object-Oriented Software*, Addison Wesley, 1999.
4. Carmen De Sabata, Ciprian Chirilă, Călin Jebelean, *Laboratorul de Programare Orientată pe Obiecte*, Lucrarea 5 - Relația de moștenire - Aplicații, UPT 2002, variantă electronică.

Lecția 6

Polimorfismul

6.1 Ce este polimorfismul?

Prin intermediul magistralei USB (Universal Serial Bus) se pot conecta la un calculator diverse dispozitive periferice precum aparate foto sau camere video digitale, toate produse de diverse firme. Magistrala care conectează dispozitivul periferic la un calculator nu cunoaște tipul dispozitivului periferic ci doar faptul că acesta este un dispozitiv periferic. Ce s-ar întâmpla însă dacă magistrala ar cunoaște diferitele particularități de transfer de date ale fiecărui dispozitiv? Răspunsul este simplu: în loc de o funcție generală pentru transferul datelor între calculator și diverse periferice, aceasta ar trebui să ofere mai multe funcții de transfer specifice pentru fiecare tip de periferic în parte!!! Ca efect, atunci când ar apare un nou tip de periferic, magistrala USB ar trebui modificată astfel încât să fie posibil și transferul între calculator și noul dispozitiv!!! Evident acest lucru nu e convenabil. În cele ce urmează, vom arăta printr-un exemplu simplu cum anume putem modela într-un limbaj orientat pe obiecte o magistrală USB care să poată transfera date între un calculator și un dispozitiv periferic, indiferent de tipul concret sau de particularitățile de transfer de date ale perifericului.

După cum am precizat anterior, o magistrală USB realizează transferul de date dintre un calculator și un dispozitiv periferic. Anterior s-a precizat că această magistrală nu cunoaște exact tipul dispozitivului periferic implicat în transferul de date, ci doar faptul că acesta este un *dispozitiv periferic*. Spuneam în Secțiunea 1.1.1 că “*simplificăm sau abstractizăm obiectele reținând doar aspectele lor esențiale*”. Tot atunci am stabilit că aspectele esențiale ale unui obiect sunt relative la punctul de vedere din care este văzut obiectul. Concret, pentru magistrala USB este important ca dispozitivul implicat în transferul de date să conțină servicii pentru stocarea și furnizarea de date. Evident, pentru un utilizator al unei camere video este important ca aceasta, pe lângă posibilitatea conectării la un calculator în vederea descărcării filmelor, să și filmeze. Având în vedere aspectele esențiale din punctul de vedere al magistrlei USB, definim clasa *Device* de mai jos ale cărei instanțe furnizează servicii de stocare, respectiv furnizare de date.

```
class Device {  
  
    private String information;  
  
    public Device() {  
        information = "";  
    }  
  
    public Device(String information) {  
        this.information = information;  
    }  
  
    public void store(String information) {  
        this.information = information;  
    }  
  
    public String load() {  
        return information;  
    }  
}
```

Orice utilizator al oricărui dispozitiv periferic de genul aparat foto sau cameră video dorește la un moment dat să descarce informația conținută de dispozitiv pe calculator. Datorită acestui fapt e absolut normal ca orice dispozitiv periferic concret să fie modelat de o clasă ce extinde clasa *Device* adăugând în același timp servicii specifice respectivului dispozitiv ca în exemplul de mai jos.

```
class PhotoDevice extends Device {  
  
    public PhotoDevice(String information) {  
        super(information);  
    }  
  
    public void takePicture() {  
        System.out.println("TakePicture...");  
        //String picture = ...  
        //Se va apela this.store(picture) pentru stocarea pozei  
    }  
}
```

```
class VideoDevice extends Device {  
  
    private String producer;
```

```
public VideoDevice(String information, String producer) {
    super(information);
    this.producer = producer;
}

public void film() {
    System.out.println("Film...");
    //String film = ...
    //Se va apela this.store(film) pentru stocarea filmului
}
}
```

Pentru cealaltă parte implicată în transferul de date, și anume calculatorul, definim clasa *PC* de mai jos.

```
class PC {

    private String memory = "";
    private String registry;

    public void store(String information) {
        memory += information;
        registry = information;
    }

    public String load() {
        return registry;
    }
}
```

În fine, clasa *USB* oferă servicii pentru transferul de date bidirecțional între un calculator și un dispozitiv periferic. Codul său e prezentat mai jos.

```
class USB {

    public void transferPCToDevice(PC pc, Device device) {
        String data;
        data = pc.load();
        device.store(data);
        System.out.println("PC -> Device " + data);
    }

    public void transferDeviceToPC(PC pc, Device device) {
        String data;
        data = device.load();
    }
}
```

```

    pc.store(data);
    System.out.println("Device -> PC" + data);
}
}

```

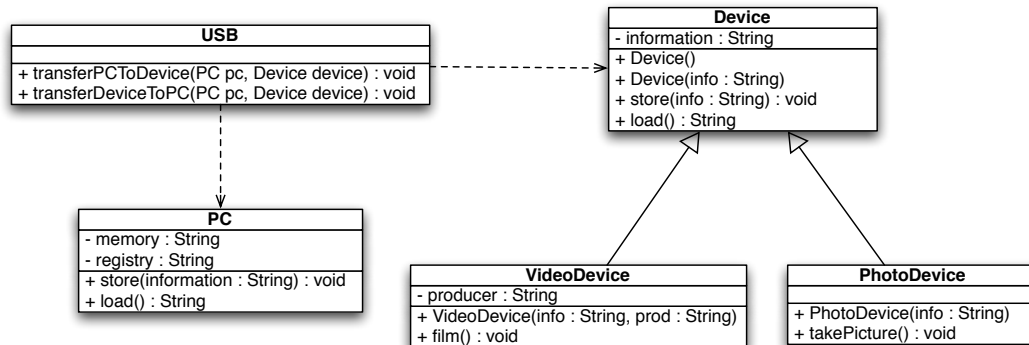


Figura 6.1: DIAGRAMA DE CLASE PENTRU EXEMPLUL DISCUTAT.



Săgețile punctate din figura de mai sus denotă relații de dependență. Relația de dependență este diferită de relația de agregare prezentată în Figura 5.1. După cum se poate observa, clasa *USB* nu conține atribute de tip *PC*, respectiv *Device* ci ea face uz de obiecte de tip *PC*, *Device* prin intermediul parametrilor *pc*, *device* aferenți metodelor *transferPCToDevice*, respectiv *transferDeviceToPC*.



Magistrala USB trebuie să poată transfera date între calculator și orice tip de dispozitiv periferic. E posibil acest lucru având în vedere că interfața magistralei oferă doar două servicii? Răspunsul este DA, deoarece *device* poate referi orice obiect instanță a clasei *Device* sau a oricărei clase ce moștenește clasa *Device*!!!

Să vedem acum un exemplu de utilizare a claselor definite până acum. În Secțiunea 5.5 am văzut că putem utiliza o instanță a unei subclase în locul unui obiect al superclasei sale. Datorită acestui fapt, clientul de mai jos e corect. Putem observa în acest client, că obiectul de tip *USB* este utilizat atât pentru a transfera date de la calculator la un aparat foto, cât și pentru a transfera date de la calculator la o cameră video. Prin urmare, bazându-ne pe moștenirea de tip, am modelat o magistrală ce poate transfera date între un calculator și orice tip concret de dispozitiv periferic.

```
class ClientUSB {  
  
    public static void main(String[] args) {  
  
        Device photo, video;  
        PC pc;  
        USB usb;  
        photo = new PhotoDevice("initialPhotoData");  
        video = new VideoDevice("initialVideoData", "company");  
        pc = new PC();  
        usb = new USB();  
  
        usb.transferPCToDevice(pc, photo);  
        usb.transferDeviceToPC(pc, video);  
    }  
}
```

Din păcate lucrurile nu se opresc aici. Să presupunem acum că în cazul camerelor video se dorește ca la descărcarea filmelor, pe lângă informația stocată de cameră, să se transmită și numele producătorului acesteia. Prin urmare, implementarea metodei *load* din clasa *Device* nu e potrivită pentru camerele video și s-ar părea că magistrala noastră USB ar trebui să trateze într-un mod particular transferul de date de la o cameră video la un calculator. Nu e deloc așa !!!

6.1.1 Suprascrierea metodelor

Definiție 8 *Atunci când într-o subclasă modificăm implementarea unei metode moștenite de la o superclasă spunem că suprascrim sau redefinim metoda respectivă.*

```
class VideoDevice extends Device {  
    ...  
    public String load() {  
        return producer + " " + super.load();  
    }  
    ...  
}
```

În exemplul de mai sus metoda *load* din clasa *VideoDevice*, redefineste metoda *load* din clasa *Device*. Mai mult, noua metodă îndeplinește și cerința de mai sus ca, în cazul camerelor video, pe lângă filmul propriu-zis să se transfere și numele producătorului echipamentului.

Atenție

Redefinirea unei metode dintr-o superclasă într-o subclasă înseamnă declararea unei metode în subclasă cu EXACT aceeași semnătură ca a

metodei din superclasă.

Important

Spunem despre o metodă suprascrisă că este o specializare (rafinare de comportament) a metodei din clasa de bază doar în cazul în care aceasta apelează metoda din superclasă. Este bine ca o metodă suprascrisă să fie o specializare. Implementarea metodei *load* din clasa *VideoDevice* este o specializare deoarece ea apelează metoda *load* moștenită de la clasa *Device* prin instrucțiunea *super.load()*.



În Secțiunea 3.4.2 am spus că este necesară o *modificare* a metodei *toString*. De fapt, modificarea despre care era vorba nu era altceva decât *suprascrierea* unei metode moștenite din clasa *Object*.

6.1.2 Legarea dinamică. Polimorfismul.

În secțiunea anterioară am redefinit metoda *load* în clasa *VideoDevice*. Acum haideti să vedem cum poate fi aceasta apelată. Un prim mod de apel al metodei este cel de mai jos, prin intermediul unei referințe de tip *VideoDevice*.

```
VideoDevice video = new VideoDevice("myVideo","XCompany");  
System.out.println(video.load()); //Se va afisa "XCompany myVideo"
```

Oare ce se va afișa însă la execuția codului de mai jos?

```
Device otherVideo = new VideoDevice("myVideo","XCompany");  
System.out.println(otherVideo.load());
```

La prima vedere am fi tentați să spunem că se va afișa doar “myVideo” deoarece referința *otherVideo* este de tip *Device* și se va executa codul metodei *load* din clasa respectivă. Lucrurile nu stau însă deloc așa iar pe ecran se va afișa “XCompany myVideo”!!! Explicația este că, de fapt, nu se apelează metoda *load* existentă în clasa *Device* ci metoda *load* existentă în clasa *VideoDevice*.

Definiție 9 *Asocierea dintre apelul la un serviciu (apel de metodă) și implementarea acestuia (implementarea metodei ce se va executa la acel apel) se numește legare.*

În limbajele de programare există două tipuri de legare:

Legare statică (early binding) : asocierea dintre un serviciu și implementarea acestuia se realizează la compilare programului. Cu alte cuvinte, se cunoaște din momentul compilării apelului metodei care este implementarea metodei ce se va executa la acel apel.

Legare dinamică (late binding) : asocierea dintre un serviciu și implementarea acestuia se realizează la NUMAI la execuția programului. Cu alte cuvinte, se

va cunoaște doar în momentul execuției apelului care implementare a unei metode se va executa la respectivul apel.

În Java, legarea apelurilor la metode nestatice se realizează în majoritatea cazurilor dinamic!!! Aceasta este explicația tipăririi mesajului “XCompany myVideo” la rularea exemplului de mai sus. La execuția apelului metodei *load*, suportul de execuție Java vede că referința *otherVideo* indică un obiect *VideoDevice* și ca urmare execută implementarea metodei *load* din clasa *VideoDevice*. Dacă referința ar fi indicat o instanță a clasei *Device* atunci s-ar fi apelat implementarea metodei *load* din clasa *Device*.

Să revenim acum la codul din clasei *USB* și mai exact la metoda *transferDeviceToPC*. Obiectul referit de parametrul *device* este cunoscut magistralei USB doar prin intermediul interfeței sale (totalitatea metodelor publice) definite în cadrul clasei *Device*. Atunci când aceasta solicită operația *load* obiectului referit de *device*, modul în care operația va fi executată depinde de tipul concret al obiectului referit de *device*. Dacă obiectul e instanță a clasei *VideoDevice* se va executa metoda *load* din această clasă și prin urmare se va transfera și numele producătorului echipamentului. Altfel se va executa metoda *load* din clasa *Device*. Prin urmare, clasa *USB* poate fi folosită în continuare nemodificată pentru a transfera date între orice fel de echipament periferic și calculator.

Polimorfismul este ansamblul format de moștenirea de tip și legarea dinamică. El ne permite să tratăm uniform obiecte de tipuri diferite (prin moștenirea de tip) și în același timp în mod particular fiecare tip de obiect dacă e necesar (prin legarea dinamică).

6.1.3 Clase și metode abstracte

Informația stocată de o cameră video este, în general, un film care este rezultatul serviciului de filmare oferit de cameră. Putem transfera filme de pe calculator spre camera video dar scopul pentru care aceasta a fost proiectată este de a filma și nu de a păstra informații. Un dispozitiv periferic care nu poate face nimic în afară de transferul bidirecțional al datelor nu este de nici un folos (chiar și stilourile de memorie, în afară de transferul bidirecțional al datelor, mai oferă și serviciul de scriere protejată). Orice dispozitiv trebuie să fie *ceva*, în cazul nostru o cameră video sau foto.

Clasa *Device* a fost creată de fapt pentru a stabili o *interfață comună* pentru toate subclasele sale. Întrucât un obiect al clasei *Device* nu este de fapt de nici un folos, trebuie ca operația de instanțiere a acestei clase să nu fie posibilă.

Definiție 10 *O clasă abstractă este o clasă care nu poate fi instanțiată.*

Declarăm o clasă ca fiind abstractă dacă prefixăm cuvântul cheie *class* de cuvântul cheie *abstract*, ca mai jos:

```
abstract class Device {  
    ...  
}
```



În exemplul de mai sus compilatorul nu ar fi semnalat o problemă dacă clasa *Device* nu ar fi fost declarată abstractă. Faptul că ea a devenit abstractă împiedică instanțierea ei accidentală și mărește gradul de înțelegere al codului; de fiecare dată când vedem o clasă abstractă știm că ea este o interfață comună pentru toate subclasele sale.

Să considerăm un alt exemplu. Cercul, pătratul și romb sunt obiecte figuri geometrice și fiecare din aceste figuri știe să-și calculeze aria. Prin urmare clasele lor ar putea fi derivate dintr-o clasă abstractă *FiguraGeometrica*. Mai mult, întrucât fiecare figură geometrică are o arie, este normal ca *FiguraGeometrica* să conțină metoda *public float arie()*. Se pune însă problema ce implementare va avea această metodă în clasa de bază deoarece fiecare figură geometrică concretă are un mod propriu de calcul al ariei? O primă variantă este prezentată mai jos.

```
abstract class FiguraGeometrica {  
  
    public float arie() {  
        return 0;  
    }  
    ...  
}
```

Din păcate nici o figură geometrică nu poate avea aria 0, deci metoda *arie* de mai sus nu va fi specializată de nici o subclasă a clasei *FiguraGeometrica*. Mai exact, fiecare subclasă a clasei *FiguraGeometrica* va reimplementa total metoda *arie*. Cu alte cuvinte, această implementare a metodei *arie* este inutilă și nu folosește nimănui. În plus, este posibil ca în interiorul unei subclase să uităm să suprascriem metoda *arie* și atunci în mod cert aria nu va fi calculată corect.

Varianta corectă în acest caz este ca metoda *arie* să fie abstractă, ea neavând nici o implementare în clasa de bază *FiguraGeometrica*.

```
abstract class FiguraGeometrica {  
  
    public abstract float arie();  
  
    ...  
}
```

Atenție

Spuneam în Secțiunea 3.1 că în anumite condiții corpul unei metode poate lipsi. O metodă abstractă nu are niciodată o implementare, deci corpul acesteia întotdeauna lipsește. Datorită acestui fapt compilatorul ar fi semnalat o eroare dacă am fi încercat să-i dăm o implementare metodei *arie*.

Atenție

O clasă poate fi abstractă chiar dacă aceasta nu conține nici o metodă abstractă.

Atenție

Dacă o clasă conține cel puțin o metodă abstractă, atunci respectiva clasă va trebui și ea declarată ca fiind abstractă, în caz contrar semnalându-se o eroare la compilare.

Atenție

O subclasă a unei clase abstracte trebuie să implementeze toate metodele abstracte ale supercalsei sale, în caz contrar fiind necesar ca și subclasa să fie abstractă. Acest fapt este normal fiindcă nu pot fi instanțiate clase care conțin servicii neimplementate!



Într-o diagramă de clase UML, clasele și metodele abstracte se evidențiază ca în Figura 6.2. Evident, când desenăm de mână o diagramă, este cam greu să scriem italic. În astfel de cazuri, se practică marcarea explicită a clasei/metodei abstracte folosind notația {abstract}.

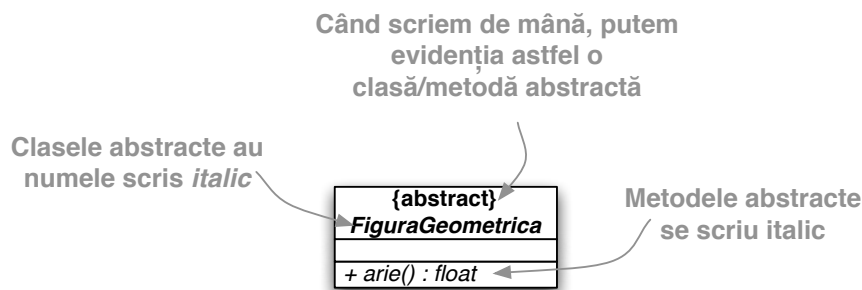


Figura 6.2: REPREZENTAREA UNEI CLASE/METODE ABSTRACTE ÎN UML.

6.2 Principiul de Proiectare Închis-Deschis

Atunci când vrem să realizăm un sistem software, nu putem trece direct la faza de scriere a codului sursă. Procesul de realizare al unui program este compus din mai multe faze, printre care și așa numita fază de proiectare. Proiectarea unui sistem software se face

pe baza unor principii iar *Principiul Închis-Deschis*, sau *The Open-Closed Principle*, este unul dintre cele mai importante.

Definiție 11 *Principiul OCP ne sugerează să proiectăm un program astfel încât entitățile software (clase, module, funcții, etc) să fie deschise pentru extensii dar închise pentru modificări [3].*

Modulele care se conformează cu acest principiu au două caracteristici principale:

Deschise pentru extensii - în acest caz ce se poate extinde nu este altceva decât comportamentul modulelor.

Închise pentru modificări - extinderea comportamentului modulelor nu duce la modificări asupra codului respectivelor modulelor.

Clasa *USB* a fost proiectată ținând cont de acest principiu. Comportamentul său poate fi extins prin adăugarea de noi dispozitive (de exemplu introducând o subclasă *Imprimanta* a clasei *Device*) în sensul că ea poate fi utilizată atunci pentru a transfera informații între un calculator și un nou tip de periferic (adică o imprimantă). În același timp, clasa *USB* va rămâne nemodificată la nivel de cod sursă. Haideți să vedem ce s-ar fi întâmplat dacă nu ar fi fost respectat principiul OCP. Avem mai jos o variantă a clasei *USB* care nu respectă acest principiu (din economie de spațiu am reprodus doar metodele de transfer de la calculator la periferic și nu și invers).

```
class USB {  
  
    public void transferPCToDevice(PC pc, PhotoDevice device) {  
        String data;  
        data = pc.load();  
        device.store(data);  
        System.out.println("PC -> Device " + data);  
    }  
  
    public void transferPCToDevice(PC pc, VideoDevice device) {  
        String data;  
        data = device.load();  
        pc.store(data);  
        System.out.println("PC -> Device " + data);  
    }  
}
```

Problema acestei variante a clasei *USB* constă în faptul că ea cunoaște diferitele tipuri de dispozitive periferice existente iar în momentul introducerii de noi subclase a clasei *Device*, de exemplu *NewDevice*, clasei *USB* va trebui să i se adauge metoda:

```
public void transferPCToDevice(PC pc, NewDevice device) {
    ...
    System.out.println("PC -> Device " + data);
}
```



Și totuși, care este marea problemă? Doar nu este așa de greu să adăugăm o metodă cu același conținut ca și restul, având doar un parametru schimbat! Ei bine, problema este mult mai gravă decât pare la prima vedere. Gândiți-vă că de fiecare dată când achiziționați un nou dispozitiv ce trebuie să comunice prin intermediul magistralei USB cu propriul calculator, trebuie să mergeți întâi la un service de calculatoare pentru a modifica magistrala USB!!! Exact așa stau lucrurile și aici: la fiecare creare a unei clase derivate din clasa *Device* clasa *USB* va trebui modificată și recompilată!!!

Se poate observa că implementările celor două metode *transferPCToDevice* sunt identice. Spunem că între cele două metode există o *duplicare de cod*. Duplicarea de cod existentă în cadrul unui sistem este o sursă serioasă de probleme. Presupunem că la un moment dat în cadrul transferului se dorește să se afișeze pe ecran în loc de "PC -> Device" un alt șir de caractere. Este evident că va trebui modificat codul în mai multe locuri, nu doar în unul singur.

Să vedem acum o altă variantă a clasei *USB* în care implementarea metodei *transferPCToDevice* arată în felul următor:

```
class USB {

    public void transferPCToDevice(PC pc, Device device) {
        String data;
        data = pc.load();
        device.store(data);
        if (device instanceof PhotoDevice)
            System.out.println("PC -> PhotoDevice " + data);
        else if (device instanceof VideoDevice)
            System.out.println("PC -> VideoDevice " + data);
    }
}
```

Evident că această implementare nu respectă OCP, la adăugarea unui nou tip de dispozitiv fiind necesară modificarea metodei.

Sfat

Încercați să scrieți programe astfel încât acestea să nu conțină duplicare de cod și nici secvențe if-then-else care verifică tipul concret al unui obiect. Duplicarea de cod și secvențele lungi if-then-else de acest fel sunt semne că undeva nu utilizați polimorfismul deși ar trebui.

6.3 Supraîncărcarea versus suprascrierea metodelor

În exemplul de mai jos, în clasa *D*, în loc să suprascriem metoda *oMetoda* am supraîncărcat-o.

Important

În literatură *supraîncărcarea* metodelor este cunoscută sub numele de *overloading* iar *suprascrierea* sub numele de *overriding*.

```
class B {  
    public void oMetoda(Object o) {  
        System.out.println("BBBB.oMetoda");  
    }  
    ...  
}  
  
class D extends B{  
    public void oMetoda(B b) {  
        System.out.println("DDDD.oMetoda");  
    }  
    ...  
}
```

Din punctul de vedere al compilatorului acest fapt nu reprezintă o problemă dar comportamentul metodei supraîncărcate s-ar putea să nu fie acela dorit de noi.

```
B b = new B();  
B d = new D();  
d.oMetoda(b); //Se va afișa BBBB.oMetoda
```

Poate ce am fi dorit noi să se afișeze este *DDDD.oMetoda* dar acest fapt nu se întâmplă deoarece clasa *B* nu conține nici o metodă cu semnătura *oMetoda(B)*, în acest caz apelându-se metoda *oMetoda(Object)* din clasa de bază.



Pentru ca o metodă dintr-o clasă derivată să se apeleze polimorfic **OBLIGATORIU** trebuie ca aceasta să *suprascrie* și nu să *supraîncarce* o metodă din clasa de bază.

6.4 Constructorii si polimorfismul

Presupunând că definim clasele de mai jos, oare ce se va afișa (și ce am dori să se afișeze) în urma execuției următoarei secvențe?

```
SubClasa ss = new SubClasa();  
System.out.println("Valoarea este " + ss.getValoare());
```

```
class SuperClasa {  
  
    protected int valoareSuperClasa;  
  
    public SuperClasa() {  
        valoareSuperClasa = valoareImplicita();  
    }  
  
    public int valoareImplicita() {  
        return 10;  
    }  
  
    public int getValoare() {  
        return valoareSuperClasa;  
    }  
}  
  
class SubClasa extends SuperClasa {  
  
    private int valoareSubClasa;  
  
    public SubClasa() {  
        valoareSubClasa = 20;  
    }  
  
    public int valoareImplicita() {  
        return valoareSubClasa;  
    }  
}
```

Poate că noi ne-am dori ca efectul să fie tipărirea șirului de caractere “Valoarea este 20”, numai că în loc de acesta se va tipări “Valoarea este 0”!!!



Dacă în constructorul unei superclase se apelează o metodă care este suprascrisă în subclasă, la crearea unui obiect al subclasei există riscul ca metoda respectivă să refere câmpuri încă neinițializate în momentul apelului.

Pentru exemplul de mai sus să urmărim ce se întâmplă la instanțierea unui obiect al clasei *SubClasa*.

1. se inițializează câmpurile cu valorile implicite corespunzătoare tipurilor lor, deci în

cazul nostru cu 0.

2. se apelează constructorul *SubClasa* care apelează constructorul *SuperClasa*.
3. constructorul *SuperClasa* apelează metoda suprascrisă *valoareImplicita*; datorită faptului că metoda este suprascrisă, implementarea apelată este cea din *SubClasa* și nu cea din *SuperClasa* iar fiindcă atribuirea *valoareSubClasa = 20* încă nu s-a realizat metoda va întoarce valoarea 0!!!
4. se revine în constructorul *SubClasa* și abia acum se execută atribuirea din acest constructor.

6.5 Tablourile și polimorfismul

În exemplul de mai jos am creat un tablou ale cărui elemente sunt referințe de tipul *Device*. Faptul că am scris *new Device[10]* NU înseamnă că am creat 10 obiecte de tip *Device* ci doar că am instanțiat un tablou de dimensiune 10, fiecare intrare a tabloului fiind o referință de tip *Device*.

```
Device[] devices = new Device[10];
```

În acest moment, intrările tabloului nu referă obiecte și se pune problema inițializării lor. Ce fel de elemente putem adăuga în tablou, având în vedere că *Device* este o clasă abstractă (presupunând că am declarat-o așa)?

```
devices[0] = new VideoDevice("video","company");
```

Răspunsul la problemă este simplu. Ținând cont de moștenirea de tip, și în acest caz putem utiliza o instanță a unei subclase în locul unei instanțe a superclasei sale și deci, codul de mai sus este absolut corect.

6.6 Exercițiu rezolvat

Joc de strategie

Într-un joc de strategie simplificat există mai multe feluri de unități de luptă. Indiferent de felul său concret, pe un obiect unitate de luptă se poate apela din exterior:

- o metodă denumită *rănire* ce primește ca parametru o valoare întreagă
- o metodă denumită *lovește* care primește ca parametru o referință la o unitate de luptă de orice fel
- o metodă denumită *esteVie* care întoarce un boolean prin care se poate testa dacă unitatea mai este sau nu în viață

Implementarea acestor metodelor depinde de felul concret al obiectului unitate de luptă după cum se detaliază în cele ce urmează. Felurile concrete de unități de luptă sunt:

- Arcaș - când un arcaș este rănit (adică și mai exact, se apelează pe el metoda `rănire`) atunci, dacă e viu viața sa se decrementează cu valoarea dată ca argument metodei. Dacă e mort nu se întâmplă nimic (nu mai poate muri odată și deci nu mai are rost decrementarea). Viața inițială a unui arcaș este de 100 și el devine mort când viața sa este mai mică sau egală cu 0. Când un arcaș lovește o unitate de luptă și el nu e mort, efectul constă în rănirea unității date ca parametru metodei lovește cu valoarea 10. Dacă e mort rănirea se face cu valoarea 0.
- Călăreț - când un călăreț este rănit atunci, dacă e viu viața sa se decrementează cu valoarea dată ca argument metodei `rănire`. Dacă e mort nu se întâmplă nimic. Viața inițială a unui călăreț este de 200 și el devine mort când viața sa este mai mică sau egală cu 0. Când un călăreț lovește o unitate de luptă și el nu e mort, efectul constă în rănirea unității date ca parametru metodei lovește cu valoarea 15. Dacă e mort rănirea se face cu valoarea 0.
- Pluton - care reprezintă o înșiruire de unități de luptă de orice fel (arcași, călăreți, alte plutoane) într-un număr nelimitat și în orice combinație. Când un pluton este rănit efectul constă în rănirea unei unități vii din pluton (prima găsită), iar dacă nu există nici o unitate vie în pluton atunci nu se întâmplă nimic. Când un pluton lovește o unitate de luptă efectul constă în lovirea unități date ca parametru metodei de către fiecare unitate a plutonului (nu are importanță dacă membrul e viu sau mort pentru că oricum, dacă e mort, rănirea va fi în final 0). Un pluton se consideră mort dacă toate unitățile din el sunt moarte, iar dacă nu conține nici un membru se consideră implicit viu. Clasa mai definește o metodă denumită `adaugă` care primește ca parametru o referință la o unitate de luptă de orice fel și o adaugă în plutonul corespunzător. Dacă unitatea ce se dorește adăugată e moartă, metoda de adăugare întoarce false și adăugarea la pluton nu se face. Dacă plutonul la care se dorește adăugarea e mort, metoda întoarce false iar adăugarea la pluton nu se face. Altfel, metoda de adăugare întoarce true.

Se cere:

1. Diagrama de clase detaliată cuprinzând toate clasele descrise și orice alte clase considerate necesare.
2. Implementarea tuturor claselor descrise, inclusiv a altora considerate necesare.
3. Considerând că la moartea unui călăreț moare și calul, introduceți și implementați corespunzător în una din clase o metodă care să întoarcă numărul de cai ce au murit de la începutul rulării jocului/programului (aceste elemente nu trebuie surprinse în diagrama de clase).
4. Exemplificați, într-o metodă `main`, lupta dintre un pluton format din patru arcași și un alt pluton format dintr-un călăreț și un pluton format din alți doi arcași. După aceea tipăriți pe ecran numărul de cai decedați.

Rezolvare

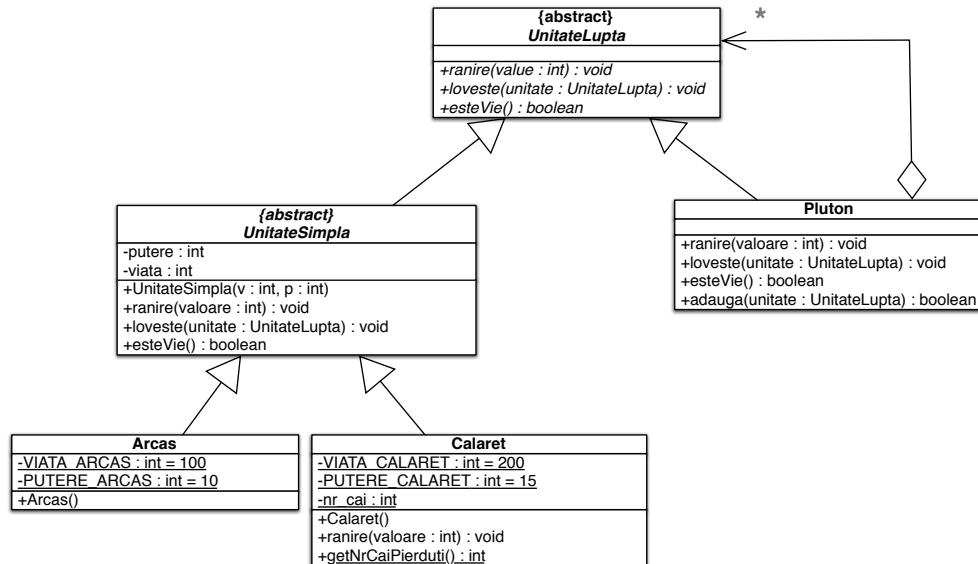


Figura 6.3: DIAGRAMA DE CLASE.

Din specificațiile problemei putem observa că plutonul va fi compus din mai multe unități de luptă de diverse feluri. Astfel, va trebui să putem avea plutoane formate din călăreți, va trebui să putem avea plutoane formate din arcași, va trebui să putem avea plutoane formate din arcași și călăreți. Mai mult, din specificații reiese că vom putea avea plutoane formate din alte (sub) plutoane, călăreți și alte plutoane și așa mai departe.

Pentru a putea rezolva simplu această explozie de combinații posibile ne vom baza pe polimorfism, adică pe faptul că putem trata uniform diversele feluri de unități de luptă. Mai exact, dorim să putem declara variabile referință ce să poată referi uniform atât călăreți cât și arcași și plutoane. Prin urmare, toate aceste obiecte vor trebui să aibă un supertip comun pentru a putea declara astfel de variabile referință. În consecință vom defini clasa *UnitateLupta* ce va fi superclasă pentru toate clasele ce modelează obiecte care reprezintă entități luptătoare.

O problemă care s-ar putea ridica aici e următoare. Dintr-o lecție anterioară știm că *Object* este superclasă pentru toate clasele din Java. De ce nu folosim variabile referință declarate de tip *Object* pentru a referi uniform orice fel de unitate de luptă?

Motivul e simplu: din specificațiile problemei reiese că, pe lângă dorința de referire uniformă, dorim și să putem apela un set de metode (adică *ranire*, *esteVie* și *loveste*) în manieră uniformă, fără a ține cont de felul concret al unei unități luptătoare. În consecință, supertipul nostru comun trebuie să conțină ori să moștenească declarațiile acestor metode. Cum *Object* nu are cum conține aceste declarații, nu ne putem baza pe referințe de acest tip (de câte ori am dori să facem un apel la metodele de mai sus, fără a ști cu ce fel de unitate de luptă avem de interacționat, ar trebui să determinăm felul concret al unității folosind operatorul *instanceof* și va trebui să facem un *cast* corespunzător).

Acesta este și motivul pentru care clasa *UnitateLupta* conține declarațiile metodelor *ranire*, *esteVie* și *loveste*. Deoarece la acest nivel de abstractizare nu există funcționalitate similară între călăreți, arcași și plutoane, toate aceste metode vor fi declarate abstracte. În același timp, la acest nivel de abstractizare nu avem nici date comune pentru absolut toate felurile de unități de luptă și, prin urmare, nu avem ce câmpuri să includem în această clasă.

```
abstract class UnitateLupta {  
  
    public abstract void ranire(int value);  
  
    public abstract void loveste(UnitateLupta unitate);  
  
    public abstract boolean esteVie();  
  
}
```

Arcașii și călăreții au o mare parte din caracteristici și funcționalități similare. De exemplu, ambele feluri de unități de luptă au o valoare curentă pentru viața lor, au o valoare pentru puterea lor, se comportă la fel când sunt rănite și așa mai departe. Pentru a nu duplica o mare parte din implementarea călăreților și arcașilor, ne vom baza pe moștenirea de clasă și vom da factor comun aceste elemente incluzându-le în clasa abstractă *UnitateSimpla*, urmând ca aceasta să fie extinsă de clasa arcașilor și de cea a călăreților.

De ce este clasa *UnitateSimpla* abstractă ? Deoarece folosim clasa doar pentru a factoriza cod și nu are sens să instanțiem vreodată această clasă. Nu avem obiecte care sunt doar unități simple: avem fie arcași, fie călăreți, fie plutoane.

```
abstract class UnitateSimpla extends UnitateLupta {  
  
    private int viata, putere;
```

```
public UnitateSimpla(int viata, int putere) {
    this.viata = viata;
    this.putere = putere;
}

public void ranire(int valoare) {
    if(esteVie()) {
        viata = viata - valoare;
    }
}

public void loveste(UnitateLupta unitate) {
    if(esteVie()) {
        unitate.ranire(putere);
    }
}

public boolean esteVie() {
    if(viata > 0) {
        return true;
    } else {
        return false;
    }
}
}
```

Am ajuns în sfârșit să implementăm arcașii. Toate datele și funcționalitățile arcașilor sunt moștenite de la clasa *UnitateSimpla*, singura sarcină care cade pe umerii clasei *Arcas* fiind inițializarea corespunzătoare a vieții și puterii unui arcaș. Cum superclasa cere ca acest lucru să fie realizat prin constructorul ei, clasa *Arcas* va trebui să realizeze inițializarea prin apelarea acestui constructor dintr-un constructor propriu.

```
class Arcas extends UnitateSimpla {

    private static final int VIATA_ARCAS = 100;
    private static final int PUTERE_ARCAS = 10;

    public Arcas() {
        super(VIATA_ARCAS, PUTERE_ARCAS);
    }

}
```

Din punctul de vedere al inițializării vieții și puterii unui călăreț, clasa călăreților are aceleași sarcini ca și clasa arcașilor.

În plus însă, problema cere să avem o metodă prin care să determinăm oricând câți cai au decedat de la începutul rulării jocului. Momentul decesului unui cal poate fi determinat în momentul rănirii unui călăreț. În consecință, este clar că această clasă reprezintă locul ideal de implementare a funcționalității cerute.

Astfel, clasa călăreților redefinesc metoda *rănire*, determinând starea călărețului înainte și după rănirea propriu-zisă: dacă înainte de rănire călărețul e viu și după rănire nu mai e viu, înseamnă că tocmai a decedat atât el cât și calul său. În acest caz vom incrementa o variabilă statică (definită în această clasă). Ea trebuie să fie statică deoarece trebuie să determinăm numărul *total* al cailor decedați, indiferent de ce cal și implicit obiect călăreț dispăre.

În cele din urmă, pentru a pune la dispoziție metoda cerută, în clasa *Calaret* definim metoda statică *getNrCaiPierduti* care întoarce valoarea variabilei statice menționate anterior. Metoda va fi statică pentru că ea nu caracterizează un călăreț anume: ea caracterizează clasa călăreților spunând câte din instanțele sale și-au pierdut caii (mai exact câte din instanțele sale au decedat).

```
class Calaret extends UnitateSimpla {

    private static final int VIATA_CALARET = 200;
    private static final int PUTERE_CALARET = 15;

    private static int nr_cai = 0;

    public static int getNrCaiPierduti() {
        return nr_cai;
    }

    public Calaret() {
        super(VIATA_CALARET, PUTERE_CALARET);
    }

    public void ranire(int valoare) {
        boolean inainte_de_ranire = this.esteVie();
        super.ranire(valoare);
        boolean dupa_ranire = this.esteVie();
        if((inainte_de_ranire == true) && (dupa_ranire == false)) {
            nr_cai++;
        }
    }
}
```

Cea mai interesantă parte din clasa *Pluton* constă în memorarea membrilor plutonului. Pentru acest lucru definim un tablou de *UnitateLupta* pentru ca elementele sale să

poată referi conform cerințelor orice fel concret de unitate de luptă. Inițial tabloul are alocate 10 poziții.

Pe de altă parte, cerințele precizează că un pluton poate avea un număr nelimitat de membri. Prin urmare, în cadrul metodei *adauga*, vom determina dacă mai există poziții neocupate în tablou. În cazul în care nu mai există poziții libere (adică numărul curent de membri este egal cu dimensiunea alocată tabloului), vom crea un tablou de dimensiune mai mare, vom copia toate referințele din vechiul tablou în noul tablou, urmând ca noul tablou să fie folosit în locul celui vechi.

Evident, clasa pluton va trebui să implementeze comportamentul său corespunzător metodelor declarate în clasa *UnitateLupta*. Această implementare este extrem de simplă odată ce rolul supertipului comun a fost înțeles și, prin urmare, nu o vom mai discuta pe larg.

```
class Pluton extends UnitateLupta {  
  
    private UnitateLupta[] membri = new UnitateLupta[10];  
  
    private int nr_membri = 0;  
  
    public void ranire(int valoare) {  
        for(int i = 0; i < nr_membri; i++) {  
            if(membri[i].esteVie()) {  
                membri[i].ranire(valoare);  
                break;  
            }  
        }  
    }  
  
    public void loveste(UnitateLupta unitate) {  
        for(int i = 0; i < nr_membri; i++) {  
            membri[i].loveste(unitate);  
        }  
    }  
  
    public boolean esteVie() {  
        if(nr_membri == 0) {  
            return true;  
        }  
        for(int i = 0; i < nr_membri; i++) {  
            if(membri[i].esteVie()) {  
                return true;  
            }  
        }  
    }  
}
```

```
        return false;
    }

    public boolean adauga(UnitateLupta unitate) {
        if(!unitate.esteVie() || !this.esteVie()) {
            return false;
        }
        if(nr_membri == membri.length) {
            UnitateLupta[] tmp = new UnitateLupta[membri.length + 10];
            for(int i = 0; i < membri.length; i++) {
                tmp[i] = membri[i];
            }
            membri = tmp;
        }
        membri[nr_membri] = unitate;
        nr_membri++;
        return true;
    }
}
```

Clasa *Main* implementează metoda *main* care realizează operațiile cerute în ultima parte a cerințelor problemei. Astfel, metoda construiește un pluton ce conține patru arcași (referit de *pluton1*), apoi un pluton ce conține doi arcași (referit de *pluton3*) și în fine, un pluton (*pluton2*) ce conține un călăreț și plutonul anterior (*pluton3*).

Pentru exemplificarea luptei, fiecare pluton lovește succesiv pe celălalt pluton, până în momentul în care unul din plutoane nu mai e viu. Pentru a fi mai interesantă simularea, se va determina aleator care pluton lovește prima dată. În final se tipărește pe ecran starea plutoanelor și învingătorul, și apoi se tipărește numărul cailor decedați.

```
public class Main {

    public static void main(String argv[]) {

        Pluton pluton1, pluton2, pluton3;

        pluton1 = new Pluton();
        pluton1.adauga(new Arcas());
        pluton1.adauga(new Arcas());
        pluton1.adauga(new Arcas());
        pluton1.adauga(new Arcas());
    }
}
```

```

    pluton3 = new Pluton();
    pluton3.adauga(new Arcas());
    pluton3.adauga(new Arcas());
    pluton2 = new Pluton();
    pluton2.adauga(new Calaret());
    pluton2.adauga(pluton3);

    //Pentru a determina aleator care pluton loveste primul
    //folosim metoda statica random din clasa Math ce intoarce
    //un numar aleator intre 0 si 1.
    boolean loveste_primul = (Math.random() > 0.5);
    while(pluton1.esteVie() && pluton2.esteVie()) {
        if(loveste_primul) {
            System.out.println("Loveste Pluton1");
            pluton1.loveste(pluton2);
        } else {
            System.out.println("Loveste Pluton2");
            pluton2.loveste(pluton1);
        }
        loveste_primul = !loveste_primul;
    }
    System.out.println("Pluton1 este viu:" + pluton1.esteVie());
    System.out.println("Pluton2 este viu:" + pluton2.esteVie());
    System.out.println("A castigat:" + (pluton1.esteVie() ? "pluton1" :
        pluton2.esteVie() ? "pluton2" : "nimeni"));

    System.out.println("Numar cai decedati:"
        + Calaret.getNrCaiPierduti());

}

}

```

6.7 Exerciții

1. Modificați ultima implementare a metodei *transferPCToDevice* din Secțiunea 6.2 astfel încât aceasta să respecte principiul OCP iar apelul acesteia să producă același efect. Puteți modifica oricare din clasele *Device*, *PhotoDevice* și *VideoDevice*. Care sunt beneficiile modificării?
2. Modificați corespunzător clasa *B* din Secțiunea 6.3 astfel încât apelul metodei *oMetoda* din exemplul dat să se facă polimorific.
3. La ghișeu de încasări a taxelor locale se prezintă un contribuabil. Operatorul de la ghișeu caută contribuabilul (după nume sau CNP), îi spune cât are de plătit pentru anul curent în total pentru toate proprietățile după care poate încasa bani (o sumă

totală sau parțială). Fiecare contribuabil poate deține mai multe proprietăți: clădiri și/sau terenuri. Fiecare proprietate e situată la o adresă (o adresă are stradă și număr). Suma datorată în fiecare an pentru fiecare tip de proprietate se calculează în felul următor:

- pentru clădire: $500 * \text{suprafața clădirii}(m^2)$
- pentru teren: $350 * \text{suprafața terenului}(m^2) / \text{rangul localității în care se află terenul}$. Rangul unei localități poate fi 1, 2, 3 sau 4.

Contribuabilul, indiferent dacă plătește sau nu, poate solicita un fluturaș cu toate proprietățile pe care le deține alături de suma pe care trebuie să o plătească în anul curent pentru o proprietate (fluturașul arată la fel indiferent dacă pentru anul în curs contribuabilul a achitat ceva sau nu). Fluturașul are următoarea structură:

```
Contribuabil: Ion Popescu

Proprietati
  Cladire: Strada V Parvan Nr. 2
          Suprafata: 20
          Cost: 10000

  Teren:   Strada V. Parvan Nr. 2
          Suprafata: 10, Rang: 1
          Cost: 3500

  Cladire: Strada Lugoj Nr. 4
          Suprafata: 25
          Cost: 12500

Suma totala: 26000
```

Se cere:

- să se construiască diagrama UML pentru clasele necesare la realizarea operațiilor descrise anterior.
 - să se implementeze o parte din clasele identificate mai sus astfel încât să poată fi executată operația: operatorul, după ce a găsit contribuabilul Ion Popescu, afișează fluturașul corespunzător acestui contribuabil. În metoda *main* se instanțiază clasa ce modelează conceptul de contribuabil, se setează proprietățile aferente acestuia după care se face afișarea lor.
4. Se cere să se modeleze o garnitură de tren. Se va defini în acest scop o clasă *Tren*. Un obiect de tip *Tren* conține mai multe referințe spre obiecte de tip *Vagon* care sunt păstrate într-un tablou. Un vagon poate fi de 3 tipuri: *CalatoriA*, *CalatoriB* și *Marfa*. Despre garnitura de tren și vagoane mai cunoaștem următoarele:
- un tren poate conține maxim 15 vagoane, indiferent de tipul vagoanelor. Vagoanele sunt atașate trenului la crearea lui.

- un vagon de tip CalatoriA
 - are capacitatea de 40 pasageri și 300 colete poștale.
 - furnizează două servicii pentru deschiderea, respectiv închiderea automată a ușilor.
- un vagon de tip CalatoriB
 - are capacitatea de 50 pasageri și 400 colete poștale.
 - furnizează două servicii pentru deschiderea, respectiv închiderea automată a ușilor.
 - fiind un vagon mai nou, furnizează un serviciu automat pentru blocarea geamurilor.
- un vagon de tip Marfa
 - are capacitatea de 400 colete poștale.
 - nu furnizează servicii pentru deschiderea, respectiv închiderea automată a ușilor, aceste operații executându-se manual. Atenție: se interzice existența metodelor pentru deschiderea, respectiv închiderea automată a ușilor în clasa ce modelează acest tip de vagon.

Se cere:

- să se construiască diagrama UML pentru clasele identificate pe baza descrierii anterioare.
 - să se implementeze clasa care modelează conceptul de vagon împreună cu eventualele sale clase derivate. Se menționează că apelurile serviciilor pentru deschiderea, respectiv închiderea ușilor, blocarea geamurilor vor afișa pe ecran un mesaj corespunzător, spre exemplu, apelul serviciului pentru blocarea geamurilor ar putea tipări “Geamurile s-au blocat”. Implementarea se va face astfel încât valorile asociate numărului de pasageri, colete să nu fie stocate în diverse câmpuri ale vagoanelor.
 - să se implementeze o metodă pentru determinarea egalității dintre două trenuri, presupunându-se că două trenuri sunt egale dacă pot transporta același număr de colete, precum și o metodă pentru afișarea tipurilor de vagoane existente într-un tren (ATENȚIE: Tipul unui vagon trebuie determinat prin apeluri polimorfice).
 - să se scrie o metodă *main* pentru exemplificarea apelurilor celor 2 metode definite la punctul precedent.
5. Considerăm o colecție de greutăți în cadrul căreia elementele sunt reținute sub forma unui tablou. Fiecare greutate are o anumită capacitate care poate fi obținută apelând metoda *public int capacitate()* pe care o are fiecare greutate. Greutățile pot fi de următoarele tipuri:
- simple, a căror capacitate este setată prin constructor.

- duble, adică formate din 2 greutateți ce sunt stocate în două câmpuri de tip *Greutate*. Aceste greutateți sunt setate prin constructor dar pot să fie modificate pe parcursul existenței obiectelor de acest tip prin intermediul a două metode accesore (*public void setGreutate1(Greutate g)*, *public void setGreutate2(Greutate g)*). Capacitatea acestui tip de greutate e egală cu suma capacităților celor două greutateți conținute. Capacitatea acestui tip de greutate nu va fi reținută într-un atribut, ci va fi calculată de fiecare dată când unui obiect de acest tip i se va solicita serviciul *capacitate()*.
- multiple, care reprezintă o înșiruire de greutateți simple, duble, și/sau eventual alte greutateți multiple. Cu alte cuvinte, o greutate multiplă reprezintă o înșiruire de greutateți. Capacitatea unei greutateți de acest tip este egală cu suma capacităților greutateților componente. Componentele acestui tip de greutate se setează prin constructorul clasei, dar se poate alege și o altă modalitate de inserare a componentelor. Ca și în cazul clasei descrise anterior, capacitatea acestui tip de greutate nu va fi reținută într-un atribut, ci va fi calculată de fiecare dată când unui obiect de acest tip i se va solicita serviciul *capacitate()*.

Sistemul mai cuprinde și clasa *ColectieGreutati* care conține un tablou de greutateți (acestea reprezintă conținutul efectiv al colecției). Clasa *ColectieGreutati* va conține următoarele metode:

- *public void adauga(Greutate g)*: are rolul de a adăuga elemente în tabloul de greutateți. Presupunem că o colecție de greutateți are o capacitate maximă de greutateți care se setează prin intermediul constructorului.
- *public double medie()*: returnează greutatea medie a colecției (capacitate/numar de greutati).

Se cere:

- diagrama UML pentru clasele prezentate mai sus.
- implementarea claselor prezentate în diagramă.
- o metodă *main* în care se va crea un obiect *ColectieGreutati*, câteva greutateți simple, duble și multiple care vor fi adăugate colecției de greutateți. Se va afișa greutatea medie a colecției.

Bibliografie

1. Bruce Eckel. *Thinking in Java, 4th Edition*. Prentice-Hall, 2006. Capitolul Polymorphism.
2. Radu Marinescu, Carmen De Sabata. *Ingineria Programării 1. Îndrumător de laborator*. Casa Cărții de Știință, 1999. Lucrarea 6, Interfețe și polimorfism.
3. Robert C. Martin. *Agile Software Development. Principles, Patterns and Practices*. Prentice Hall, 2003. Capitolul 9, OCP: The Open-Closed Principle.

Lecția 7

Interfețe

Să presupunem că trebuie să scriem o clasă ce folosește la gestionarea jucătorilor unei echipe de fotbal. O parte din această clasă este prezentată mai jos.

```
class EchipaFotbal {  
  
    private String nume;  
    private JucatorFotbal capitan;  
    private JucatorFotbal[] jucatori = new JucatorFotbal[10];  
    ...  
}
```

Ce poate fi entitatea *JucatorFotbal*? Ea poate fi o clasă (posibil subclasă a unei alte clase și/sau abstractă) sau o *interfață*. Dacă despre clase, subclase și clase abstracte s-a vorbit pe parcursul mai multor lecții anterioare, în această lecție se va vorbi despre interfețe.

7.1 Definirea unei interfețe. Componenta unei interfețe

Principial, o interfață poate fi văzută ca o clasă pur abstractă, ca o clasă ce conține doar metode abstracte. Definirea unei interfețe se face asemănător cu a unei clase înlocuind cuvântul cheie *class* cu cuvântul cheie *interface*.

```
interface JucatorFotbal {  
    ...  
}
```

Spre deosebire de o clasă, o interfață poate conține doar atribute declarate *static final* și doar metode ale căror corpuri sunt vide. În același timp, membrii unei interfețe sunt doar publici.

```
interface JucatorFotbal {  
    int categorie = 5;  
    void joacaFotbal();  
}
```

În exemplul de mai sus membrii interfeței nu au fost declarați explicit ca fiind publici, deoarece acest aspect este implicit atunci când definim o interfață. Cu alte cuvinte exemplul de mai jos este identic cu cel de sus. La fel se întâmplă și cu câmpurile care sunt implicit declarate *final static*.

```
interface JucatorFotbal {  
    public int categorie = 5;  
    public void joacaFotbal();  
}
```

Important

Toți membrii unei interfețe sunt publici chiar dacă specificatorul de acces *public* este omis în definirea interfeței. O interfață nu poate conține membrii declarați *private* sau *protected*.

Important

Toate atributele unei interfețe sunt *static final* chiar dacă ele nu sunt declarate astfel în definirea interfeței. Datorită acestui fapt orice atribut al unei interfețe trebuie inițializat.

Important

O interfață poate extinde oricâte alte interfețe utilizând cuvântul cheie *extends*.

7.2 Implementarea unei interfețe

După cum spuneam anterior, o interfață poate fi văzută ca fiind o clasă pur abstractă. După cum știm, o clasă abstractă nu poate fi instanțiată și deci e normal ca acest lucru să fie valabil și în cazul interfețelor. În acest context, ce poate referi *jf* din declarația de mai jos?

```
JucatorFotbal jf;
```

Referința *jf* poate referi orice obiect instanță a unei clase care *implementează* interfața *JucatorFotbal*. A implementa o anumită interfață înseamnă a realiza într-un anumit mod funcționalitatea precizată de interfața respectivă.

În Java, implementarea unei interfețe de către o clasă se face folosind cuvântul cheie *implements* urmat de numele interfeței implementate, ca în exemplele de mai jos.

```

class JucatorBunFotbal implements JucatorFotbal {

    public void joacaFotbal() {
        System.out.println("Eu joc bine fotbal.");
    }
}

```

În UML, implementarea de către o clasă a unei interfețe se numește *realizare*. În Figura 7.1 se exemplifică modul de reprezentare a interfețelor și a relațiilor de realizare pe baza codului prezentat anterior.

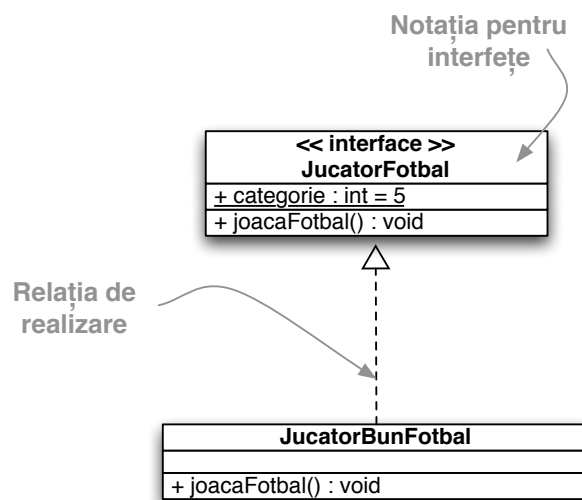


Figura 7.1: REPREZENTAREA UML A INTERFEȚEI ȘI A RELAȚIEI DE REALIZARE.

Important În momentul în care o clasă implementează o interfață aceasta fie implementează toate metodele din interfață, fie e declarată abstractă.

Important O interfață poate avea oricâte implementări. Două clase distincte pot implementa o aceeași interfață.

Important O clasă poate extinde o singură clasă dar poate implementa oricâte interfețe.

Important Dacă în clasa *JucatorFoarteBun* de mai jos specificatorul de acces al metodei nu ar fi fost public, compilatorul ar fi semnalat o eroare la compilare. Motivul este foarte simplu: în momentul în care clasa implementează o interfață,

ea este obligată să furnizeze clienților toate serviciile specificate de interfața implementată.

```
class JucatorFoarteBunFotbal implements JucatorFotbal {

    public void joacaFotbal() {
        System.out.println("Eu joc foarte bine fotbal.");
    }
}

interface JucatorTenis {
    void joacaTenis();
}

class JucatorFotbalTenis implements JucatorFotbal, JucatorTenis {

    public void joacaFotbal() {
        System.out.println("Eu joc fotbal.");
    }

    public void joacaTenis() {
        System.out.println("Eu joc tenis.");
    }
}

class JucatorFoarteBunFotbalTenis
    extends JucatorFoarteBunFotbal implements JucatorTenis {

    public void joacaTenis() {
        System.out.println("Eu joc tenis.");
    }
}
```

Având în vedere cele precizate mai sus, este momentul să atașăm referinței *jf* diferite obiecte.

```
//jf refera instanta unei clase care implementeaza interfata JucatorFotbal
jf = new JucatorBunFotbal();

//jf refera instanta unei alte clase care implementeaza JucatorFotbal
jf = new JucatorFoarteBunFotbal();

//jf refera instanta unei clase care implementeaza interfetele
//JucatorFotbal si JucatorTenis
jf = new JucatorFotbalTenis();
jf.joacaFotbal();
```

```
//Prin intermediul referintei jf pot fi accesate doar serviciile
//definite de interfața JucatorFotbal
jf.joacaTenis(); //EROARE
((JucatorFotbalTennis)jf).joacaTenis(); //CORECT dar nu se recomanda

//jf refera o instanța a unei clase ce extinde o clasă
//ce implementează interfața JucatorFotbal
jf = new JucatorFoarteBunFotbalTennis();
```

În toate exemplele anterioare ceea ce e important constă în faptul că *jf* poate referi o instanță a oricărei clase ce implementează interfața *JucatorFotbal*, o instanță care pune la dispoziție serviciile din interfața *JucatorFotbal*.

7.3 Tipurile unui obiect

În Lecția 1 spuneam că ansamblul tuturor operațiilor primitive oferite de un obiect se numește interfața obiectului respectiv și că numele unei interfețe denotă un tip. Tot atunci am precizat că e posibil ca un obiect să aibe mai multe tipuri, acest aspect fiind delimitat de perspectiva din care este văzut obiectul. Până acum am văzut situații în care un obiect avea mai multe tipuri diferite: o instanță a unei clase avea atât tipul subclasei cât și al tuturor superclaselor sau supertipurilor acelei clase. Deoarece în Java o clasă poate extinde cel mult o clasă, utilizând doar clase, între oricare două tipuri a unui obiect există tot timpul o relație supertip-subtip. Datorită faptului că o clasă poate implementa mai multe interfețe, putem modela și obiecte ce au două sau mai multe tipuri între care nu există o astfel de relație.



Din punctul de vedere al organizatorilor unui campionat de fotbal este necesar ca toți participanții să știe să joace fotbal, nefiind necesară practicarea de către aceștia și a altor sporturi. În același timp, la un campionat de fotbal pot participa și jucători de fotbal ce știu juca și tenis, acest aspect rămânând necunoscut organizatorilor.

În exemplul de mai jos, referinței *JucatorFotbal jf* i s-a atașat un obiect instanță a unei clase ce implementează, pe lângă interfața *JucatorFotbal* și interfața *JucatorTennis*. Utilizând această referință putem apela metode (servicii) definite în interfața *JucatorFotbal*, dar în nici un caz nu putem apela direct metoda *joacaTennis*. Acest lucru se datorează faptului că referința e de tip *JucatorFotbal* iar compilatorul vede că această interfață nu definește un serviciu *joacaTennis* semnaland o eroare. Dacă totuși dorim acest lucru, va trebui să utilizăm operatorul de *cast*, convertind referința într-o referință de tipul *JucatorTennis*. Acest lucru nu e indicat a se realiza deoarece dacă obiectul referit de *jf* nu știe “juca tenis” (clasa sa nu implementează interfața *JucatorTennis* dar implementează *JucatorFotbal* sau altfel spus obiectul referit nu e de tip *JucatorTennis* dar e

de tip *JucatorFotbal*) se va genera o eroare la execuția programului.

```
jf = new JucatorFotbalTenis();
jf.joacaFotbal();           //Apel corect
jf.joacaTenis();            //Eroare de compilare
((JucatorTenis)jf).joacaTenis(); //Corect, dar poate fi foarte riscant
```



Un obiect este de un anumit tip dacă acesta furnizează toate serviciile specificate de acel tip. Un obiect instanță a clasei *JucatorFotbalTenis* are tipurile: *JucatorFotbalTenis*, *JucatorFotbal*, *JucatorTenis* și, evident, *Object*.

7.4 Conflicte

Având în vedere că o clasă poate implementa oricâte interfețe, e posibil să apară uneori conflicte între membrii diferitelor interfețe implementate de o clasă. La compilare primului exemplu de mai jos va fi semnalată o eroare deoarece în cadrul metodei *oMetoda* nu se știe care atribut *C* este utilizat. În cazul celui de-al doilea exemplu situația e asemănătoare.

```
interface A {
    int C = 5;
}

interface B {
    int C = 5;
}

class Clasa implements A,B {

    public void oMetoda(){
        int c = C + 5; //EROARE, dar se poate rezolva
                       //scriind A.C sau B.C functie de
                       //care camp C se doreste a fi utilizat
    }
}

interface AA {
    void f();
}
```

```
interface BB {
    int f();
}

class CC implements AA, BB {
    //Conflict - doua metode nu pot diferi doar prin tipul returnat
}
```

7.5 Clase abstracte versus interfețe

Când e mai bine să folosim clase abstracte și când interfețe? În această secțiune vom prezenta comparativ avantajele și dezavantajele folosirii uneia versus celeilalte.

Presupunem că vrem să descriem mai multe instrumente muzicale, printre care se află și instrumentele vioară și pian. Cu ajutorul oricărui tip de instrument muzical se poate cânta iar într-o orchestră pot să apară la un moment dat diferite tipuri de instrumente. În acest context pentru descrierea instrumentelor într-un limbaj de programare orientat pe obiecte putem avea o entitate numită *Instrument*. În Java, această entitate poate fi atât o clasă abstractă cât și o interfață.

```
abstract class AInstrument {
    public abstract void canta();
}

interface IInstrument {
    void canta();
}
```

Definirea instrumentelor vioară și pian, atât pentru cazul în care se extinde o clasă abstractă cât și pentru cazul în care se implementează o interfață e prezentată mai jos.

```
class AVioara extends AInstrument {

    public void canta() {
        System.out.println("Vioara canta.");
    }
}

class IVioara implements IInstrument {

    public void canta() {
        System.out.println("Vioara canta.");
    }
}
```

Dar după o anumită perioadă de timp, toate instrumentele care apar pe piață știu să se acordeze automat. Este evident că toate instrumentele trebuie să furnizeze o metodă *public void acordeaza()* orchestrei din care fac parte.

```
class APian extends AInstrument {  
  
    public void canta() {  
        System.out.println("Pianul canta.");  
    }  
}  
  
class IPian implements IInstrument {  
  
    public void canta() {  
        System.out.println("Pianul canta.");  
    }  
}
```

Pentru varianta în care se extinde o clasă abstractă, modificările necesare sunt minime. E nevoie de adăugarea unei metode care să furnizeze serviciul pentru acordarea automată în clasa de bază, clasele *Avioara* și *APian* rămânând nemodificate.

```
abstract class AInstrument {  
  
    public abstract void canta();  
  
    public void acordeaza() {  
        System.out.println("Acordare automata.");  
    }  
}  
  
interface IInstrument {  
    void canta();  
    void acordeaza();  
}  
  
class IVioara implements IInstrument {  
  
    public void canta() {  
        System.out.println("Vioara canta.");  
    }  
  
    public void acordeaza() {  
        System.out.println("Acordare automata.");  
    }  
}
```

Pentru varianta în care se implementează o interfață, modificările necesare sunt mai mari decât în cazul anterior. În primul rând trebuie adăugată metoda *public void acordeaza()* în interfața *IInstrument* iar apoi această metodă trebuie implementată în clasele *IVioara* și *IPian*.

```
class IPian implements IInstrument {  
  
    public void canta() {  
        System.out.println("Pianul canta.");  
    }  
  
    public void acordeaza() {  
        System.out.println("Acordare automata.");  
    }  
}
```

Atenție

Introducerea unei metode într-o interfață necesită modificarea tuturor claselor neabstracte care implementează respectiva interfață.

7.6 Exercițiu rezolvat

Expresii Derivate

Într-un sistem de prelucrare de expresii matematice (evident simplificat) există mai multe feluri de expresii ce vor fi detaliate mai jos. Indiferent de felul concret al unui obiect expresie, pe acesta se poate apela din exteriorul său o metodă denumită *calculDerivată* care întoarce o referință la un alt obiect expresie reprezentând derivata de ordinul întâi a expresiei pe care s-a apelat metoda sus numită. Modul de construire a expresiei derivate se prezintă în detaliu mai jos împreună cu diferitele feluri de expresii:

1. Constantă - reprezintă o expresie constantă (ex. 1). Valoarea constantei este dată ca parametru la crearea unui astfel de obiect și va fi păstrată intern de obiectul constantă creat. Derivata unei constante este o expresie constantă a cărei valoare este 0. Clasa mai definește o metodă ce întoarce un String care e reprezentarea șir de caractere a expresiei (ex. "1").
2. Variabilă - reprezentând variabila "x". Derivata unei expresii variabilă este o expresie constantă a cărei valoare este 1. Clasa mai definește o metodă ce întoarce un String care e reprezentarea șir de caractere a expresiei (în acest caz tot timpul "x").
3. Sumă - reprezentând o expresie sumă între două expresii de orice fel (ex. $1 + x$). Expresiile care sunt însumate vor fi date ca argumente la crearea unui obiect sumă și vor fi memorate intern de obiectul creat. Derivata unei sume este o expresie construită după formula $(a + b)' = a' + b'$. Clasa mai definește o metodă ce întoarce un String care e reprezentarea șir de caractere a expresiei. Aceasta e

formată din reprezentarea String a expresiei din stânga sumei urmată de “+” și apoi de reprezentarea String a expresiei din dreapta. Pentru a evita probleme de precedență a operatorilor, reprezentarea ca șir de caractere se va pune între paranteze (ex. “(1+x)”).

4. Înmulțire - reprezentând o expresie înmulțire între două expresii de orice fel (ex. $2 * x$). Expresiile care sunt înmulțite vor fi date ca argumente la crearea unui astfel de obiect și vor fi memorate intern de către acesta. Derivata unei înmulțiri este o expresie construită după formula $(a*b)' = a'*b + a*b'$. Clasa mai definește o metodă ce întoarce reprezentarea sub formă de String a expresiei. Aceasta e construită ca în cazul sumei dar cu semnul “*” în loc de “+” (ex. “(2 * x)”).

Notă: Nu trebuie să efectuați simplificări matematice în expresia derivată.

Se cere:

1. Diagrama UML detaliată a claselor prezentate, împreună cu eventuale alte clase / interfețe care se consideră necesare.
2. Implementarea tuturor claselor.
3. Pentru exemplificare, se va construi într-o metodă main structura de obiecte corespunzătoare expresiei matematice “1 + x*x”. După construcție se va tipări pe ecran expresia, derivata de ordinul întâi al expresiei și derivata de ordinul doi a aceleiași expresii.

Rezolvare

Din specificațiile problemei (cât și din cunoștințele noastre de matematică) putem observa că o expresie sumă poate reprezenta suma a două alte (sub) expresii de orice fel. Astfel putem avea o sumă între două constante, între două variabile, între două expresii ce reprezintă înmulțiri, între o constantă și o altă sumă, etc. În mod similar, putem avea diferite combinații de feluri de operanzi și în cazul înmulțirii.

În plus, dacă ne gândim cum s-ar putea extinde pe viitor această aplicație, este clar că una din direcțiile de îmbunătățire foarte probabile constă în adăugarea de noi feluri de expresii precum împărțire, radical, etc. La rândul lor, aceste feluri de expresii vor trebui să poată avea orice fel de operanzi și vor trebui să poată fi incluse în sume, înmulțiri, etc. Prin urmare, numărul de combinații pe care va trebui să-l gestionăm pe viitor va fi cu mult mai mare decât putem observa acum într-o primă variantă a programului (număr care și acum este destul de mare).

Pentru a putea rezolva simplu această explozie de combinații ne vom baza pe polimorfism, ne vom baza pe faptul că putem trata uniform diversele feluri de expresii matematice. Mai exact, dorim să putem declara variabile referință ce să poată referi uniform atât constante cât și sume, înmulțiri și variabile. Prin urmare, toate aceste obiecte vor

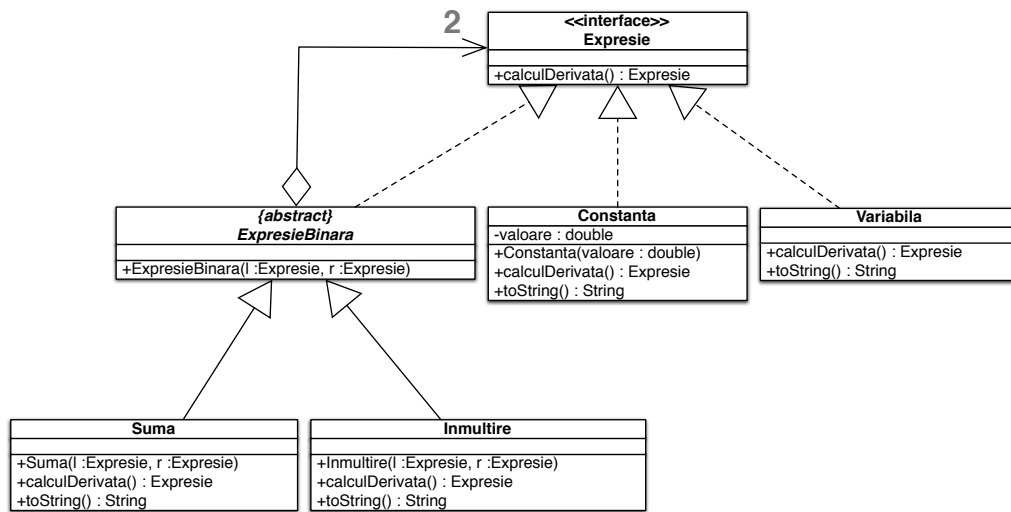


Figura 7.2: DIAGRAMA DE CLASE.

trebuie să aibă un supertip comun pentru a putea declara referințe cu o astfel de proprietate.

În același timp, dacă ne gândim la formula de derivare a unei sume, putem observa că, pentru a construi derivata sumei, avem nevoie de expresiile derivate ale operanzilor săi. Același lucru poate fi observat și în cazul înmulțirii. Pe de altă parte, anterior am spus că o sumă/înmulțire poate avea ca operanzi expresii de diverse feluri (adică constante, variabile, etc.). În consecință, suma și înmulțirea vor trebui să poată calcula uniform derivata operanzilor lor, vor trebui să poată apela uniform metoda *calculDerivata* indiferent de felul concret al unei expresii operand.

Având în vedere toate aceste observații am definit interfața *Expresie* ce va fi implementată de toate clasele ce modelează obiecte expresii matematice. Interfața conține metoda *calculDerivata* care va întoarce o referință de tip *Expresie*. Motivul utilizării acestui tip returnat e simplu: derivata unei expresii oarecare poate fi o sumă de alte subexpresii, poate fi o constantă, etc. În general ar putea fi orice fel de expresie.

```

interface Expresie {

    Expresie calculDerivata();

}

```

Implementarea clasei *Constanta* este extrem de simplă și prin urmare nu o vom discuta în detaliu. Singurul lucru mai interesant constă în calculul derivatei sale. În acest sens se va crea o instanță a clasei *Constanta* reprezentând valoarea 0, iar metoda *calculDerivata* întoarce o referință către acest obiect nou creat.

```
class Constanta implements Expresie {  
  
    private double valoare;  
  
    public Constanta(double valoare) {  
        this.valoare = valoare;  
    }  
  
    public Expresie calculDerivata() {  
        return new Constanta(0);  
    }  
  
    public String toString() {  
        return valoare + "";  
    }  
  
}
```

În mod similar se va calcula și derivata unei variabile: se va crea o instanță a clasei *Constanta* reprezentând valoarea 1, iar metoda *calculDerivata* întoarce o referință către acest obiect nou creat.

```
class Variabila implements Expresie {  
  
    public Expresie calculDerivata() {  
        return new Constanta(1);  
    }  
  
    public String toString() {  
        return "x";  
    }  
  
}
```

Din specificațiile problemei putem observa că suma și înmulțirea au în comun un lucru: ambele expresii au doi operanzi. Prin urmare ne-am bazat pe moștenirea de clasă și am factorizat aceste date într-o clasă abstractă *ExpresieBinara*. Ea implementează interfața *Expresie* și va fi extinsă atât de clasa *Suma* cât și de clasa *Inmultire*.

Rezoluți

Clasa *ExpresieBinara* poate factoriza într-o formă comună și implementarea metodei *toString* pe care fiecare subclasă o redefineste. Această

factorizare nu a fost realizată aici fiind lăsată ca exercițiu.

De ce este clasa *ExpresieBinara* declarată ca fiind abstractă? Deoarece nu are sens să o putem instanția, rolul său fiind doar factorizarea de cod comun. Nu avem obiecte care sunt pur și simplu expresii binare. Avem fie sume fie înmulțiri.

```
abstract class ExpresieBinara implements Expresie {  
  
    protected Expresie st,dr;  
  
    public ExpresieBinara(Expresie st, Expresie dr) {  
        this.st = st;  
        this.dr = dr;  
    }  
}
```

O primă sarcină care trebuie realizată de implementarea clasei *Suma* constă în setarea operanzilor conform specificațiilor problemei. Acest lucru se realizează prin constructorul clasei care, la rândul său, memorează operanzii obiectului după cum impune superclasa (adică prin apelarea constructorului din superclasa *ExpresieBinara*).

Un alt lucru pe care îl vom discuta se referă la modul în care se va construi expresia derivată a unei sume. Astfel, metoda *calculDerivata* va crea un nou obiect sumă care are ca și operanzi derivata operandului din stânga sumei, respectiv derivata operandului din dreapta. Derivatele operanzilor se determină simplu: prin apelarea pe obiectele corespunzătoare a metodei *calculDerivata*. În cele din urmă, metoda *calculDerivata* din clasa *Suma* întoarce o referință la obiectul sumă nou creat. Se poate observa că modul de construcție al derivatei reflectă exact formula de derivare cunoscută.

```
class Suma extends ExpresieBinara {  
  
    public Suma(Expresie st, Expresie dr) {  
        super(st,dr);  
    }  
  
    public Expresie calculDerivata() {  
        return new Suma(st.calculDerivata(),dr.calculDerivata());  
    }  
  
    public String toString() {  
        return "(" + st.toString() + " + " + dr.toString() + ")";  
    }  
}
```

Modul de implementare a clasei *Inmultire* este similar cu a clasei *Suma*. Din acest motiv nu vom mai descrie implementarea obiectelor înmulțire.

```
class Inmultire extends ExpresieBinara {  
  
    public Inmultire(Expresie st, Expresie dr) {  
        super(st,dr);  
    }  
  
    public Expresie calculDerivata() {  
        Expresie t1 = new Inmultire(st,dr.calculDerivata());  
        Expresie t2 = new Inmultire(st.calculDerivata(),dr);  
        return new Suma(t1,t2);  
    }  
  
    public String toString() {  
        return "(" + st.toString() + " * " + dr.toString() + ")";  
    }  
  
}
```

În metoda *main* construim în primul rând expresia cerută. Astfel, vom crea un obiect constantă reprezentând valoarea 1 (referită de *c1*), două variabile (referite de *v1* respectiv *v2*), o înmulțire (referită de *i1*) care va avea ca operanzi variabilele create anterior (referite de *v1* respectiv *v2*) și, în final, o sumă (referită de *exp*) ce are ca operanzi constanta (*c1*) respectiv suma creată anterior (referită de *i1*).

În continuare tipărim pe ecran expresia. Se poate observa că pur și simplu tipărim referința *exp*. Acest lucru e posibil deoarece pentru toate clasele am redefinit corespunzător metoda *toString* declarată în clasa *Object*.

Pentru calculul derivatei de ordinul unu vom apela pe referința *exp* metoda *calculDerivata* și salvăm valoarea întoarsă în variabila *deriv1* (care este tot o expresie). Pentru calculul derivatei de ordinul doi se va apela *calculDerivata* pe *deriv1* (adică pe derivata de ordinul unu). Evident, se tipăresc aceste derivate conform cerințelor.

```
class Main {  
  
    public static void main(String[] args) {  
        Constanta c1 = new Constanta(1);  
        Variabila v1 = new Variabila();  
        Variabila v2 = new Variabila();  
        Inmultire i1 = new Inmultire(v1,v2);  
        Suma exp = new Suma(c1,i1);  
    }  
}
```

```
        System.out.println(exp);

        Expresie deriv1 = exp.calculDerivata();
        System.out.println(deriv1);

        Expresie deriv2 = deriv1.calculDerivata();
        System.out.println(deriv2);
    }
}
```

Rezolvați

Rezolvați problema fără a face uz de polimorfism și comparați complexitatea unei astfel de implementări (incorecte din punct de vedere al programării orientate pe obiecte) cu cea bazată pe polimorfism. Apoi adăugați un nou fel de expresie în ambele rezolvări (de exemplu radical). Comparați dificultățile întâmpinate în cele două variante.

7.7 Exerciții

1. Creați o interfață cu trei metode. Implementați doar două metode din interfață într-o clasă. Va fi compilabilă clasa?
2. Se cere să se modeleze folosind obiecte și interfețe un sistem de comunicație prin mesaje între mai multe persoane. Pentru simplitate, considerăm că sunt disponibile doar două modalități de comunicare: prin e-mail sau prin scrisori (poștă clasică).

O persoană X nu poate trimite un mesaj unei persoane Y direct, deoarece, de regulă, distanțele între persoane sunt prea mari. De aceea, pentru orice mesaj, persoana X trebuie să apeleze la un obiect transmițător, care știe să trimită mesaje către diverși destinatari, în speță, și către persoana Y.

Practic, dacă X dorește să-i trimită un e-mail lui Y va apela la un obiect *EMailTransmitter*, iar dacă dorește să-i trimită o scrisoare lui Y va opta pentru un obiect *MailTransmitter*. X va prezenta, în fiecare caz, identitatea proprie (vezi this), identitatea destinatarului, și mesajul pe care dorește să-l transmită.

Evident, cele două moduri de comunicare trebuie să difere prin ceva, altfel nu s-ar justifica existența a două tipuri de obiecte. La transmiterea unui e-mail, destinatarul mesajului este notificat imediat de obiectul intermediar (*EMailTransmitter*). Notificarea unei persoane implică execuția unei metode a clasei care modelează o persoană, fiind sarcina acestei metode de a cere obiectului *EMailTransmitter* în cauză să furnizeze mesajul respectiv. La transmiterea unei scrisori, destinatarul nu este notificat imediat. Chiar și în realitate, o scrisoare nu este transmisă spre destinație imediat ce a fost depusă în cutia poștală, ci doar în anumite momente ale zilei

(dimineața, seara, etc). În aplicația noastră, considerăm că destinatarii scrisorilor sunt notificați doar atunci când cutia poștală *se umple* cu mesaje. Adică obiectul *MailTransmitter* care modelează sistemul clasic de poștă va avea un buffer de mesaje (de exemplu, un tablou) cu N elemente. Destinatarii mesajelor sunt notificați doar atunci când în buffer s-au adunat N mesaje. Notificarea unui destinatar presupune același lucru ca și în cazul poștei electronice, pentru o tratare unitară a celor două cazuri. După ce toți destinatarii au fost notificați și și-au *ridicat* scrisorile, buffer-ul va fi golit.

Sistemul trebuie să fie alcătuit din module slab cuplate (loosely coupled), ceea ce va duce la posibilitatea extinderii sale facile. Cu alte cuvinte, trebuie să ținem cont că, în viitor, ar putea fi nevoie să folosim și alte modalități de comunicare prin mesaje (fax, SMS, etc.), care trebuie să poată fi integrate în sistemul nostru fără a fi nevoie de prea multe modificări.

Cerințe. Sugestii de implementare Odată ce sistemul este modelat, el trebuie să permită o abordare de forma celei de mai jos:

```
// 4 persoane
Person p1=new Person("Paul");
Person p2=new Person("Andreea");
Person p3=new Person("Ioana");
Person p4=new Person("Gabriel");

//cream sistemul de transmitere prin e-mail-uri
Transmitter email=new EmailTransmitter();

//cream sistemul de transmitere prin scrisori, cu un buffer de 2 scrisori
Transmitter mail=new MailTransmitter(2);

//p1 doreste sa trimita un e-mail catre p2
p1.setTransmitter(email);
p1.send(p2,"Scrie-i Ioanei sa-mi dea adresa ei de e-mail!");

//p2 trimite o scrisoare catre p3. Scrisoarea nu va ajunge imediat,
//deoarece deocamdata este singura in "cutia postala"
p2.setTransmitter(mail);
p2.send(p3,"Paul zice sa-i trimiti adresa ta de e-mail");

//p4 trimite o scrisoare catre p1. Fiind a doua scrisoare,
//buffer-ul de scrisori se va umple si ambele scrisori vor fi trimise
p4.setTransmitter(mail);
p4.send(p1,"Ce mai faci?");
```

```
//p3 a primit in acest moment scrisoarea de la p2 si poate raspunde
//prin e-mail lui p1
p3.setTransmitter(email);
p3.send(p1,"Adresa mea de e-mail este: ioana@yahoo.com");
```

Transmitter se cere să fie văzută ca o interfață, sub forma:

```
public interface Transmitter
{
    public void store(Message message);
    public Message retrieve(Person receiver);
}
```

În codul anterior, *Message* este o clasă care modelează mesaje, adică conține referințe spre cele două persoane implicate în mesaj (sender-ul și receiver-ul) și conține mesajul efectiv sub forma unui obiect *String*.

Clasele *EMailTransmitter* și *MailTransmitter* vor implementa interfața *Transmitter*, adică trebuie să furnizeze implementări specifice pentru metodele *store* și *retrieve*.

Fiecare persoană va conține o referință spre obiectul *Transmitter* prin care vrea să transmită mesaje. Evident, acest obiect poate fi schimbat prin metoda *setTransmitter* pe care o are fiecare persoană.

Întreaga filosofie a aplicației se rezumă la următoarele două reguli:

- în cadrul operației *send*, se va folosi metoda *store* a obiectului *Transmitter* curent.
- când o persoană este notificată că a primit un mesaj, respectiva persoană va primi ca parametru al metodei de notificare și o referință spre obiectul *Transmitter* care a făcut notificarea. Persoana destinatar va cere obiectului *Transmitter*, prin intermediul metodei *retrieve*, să-i furnizeze mesajul. Evident, trebuie să-și prezinte identitatea, deoarece un transmițător de scrisori va avea în buffer mai multe scrisori, și trebuie s-o furnizeze pe aceea care corespunde destinatarului. După ce destinatarul a primit mesajul, el îl va afișa pe monitor.

Astfel, codul de mai sus va produce, de exemplu, textul:

- Paul said to Andreea (EMAIL) : "Scrie-i Ioanei sa-mi dea adresa ei de e-mail"
- Andreea said to Ioana (MAIL) : "Paul zice sa-i trimiti adresa ta de e-mail"
- Gabriel said to Paul (MAIL) : "Ce mai faci?"
- Ioana said to Paul (EMAIL) : "Adresa mea de e-mail este: ioana@yahoo.com"

3. Să se scrie un program Java care modelează activitatea unui ghișeu bancar. Sistemul este format din următoarele entități:

ContBancar cu următoarele atribute:

- `numarCont(String)`
- `suma(float)`

Client cu următoarele atribute:

- `nume(String)`
- `adresa(String)`
- `conturi`(tablou de elemente de tip `ContBancar`; un client trebuie să aibe cel puțin un cont, dar nu mai mult de 5)

Conturile bancare pot fi de mai multe feluri: în LEI și în EURO. Conturile în EURO și numai ele au o dobândă fixă, 0.3 EURO pe zi, dacă suma depășește 500 EURO sau 0 în caz contrar, deci acest tip de cont trebuie să ofere serviciul *public float getDobanda()*. Pot exista transferuri între conturile în LEI și numai între ele, în sensul că un cont de acest tip trebuie să ofere serviciul *public void transfer(ContBancar contDestinatie, float suma)*. Toate conturile implementează o interfață *SumaTotala* care are o metodă *public float getSumaTotala()*. Pentru conturile în lei suma totală este chiar suma existentă în cont iar pentru conturile în EURO este `suma*36.000`.

Banca cu următoarele atribute:

- `clienti`(tablou de elemente de tip `Client`)
- `codBanca(String)`

Conturile, pe lângă implementarea interfeței *SumaTotala*, vor avea metode pentru setarea respectiv citirea atributelor ca unică modalitate de modificare (din exterior) a conținutului unui obiect de acest tip precum și metodele *public float getDobanda()*, *void transfer(ContBancar contDestinatie, float suma)* dar numai acolo unde este cazul.

Clasa *Client* va conține un set de metode pentru setarea respectiv citirea atributelor ca unică modalitate de modificare (din exterior) a conținutului unui obiect *Client*, un constructor prin intermediul căruia se vor putea seta numele, adresa clientului precum și conturile deținute de acesta; clasa trebuie să ofere și o metodă pentru afișare.

Clasa *Banca* va implementa metode pentru efectuarea următoarelor operații, în contextul în care nu pot exista mai mulți clienți cu același nume.

- adăugarea unui client nou *public void add(Client c)*
- afișarea informațiilor despre un client al cărui nume se transmite ca parametru *public void afisareClient(String nume)* în următoarea formă:
 - nume adresa
 - pentru fiecare cont deținut, se va afișa doar suma totală pe o linie separată

În afara metodelor enumerate mai sus, clasele vor ascunde față de restul sistemului toate metodele și atributele conținute.

4. Fie o clasă *Project* care modelează un proiect software. Proiectele vor avea neapărat un manager. La un proiect se pot adăuga oricând participanți, folosind metoda *public void addMember(Member m)*. Orice proiect are un titlu (String), un obiectiv (String) și niște (unul sau mai multe, vezi mai jos) fonduri (long). Managerul și toți participanții vor fi programatori care au o vârstă (int) și un nume (String). Un programator poate participa în mai multe proiecte.

Există trei tipuri de proiecte: comerciale, militare și open-source. Cele comerciale și militare au un dead-line (String) și un număr de maxim 15 membri, cele open-source au un mailing-list (String) și număr nelimitat de membri. Cele militare au și o parolă (String), iar cele comerciale au fonduri de marketing (long) egale cu jumătate din fondurile normale și un număr de echipe (int) mai mic decât numărul de membri.

Toate proiectele implementează o interfață *Risky* care are o metodă *public double getRisk()*. Această metodă calculează riscurile legate de un proiect.

La cele militare, riscul este numărul membrilor / lungimea parolei / fonduri.

La cele comerciale, riscul este numărul echipelor * 3 / numărul membrilor / fonduri - fonduri de marketing.

La proiectele open-source, riscul este numărul membrilor / fonduri.

Clasa *InvestmentCompany* va modela o firmă care, date fiind niște proiecte în număr nelimitat, calculează care este proiectul cel mai puțin riscant. Va pune deci la dispoziție o metodă *public void addProject(Project p)* și o metodă *public Project getBestInvestment()*. Clasa *InvestmentCompany* va avea și o metodă *public static void main(String[] args)* care să exemplifice folosirea clasei.

Cerințe:

- Specificați clasele de care aveți nevoie și desenați ierarhia de clase dacă este cazul. Implementați clasele.
- Unde (în ce clase) ați ales să implementați interfața *Risky* și de ce ați făcut această alegere?

Bibliografie

1. Bruce Eckel. *Thinking in Java, 4th Edition*. Prentice-Hall, 2006. Capitolul Interfaces.
2. Martin Fowler. *UML Distilled, 3rd Edition*. Addison-Wesley, 2003.
3. Carmen De Sabata, Ciprian Chirilă, Călin Jebelean, *Laboratorul de Programare Orientată pe Obiecte*, Lucrarea 7 - Interfețe - Aplicații, UPT 2002, variantă electronică.

Lecția 8

Tratarea excepțiilor

Un program comercial, fie el scris în Java sau în orice alt limbaj de programare, trebuie să țină cont de posibilitatea apariției la execuție a unor anumite situații neobișnuite: fișierul din care se dorește a se citi o informație nu există, utilizatorul a introdus un șir de caractere de la tastatură dar care nu reprezintă un număr așa cum a cerut programul, ș.a.m.d. În această lucrare vom studia un mecanism dedicat tratării acestor situații excepționale.

8.1 Excepțiile și tratarea lor

8.1.1 Ce este o excepție?

Ideal, toate situațiile neobișnuite ce pot să apară pe parcursul execuției unui program trebuie detectate și tratate corespunzător de acesta. Motivul? Există o sumedenie. De exemplu, un program ar putea cere utilizatorului să introducă de la tastatură un număr, dar acesta să introducă un șir de caractere ce nu reprezintă un număr. Un program bine gândit va detecta această problemă, va anunța greșeala și probabil va cere utilizatorului reintroducerea numărului. Un program care se oprește subit în momentul în care utilizatorul face o greșeală, fără nici un avertisment și fără nici un mesaj clar care să anunțe problema, va “băga în ceață” orice utilizator.

Exemplul de mai sus este doar un caz particular de situație neobișnuită sau de excepție. Situațiile neobișnuite pot apare pe parcursul execuției programului și din motive independente de utilizator. De exemplu, am putea avea într-un program o metodă care trebuie să tipărească pe ecran elementul i al unui tablou de întregi, valoarea lui i și tabloul fiind specificați prin parametrii metodei. Dacă valoarea indexului depășește limitele tabloului, vorbim tot de o situație neobișnuită, deși ea are un iz de eroare de programare. Și aceste situații ar trebui detectate de un program chiar dacă utilizatorul nu prea are ce face în cazul unei erori de programare. Tratarea unei astfel de situații ar putea implica de exemplu crearea unui raport de eroare de către utilizator la cererea

programului. Tot este mai bine decât oprirea subită a programului.

Atenție

În general, o eroare este o excepție, dar o excepție nu reprezintă neapărat o eroare de programare.

Din exemplele de până acum s-ar putea crede că tratarea unei excepții implică într-o formă sau alta intervenția utilizatorului. Nu este deloc așa. O practică în industria aero-spațială (e drept destul de primitivă) este ca o aceeași parte dintr-un program să fie scrisă de două ori de programatori diferiți. Dacă la execuție programul de control al sistemului de zbor detectează o eroare de programare într-un exemplar al respectivei părți, el va cere automat celui de-al doilea exemplar să preia responsabilitățile primului exemplar. Acesta este doar un exemplu în care chiar și unele erori de programare pot fi tratate de un program. Să nu mai vorbim de situații mai simple. De exemplu, pentru a îndeplini anumite funcționalități un program trebuie să acceseze serviciile puse la dispoziție de un server. Se poate întâmpla ca la un moment dat respectivul server să fie suprasolicitat și să nu poată răspunde la cererea programului. Și o astfel de situație neobișnuită poate fi văzută ca o excepție la apariția căreia programul ar putea încerca, de exemplu, să acceseze automat un alt server care pune la dispoziție aceleași servicii.

În urma acestei discuții putem defini o excepție în felul următor:

Definiție 1 *O excepție reprezintă un eveniment deosebit ce poate apare pe parcursul execuției unui program și care necesită o deviere de la cursul normal de execuție al programului.*

8.1.2 Metode clasice de tratare a excepțiilor

Pentru a ilustra modul clasic de tratare a situațiilor neobișnuite și a deficiențelor sale vom considera un exemplu ipotetic. Să presupunem un program care manipulează șiruri de numere reale. Clasa de mai jos modelează un astfel de șir. Pentru simplitate vom considera că asupra unui șir putem efectua două operații: adăugarea unui număr la sfârșitul său, respectiv extragerea din șir al primului număr de după prima apariție în șir a unui număr dat. O cerință suplimentară asupra acestei clase este ca diferite situații anormale să poată fi puse în evidență de apelanții metodelor ei.

```
class SirNumereReale {  
  
    private double[] sir = new double[100];  
    private int nr = 0;  
    private boolean exceptie = false;  
  
    public boolean testExceptie() {  
        return exceptie;  
    }  
}
```

```
}

public boolean adauga(double x) {
    if(nr == sir.length) {
        return false;
    }
    sir[nr] = x;
    nr++;
    return true;
}

public double extrageDupa(double x) {
    int i;
    for(i = 0; i < nr; i++) {
        if(sir[i] == x) {
            if(i + 1 < nr) {
                double result = sir[i + 1];
                for(int j = i + 1; j < nr - 1; j++) {
                    sir[j] = sir[j + 1];
                }
                nr--;
                exceptie = false;
                return result;
            } else {
                exceptie = true;
                return -2;
            }
        }
    }
    exceptie = true;
    return -1;
}
```

Un lucru interesant de observat este că situațiile neobișnuite sunt tratate utilizând convenții. Astfel, apelantul metodei de adăugare își va putea da seama că operația a eșuat testând valoarea returnată de metodă. În cazul celei de-a doua metode lucrurile sunt mai complicate. Este nevoie de un fanion care să fie testat după fiecare apel la metoda de extragere. Dacă el indică faptul că avem o situație neobișnuită atunci, pe baza valorii returnate de metodă, apelantul metodei își poate da seama ce s-a întâmplat: valoarea -1 spune apelantului că în acel șir nu există numărul dat ca parametru, iar valoarea -2 îi spune că după numărul dat ca parametru nu mai există nici un număr. Necesitatea fanionului e evidentă: valoarea -1 poate reprezenta chiar un număr din șir. Distincția între valoarea -1 ca număr din șir și valoarea -1 ca și cod de identificare a problemei apărute este realizată prin intermediul fanionului.

La prima vedere, aceste convenții par bune. Din păcate, ele fac utilizarea unui obiect șir destul de dificilă. Să considerăm acum că trebuie să scriem o metodă care primește ca parametru un număr și un șir și trebuie să extragă primele 10 numere ce apar în șir după prima apariție a numărului dat ca parametru și să le calculeze media. Dacă nu există 10 numere după numărul dat se va returna 0. Este important de remarcat că aceasta nu e o convenție de codificare a unei situații neobișnuite ci doar o cerință pe care trebuie să o implementeze metoda. Codul metodei e prezentat mai jos.

```
class Utilitar {  
  
    public static double medie(double x, SirNumereReale sir) {  
        double medie = 0;  
        for(int i = 0; i < 10; i++) {  
            double tmp = sir.extrageDupa(x);  
            if(!sir.testExceptie()) {  
                medie+= tmp;  
            } else {  
                medie = 0;  
                break;  
            }  
        }  
        return medie / 10;  
    }  
}
```

La prima vedere, acest cod este corect. Din păcate nu este așa și codul totuși e compilabil! Ce se întâmplă dacă parametrul x nu există în șirul dat? Evident, undeva în sistem această problemă va fi sesizată sub forma unei erori de programare. Unde? Depinde. Poate în apelantul metodei *medie*, poate în apelantul apelantului metodei *medie*, poate la un milion de apeluri distanță. Cine este de vină? Apelantul metodei *medie* va da vina pe cel care a implementat metoda *medie* deoarece el trebuia să returneze 0 doar dacă nu existau 10 numere în șir după parametrul x . Are dreptate. Programatorul ce a implementat metoda *medie* dă vina pe specificațiile primite care nu spuneau nimic de faptul că parametrul x ar putea să nu apară în șir. Are dreptate. Managerul de proiect dă vina însă tot pe el pentru că nu a sesizat această inconsistență și nu a anunțat-o. Programatorul replică prin faptul că a considerat că metoda se apelează totdeauna cu un șir ce-l conține pe x . După îndelungi vociferări, în urma cărora tot se va găsi un vinovat, codul va fi modificat ca mai jos.

```
class Utilitar {  
  
    private static notFound = false;  
  
    public static boolean elementLipsa() {
```

```
        return notFound;
    }

    public static double medie(double x, SirNumereReale sir) {
        double medie = sir.extrageDupa(x);
        notFound = false;
        if(sir.testExceptie()) {
            if(medie == -1) {
                notFound = true;
                return 0; //desi nu inseamna nimic
            } else {
                notFound = false;
                return 0;
            }
        }
        for(int i = 1; i < 10; i++) {
            double tmp = sir.extrageDupa(x);
            if(!sir.testExceptie()) {
                medie+= tmp;
            } else {
                medie = 0;
                break;
            }
        }
        return medie / 10;
    }
}
```

După cum se poate vedea din exemplul de mai sus, tratarea clasică a excepțiilor ridică o serie de probleme. Pe de-o parte codul poate deveni destul de greu de urmărit datorită nenumăratelor teste care trebuie realizate și care se întretes în codul esențial. Cea mai mare parte din codul metodei *medie* testează valoarea fanioanelor pentru a detecta situații neobișnuite și pentru a le trata corespunzător. Aceste teste se întrepătrund cu ceea ce face metoda de fapt: ia 10 numere din șir ce apar după parametrul *x* și le calculează media. Efectul este că metoda va fi foarte greu de înțeles pentru o persoană care vede codul prima dată. Pe de altă parte, cea mai mare problemă a modului clasic de tratare a excepțiilor este că el se bazează pe convenții: după fiecare apel al metodei *extrageDupa* trebuie verificat fanionul pentru a vedea dacă nu a apărut vreo situație neobișnuită. Din păcate convențiile se încalcă iar compilatorul nu poate să verifice respectarea lor.

Revenind la exemplul nostru, mai exact la programatorul ce a implementat metoda *medie*, el poate aduce un argument incontestabil în apărarea sa: dacă programul este scris în Java, atunci programatorul ce a implementat clasa *SirNumereReale* de ce nu a folosit mecanismul excepțiilor verificate pentru a anunța situațiile neobișnuite?

8.2 Tratarea excepțiilor în Java

8.2.1 Definirea excepțiilor

După cum am văzut deja, în Java aproape orice este văzut ca un obiect. Prin urmare nu ar trebui să ne mire faptul că o excepție nu este altceva decât un obiect care se definește aproape la fel ca orice alt obiect. Singura condiție suplimentară ce trebuie satisfăcută este să fie moștenită direct ori indirect clasa predefinită *Throwable* de către clasa obiectului nostru excepție. În practică, la definirea excepțiilor NU se extinde însă această clasă, ci se utilizează clasa *Exception*, o subclasă a clasei *Throwable*. Să vedem acum cum definim excepțiile pentru situațiile speciale din exemplul de la începutul lecției.

```
class ExceptieSirPlin extends Exception {

    public ExceptieSirPlin(String mesaj) {
        super(mesaj);
    }
}

class ExceptieNumarAbsent extends Exception {

    private double nr;

    public ExceptieNumarAbsent(double nr) {
        super(''Numarul '' + nr + '' nu apare in sir!'');
        this.nr = nr;
    }

    public int getNumarAbsent() {
        return nr;
    }
}

class ExceptieNuExistaNumere extends Exception {

    public ExceptieNuExistaNumere() {
        super(''Nu mai exista numere dupa numarul dat!'');
    }
}
```

8.2.2 Clauza throws

Încă din prima lecție am spus că obiectele ce compun un sistem software interacționează prin apeluri de metode. Din perspectiva unui obiect care apelează o metodă a unui alt obiect, la revenirea din respectivul apel putem fi în două situații: metoda s-a executat în

mod obișnuit sau metoda s-a terminat în mod neobișnuit datorită unei excepții. Clauza *throws* apare în antetul unei metode și ne spune ce tipuri de excepții pot conduce la **terminarea** neobișnuită a respectivei metode. Mai jos prezentăm modul în care specificăm faptul că metodele clasei *SirNumereReale* pot să se termine datorită unor situații neobișnuite și care sunt acele situații.

```
class SirNumereReale {  
  
    //Implementarea clasei nu conteaza pentru client.  
    //El cunoaste doar interfata obiectului data de metodele  
    //publice ale clasei.  
  
    public void adauga(double x) throws ExceptieSirPlin {  
        ...  
    }  
  
    public double extrageDupa(double x)  
        throws ExceptieNumarAbsent, ExceptieNuExistaNumere {  
        ...  
    }  
}
```

Clauza *throws* poate fi văzută ca o specificare suplimentară a tipului returnat de o metodă. De exemplu, metoda *extrageDupa* returnează în mod obișnuit o valoare *double* reprezentând un număr din sir. Clauza *throws* spune că în situații excepționale, metoda poate returna o referință la un obiect *ExceptieNumarAbsent*, o referință la un obiect *ExceptieNuExistaNumere* sau o referință la un obiect a cărui clasă (tip) moștenește clasele (tipurile) *ExceptieNumarAbsent* sau *ExceptieNuExistaNumere*.

Atenție

Faptul că în clauza *throws* a unei metode apare un tip A înseamnă că respectiva metodă se poate termina fie cu o excepție de tip A, fie cu orice excepție a cărei clasă moștenește direct ori indirect clasa A.

8.2.3 Interceptarea excepțiilor

Până acum am vorbit despre modul de definire a unei excepții și de modul de specificare a faptului că o metodă poate să se termine cu excepție. Dar cum poate afla un obiect ce apelează o metodă ce se poate termina cu excepție dacă metoda s-a terminat normal sau nu? Răspunsul e simplu: dacă îl interesează acest lucru trebuie să utilizeze un bloc *try-catch-finally*. Structura generală a unui astfel de bloc este prezentată mai jos.

```
try {  
    //Secventa de instructiuni ce trateaza situatia normala  
    //de executie, dar in care ar putea apare exceptii
```

```
} catch(TipExceptie1 e) {  
    //Secventa de instructiuni ce se executa cand in sectiunea  
    //try apare o exceptie de tip TipExceptie1 sau de un subtip de-al sau  
    //Parametrul e o referinta la exceptia prinsa  
} catch(TipExceptie2 e) {  
    //Secventa de instructiuni ce se executa cand in sectiunea  
    //try apare o exceptie de tip TipExceptie2 sau de un subtip de-al sau  
    //Parametrul e o referinta la exceptia prinsa  
}  
//... Pot apare 0 sau mai multe sectiuni catch  
} catch(TipExceptieN e) {  
    //Secventa de instructiuni ce se executa cand in sectiunea  
    //try apare o exceptie de tip TipExceptieN sau de un subtip de-al sau  
    //Parametrul e o referinta la exceptia prinsa  
} finally {  
    //Secventa de instructiuni ce se executa in orice conditii la  
    //terminarea executiei celorlalte sectiuni  
    //Sectiunea finally e optionala si e numai una  
}  
//aici urmeaza alte instructiuni (*)
```

Din punct de vedere static, secțiunile blocului *try-catch-finally* au următoarea utilizare. În secțiunea *try* se introduce codul pe parcursul căruia pot apare excepții. În fiecare secțiune *catch* se amplasează secvența de instrucțiuni ce trebuie să se execute în momentul în care în secțiunea *try* a apărut o excepție de tipul parametrului secțiunii *catch* sau de un subtip de-al său. În secțiunea *finally* se amplasează cod ce trebuie neapărat să se execute înaintea părăsirii blocului *try-catch-finally*, indiferent de faptul că în secțiunea *try* a apărut sau nu vreo excepție și indiferent dacă ea a fost interceptată ori nu de vreo secțiune *catch*.

Cum funcționează la execuție un bloc *try-catch-finally*? În situația în care nu apare nici o excepție în secțiunea *try* lucrurile sunt simple: se execută secțiunea *try* ca orice alt bloc de instrucțiuni, apoi se execută secțiunea *finally* dacă ea există și apoi se continuă cu prima instrucțiune de după blocul *try-catch-finally*. Prin urmare secțiunile *catch* NU se execută dacă nu apare vreo excepție în secțiunea *try*.

În continuare, pentru a înțelege cum funcționează un bloc *try-catch-finally* atunci când apar excepții, trebuie să înțelegem ce se întâmplă la execuție în momentul apariției unei excepții. În primul rând trebuie să spunem că la execuția programului, dacă se emite o excepție, execuția “normală” a programului e întreruptă până în momentul în care excepția respectivă este tratată (interceptată). Prinderea unei excepții înseamnă executarea unei secțiuni *catch* dintr-un bloc *try-catch-finally*, secțiune *catch* asociată acelei excepții. Din momentul emiterii și până în momentul prinderii se aplică în mod repetat un algoritm pe care-l vom prezenta în continuare. Algoritmul se aplică mai

întâi pentru metoda în care a apărut excepția. Dacă în urma aplicării algoritmului se constată că excepția nu e tratată în acea metodă, algoritmul se repetă pentru metoda ce a apelat metoda curentă. Acest lucru se tot repetă până se ajunge la o metodă ce prinde excepția. Dacă mergând înapoi pe stiva de apeluri se ajunge până în metoda *main* și nici aici nu este prinsă excepția, mașina virtuală Java va afișa pe ecran mesajul excepției, șirul de metode prin care s-a trecut, după care va opri programul.

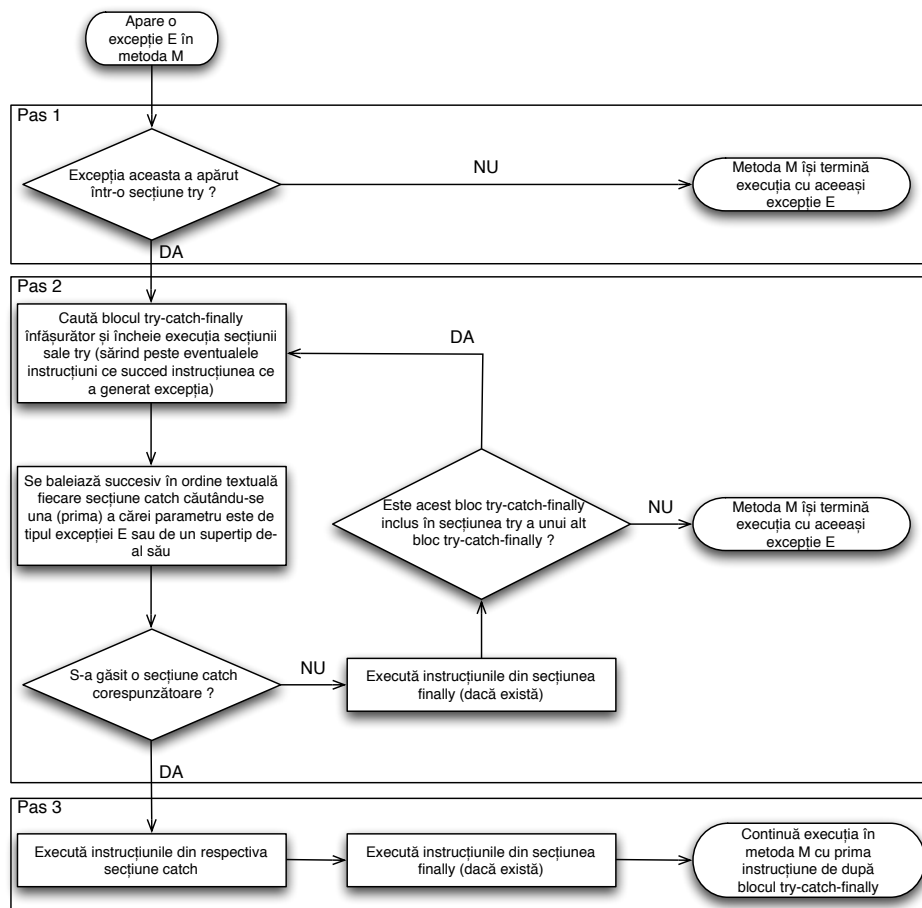


Figura 8.1: CUM SE DECIDE DACĂ METODA M PRINDE EXCEPȚIA E SAU SE TERMINĂ ȘI EA CU ACEEAȘI EXCEPȚIE E.

Să vedem acum algoritmul despre care am vorbit mai sus. El se aplică unei metode în momentul în care apare una din situațiile de mai jos:

- s-a executat o instrucțiune care a emis explicit ori implicit o excepție.
- s-a executat un apel la o metodă iar ea s-a terminat cu o excepție.

În primul pas al algoritmului se verifică dacă respectiva instrucțiune/apel apare într-o secțiune *try* a unui bloc *try-catch-finally*. Dacă nu, respectiva metodă se termină automat cu aceeași excepție deoarece ea nu interceptează respectiva excepție. Altfel se trece la pasul doi.

În al doilea pas se caută cel mai apropiat bloc *try-catch-finally* (blocurile *try-catch-finally* pot fi încuibate în sensul că un bloc apare în secțiunea *try* a unui bloc înfășurător). Odată găsit, algoritmul baleiază în ordine textuală fiecare secțiune *catch* și verifică dacă tipul concret al excepției apărute este de tipul parametrului sau de un subtip de-al tipului parametrului. La găsirea primei clauze *catch* ce respectă această regulă căutarea se oprește și se trece la pasul trei. Dacă nu se găsește o astfel de secțiune *catch* se trece automat la execuția secțiunii *finally* (dacă ea există) după care se repetă pasul doi pentru un eventual bloc *try-catch-finally* a cărui secțiune *try* include blocul *try-catch-finally* curent. Dacă nu există un astfel de bloc metoda noastră se termină cu aceeași excepție ce a declanșat algoritmul deoarece ea nu o interceptează.

În al treilea pas se poate spune cu certitudine că excepția a fost interceptată (deci algoritmul nu se mai aplică pentru apelantul acestei metode) și se trece la executarea instrucțiunilor din clauza *catch* ce a prins excepția. Apoi se execută secțiunea *finally* (dacă ea există) după care se reîncepe execuția normală a programului de la prima instrucțiune de după blocul *try-catch-finally* ce a interceptat excepția.



Dacă ați citit cu atenție veți constata că algoritmul nu spune nimic despre ce se întâmplă dacă se emite o excepție într-o secțiune *catch* sau *finally*. Lucrurile sunt simple: se aplică algoritmul ca pentru orice emisie de excepție. Situația mai puțin clară este atunci când se execută o secțiune *finally* și excepția care a declanșat algoritmul nu a fost încă tratată. Într-o astfel de situație, excepția curentă se consideră tratată și se reîncepe algoritmul pentru excepția nou emisă. Atenție însă la faptul că aceste excepții nu apar în secțiunea *try* a blocului *try-catch-finally* curent!!! Adică o excepție emisă într-o secțiune *catch* sau *finally* a unui bloc *try-catch-finally* nu poate fi prinsă de o secțiune *catch* a aceluiași bloc *try-catch-finally*!



Atenție la ordinea de amplasare a clauzelor *catch*!!! Dacă ne uităm la fragmentul de cod de mai jos am putea spune că totul e bine. Nu e așa. În a doua clauză *catch* nu se ajunge niciodată! Deși în secțiunea *try* apare o excepție de tip *oExcepție*, ea va fi prinsă totdeauna de prima secțiune *catch* deoarece *oExcepție* e subtip de-al lui *Exception*. Din fericire, compilatorul sesizează astfel de probleme.

```
class oExcepție extends Exception {}
```

```
try {  
    //Aici apare o exceptie de tip oExceptie  
} catch(Exception e) {  
    ...  
} catch(oExceptie e) {  
    ...  
}
```

Să vedem acum cum va arăta codul metodei *medie* prezentate la începutul lecției când folosim excepții.

```
class Utilitar {  
  
    public static double medie(double x, SirNumereReale sir)  
        throws ExceptieNumarAbsent {  
        double medie = 0;  
        try {  
            for(int i = 0; i < 10; i++) {  
                medie+=sir.extrageDupa(x);  
            }  
            medie = medie / 10;  
        } catch(ExceptieNuExistaNumere e) {  
            medie = 0;  
        }  
        return medie;  
    }  
}
```

Să vedem ce se întâmplă la execuția acestei metode. Presupunem inițial că numărul x există în șir și există 10 numere după el. Prin urmare metoda *extrageDupa* nu se va termina cu excepție, secțiunea *try* se va executa ca orice alt bloc de instrucțiuni iar după terminarea sa se continuă cu prima instrucțiune de după blocul *try-catch-finally*; adică se va executa instrucțiunea *return* și metoda noastră se termină normal.

Să vedem acum ce se întâmplă dacă după numărul x mai există în șir doar un număr. Prin urmare, la al doilea apel al lui *extrageDupa* metoda se va termina cu o excepție de tip *ExceptieNuExistaNumere*. Aplicând algoritmul prezentat anterior se constată că excepția a apărut într-o secțiune *try*. Se baleiază apoi secțiunile *catch* și se constată că prima este destinată prinderii excepțiilor de tipul celei emise. Prin urmare se trece la execuția corpului ei și *medie* va deveni 0. Apoi se trece la execuția primei instrucțiuni de după *try-catch-finally* și se execută instrucțiunea *return*. Este important de observat că în momentul în care *extrageDupa* s-a terminat cu excepție NU SE MAI EXECUTĂ NIMIC DIN SECȚIUNEA TRY. Pur și simplu se abandonează secțiunea *try* indiferent ce instrucțiuni sunt acolo. În cazul nostru se abandonează inclusiv ciclul *for* care încă nu s-a terminat când a apărut excepția.

Să vedem acum ce se întâmplă dacă nu avem numărul x în șir. Evident, *extrageDupa* se termină cu o excepție *ExceptieNumarAbsent* la primul apel. Se aplică algoritmul prezentat mai sus și se constată că această excepție nu e tratată de blocul *try-catch-finally* înconjurător deoarece nu există o clauză *catch* care să prindă această excepție. Prin urmare, metoda *medie* se va termina cu aceeași excepție. Acesta e și motivul pentru care apare tipul acestei excepții în clauza *throws* a metodei. Dacă nu am pune această clauză compilatorul ar da o eroare de compilare. Motivul e simplu: el vede că metoda *extrageDupa* poate să se termine cu *ExceptieNumarAbsent*, dar metoda *medie* nu o interceptează (nu există o secțiune *catch* corespunzătoare). Prin urmare, compilatorul vede că metoda *medie* s-ar putea termina cu această excepție și îi spune programatorului să se hotărască: fie interceptează excepția în această metodă fie specifică explicit că metoda se poate termina cu *ExceptieNumarAbsent*.



Uitați-vă la codul de mai jos. Codul nu e compilabil! Motivul e simplu. Clauza *throws* spune compilatorului că metoda se poate termina cu o excepție de tipul *oExceptie* sau cu o excepție de un subtip de-al său. Nimic nu-i poate schimba această părere. Pe de altă parte în metoda *altaMetoda* se prind doar excepții de tip *altaExceptie*. Acest lucru e corect: pot ajunge astfel de excepții aici. Altceva e problematic pentru compilator: se prind doar anumite cazuri particulare de excepții *oExceptie*. Ce se întâmplă cu restul? Ca urmare, compilatorul va semnala o eroare ce va fi rezolvată de programator fie prin adăugarea unei secțiuni *catch* pentru excepții de tip *oExceptie* după secțiunea deja existentă (mai știți de ce obligatoriu după?) fie se utilizează o clauză *throws* pentru metoda *altaMetoda*.

```
class oExceptie extends Exception {}
class altaExceptie extends oExceptie {}

class Test {

    public static void oMetoda() throws oExceptie {...}

    public static void altaMetoda() {
        try { oMetoda();
        } catch(altaExceptie e) {...}
    }
}
```

Din acest exemplu reies clar avantajele utilizării excepțiilor verificate. Pe de-o parte codul metodei *medie* e partiționat clar: la o execuție normală se execută ce e în *try*; dacă apare excepția *ExceptieNuMaiExistaNumere* se execută instrucțiunile din secțiunea *catch* corespunzătoare. Pe de altă parte, există și un alt avantaj mai important: compilatorul ne obligă să spunem clar ce se întâmplă dacă apare vreo excepție într-o metodă: fie o tratăm în acea metodă fie anunțăm apelantul prin clauza *throws* că metoda noastră se poate termina cu excepții.

8.2.4 Emiterea explicită a excepțiilor

Să vedem acum cum anume se anunță explicit apariția unei situații neobișnuite. Mai exact, vom vedea cum anume se emite explicit o excepție. Pentru acest lucru se utilizează instrucțiunea *throw*. Forma sa generală este prezentată mai jos, unde *ExpresieDeTipExcepție* trebuie să producă o referință la un obiect excepție.

```
throw ExpresieDeTipExcepție;
```

Să vedem acum cum rescriem clasa *SirNumereReale* astfel încât ea să emită excepții la întâlnirea situațiilor neobișnuite.

```
class SirNumereReale {

    private double[] sir = new double[100];
    private int nr = 0;

    public void adauga(double x) throws ExceptieSirPlin {
        if(nr == sir.length) {
            throw new ExceptieSirPlin("'Sirul este plin!'");
        }
        sir[nr] = x;
        nr++;
    }

    public double extrageDupa(double x)
        throws ExceptieNumarAbsent, ExceptieNuExistaNumere {
        int i;
        for(i = 0; i < nr; i++) {
            if(sir[i] == x) {
                if(i + 1 < nr) {
                    double result = sir[i + 1];
                    for(int j = i + 1; j < nr - 1; j++) {
                        sir[j] = sir[j + 1];
                    }
                    nr--;
                    return result;
                } else {
                    throw new ExceptieNuExistaNumere();
                }
            }
        }
        throw new ExceptieNumarAbsent(x);
    }
}
```

8.2.5 Situații în care se pot emite implicit excepții

Multe instrucțiuni Java pot emite excepții într-o manieră implicită în sensul că nu se utilizează instrucțiunea *throw* pentru emiterea lor. Aceste excepții sunt emise de mașina virtuală Java în momentul detecției unei situații anormale la execuție.

Spre exemplu, să considerăm un tablou cu 10 intrări. În momentul în care vom încerca să accesăm un element folosind un index mai mare ca 9 (sau mai mic ca 0) mașina virtuală Java va emite o excepție de tip *IndexOutOfBoundsException*. Este important de menționat că toate excepțiile ce sunt emise de mașina virtuală se propagă și se interceptează exact ca orice alt fel de excepție definită de un programator. În exemplul de mai jos prezentăm un mod (oarecum abuziv) de inițializare a tuturor intrărilor unui tablou de întregi cu valoarea 1.

```
class ExempluUnu {  
  
    public static void main(String argv[]) {  
        int[] tab = new int[10];  
        int i = 0;  
        try {  
            while(true) {  
                tab[i++] = 1;  
            }  
        } catch(IndexOutOfBoundsException e) {  
            //Aici se ajunge in momentul in care incerc sa  
            //accesez elementul 10 al tabloului  
            System.out.println("Am initializat tot tabloul");  
        }  
    }  
}
```

Există multe alte instrucțiuni care pot emite implicit excepții. Să considerăm codul de mai jos. Metoda returnează rezultatul împărțirii primului parametru la al doilea. Toată lumea știe că împărțirea la 0 este o eroare. Dacă la un apel al metodei, *b* ia valoarea 0 atunci operația de împărțire va emite (de fapt mașina virtuală) o excepție de tip *ArithmeticException*. Este interesant de observat că aplicând algoritmul de propagare al excepțiilor prezentat anterior, metoda *divide* se va putea termina cu o excepție *ArithmeticException*. Întrebarea e de ce compilează acest cod din moment ce nu avem clauza *throws*? Răspunsul va fi dat în secțiunea următoare.

```
class ExempluDoi {
```

```
public static double divide(int a, int b) {  
    return a / b;  
}  
}
```

8.2.6 Clasificarea excepțiilor

În Java excepțiile se clasifică în excepții verificate și excepții neverificate. Dacă o metodă se poate termina cu o excepție verificată, tipul excepției respective (sau un supertip de-al său) trebuie să apară în clauza *throws* a respectivei metode. Altfel, la compilarea codului respectivei metode se va semnala o eroare. Dacă o metodă se poate termina cu o excepție neverificată, nu este obligatorie prezența tipului său (sau a unui supertip de-al său) în clauza *throws* a metodei iar codul său va fi totuși compilabil.

Excepțiile *IndexOutOfBoundsException*, *ArithmeticException* și alte excepții ce pot fi emise implicit de mașina virtuală sunt excepții neverificate. Programatorul poate și el defini și emite prin instrucțiunea *throw* excepții neverificate. Pentru a modela o astfel de excepție clasa sa trebuie să moștenească direct ori indirect clasa *RuntimeException*. Toate celelalte excepții a căror clasă nu au ca superclase directe ori indirecte clasele *RuntimeException* sau *Error* sunt excepții verificate.

8.3 Exercițiu rezolvat

Calcul număr de excepții create

Să se creeze o clasă *MyException* derivată din clasa *Exception* ce conține:

- un constructor ce are ca parametru un șir de caractere ce va fi furnizat de serviciul *getMessage()* al excepției. Serviciul *getMessage()* nu va fi suprascris.
- o metodă ce returnează de câte ori a fost instanțiată, atât ea (clasa *MyException*) cât și orice subclasă a sa. După ce ați implementat metoda precum și mecanismul de numărare, explicați datorită cărui fapt metoda returnează și câte instanțe ale subclaselor au fost create.

Creați într-o metodă *main* trei obiecte de tip *MyException* care vor fi atașate pe rând aceleiași referințe. Apelați pentru fiecare obiect creat două servicii furnizate de acesta.

Rezolvare

```
class MyException extends Exception {  
  
    private static int instanceNo = 0;
```

```
//acest constructor se apeleaza la fiecare instantiere a clasei,  
//precum si a eventualelor subclase  
public MyException(String message) {  
    super(message);  
    instanceNo++;  
}  
  
public static int getInstanceNo() {  
    return instanceNo;  
}  
  
public static void main(String[] argv) {  
    MyException e;  
    //serviciile ce se doresc a fi apelate sunt  
    //mostenite de la clasa Exception  
    //getMessage(), toString(), printStackTrace(), ...  
  
    //se poate apela si getInstanceNo(), dar aceasta fiind statica  
    //nu se recomanda apelul prin intermediul referintei unui obiect  
  
    e = new MyException("primul caz");  
    System.out.println(e.getMessage());  
    System.out.println(e.getInstanceNo());  
  
    e = new MyException("al doilea caz");  
    System.out.println(e.getMessage());  
    System.out.println(e.toString());  
  
    e = new MyException("al treilea caz");  
    e.printStackTrace();  
    System.out.println(e.toString());  
}  
}
```

8.4 Exerciții

1. Ce se va afișa pe ecran la execuția programului de mai jos? Explicați de ce.

```
class L1 extends Exception {  
    public String toString() {  
        return "L1";  
    }  
}
```

```
class L2 extends Exception {
    public String toString() {
        return "L2";
    }
}

class Test {

    public static void main(String argv[]) {
        try {
            int i;
            for(i = 0; i < 4; i++) {
                if(i == 0) throw new L1();
                else throw new L2();
            }
        } catch(L1 e) {
            System.out.println(e);
        } catch(L2 e) {
            System.out.println(e);
        }
    }
}
```

2. Se dă codul de mai jos. Definiți excepțiile verificate *E1* și *E2* astfel încât codul clasei *Exemplu* să fie compilabil fără a-i aduce niciun fel de modificări.

```
class Exemplu {

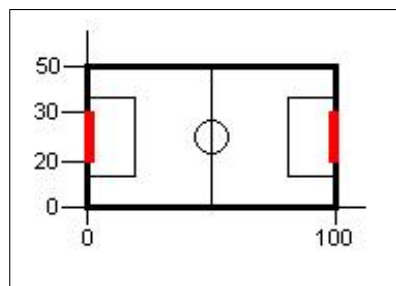
    public void doSomething(int i) {
        try {
            if(i == 0) throw new E1();
            else throw new E2();
        } catch(E1 e) {
            System.out.println("Prins");
        }
    }
}
```

3. Clasele *E1* și *E2* sunt excepții. Definiți aceste excepții în așa fel încât codul de mai jos să fie compilabil fără a-i aduce niciun fel de modificare.

```
class Exemplu {
```

```
public void doSomething(int i) {  
    if(i == 0) throw new E1();  
    else throw new E2();  
}  
}
```

4. Să se scrie un program Java care modelează un meci de fotbal simplificat. Meciul se desfășoară pe terenul de fotbal din figura de mai jos, teren ale cărui dimensiuni sunt indicate în aceeași figură.



Programul conține o clasă *Minge* care are două atribute reprezentând poziția (X,Y) curentă a mingii. Aceste coordonate se setează prin parametrii constructorului acestei clase. Clasa mai conține două metode care permit determinarea coordonatei X respectiv Y a mingii, și o metodă *suteaza()*. Această metodă generează două noi valori pentru poziția mingii și, în anumite situații, își va termina execuția cu excepții verificate. Astfel:

- dacă mingea ajunge într-o poziție (X,Y) cu proprietatea $Y=0$ sau $Y=50$, se va genera o excepție de tip *Out*.
- dacă mingea ajunge într-o poziție (X,Y) cu proprietatea $X=0$ sau $X=100$ și e gol (adică $Y \geq 20$ și $Y \leq 30$) se va genera o excepție de tip *Gol*.
- dacă mingea ajunge într-o poziție (X,Y) cu proprietatea $X=0$ sau $X=100$, dar nu e gol sau out (adică $0 < Y < 20$ sau $30 < Y < 50$), atunci se va genera o excepție de tip *Corner*.

Programul mai conține o clasă *Joc* care va avea atribute pentru numele echipelor (setate la crearea unui obiect *Joc*), pentru numărul de goluri corespunzător fiecărei echipe și pentru numărul total de out-uri și cornere pe întregul meci. Clasa mai definește o metodă ce întoarce reprezentarea sub formă de șir de caractere a unui obiect *Joc*, reprezentare ce include numele echipelor, scorul și statisticile descrise anterior. Desfășurarea propriu-zisă a jocului se realizează de metoda *simuleaza()* din cadrul aceleiași clase.

Simularea constă în crearea unei mingi inițiale urmată de efectuarea unui număr de 1000 de șuturi. Pentru fiecare poziție ocupată de minge în timpul simulării se va afișa un mesaj de forma “Nume echipa 1 - Nume echipa 2 : Mingea se află la coordonatele (X,Y)”. Tratarea situațiilor excepționale ce pot să apară constă în modificarea corespunzătoare a scorului și a statisticilor, afișarea unui mesaj corespunzător pe ecran, precum și de înlocuirea mingii curente cu una nouă, plasată după caz astfel:

- în caz de gol, se va crea o nouă minge amplasată la mijlocul terenului (X=50 și Y=25).
- în caz de out, se va crea o nouă minge plasată la aceeași poziție ca vechea minge.
- în caz de corner, noua minge se va plasa în colțul corespunzător al terenului.

Pentru exeplicare, într-o metodă main se vor crea două obiecte *Joc*, se vor simula ambele jocuri și, în final, se vor afișa pe ecran rezultatele și statisticile jocurilor.

NOTĂ

Pentru a obține rezultate interesante se pune la dispoziție clasa de mai jos care generează perechi (X,Y) reprezentând puncte de pe teren sau de pe frontierele acestuia.

```
import java.util.Random;
import java.util.Date;

class CoordinateGenerator {

    private Random randomGenerator;

    public CoordinateGenerator() {
        Date now = new Date();
        long sec = now.getTime();
        randomGenerator = new Random(sec);
    }

    public int generateX() {
        int x = randomGenerator.nextInt(101);
        if(x < 5) {
            x = 0;
        } else if(x > 95) {
            x = 100;
        }
    }
}
```

```
    } else {  
        x = randomGenerator.nextInt(99) + 1;  
    }  
    return x;  
}  
  
public int generateY() {  
    int y = randomGenerator.nextInt(101);  
    if(y < 5) {  
        y = 0;  
    } else if(y > 95) {  
        y = 50;  
    } else {  
        y = randomGenerator.nextInt(49) + 1;  
    }  
    return y;  
}  
}
```

Bibliografie

1. James Gosling, Bill Joy, Guy L. Steele Jr., Gilad Bracha, *Java Language Specification*, <http://java.sun.com/docs/books/jls>, 2005.
2. Carmen De Sabata, Ciprian Chirilă, Călin Jebelean, *Laboratorul de Programare Orientată pe Obiecte*, Lucrarea 8 - Tratarea excepțiilor - Aplicații, UPT 2002, variantă electronică.

Lecția 9

Pachete

În prima lucrare s-a accentuat faptul că un program orientat pe obiecte este compus din obiecte care interacționează între ele prin apeluri de metode sau, altfel spus, prin transmitere de mesaje. După cum probabil s-a remarcat deja, aceasta e o viziune dinamică asupra unui program orientat pe obiecte, mai exact o viziune asupra a ceea ce se întâmplă la execuția programului. Din punct de vedere static programul, sau mai exact codul său sursă, este organizat sub forma unui set de clase care prezintă implementarea obiectelor. Odată ce dimensiunea unui sistem software crește, numărul de tipuri distincte de obiecte crește și deci și numărul de clase/interfețe. Astfel, se pune problema organizării sau grupării eficiente a claselor/interfețelor. În această lucrare vom prezenta anumite aspecte de principiu legate de această organizare și facilitățile puse la dispoziție de Java pentru a o pune în practică.

9.1 Modularizarea

9.1.1 Arhitectura programelor orientate pe obiecte

În general, pentru a putea înțelege și construi ușor un sistem complex de orice natură, acesta e văzut ca fiind compus fizic dintr-un ansamblu de subsisteme interconectate între ele. Într-o viziune similară, un sistem software de mari dimensiuni este văzut ca fiind compus fizic dintr-un ansamblu de subsisteme software sau *module*. Este important de reținut că modulele sunt utilizate pentru a reda *arhitectura fizică* a sistemului software, pentru a reda părțile sale constituente.



Pentru a exemplifica ce înseamnă arhitectura fizică a unui sistem să ne gândim la sistemele de calcul (PC-urile) pe care le avem noi acasă. Care sunt părțile lor constituente? Dintre modulele cuprinse fizic de un calculator putem aminti procesorul, placa de bază, placa de rețea, placa video, placa de sunet. Evident există multe astfel de module într-un calculator.

Dar ce este efectiv un modul software? Ce conține un astfel de modul? Răspunsul la această întrebare depinde din păcate de limbajul de programare la care ne raportăm. În Java, un modul poate fi implementat printr-un *pachet* care trebuie văzut ca un container în care se depune un grup de clase și interfețe. Spre deosebire de modulele care descriu arhitectura fizică a unui program, clasele și interfețele descriu *arhitectura logică* a modulelor ce le conțin și deci implicit a sistemului software.



În exemplul anterior am spus că un calculator este compus fizic din mai multe module. Un astfel de modul este, de exemplu, placa de sunet. Pe placa de sunet găsim mai multe obiecte: tranzistoare, rezistențe, condensatoare, circuite digitale, etc. Toate acestea interacționează între ele urmând o anumită logică pentru a îndeplini principalele funcții ale unei plăci de sunet: redare sunet, înregistrare sunet, comunicare cu placa de bază, etc. Într-un mod similar, clasele dintr-un modul descriu implementarea și interacțiunile obiectelor care dau naștere funcționalității sau comportamentului respectivului modul.

În acest context putem defini activitatea de modularizare. Principial modularizarea reprezintă procesul de structurare al unui sistem software în module. Este foarte important de menționat că această structurare NU se face ad-hoc, ci pe baza unor principii de modularizare. Structurarea unui program orientat pe obiecte în module este o problemă poate la fel de dificilă ca identificarea setului corespunzător de obiecte care va exista în respectivul program și depășește cu mult scopul acestei lecții. Noi ne vom rezuma aici doar la anumite aspecte elementare. Astfel, putem vorbi de modularizare doar dacă rezultatul acestei activități prezintă așa numita proprietate de modularitate.

9.1.2 Proprietatea de modularitate

Proprietatea de modularitate este un concept de bază în domeniul tehnologiei orientate pe obiecte și nu numai.

Definiție 2 *Modularitatea este proprietatea unui sistem care a fost descompus într-un set de module coezive și slab cuplate.*

În definiția de mai sus apar doi termeni foarte importanți în contextul orientării pe obiecte a programelor: *cuplajul* și *coeziunea*. Spunem că două module sunt cuplate atunci când ele sunt “conectate” între ele. Două module sunt slab cuplate atunci când gradul lor de interconectare este redus sau altfel spus, atunci când modificări aduse unui modul nu implică modificarea celuilalt. Cât privește cea de-a doua noțiune, spunem că un modul e coeziv dacă elementele conținute de el, în cazul nostru clasele, au puternice legături între ele care să justifice gruparea lor la un loc.



De multe ori ați auzit spunându-se că sistemele de calcul pe care le avem noi acasă (PC-urile) sunt modulare. De ce sunt ele modulare? Pentru că ele sunt fizic compuse dintr-un set de module coezive și slab cuplate.

Dintre aceste module amintim placa de baza, procesorul, placa video, placa de rețea, etc. Faptul că un modul este coeziv înseamnă că absolut tot ceea ce se găsește în el este utilizat pentru îndeplinirea funcțiilor specifice modulului. De exemplu, toate componentele electronice de pe o placă video sunt destinate îndeplinirii funcțiilor de afișare pe ecranul monitorului. Faptul că două module sunt cuplate înseamnă că îndeplinirea funcțiilor proprii de către un modul depinde de utilizarea celui alt. De exemplu, placa video și placa de rețea nu sunt cuplate pentru că fiecare poate să-și îndeplinească funcțiile independent una de alta. Pe de altă parte, placa video și placa de bază sunt cuplate pentru că nici una nu poate funcționa corect fără cealaltă. Totuși este important de menționat că placa video și placa de bază sunt slab cuplate, deoarece este ușor să înlocuim un model de placă video cu un alt model ceva mai performant fără a trebui să schimbăm și placa de bază.

9.1.3 Scopul modularizării

În dicuția de până acum am prezentat conceptul de modularizare. Dar care este scopul? De ce vrem noi să modularizăm programele în general sau programele orientate pe obiecte în particular?



Imaginați-vă ce s-ar întâmpla de exemplu dacă pentru a testa o singură clasă într-un sistem ar trebui să compilăm întregul său cod, chiar și acele părți care nu au logic nici o legătură cu clasa noastră. Ar fi cam ineficient mai ales dacă compilarea durează câteva ore. Mai rău, ar fi foarte neplăcut ca pentru a testa aceeași clasă ar trebui să înțelegem o mare parte din restul sistemului. Aceasta ar necesita discuții cu alte echipe de programatori care s-au ocupat de implementarea diferitelor alte părți din sistem lucru care ar putea fi foarte ineficient mai ales dacă echipele sunt pe continente diferite (și nici deplasarea până în căminele vecine nu e prea plăcută dacă are loc de 50 de ori pe zi).

Ei bine, scopul general este unul simplu: reducerea complexității sistemului și implicit a costurilor de construcție permițând modulelor să fie proiectate, implementate și revizuite în mod independent. Această independență poate permite modulelor să fie compilate și testate individual. Prin urmare, este posibil ca diferite module să fie construite de echipe diferite de programatori care să lucreze independent. Mai mult, o echipă va lucra mult mai eficient deoarece ea poate să se concentreze exclusiv asupra funcționalității oferite de modulul de care se ocupă fără a fi nevoită să înțeleagă sistemul în ansamblul său.



Calculatorul de acasă a fost gândit să fie modular. Astfel, placa de bază poate fi construită separat, placa de rețea separat, placa video separat, etc. Ce s-ar întâmpla dacă pentru a construi o placă de rețea ar trebui (contraintuitiv) să se cunoască în detaliu cum funcționează placa video? Probabil nu ar fi prea mulți producători de plăci de rețea și acestea ar fi cam scumpe.

9.1.4 De la vorbe la fapte

Din discuția de până acum reiese importanța modularizării. Dar cum atingem acest scop? După cum am mai spus, o discuție despre acest lucru depășește scopul acestei lecții. Totuși, pentru a înțelege rolul anumitor construcții de limbaj din Java vom menționa un set restrâns de reguli de modularizare:

- implementarea unui modul trebuie să depindă numai de interfețele altor module.
- interfața unui modul trebuie să cuprindă aspecte care sunt puțin probabil a se modifica.
- implementarea unui modul trebuie ascunsă de celelalte module.



Explicarea acestor reguli se face cel mai bine printr-un exemplu. Astfel, prima regulă spune că modul în care este implementată funcționalitatea unei plăci de bază nu trebuie să depindă de modul de implementare a unei plăci video. Ea trebuie să depindă numai de interfața unei plăci video. Așa stau lucrurile într-un calculator: placa de bază a fost construită știind doar că placa video are o interfață de comunicare PCI. A doua regulă spune că interfața unei plăci video trebuie să fie stabilă. Cu alte cuvinte, modul de comunicare al plăcii video nu trebuie să se schimbe des. Efectul acestor două reguli e simplu: într-un calculator putem schimba un model de placă video cu alt model mai performant fără a schimba placa de bază! Evident, dacă interfața plăcii video se schimbă, de exemplu e o placă cu interfață AGP, trebuie să schimbăm și placa de bază (presupunând că ea nu are o magistrală AGP). Ultima regulă este un efect al aplicării principiului ascunderii informației la nivelul modulelor. Dacă un modul depinde sau este interesat doar de interfața modulelor cu care comunică înseamnă că el nu e interesat de implementarea lor. Ca urmare, aplicând principiul ascunderii informației la nivelul modulelor, trebuie să ascundem implementarea modulelor de orice alt modul.

9.2 Modularizarea programelor Java

Limbajul Java oferă suport pentru modularizarea programelor prin noțiunea de *pachet*. Din punct de vedere fizic, un program Java este compus dintr-o colecție de pachete care la rândul lor sunt compuse fizic dintr-un număr de *unități de compilare*. Într-un sistem de calcul obișnuit, unui pachet îi corespunde un director, în timp ce unei unități

de compilare îi corespunde un fișier cod sursă. Este bine ca toate fișierele ce aparțin unui pachet să fie amplasate în directorul ce corespunde respectivului pachet.

Important

Directorul părinte al directorului ce corespunde unui pachet va fi numit pe parcursul acestei lecții *directorul de lucru* sau directorul *src*.

Toate clasele și interfețele declarate în unitățile de compilare care aparțin unui pachet sunt membri ai respectivului pachet. În continuare vom discuta câteva aspecte legate de lucrul cu clase și interfețe în contextul pachetelor.

9.2.1 Declararea pachetelor

După cum am spus înainte, un pachet este fizic compus din mai multe fișiere sursă, acestea din urmă conținând declarațiile claselor și interfețelor membre ale pachetului respectiv. Prin urmare, este logic să putem specifica apartenența unui fișier la un pachet. Acest lucru se realizează prin clauza *package* care poate fi plasată *numai pe prima linie* a fișierului.

```
//Acest cod este continut de un singur fisier, sa spunem fisier1.java
//si trebuie plasat intr-un director numePachet, mai exact src/numePachet
package numePachet;

class ClasaA {
    ...
}

class ClasaB {
    ...
}

class ClasaC {
    ...
}
```

Să considerăm exemplul de mai sus. Clauza *package* de la începutul fișierului indică faptul că *fișier1.java* aparține pachetului *numePachet* și prin urmare clasele *ClasaA*, *ClasaB* și *ClasaC* sunt membre ale acestui pachet. Modul de reprezentare UML al pachetelor și al apartenenței unei clase la un pachet este exemplificat în Figurile 9.1 și 9.2

Important

Clauza *package* poate apare numai pe prima linie dintr-un fișier. Prin urmare este clar că un fișier sursă NU poate conține clase ce sunt membre ale unor pachete diferite. Pe de altă parte, clasele declarate în fișiere diferite pot fi membre ale aceluiași pachet dacă respectivele fișiere au pe prima linie o clauză *package* cu nume de pachet identic.

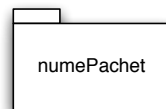


Figura 9.1: REPREZENTAREA UML A PACHETELOR.

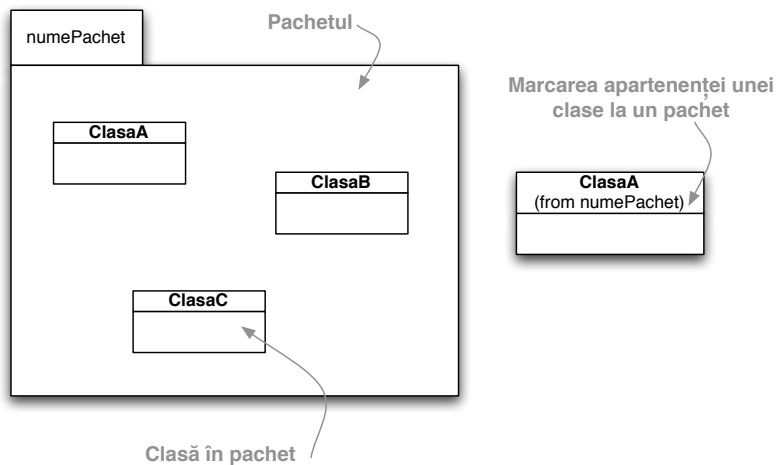


Figura 9.2: MODALITĂȚI DE REPREZENTARE A CLASELOR DIN PACHETE.



Până acum am scris programe Java fără să utilizăm clauza *package*. Cărui pachet aparțineau clasele declarate? Dacă nu se utilizează clauza *package* într-un fișier, se consideră implicit că toate declarațiile conținute de fișier aparțin unui pachet ce nu are nume sau care e anonim. Fizic, el corespunde directorului de lucru pe care l-am mai numit `src`.

9.2.2 Subpachetele. Numele complet al unui pachet

Java permite ca un pachet să fie fizic descompus în mai multe pachete (sau subpachete), subpachetele în alte pachete (sau subsubpachete), etc. Datorită acestui fapt putem avea două pachete cu același nume conținute în pachete diferite. Prin urmare, pentru a identifica complet un pachet, nu este suficient să utilizăm numele său ci numele său complet. Acesta e format din numele “mic” al pachetului și *numele complet* al pachetului ce-l conține.

```
package numePachet.numeSubPachet.numeSubSubPachet;
```

Ca și exemplu, plasând clauza *package* de mai sus într-un fișier, toate clasele și interfețele respectivului fișier vor fi membre ale pachetului *numeSubSubPachet* care e conținut de pachetul *numeSubPachet* care la rândul său e conținut de pachetul *numePachet* care la rândul său e conținut de pachetul anonim.



Ce director îi va corespunde pachetului *numeSubSubPachet*? Dacă pachetului *numePachet* îi corespunde un director *src/numePachet* e natural ca pachetului *numeSubPachet* să-i corespundă un director *src/numePachet/numeSubPachet*. Urmând aceeași logică, pentru pachetul *numeSubSubPachet* vom avea un director *src/numePachet/numeSubPachet/numeSubSubPachet*.

9.2.3 Numele complet al claselor

În programele scrise până acum nu am făcut uz de facilitățile de modularizare puse la dispoziție de Java. Mai exact, toate clasele erau membre ale pachetului anonim. Prin urmare, o clasă era complet identificată prin numele său. În contextul pachetelor, o clasă nu mai poate fi complet identificată doar prin numele său deoarece două clase pot avea același nume dacă sunt membre ale unor pachete diferite. Ca și în cazul subpachetelor, o clasă e complet identificată prin numele ei *complet* care e format din numele ei “mic” și numele complet al pachetului ce o conține.

```
//Tot exemplul este continut intr-un singur fisier, sa zicem fisier2.java
//si trebuie plasat in directorul src/numePachetulNostru/numePachetulMeu
package numePachetulNostru.numePachetulMeu;

class ClasaD {

    public static void main(String argv[]) {
        ...
    }
}
```

Ca și exemplu, clasa de mai sus este complet identificată prin următoarea succesiune de identificatori separați prin punct.

```
numePachetulNostru.numePachetulMeu.ClasaD
```

Atenție

Să presupunem că dorim să lansăm în execuție programul care începe în metoda *main* din exemplul de mai sus. Cum procedăm? Evident, folosind

numele complet al clasei *ClasaD*. Mai exact vom utiliza comanda *java numePachetul-Nostru.numePachetulMeu.ClasaD* aflându-ne în directorul de lucru (src).

9.2.4 Vizibilitatea conținutului pachetelor

Așa cum am amintit în discuția de la începutul acestei lecții, implementarea unui pachet trebuie ascunsă de alte pachete și doar interfața unui pachet trebuie să fie vizibilă altor pachete. Cu alte cuvinte, un pachet trebuie să permită accesarea din exteriorul lui doar a claselor și interfețelor ce țin de interfața pachetului, în timp ce clasele și interfețele ce țin de implementarea lui trebuie ascunse. Java permite specificarea apartenenței unei clase/interfețe la interfața unui pachet prin utilizarea specificatorului de acces *public* înaintea declarației respectivei clase.

```
//Tot exemplul este continut intr-un singur fisier,  
//ClasaDinInterfataPachetului.java  
//care trebuie plasat in directorul src/unPachet  
package unPachet;  
  
public class ClasaDinInterfataPachetului {  
    ...  
}
```

Utilizând *public* în exemplul de mai sus specificăm faptul că respectiva clasă aparține interfeței pachetului și poate fi accesată sau utilizată în exteriorul pachetului *unPachet*. Dacă specificatorul *public* ar fi lipsit, clasa de mai sus ar fi fost considerată ca ținând de implementarea pachetului și nu ar fi fost accesibilă din afara pachetului *unPachet*.



În programele de până acum nu am folosit niciodată specificatorul *public* înaintea claselor pe care le-am declarat. De ce puteam totuși să le utilizăm? Foarte simplu. Pentru că noi nu am utilizat nici clauza *package* pentru a le distribui în pachete. Ca urmare, toate clasele făceau practic parte din același pachet (pachetul anonim), iar în interiorul unui pachet putem accesa sau utiliza orice clasă sau interfață declarată în acel pachet.

9.2.5 Accesarea conținutului pachetelor

Până acum am văzut cum implementăm efectiv gruparea claselor și a interfețelor în pachete și regulile de vizibilitate a claselor și interfețelor la nivelul pachetelor. Dar cum accesăm efectiv aceste entități? Evident acest lucru depinde de “locul” din care dorim să le accesăm. În principiu, clasele dintr-un pachet pot fi accesate din același pachet prin numele lor “mic” iar din afara pachetului “doar” prin numele lor complet (evident ele trebuie să fie accesibile sau, altfel spus, declarate publice).

```
//Tot exemplul este continut intr-un singur fisier, A1.java
//care trebuie plasat in directorul src/pachet1
package pachet1;

public class A1 {
    ...
}
```

```
//Tot exemplul este continut intr-un singur fisier, sa zicem fisier1.java
//care trebuie plasat in directorul src/pachet1
package pachet1;

class B1 {

    public void metodaB1() {
        A1 ob = new A1();
        //Se acceseaza clasa A1 membra a pachetului pachet1
        //Accesul se face prin numele ''mic''
    }
}
```

```
//Tot exemplul este continut intr-un singur fisier, sa zicem fisier2.java
//care trebuie plasat in directorul src/pachet2
package pachet2;

class A2 {

    public void metodaA2() {
        pachet1.A1 ob = new pachet1.A1();
        //Se acceseaza clasa A1 membra a pachetului pachet2
        //Accesul se face prin numele complet
    }
}

class B2 extends pachet1.A1 {
    ...
}
```

Important

Dacă am accesa clasa A1 în *fisier2.java* utilizând doar numele ei “mic”, compilatorul ar semna o eroare.

Totuși, necesitatea utilizării numelui complet al unei clase/interfețe publice pentru a o accesa dintr-un alt pachet este destul de incomod. Această “deficiență” poate fi eliminată

prin utilizarea clauzelor *import*. Ele pot apare numai la începutul unui fișier sursă și pot fi precedate doar de o eventuală clauză *package*.

Astfel, în cadrul celui de-al treilea fișier din exemplul de mai sus, putem să accesăm clasa *A1* din pachetul *pachet1* utilizând numai numele clasei (adică *A1*) dacă introducem imediat după clauza *package* o clauză *import* de forma:

```
//Aceasta forma a clauzei import se mai numeste single type import
import pachet1.A1;
```

Mai mult, clauza *import* ar putea avea și forma:

```
//Aceasta forma a clauzei import se mai numeste type import on demand
import pachet1.*;
```

Dacă folosim a doua formă a clauzei *import*, putem accesa în același fișier orice clasă (sau interfață) publică din pachetul *pachet1*, folosind doar numele lor “mic”. Utilizând clauze *import*, conținutul celui de-al treilea fișier va putea avea forma de mai jos, fără să apară nici o eroare de compilare.

```
//Tot exemplul este continut intr-un singur fisier, fisier2.java
//care trebuie plasat in directorul src/pachet2
package pachet2;
import pachet1.*;

class A2 {

    public void metodaA2() {
        A1 ob = new A1();
    }
}

class B2 extends A1 {
    ...
}
```



Toate clasele predefinite din Java sunt distribuite în pachete. În particular, clasele *String*, *Math*, clasele înfășurătoare, etc. pe care le-am utilizat deja aparțin unui pachet denumit *java.lang*. Totuși, noi nu am utilizat clauze *import* pentru a putea accesa aceste clase din alte pachete (în particular, din pachetul anonim cu care am lucrat până acum). Ei bine, pachetul *java.lang* este tratat într-un mod special de compilatorul Java, considerându-se implicit că fiecare fișier sursă Java conține o clauză *import java.lang.**;

Atenție

Atenție la eventuale conflicte de nume! Astfel, putem avea două pachete diferite care conțin câte o clasă cu același nume. Pentru detalii suplimentare legate de utilizarea clauzei `import` în astfel de situații consultați specificațiile limbajului Java.



În exemplul de mai sus *pachet2* utilizează clase grupate logic în *pachet1*. Se spune că *pachet2* depinde de *pachet1*. Figura 9.3 exemplifică modul de reprezentare UML a relației de dependență între două pachete.

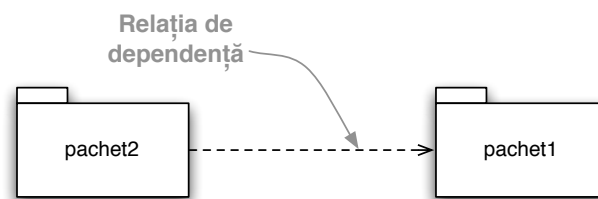


Figura 9.3: REPREZENTAREA UML A DEPENDENȚEI ÎNTRE PACHETE.

9.2.6 Vizibilitatea membrilor claselor

În lecțiile anterioare am văzut că drepturile de access la membrii unui clase pot fi specificați explicit prin specificatori de access. Tot atunci am discutat regulile de vizibilitate date de specificatorii *private*, *public* și *protected*. În continuare vom extinde aceste reguli în contextul pachetelor.

Atunci când nu se specifică explicit specificatorul de access la membrii unei clase (câmpuri sau metode) se consideră implicit că accesul la respectivul membru este de tip *package*. Aceasta înseamnă că respectivul membru este accesibil de oriunde din interiorul pachetului ce conține clasa membrului respectiv, dar numai din interiorul respectivului pachet. Este important de menționat că NU există un specificator de access *package*. Dacă dorim ca accesul la membrul unei clase să fie de tip *package*, pur și simplu nu punem nici un specificator de access.

Pe de altă parte, în lecția legată de relația de moștenire am discutat regulile de vizibilitate date de specificatorul de acces *protected*. Acolo am spus că *în general* un membru *protected* al unei clase este “asemănător” unui membru *private* dar care este accesibil și din toate clasele ce moștenesc clasa membrului respectiv. Am accentuat “în general” pentru că acesta este comportamentul teoretic al specificatorului *protected*. În plus față de această regulă, în Java un membru declarat *protected* este accesibil de oriunde din interiorul pachetului ce conține clasa membrului respectiv.

Sfat

Încercați să evitați această particularitate a limbajului Java.

Prin urmare, regulile de acces ale unei clase B la membrii unei clase A se extind în modul următor:

- Dacă clasele A și B sunt conținute în același pachet atunci B are acces la toți membrii clasei A ce nu sunt declarați *private*.
- Dacă clasele A și B sunt conținute de pachete distincte și B nu moștenește A atunci B are acces la toți membrii clasei A ce sunt declarați *public*.
- Dacă clasele A și B sunt conținute de pachete distincte și B moștenește A atunci B are acces la toți membrii clasei A ce sunt declarați *public* sau *protected*.

Atenție

În cazul în care clasa A nu este declarată ca aparținând interfeței pachetului ce o conține, atunci ea nu este accesibilă din afara respectivului pachet și implicit nici un membru al respectivei clase nu poate fi accesat din exteriorul pachetului în care ea se află.

Aceste reguli de vizibilitate sunt exemplificate în porțiunile de cod de mai jos.

```
//Tot exemplul este continut intr-un singur fisier, A1.java
//care trebuie plasat in directorul src/pachet1
package pachet1;

public class A1 {

    private int x;
    public int y;
    protected int z;
    int t;
}
```

```
//Tot exemplul este continut intr-un singur fisier, fisier1.java
//care trebuie plasat in directorul src/pachet1
package pachet1;

class B1 {
```

```
public void metodaB1() {
    A1 ob = new A1();
    ob.x = 1; //Eroare
    ob.y = 1; //Corect
    ob.z = 1; //Corect
    ob.t = 1; //Corect
}
}
```

```
//Tot exemplul este continut intr-un singur fisier, fisier2.java
//care trebuie plasat in directorul src/pachet2
package pachet2;

class A2 {

    public void metodaA2() {
        pachet1.A1 ob = new pachet1.A1();
        ob.x = 1; //Eroare
        ob.y = 1; //Corect
        ob.z = 1; //Eroare
        ob.t = 1; //Eroare
    }
}

class B2 extends pachet1.A1 {

    public B2() {
        x = 1; //Eroare
        y = 1; //Corect
        z = 1; //Corect
        t = 1; //Eroare
    }

    public void metodaB2() {
        pachet1.A1 ob = new pachet1.A1();
        ob.x = 1; //Eroare
        ob.y = 1; //Corect
        ob.z = 1; //Eroare (Da! E eroare!)
        ob.t = 1; //Eroare
    }
}
```

Atenție

Din punctul de vedere al regulilor de vizibilitate, un pachet și un subpachet al său se comportă ca și cum ar fi două pachete oarecare!



Poate vă întrebați de ce în exemplul de mai sus nu e posibil să accesăm câmpul *protected z* din clasa *pachet1.A1* în metoda *metodaB2*. Doar *B2* extinde clasa *pachet1.A1*. Aceasta pentru că, din punct de vedere pur teoretic, un membru *protected* poate fi accesat din clasa sa și din orice subclasă a clasei sale dar în al doilea caz doar pentru instanța *this*. Dacă am pune toate clasele de mai sus în același pachet accesul ar fi posibil. Aceasta pentru că specificatorul *protected* din Java diferă de specificatorul *protected* din teorie permițând accesul, de oriunde din interiorul pachetului ce conține clasa respectivului membru. În cazul nostru, membrul *z* e *protected* și ar putea fi accesat din orice clasă ce aparține aceluiași pachet ca și clasa sa (repetăm: dacă am pune toate clasele în același pachet).



În UML, vizibilitatea membrilor ce au politică de access de tip package se marchează cu simbolul `~`. Figura 9.4 exemplifică modul de reprezentare a clasei *A1* din exemplul anterior.

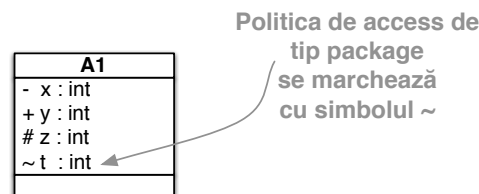


Figura 9.4: VIZIBILITATEA DE TIP PACKAGE ÎN UML.

9.2.7 Gestionarea fișierelor .java și .class

Până acum am discutat modul în care realizăm distribuirea claselor și a interfețelor pe care le declarăm într-o aplicație în diferite pachete. Dar cum repartizăm declarațiile claselor și interfețelor pe fișiere sursă? La prima vedere am fi tentați să spunem că din moment ce repartizarea pe pachete se face la nivel de fișier sursă, am putea declara toate clasele și interfețele unui pachet în același fișier sursă. Ei bine acest lucru nu e posibil totdeauna. Motivul e simplu: orice clasă sau interfață publică dintr-un pachet trebuie declarată într-un fișier al cărui nume e format din numele clasei/interfeței urmat de terminația *.java*.

Important

Nerespectarea acestei reguli duce la o eroare de compilare. Acesta e motivul pentru care, în exemplele de până acum, toate clasele publice erau declarate într-un fișier a cărui nume respecta regula anterioară.

Atenție

Într-un fișier putem declara mai multe clase și interfețe, dar datorită regulii de mai sus într-un fișier putem avea maxim o clasă sau o interfață

publică. Restul claselor/interfețelor nu vor putea fi publice și implicit nu vor putea fi accesate din afara pachetului.

Pe de altă parte, odată ce compilăm un fișier cu cod sursă, vom obține câte un fișier *.class* pentru fiecare clasă și interfață din fișierul sursă compilat. În mod obligatoriu aceste fișiere *.class* trebuie amplasate într-o structură de directoare care să reflecte direct pachetul din care fac ele parte. Să considerăm un exemplu.

```
//Tot exemplul este continut intr-un singur fisier A1.java
package pachet1;

public class A1 {
    ...
}

class B1 {
    ...
}
```

```
//Tot exemplul este continut intr-un singur fisier cu nume oarecare
//din moment ce nu avem declaratii publice
package pachet2;

class A2 {
    ...
}
```

```
//Tot exemplul este continut intr-un singur fisier cu nume oarecare
//din moment ce nu avem declaratii publice
package pachet2.subpachet21;

class B21 {
    ...
}
```

După ce compilăm aceste fișiere cu cod sursă, fișierele *.class* obținute trebuie repartizate după cum urmează:

- Fișierele *A1.class* și *B1.class* într-un director denumit *pachet1*.
- Fișierele *A2.class* într-un director denumit *pachet2*.
- Fișierele *B21.class* într-un director denumit *subpachet1* care este subdirector al directorului *pachet2*.

Această distribuire a fișierelor rezultate în urma compilării nu trebuie însă făcută manual. Ea este realizată automat de compilator dacă se utilizează un argument al compilatorului Java ca în exemplul de mai jos. Argumentul *-d numeDirector* îi spune compilatorului să distribuie automat fișierele *.class* generate într-o structură de directoare corespunzătoare a cărei rădăcină este directorul *numeDirector*.

```
javac -d numeDirector nume_fisier_compilat
```

Pentru a compila toată aplicația va trebui să compilăm toate fișierele sursă ce o compun. În acest scop, putem da ca argumente comenzii *javac* o listă de fișiere, listă care poate fi creată utilizând și nume generice (*.java). Pe de altă parte am putea compila toată aplicația compilând pe rând fiecare pachet. În acest caz foarte importante sunt opțiunile de compilare *-sourcepath numeDirector(e)* și *-classpath numeDirector(e)*.

Astfel, în momentul în care compilatorul găsește în pachetul compilat un acces la o clasă din afara pachetului, va trebui cumva să știe ce conține acea clasă, mai exact, are nevoie de fișierul *.class* asociat ei. Opțiunea *-classpath numeDirector(e)* îi furnizează compilatorului o listă de directoare ^{1 2} care conțin pachete (directoare) cu fișiere *.class*. Pe baza acestei liste și pe baza numelui complet al clasei referite compilatorul poate localiza fișierul *.class* necesar.

Problemele apar în momentul în care există dependențe circulare: pachetul *alpha* trebuie compilat înainte de *beta* pentru că utilizează o clasă din al doilea pachet, iar al doilea pachet trebuie compilat înainte de primul pentru că el utilizează o clasă din pachetul *alpha*. Ce e de făcut? Deși aceste dependențe circulare indică probleme serioase de modularizare (ambele pachete trebuie compilate simultan) compilatorul Java ne ajută prin opțiunea *-sourcepath numeDirectoare(e)*. Ea indică o listă de directoare ce conțin pachete (directoare) cu fișiere sursă. Astfel, dacă de exemplu compilăm tot pachetul *alpha*, în momentul în care compilatorul are nevoie de fișierul *.class* al unei clase din *beta* (care nu a fost încă compilat!), compilatorul va căuta pe baza listei *sourcepath* și pe baza numelui complet al clasei referite fișierul sursă care conține acea clasă și-l va compila și pe el.

Important

Dacă compilatorul nu poate obține de nicăieri fișierul *.class* al unei clase utilizate în pachetul compilat va genera o eroare de compilare.

¹lista poate conține și fișiere *.jar*, acestea reprezentând de fapt un director împachetat folosind utilitarul *jar*

²în Windows separatorul elementelor listei este ; iar în Linux :



Din discuția de mai sus s-ar părea că e destul de complicat să compilăm individual fiecare pachet al unei aplicații și ar fi mai bine să compilăm totul odată. Răspunsul e simplu: e complicat dar necesar mai ales când echipe diferite de programatori dezvoltă individual pachete diferite ale unei aplicații uriase. Java a introdus pachetele pentru a ajuta la dezvoltarea de aplicații mari permițând diferitelor echipe printre altele să-și compileze “individual partea” lor de aplicație.



Compilarea individuală a unui pachet e posibilă doar dacă sistemul a fost modularizat corespunzător (vezi regulile de modularizare). Dacă, de exemplu, interfața unui pachet nu e stabilă atunci e foarte probabil să apară conflicte (și la propriu și la figurat) între echipele ce dezvoltă respectivul modul și cele care dezvoltă module ce depind de respectiva interfață. Practic, ele nu vor mai fi echipe independente. La fel se întâmplă și în cazul dependențelor circulare. De aici se poate deduce o altă regulă de modularizare: nu permiteți cicluri în graful de dependență între module.

9.3 Exerciții

1. Fie codul de mai jos. Presupunem că dorim să mutăm doar clasa *B* în *pachet2* și, în continuare, *B* moștenește *A*. Arătați și explicați ce modificări trebuie efectuate pentru ca programul să fie în continuare compilabil.

```
//Continut fisier x.java
package pachet1;
class A {}
```

```
//Continut fisier y.java
package pachet1;
class B extends A {}
```

2. Să se scrie o clasă ce modelează conceptul de stivă. Elementele ce pot aparține stivei la un moment dat sunt niște figuri geometrice. Fiecare figură e caracterizată print-un punct de coordonate $O(x,y)$ și de alte elemente specifice funcție de tipul figurii. Concret, putem avea figuri de tip *Cerc* și *Pătrat*.

Un obiect de tip *Cerc* are punctul de coordonate $O(x,y)$ care reprezintă originea cercului; are un atribut pentru stocarea razei cercului; este egal cu un alt obiect *Cerc* dacă cele două obiecte au aceeași origine și razele de lungimi egale; e afișat sub forma - “Cerc:” urmat de coordonatele originii și de rază.

Un obiect de tip *Pătrat* are punctul de coordonate $O(x,y)$ care reprezintă colțul din

stânga-sus al pătratului; are punctul de coordonate $P(x,y)$ care reprezintă colțul din stânga-jos al pătratului; este egal cu un alt obiect Pătrat dacă cele două obiecte au aceeași arie; e afișat sub forma - "Patrat:" urmat de coordonatele stânga-sus și de latura pătratului.

Singura modalitate pentru setarea atributelor figurilor descrise mai sus e prin intermediul constructorilor!!!

Dimensiunea stivei este setată prin intermediul unui constructor ce primește ca parametru un *int*. Dacă parametrul primit este negativ atunci acest constructor va genera o eroare "neverificată". Clasa care modelează stiva pune la dispoziția unui client:

- (a) o metodă pentru introducerea unui element în stivă. Această metodă primește ca parametru elementul ce se dorește a fi introdus. Dacă nu mai există loc în stivă pentru introducerea unui nou element, se va genera o excepție prin care clientul unui obiect de acest tip (deci, clientul stivei) va fi informat de faptul că momentan nu mai pot fi introduse elemente. Întrucât nu vrem ca să existe în stivă mai multe figuri identice, încercarea de a introduce o figură care e deja stocată va genera o excepție ce va furniza mesajul "Elementul" + (parametrul primit) + "este deja stocat".
- (b) o metodă pentru returnarea și ștergerea ultimului element introdus în stivă. Dacă stiva este goală, acest lucru va fi semnalat tot printr-o excepție.
- (c) o metodă pentru afișarea conținutului stivei. Elementele vor fi afișate în ordinea UltimulStocat...PrimulStocat.

În vederea creării de figuri geometrice se vor defini mai multe pachete, unele pentru citire și altele pentru scriere.

- (a) pachetul *reader.input* - va conține o interfață ce va avea cel puțin două metode: una pentru citirea atributelor unui obiect de tip Figura și alta pentru citirea unei opțiuni (metoda care citește o figură va returna un obiect de tip Figura iar metoda pentru citirea unei opțiuni va returna un *int*, 1, 2, 3 sau 4).
- (b) pachetul *reader.output* - va conține o interfață cu o singură metodă ce va primi ca parametru un obiect stivă; implementările acestei metode nu vor face altceva decât să afișeze corespunzător stiva pe ecran sau într-un fișier.
- (c) pachetul *reader.input.text* - va avea o clasă care va implementa interfața din pachetul *reader.input* astfel încât citirile să se facă de la tastatură.
- (d) pachetul *reader.output.text* - va avea o clasă care va implementa interfața din pachetul *reader.output* astfel încât afișarea stivei să se facă pe ecran, în mod text.

Se va construi cel puțin un pachet care va conține clasele din prima parte a problemei. Se va scrie o metodă *main* în care se va instanția o stivă precum și două obiecte

de tipul interfețelor definite în pachetele *reader.input*, respectiv *reader.output*. Prin intermediul unui obiect ce implementează interfața din pachetul *reader.input*, se vor citi opțiuni (până la citirea opțiunii 4) și în funcție de opțiunile citite se vor apela cele trei metode ale stivei.

Bibliografie

1. Grady Booch, *Object-Oriented Analysis And Design With Applications*, Second Edition, Addison Wesley, 1997.
2. Martin Fowler. *UML Distilled, 3rd Edition*. Addison-Wesley, 2003.
3. Sun Microsystems, *Java Language Specification*. <http://java.sun.com/docs/books/jls/>, Capitolul 9, 2000.

Lecția 10

Colecții de obiecte

Presupunem că trebuie scrisă o aplicație care să gestioneze informații despre angajații unei companii dezvoltatoare de software. În companie există două feluri de angajați: contabili și programatori. În acest context se pune problema modului în care se pot stoca informațiile despre angajații companiei.

```
class Angajat {  
  
    private String nume, prenume;  
    private String departament;  
  
    public Angajat(String nume, String prenume, String departament) {  
        this.nume = nume;  
        this.prenume = prenume;  
        this.departament = departament;  
    }  
  
    public void schimbaDepartamentul(String departament) {  
        this.departament = departament;  
    }  
  
    public String toString() {  
        return "Nume:" + nume + " Departament:" + departament;  
    }  
  
    public boolean equals(Object o) {  
        return (o instanceof Angajat) && ((Angajat)o).nume.equals(nume)  
            && ((Angajat)o).prenume.equals(prenume);  
    }  
    ...  
}
```

Doi angajați se consideră identici din punct de vedere al conținutului dacă aceștia au același nume, respectiv același prenume.

```
class Contabil extends Angajat {  
    public Contabil(String nume, String prenume) {  
        super(nume, prenume, "contabilitate");  
    }  
    ...  
}  
  
class Programator extends Angajat {  
    public Programator(String nume, String prenume) {  
        super(nume, prenume, "dezvoltare sisteme");  
    }  
    ...  
}
```

10.1 Să ne amintim...tablourile

După cum am văzut într-una din lecțiile anterioare, o variantă pentru stocarea unei colecții de obiecte de același tip o reprezintă tablourile.

```
Angajat[] a1 = new Angajat[2];  
  
a1[0] = new Contabil("Popescu","Mircea");  
a1[1] = new Programator("Ionescu","Mihai");  
  
//sau  
a1 = new Angajat[] { new Contabil("Popescu","Mircea"),  
                     new Programator("Ionescu","Mihai")};
```

Spre deosebire de C sau C++ unde o metodă putea returna un pointer spre un tablou, în Java o metodă poate returna o referință la un obiect tablou, ca mai jos.

```
Angajat[] creazaAngajati() {  
    Angajat[] angajati = ...  
    ...  
    return angajati;  
}
```

În multe cazuri, după ce un tablou a fost creat, asupra sa se doresc a se efectua operații de genul căutare, tipărire, testarea egalității dintre două tablouri din punctul de vedere al conținutului stocat, etc.

Dacă încercăm să “tipărim” un obiect tablou ca mai jos, vom observa că ceea ce se va tipări va fi reprezentarea implicită a fiecărui obiect sub formă de șir de caractere, adică numele clasei instanțiate precum și codul hash al obiectului tipărit.

```
//Se va tipari LAngajat;@11b86e7
System.out.println(a1);
```

Totuși, de obicei, prin tipărirea unui tablou se înțelege tipărirea tuturor elementelor existente în cadrul tabloului. Ei bine, această operație poate fi realizată apelând metoda statică *String toString(Object[] a)* a clasei *Arrays* din pachetul *java.util*.

```
//Se va tipari
//[Nume:Popescu Departament:contabilitate,
// Nume:Ionescu Departament:dezvoltare sisteme]
System.out.println(Arrays.toString(a1));
```

Dacă în clasa *Angajat* nu am fi suprascris metoda *toString*, în loc de afișarea de mai sus, pentru fiecare obiect angajat stocat în cadrul tabloului s-ar fi afișat numele clasei *Angajat* urmat de codul hash corespunzător fiecărui obiect. Acest lucru s-ar fi întâmplat deoarece metoda *toString* din clasa *Arrays* apelează pentru fiecare obiect conținut metoda *toString* corespunzătoare.

O detaliere a unor metode existente în clasa *Arrays* se află în Tabelul 10.1¹. Referitor la prima metodă din cadrul Tabelului 10.1 este important de spus că două tablouri sunt considerate a fi egale din punct de vedere al conținutului dacă ambele tablouri conțin același număr de elemente iar elementele conținute sunt egale, mai mult, ele fiind stocate în aceeași ordine. Două obiecte referite de *o1* și *o2* sunt egale dacă (*o1==null* ? *o2==null* : *o1.equals(o2)*).



În Secțiunea 3.4.2 am spus că nu este bine să avem într-o clasă o metodă de genul *public void afiseaza()*. Pe lângă faptul că absența metodei *toString()* ar necesita pentru fiecare mediu nou de afișare (fișier, casetă de dialog) introducerea unei noi metode care să afișeze obiectul receptor pe noul mediu, aceasta ar face imposibilă și folosirea metodelor din clasa *Arrays* în scopul în care acestea au fost create.

Una dintre problemele principale existente atunci când utilizăm tablourile ca suport de stocare a colecțiilor de elemente este dimensiunea fixă a capacității de stocare. Atunci când dorim să modificăm dimensiunea unui tablou, trebuie să creem întâi un alt tablou iar apoi să copiem elementele din vechiul tablou în noul tablou. În acest scop putem folosi metoda *public static void arraycopy* a clasei *System*. Parametrii acestei metode

¹În clasa *Arrays* aceste metode sunt supraîncărcate, ele existând și pentru fiecare tip primitiv.

Prototipul	Descriere
boolean equals(Object[] a, Object[] a2)	Testează egalitatea dintre două tablouri din punct de vedere al conținutului
String toString(Object[] a)	Returnează un șir de caractere ce conține reprezentările sub formă de șiruri de caractere a tuturor obiectelor stocate în tabloul referit de <i>a</i>
void sort(Object[] a)	Sortează elementele tabloului referit de <i>a</i> . Pentru a putea fi sortate, toate elementele din tablou trebuie să fi implementat în prealabil interfața <i>Comparable</i>

Tabelul 10.1: CÂTEVA METODE STATICE DEFINITE DE CLASA ARRAYS.

sunt prezentați în detaliu la adresa

<http://java.sun.com/j2se/1.5.0/docs/api/java/lang/System.html>.

```
//numarul de angajati a crescut
Angajat[] a = new Angajat[4];
//copiem continutul vechiului tablou in noul tablou
System.arraycopy(a1,0,a,0,a1.length);
//a1 va referi noul tablou
a1 = a;
```

Până în acest moment tabloul de angajați nu s-a aflat într-o relație de agregare cu nici un alt obiect. Însă se pune problema de apartenență a colecției de angajați la un obiect. În acest caz, angajații aparțin unei companii.

```
class Companie {

    private String nume;
    private Angajat[] angajati;
    private int nrAngajati = 0;

    public Companie(String nume, int nrAngajati) {
        this.nume = nume;
        angajati = new Angajat[nrAngajati];
    }

    public void addAngajat(Angajat a) {
        if(nrAngajati<angajati.length)
```

```

        angajati[nrAngajati++]=a;
    else {
        //Marim numarul maxim de angajati al companiei
        Angajat[] ang = new Angajat[angajati.length+10];
        System.arraycopy(angajati,0,ang,0,angajati.length);
        angajati = ang;
        angajati[nrAngajati++]=a;
    }
}
...
}

```

Ce s-ar întâmpla dacă, la un moment dat, ar trebui să se creeze un concurs între angajații tuturor companiilor producătoare de software? Este evidentă necesitatea stocării unei colecții de angajați într-o altă clasă, posibil numită *Joc*.

```

class Joc {

    private Angajat[] angajati;
    private int nrParticipanti = 0;

    public Joc(int nrParticipanti) {
        angajati = new Angajat[nrParticipanti];
    }

    public void addAngajat(Angajat a) {
        if(nrParticipanti<angajati.length)
            angajati[nrParticipanti++]=a;
        else {
            //Marim numarul maxim de angajati participanti
            Angajat[] ang = new Angajat[angajati.length+10];
            System.arraycopy(angajati,0,ang,0,angajati.length);
            angajati = ang;
            angajati[nrParticipanti++]=a;
        }
    }
    ...
}

```

Se observă că între implementările celor două clase de mai sus, *Companie* respectiv *Joc*, există o secvență de *cod duplicat*. În Secțiunea 6.2 se spunea că e bine să scriem programe astfel încât acestea să nu conțină duplicare de cod. O variantă posibilă de eliminare a duplicării de cod ar fi crearea unei clase *ListaAngajati* a cărei funcționalitate să se rezume strict la stocarea unei colecții de angajați.

```
class ListaAngajati {  
  
    private Angajat[] angajati;  
    private int nrAngajati = 0;  
  
    public ListaAngajati(int nr) {  
        angajati = new Angajat[nr];  
    }  
  
    public void addAngajat(Angajat a) {  
        if(nrAngajati<angajati.length)  
            angajati[nrAngajati++]=a;  
        else {  
            //Marim numarul maxim de angajati participanti  
            Angajat[] ang = new Angajat[angajati.length+10];  
            System.arraycopy(angajati,0,ang,0,angajati.length);  
            angajati = ang;  
            angajati[nrAngajati++]=a;  
            ...  
        }  
    }  
    ...  
}
```

În acest caz un obiect de tip *Companie* va agrega, în loc de un tablou de angajați ale cărui elemente să trebuiască a fi gestionate în interiorul clasei, un obiect *ListaAngajati* care se va ocupa de gestiunea angajaților.

```
class Companie {  
    private String nume;  
    private ListaAngajati lista;  
  
    public Companie(String nume, int nrAngajati) {  
        this.nume = nume;  
        lista = new ListaAngajati(nrAngajati);  
    }  
  
    public void addAngajat(Angajat a) {  
        lista.addAngajat(a);  
    }  
    ...  
}
```



Conform cu cele precizate mai sus, de fiecare dată când e nevoie de o colecție de obiecte trebuie să creăm mai întâi o clasă care să se ocupe de gestionarea (adăugarea, ștergerea, etc.) obiectelor din interiorul colecției. Din fericire, Java pune la dispoziție un suport pentru lucrul cu colecții de obiecte, acest lucru făcând inutilă crearea în acest scop a propriilor clase.

10.2 Suportul Java pentru lucrul cu colecții de obiecte

Anterior am precizat că Java ne pune la dispoziție un suport pentru lucrul cu colecții de obiecte – în cadrul secțiunii de față se dorește prezentarea unor aspecte legate de suportul menționat.

10.2.1 O privire de ansamblu

Până acum am folosit din abundență termenul de *colecție de obiecte* dar nu am dat o definiție concretă a acestui termen. Acum e momentul să facem acest lucru.

Definiție 3 *O colecție este un grup de obiecte.*

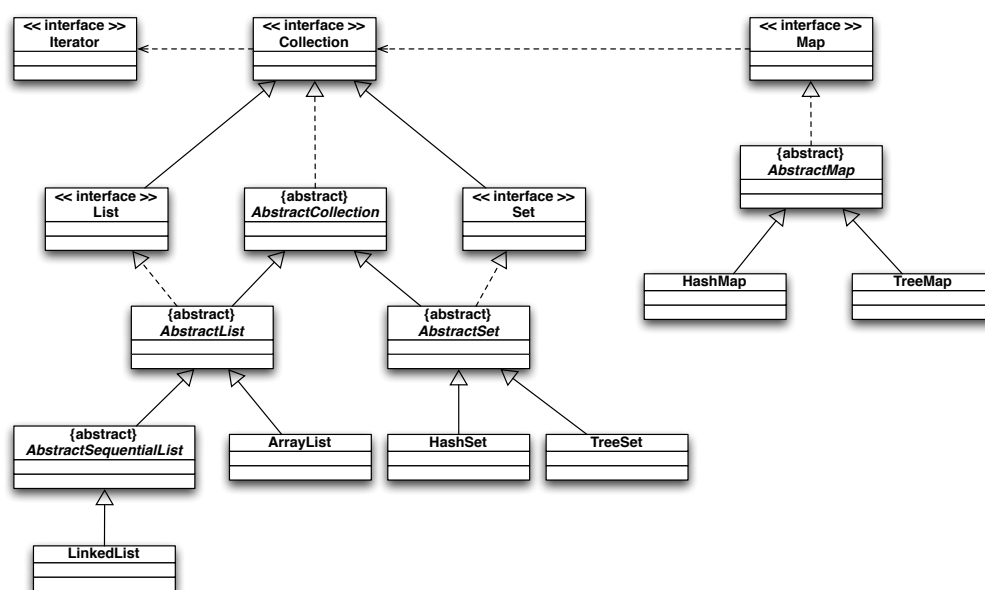


Figura 10.1: O VEDERE SIMPLIFICATĂ ASUPRA SISTEMULUI DE COLECȚII JAVA 1.4.

Clasele și interfețele pe care le oferă Java în vederea lucrului cu colecții de obiecte se află în pachetul *java.util* și, în consecință, pentru a le putea folosi ușor trebuie să folosim clauze import corespunzătoare, de exemplu ca mai jos:

```
import java.util.*;
```

În Figura 10.1 sunt reprezentate câteva interfețe și clase din acest pachet. Toate clasele concrete care au ca supertip interfața *Collection* implementează conceptul de colecție de obiecte.

Toate clasele concrete care au ca supertip interfața *List* implementează conceptul de listă. O listă este o colecție de obiecte în care fiecare obiect este reținut într-o ordine bine precizată.

Toate clasele concrete care au ca supertip interfața *Set* implementează conceptul de mulțime. O mulțime este o colecție de obiecte în care un obiect este reținut o singură dată.

Toate clasele concrete care au ca supertip interfața *Map* implementează conceptul de dicționar cheie-valoare.

10.2.2 Interfața Collection

Având în vedere că interfața *Collection* este implementată de ambele tipuri de colecții, liste și mulțimi, se impune detalierea în Tabelul 10.2 a metodelor existente în această interfață.

10.2.3 Liste

Se observă că în interiorul interfeței *Collection* nu există metode care să întoarcă un element de pe o poziție specificată. Acest lucru se datorează faptului că interfața este implementată și de mulțimi iar în interiorul mulțimilor ordinea elementelor nu este neapărat ordinea în care elementele au fost adăugate, ea neavând nici o semnificație. Interfața *List*, pe lângă metodele moștenite de la interfața *Collection*, mai are și alte metode proprii, unele dintre ele fiind prezentate în Tabelul 10.3.

Printre clasele predefinite care implementează interfața *List* sunt *ArrayList* și *LinkedList* iar în continuare vom prezenta câteva dintre caracteristicile acestor implementări.

Clasa *ArrayList* furnizează o implementare a interfeței *List*, elementele propriu-zise ale liste fiind stocate de un tablou care este redimensionat. Această implementare face ca accesul aleator al unui element să se facă foarte rapid. În schimb, ștergerea și inserarea unui element în interiorul listei este o operație consumatoare de timp în raport cu implementarea *LinkedList*.

Clasa *LinkedList* furnizează o implementare a interfeței *List*, elementele propriu-zise

Prototipul	Descriere
<code>boolean add(Object o)</code>	Asigură faptul că obiectul referit de <i>o</i> există în colecție. Returnează <code>true</code> dacă s-a modificat colecția și <code>false</code> în caz contrar. Clasele care implementează această metodă pot impune condiții legate de elementele care pot fi adăugate, spre exemplu, elementul <i>null</i> poate sau nu să fie adăugat într-o colecție
<code>boolean addAll(Collection c)</code>	Adaugă în colecție toate elementele existente în colecția referită de <i>c</i>
<code>void clear()</code>	Șterge toate elementele din colecție
<code>boolean contains(Object o)</code>	Returnează <code>true</code> dacă colecția conține cel puțin un element <i>e</i> astfel încât $(o == null ? e == null : o.equals(e))$
<code>boolean containsAll(Collection c)</code>	Returnează <code>true</code> dacă colecția conține toate elementele din colecția specificată prin <i>c</i>
<code>boolean equals(Object o)</code>	Compară colecția cu obiectul specificat
<code>int hashCode()</code>	Returnează codul hash al colecției
<code>boolean isEmpty()</code>	Returnează <code>true</code> dacă nu există nici un element în colecție
<code>Iterator iterator()</code>	Returnează un iterator care poate fi folosit pentru parcurgerea colecției
<code>boolean remove(Object o)</code>	Returnează <code>true</code> dacă s-a șters un element egal cu cel referit de <i>o</i> din colecție. Dacă colecția conține elementul <i>o</i> duplicat, se va șterge doar un singur element
<code>boolean removeAll(Collection c)</code>	Șterge toate elementele existente în colecția <i>c</i> . Returnează <code>true</code> dacă a avut loc cel puțin o ștergere
<code>boolean retainAll(Collection c)</code>	Reține în colecție doar elementele din colecția <i>c</i> (operația de intersecție din teoria mulțimilor). Returnează <code>true</code> dacă colecția s-a modificat
<code>int size()</code>	Returnează numărul de elemente din colecție
<code>Object[] toArray()</code>	Returnează un tablou ce conține toate elementele din colecție
<code>Object[] toArray(Object[] a)</code>	Returnează un tablou ce conține toate elementele din colecție. În acest caz argumentul primit este tipul elementelor din tablou

Tabelul 10.2: METODELE INTERFEȚEI COLLECTION.

Prototipul	Descriere
Object get(int index)	Returnează elementul de pe poziția <i>index</i> . Va genera o excepție <i>IndexOutOfBoundsException</i> dacă indexul satisface condiția (<i>index < 0 index >= size()</i>)
Object set(int index, Object element)	Înlocuiește elementul de pe poziția <i>index</i> cu elementul specificat. Returnează elementul vechi care se afla pe poziția dată
int lastIndexOf(Object o)	Returnează poziția ultimei apariții din listă a elementului referit de <i>o</i> sau -1 dacă acesta nu există
Object remove(int index)	Șterge elementul de pe poziția specificată din listă

Tabelul 10.3: METODE DIN INTERFAȚA LIST.

Prototipul	Descriere
ArrayList()	Construiește o listă fără nici un element având capacitatea inițială de 10 elemente. Dacă se încearcă adăugarea a mai mult de 10 elemente, dimensiunea listei se va modifica automat
ArrayList(Collection c)	Construiește o listă ce conține elementele conținute de colecția primită ca argument. Capacitatea inițială este cu 10 procente mai mare decât numărul de elemente din colecția primită ca argument
ArrayList(int initialCapacity)	Construiește o listă fără nici un element având capacitatea inițială de <i>initialCapacity</i> elemente

Tabelul 10.4: CONSTRUCTORII CLASEI ARRAYLIST.

Prototipul	Descriere
LinkedList()	Construiește o listă fără nici un element
LinkedList(Collection c)	Construiește o listă ce conține elementele conținute de colecția primită ca argument

Tabelul 10.5: CONSTRUCTORII CLASEI LINKEDLIST.

Prototipul	Descriere
<code>void addFirst(Object o)</code>	Adaugă elementul specificat la începutul listei
<code>void addLast(Object o)</code>	Adaugă elementul specificat la sfârșitul listei
<code>Object removeFirst()</code>	Șterge primul element din listă și returnează o referință spre el
<code>Object removeLast()</code>	Șterge ultimul element din listă și returnează o referință spre el
<code>Object getFirst()</code>	Returnează primul element din listă
<code>Object getLast()</code>	Returnează ultimul element din listă

Tabelul 10.6: METODE SPECIFICE CLASEI `LinkedList`.

ale listei fiind stocate sub forma unei liste înlănțuite. Acest fapt asigură un timp mai bun pentru ștergerea și inserarea unui element în interiorul listei comparativ cu *ArrayList*. În schimb, accesul aleator la un element din interiorul listei este o operație consumatoare de mai mult timp față de *ArrayList*.



Ce e mai bine să folosim, *ArrayList* sau *LinkedList*? Răspunsul diferă în funcție de operațiile frecvente care se efectuează asupra listelor.

Clasa *LinkedList* are câteva metode în plus față de cele din interfața *List*, anume metode care permit prelucrarea elementelor aflate la cele două capete ale listei. Aceste metode specifice sunt prezentate în Tabelul 10.6.

Exemplu La începutul lecției am spus că trebuie scrisă o aplicație care să gestioneze informații despre angajații unei companii dezvoltatoare de software. În acest scop a fost implementată clasa *Companie* care avea, în prima variantă de implementare, un atribut de tip tablou de angajați. Apoi am creat o clasă numită *ListaAngajati* și am înlocuit în clasa *Companie* tabloul cu un atribut de tip *ListaAngajati*. În exemplul de mai jos, în loc de folosirea unei clase proprii pentru gestionarea angajaților am folosit clasa predefinită *ArrayList*.

```
class Companie {

    private String nume;
    private ArrayList angajati;

    public Companie(String nume, int nrAngajati) {
        this.nume = nume;
    }
}
```

```
    angajati = new ArrayList(nrAngajati);  
}  
  
public void addAngajat(Angajat a) {  
    angajati.add(a);  
}  
...  
}
```

Important

Metoda *add* a clasei *ArrayList* are ca parametru o referință spre un *Object*. Dar datorită faptului că orice clasă moștenește clasa *Object* și datorită facilității oferite de moștenirea de tip, o instanță a clasei *ArrayList* poate stoca orice tip de obiecte.

Important

Se observă că nu există în interfața *Collection* metode de genul *public boolean add(tipPrimitiv i)*, unde *tipPrimitiv* desemnează unul dintre tipurile primitive existente.

Datorită mecanismului de autoboxing prezentat în Secțiunea 4.2.2 putem scrie

```
List c = new ArrayList(10);  
c.add(6);
```

Dacă dorim să testăm dacă un anumit angajat este sau nu angajat în cadrul companiei, nu trebuie decât să creăm o nouă metodă în clasa *Companie*.

```
public boolean este(Angajat a) {  
    return angajati.contains(a);  
}
```

Important

Polimorfismul este mecanismul datorită căruia metoda *contains* poate testa existența unui obiect specificat în interiorul unei colecții. Concret, pentru colecția referită de *angajati* metoda *contains* returnează *true* dacă și numai dacă în colecție există cel puțin un element *e* astfel încât (*a==null ? e==null : a.equals(e)*).



Dacă în clasa *Angajat* în loc să suprascriem metoda *equals* din clasa *Object*, am fi creat metoda *public boolean egal(Object o)* cu același conținut ca și metoda *equals* din clasa *Angajat*, nu s-ar fi putut NICIODATĂ testa existența unui angajat într-o colecție predefinită Java – nu s-ar fi testat CORECT egalitatea dintre doi angajați din punct de vedere al conținutului. Atunci când suprascriem metoda *equals* nu facem altceva decât să specificăm ce înseamnă egalitatea dintre două obiecte din punct de vedere al conținutului – în cazul a două obiecte de tip *Angajat* egalitatea înseamnă nume, respectiv prenume egale.

Dacă dorim să schimbăm departamentul angajatului aflat pe poziția *pos* în interiorul listei am putea crea în clasa *Companie* metoda de mai jos

```
public void schimbaDepartamentul(int pos, String departamentNou) {  
    Angajat a = (Angajat)angajati.get(pos);  
    a.schimbaDepartamentul(departamentNou);  
}
```

Se observă că accesul la serviciile specifice obiectelor de tip *Angajat* conținute de listă impune folosirea operatorul *cast*. Însă folosirea operatorului *cast*, pe lângă faptul că este deranjantă, poate introduce erori la rularea programului datorită neconcordanței dintre tipul real al obiectului existent în colecție și tipul spre care facem *cast*. Începând cu varianta 1.5 a limbajului Java s-au introdus tipurile generice care elimină necesitatea conversiilor de tip explicite la lucrul cu colecții. Detalii despre tipurile generice sunt prezentate în Secțiunea 10.3.

Atenție

Nu se recomandă ca în clasa *Companie* să existe metode care accesează elemente de pe anumite poziții ale colecției *angajati*. Cauza este simplă: la un moment dat, în interiorul clasei *Companie*, în loc să folosim un obiect *ArrayList* pentru stocarea informațiilor despre angajați, decidem să folosim un obiect instanță a clasei *HashSet*. Este evident că elementul de pe o anumită poziție nu ar mai fi posibil de accesat.

10.3 Genericitatea colecțiilor

Despre ce este vorba? În ultima implementare a clasei *Companie* am folosit un obiect *ArrayList* pentru stocarea informațiilor despre angajați. Nu am putut restricționa tipul elementelor pe care le poate conține lista referită de atributul *angajati* și, în consecință, următoarea instrucțiune este corectă

```
angajati.add(new Integer(20));
```

Evident că am vrea ca existența unor astfel de operații să fie semnalată printr-o eroare la compilare. Ei bine, datorită existenței tipurilor generice putem restricționa tipul elementelor conținute de o colecție.

```
class Companie {  
    private String nume;  
    private ArrayList<Angajat> angajati;
```

```
public Companie(String nume, int nrAngajati) {  
    this.nume = nume;  
    angajati = new ArrayList<Angajat>(nrAngajati);  
}  
}
```

Declarația *private ArrayList<Angajat> angajati* specifică faptul că avem o listă ce poate conține doar obiecte de tip *Angajat*. Spunem că *ArrayList* este o clasă generică care are un parametru de tip, în acest caz, *Angajat*. De asemenea, parametrul de tip este specificat și la instanțierea clasei. În acest moment, dacă încercăm să adăugăm în listă elemente de tip *Integer* compilatorul va semnala o eroare.

```
public void schimbaDepartamentul(int pos, String departamentNou) {  
    Angajat a = angajati.get(pos);  
    a.schimbaDepartamentul(departamentNou);  
}
```

Având în vedere că putem stoca în obiectul de tip *ArrayList* doar obiecte de tip *Angajat*, exemplul de mai sus este absolut corect din punct de vedere sintactic iar necesitatea instrucțiunii *cast* a dispărut.

Atenție

Nu e obligatoriu ca într-un program pe care-l scriem să folosim tipurile generice dar folosirea acestora mărește gradul de înțelegere al programului precum și robustețea acestuia.

Tipurile generice și subclasele. Considerăm următoarea secvență:

```
ArrayList<Contabil> lContabili = new ArrayList<Contabil>(); //1  
ArrayList<Angajat> lAngajati = lContabili; //2
```

Se pune problema corectitudinii linii numerotate cu 2. La prima vedere, linia 2 ar fi corectă. Dar în continuare vom arăta că lucrurile nu sunt chiar așa.

```
//Daca linia 2 ar fi corecta, am putea adauga intr-o  
//lista de contabili orice obiecte de tip Angajat, ceea ce nu am dori  
lAngajati.add(new Angajat("Mihai", "Mircea"));  
//si undeva s-ar putea atribui unei referinte Contabil un obiect Angajat  
Contabil c = lContabili.get(0);
```

Atenție

Pentru acest exemplu, clasa *Angajat* nu a fost declarată ca fiind abstractă deoarece pentru ilustrarea fenomenului era necesară instanțierea unui obiect de tipul superclasei. Dar ar fi bine ca în programe clase de genul *Angajat* să fie declarate abstracte.

Prototipul	Descriere
boolean hasNext()	Returnează true dacă mai sunt elemente de parcurs în cadrul iterației curente
Object next()	Returnează următorul element din iterație
void remove()	Șterge din colecția care a creat iteratorul ultimul element returnat de iterator

Tabelul 10.7: METODELE INTERFEȚEI ITERATOR.

10.4 Parcurgerea colecțiilor

Spuneam anterior că nu se recomandă ca în interiorul unei clase să existe metode care să acceseze elemente de pe anumite poziții ale unei colecții. Dar dacă la un moment dat e nevoie, totuși, ca un client să aibă acces la elementele colecției? Sau dacă în interiorul clasei posesoare a colecției se dorește efectuarea unei anumite operații pentru fiecare element existent? Și în acest caz, schimbarea implementării colecției în interiorul clasei va necesita modificări.

Am văzut că în interfața *Collection* există metoda *iterator()* ce returnează o referință spre un obiect *Iterator*. Iteratorul returnat permite parcurgerea colecției într-o ordine bine precizată de fiecare implementare a interfeței *Collection*.

Metodele interfeței *Iterator* sunt prezentate în Tabelul 10.7.

Exemple. Parcurgerea elementelor listei de angajați în contextul în care nu se folosesc tipurile generice se poate face ca mai jos:

```

Iterator it = angajati.iterator();
while(it.hasNext()) {
    Angajat a = (Angajat)it.next();
    a.schimbaDepartamentul("Contabilitate");
}

```

Dar dacă dorim să scăpăm de instrucțiunea *cast* putem parcurge colecția în felul următor:

```

Iterator<Angajat> it = angajati.iterator();//1
while(it.hasNext()) { //2
    Angajat a = it.next();
    a.schimbaDepartamentul("Contabilitate");
}

```

Prototipul	Descriere
<code>int compareTo(Object o)</code>	Compară obiectul curent cu cel primit pentru stabilirea unei ordini între cele două obiecte

Tabelul 10.8: METODELE INTERFEȚEI COMPARABLE.

Atenție

Dacă între liniile 1 și 2 de mai sus, s-ar fi adăugat noi elemente în colecția referită de *angajati*, ar fi trebuit obținut un nou iterator.

Instrucțiunea *foreach*. Începând cu Java versiunea 1.5 s-a introdus instrucțiunea denumită *foreach* cu ajutorul căreia o colecția de angajați se parcurge în felul următor:

```
for(Angajat current: angajati)
    System.out.println(current);
```

Tipărirea elementelor unei colecții. Putem afișa o colecție, la fel cum afișăm fiecare obiect, folosind metoda *toString()*. De fapt, în toate implementările interfeței *Collection*, metoda *toString()* este suprascrisă astfel încât aceasta să returneze reprezentările sub formă de șiruri de caractere a tuturor elementelor conținute, încadrate între [și].

10.5 Alte tipuri de colecții

10.5.1 Mulțimi

Anterior am precizat că toate clasele concrete care au ca supertip interfața *Set* implementează conceptul de mulțime, o mulțime fiind o colecție ce nu acceptă elemente duplicate. În această secțiune vom exemplifica modul de utilizare al unor clase predefinite pentru lucrul cu mulțimi de obiecte.

În interfața *Set* nu există metode în plus față de interfața *Collection*. Implementarea interfeței de către clasa *HashSet* nu garantează că elementele vor fi reținute într-o ordine particulară.

În principiu, pentru operații obișnuite cu mulțimi se va folosi clasa *HashSet*. *TreeSet* se utilizează atunci când se dorește extragerea de elemente într-o anumită ordine. În general, pentru a putea extrage elemente într-o anumită ordine, elementele colecției de tip *TreeSet* trebuie să implementeze interfața *Comparable*. Clasele predefinite *String*, clasele înfășurătoare, chiar și clasele din suportul pentru lucrul cu colecții implementează interfața *Comparable*. Obiectele ce vor fi adăugate într-o colecție *TreeSet* trebuie să fie instanțe ale unor clase ce implementează interfața *Comparable*.

Exemplu. Fie clasa:

```
class Valoare {
    private int v;

    public Valoare(int v) {
        this.v = v;
    }

    public Valoare(int v) {
        this.v = v;
    }

    public boolean equals(Object o) {
        return (o instanceof Valoare) && ((Valoare)o).v == v;
    }
}
```

După cum se vede, clasa *Valoare* suprascrie metoda *public boolean equals(Object o)*. În acest context, se pune problema afișării efectului produs de codul de mai jos.

```
Set<Valoare> set = new HashSet<Valoare>();
set.add(new Valoare(5));
set.add(new Valoare(5));
System.out.println(set);
```

Am spus că *HashSet* este o clasă ce modelează conceptul de mulțime, o mulțime neputând să conțină elemente duplicate. Atunci ar fi firesc ca tipărirea de mai sus să producă pe ecran textul *[5]* dar, în realitate, dacă rulăm codul de mai sus, vom vedea că se va tipări *[5, 5]*, mulțimea având două elemente “identice”.

De fapt, atunci când se testează dacă un element mai e în mulțime contează ca atât metoda *equals* să returneze egalitate cât și ca obiectele să aibă același cod *hash*. Prin urmare soluția în acest caz este suprascrierea metodei *hashCode*.

```
class Valoare {
    ...
    public int hashCode() {
        return v;
    }
}
```

Atenție

De obicei, la lucrul cu *HashSet* implicația *o1.equals(o2) -> o1.hashCode() == o2.hashCode()* trebuie să fie adevărată.

Prototipul	Descriere
Object put(Object key, Object value)	Asociază obiectul referit de <i>value</i> cu cheia specificată de <i>key</i> . Dacă în dicționar mai există stocată cheia <i>key</i> , atunci valoarea asociată e înlocuită
Object get(Object key)	Returnează valoarea referită de cheia specificată

Tabelul 10.9: CÂTEVA METODE DIN INTERFAȚA MAP.

10.5.2 Dicționare

Am spus anterior că toate clasele concrete care au ca supertip interfața *Map* implementează conceptul de dicționar cheie-valoare. În această secțiune prezentăm în Tabelul 10.9 câteva metode existente în interfața *Map* iar în continuare vom exemplifica modul de folosire al clasei *HashMap*.

Dacă căutarea într-un dicționar s-ar face liniar, această operație ar fi foarte ineficientă din punctul de vedere al timpului necesar efectuării ei. În cadrul dicționarelor, nu se face o căutare liniară a cheilor ci una bazată pe așa numita funcție *hash*. Fiecare cheie are un cod *hash* care e folosit ca index într-un tablou capabil să furnizeze rapid valoarea asociată cheii.

Exemplu. Fie clasa:

```
class Valoare {
    private int v;

    public Valoare(int v) {
        this.v = v;
    }

    public boolean equals(Object o) {
        return (o instanceof Valoare) && ((Valoare)o).v == v;
    }
}
```

În exemplul de mai jos, în dicționar se vor introduce doi angajați, ambii asociați cheii cu valoarea 1.

Atenție

Nu se recomandă ca într-un dicționar să existe două chei între care să existe egalitate din punct de vedere al conținutului.

```
HashMap<Valoare, Angajat> hm = new HashMap<Valoare, Angajat>();  
Valoare v1 = new Valoare(1);  
Valoare v2 = new Valoare(1);  
hm.put(v1, new Contabil("Ionescu", "Mircea"));  
hm.put(v2, new Contabil("Ion", "Mircea"));  
System.out.println(hm);
```

Dar ce se va afișa în urma execuției?

```
System.out.println(hm.get(new Valoare(1)));
```

Răspunsul este foarte simplu: *null*. Deși există elemente asociate cheii cu valoarea 1, cheile neavând același cod *hash*, elementul asociat indexului corespunzător codului *hash* al cheii din exemplul de mai sus din tabloul în care se păstrează elementele în cadrul implementării interne a clasei *HashMap* este *null*.

Soluția, și în acest caz, este suprascrierea metodei *hashCode* pentru clasa *Valoare*. Atunci, la execuția codului de mai sus se va afișa *Nume:Ion Departament:contabilitate*. În cartea *Thinking in Java*, Capitolul *Containers in Depth - Overriding hashCode()* este prezentat în detaliu un algoritm pentru generarea de coduri hash corecte!

10.6 Exerciții rezolvate

Biblioteca

Folosind clasa *ArrayList* creați o clasă *Biblioteca* ce poate stoca un număr nelimitat de obiecte de tip *Carte*. O carte are două atribute ce stochează titlul precum și autorul cărții iar afișarea acesteia pe ecran va furniza utilizatorului valorile atributelor menționate.

Clasa *Biblioteca* oferă doar două servicii, unul pentru adăugarea de elemente de tip *Carte* și altul pentru afișarea elementelor conținute. Se cere implementarea claselor menționate precum și crearea într-o metodă *main* a unei biblioteci ce are trei cărți. Cărțile ce există în bibliotecă vor fi tipărite.

Rezolvare

```
import java.util.*;  
  
class Carte {  
    private String autor, titlu;
```

```
public Carte(String autor, String titlu) {
    this.autor = autor;
    this.titlu = titlu;
}

public String toString() {
    return autor + " " + titlu;
}
}

class Biblioteca {
    private ArrayList<Carte> carti = new ArrayList<Carte>();

    public void add(Carte c) {
        carti.add(c);
    }

    public String toString() {
        //se poate si parcurge colectia cu iteratori, dar ar fi inutil
        //din moment ce exista metoda toString() din ArrayList
        return carti.toString();
    }

    public static void main(String[] argv) {
        Carte c1 = new Carte("colectiv", "abecedar");
        Carte c2 = new Carte("UPT", "Java");
        Carte c3 = new Carte("INFO", "Java");

        Biblioteca b = new Biblioteca();
        b.add(c1);
        b.add(c2);
        b.add(c3);

        System.out.println(b);
    }
}
```

Fișiere și Directoare

Respectând cerințele enunțate și principiile programării orientate pe obiecte, să se implementeze în Java interfața și clasele descrise mai jos.

Interfața *Intrare* conține:

- o metodă denumită *continut*, fără argumente și care returnează o referință *String*.

Clasa *Fisier* implementează interfața *Intrare* și conține:

- un atribut de tip *String* denumit *informatie*, specific fiecărui obiect *Fisier* în parte.
- un constructor ce permite setarea atributului anterior cu o valoare *String* dată ca parametru constructorului.
- implementarea metodei *continut* întoarce valoarea atributului *informatie* descris mai sus.

Clasa *Director* implementează interfața *Intrare* și conține:

- un atribut denumit *intrari* de tip *ArrayList<Intrare>*. Câmpul este specific fiecărei instanțe a acestei clase și se va inițializa cu un obiect listă gol.
- o metoda *adauga* cu un singur parametru; acesta trebuie să fie declarat în așa fel încât să poată referi atât obiecte a clasei *Director*, cât și obiecte a clasei *Fisier* dar să NU poată referi orice fel de obiect Java (spre exemplu, NU va putea referi un obiect *String*). Metoda introduce în lista anterioară referința primită ca parametru.
- obligatoriu în implementarea metodei *continut* se parcurge lista *intrari* și se apelează metoda *continut* pe fiecare referință din lista concatenându-se *String*-urile întoarse de aceste apeluri; metoda va returna o referință spre *String*-ul rezultat n urma concatenării.

În toată această ultimă clasă, obiectele *Fisier* și obiectele *Director* din listă trebuie să fie tratate uniform.

Rezolvare

```
interface Intrare {
    String continut();
}

class Fisier implements Intrare {
    private String informatie;
    public Fisier(String informatie) {
        this.informatie = informatie;
    }
    public String continut() {
        return informatie;
    }
}

class Director implements Intrare {
    private ArrayList<Intrare> intrari = new ArrayList<Intrare>();
```

```
public void adauga(Intrare intr) {
    intrari.add(intr);
}
public String continut() {
    String tmp = "";
    for(Intrare i : intrari) {
        tmp = tmp + i.continut();
    }
    return tmp;
}
}
```

10.7 Exerciții

1. Ce credeți că e mai bine să folosim, *ArrayList* sau *LinkedList*?
2. Scrieți un program în care se citesc de la tastatură șiruri de caractere până la citirea șirului *STOP*. Șirurile citite se vor stoca într-o colecție inițială de tip *LinkedList* ce poate conține duplicări. Creați o nouă colecție de tip *LinkedList* ce va conține elementele colecției inițiale, dar fără duplicări. Tipăriți apoi ambele colecții.
3. Să se implementeze ierarhia de clase descrisă mai jos:

- Clasa *Tip*: reprezintă un tip de date abstract
 - Date membru: nu are
 - Metode membru
 - * *public String getTip()*: returnează numele clasei sub forma unui șir de caractere precedat de șirul "Tip: "
 - * *public String toString()*: afișează valoarea atributului încapsulat de clasele derivate

Metoda *getTip* nu are inițial nici o implementare.

- Clasa *Intreg*: reprezintă tipul de date întreg (moștenește clasa *Tip*)
 - Date membru: un atribut de tip *int*
 - Metode membru
 - * *public String getTip()*
 - * *public String toString()*
- Clasa *Sir*: reprezintă tipul de date șir de caractere (moștenește clasa *Tip*)
 - Date membru: un atribut de tip *String*
 - Metode membru
 - * *public String getTip()*
 - * *public String toString()*

- Clasa *Colectie*: reprezintă tipul de date colecție de obiecte *Tip*
 - Date membru: un atribut ce stochează elementele colecției
 - Metode membru
 - * *public String getTip()*
 - * *public String toString()*
 - * o metodă care testează egalitatea dintre două colecții din punct de vedere al conținutului elementelor. Două colecții sunt considerate a fi egale din punct de vedere al conținutului dacă ambele conțin același număr de elemente iar elementele conținute sunt egale, mai mult, ele sunt stocate în aceeași ordine.
 - * o metodă pentru adăugarea de elemente în colecție

Acest tip de colecție trebuie implementat astfel încât o colecție să poată conține elemente de tip *Colectie*.

Exemple. Presupunem că avem o colecție formată din următoarele elemente: 7, 4, Eu, 12. Apelul metodei *toString* trebuie să furnizeze rezultatul (7, 4, Eu, 12).

Presupunem că avem o colecție formată din următoarele elemente: 7, 4, Eu, 12 și colecția formată din elementele 2 și 8. Apelul metodei *toString* trebuie să furnizeze rezultatul (7, 4, Eu, 12, (2, 8)). Metoda *toString* din această clasă trebuie să fie implementată urmărind următoarele cerințe

- folosirea operatorului *instanceof* e STRICT interzisă
- trebuie să existe o variabilă de tip *Iterator* în interiorul metodei

Se va scrie și o metodă *main* într-o altă clasă în care se va crea o colecție de obiecte *Tip* ce va avea cel puțin un element de tip *Colectie*, după care aceasta se va afișa. Se va testa și egalitatea elementelor dintre două colecții.

Bibliografie

1. Gilad Bracha, *Generics in the Java Programming Language*.
<http://java.sun.com/j2se/1.5/pdf/generics-tutorial.pdf>.
2. Bruce Eckel. *Thinking in Java, 4th Edition*. Prentice-Hall, 2006. Capitolul Containers in Depth.
3. Sun Microsystems Inc., *Online Java 1.5 Documentation*,
<http://java.sun.com/j2se/1.5.0/docs/api/>, 2005.

Lecția 11

Elemente de programare concurrentă

Multe sisteme software reale, scrise în Java sau în alte limbaje de programare, trebuie să trateze mai multe evenimente simultan. În această lecție vom vedea cum anume putem crea astfel de programe în Java, ce probleme specifice pot apare în astfel de programe și ce mecanisme de limbaj pune la dispoziție Java pentru a evita aceste probleme.

11.1 Concurența

În teoria programării orientate pe obiecte concurența se definește în felul următor:

Definiție 4 *Concurența este proprietatea care distinge un obiect activ de un obiect pasiv.*

Dar ce înseamnă un obiect activ și ce înseamnă un obiect pasiv? Un obiect activ este un obiect care “face ceva” fără ca alt obiect exterior lui să acționeze asupra sa prin apelarea unei metode. Un obiect este pasiv dacă el stă și nu “face nimic” dacă asupra sa nu acționează un alt obiect exterior lui.



În lumea ce ne înconjoară există o grămadă de obiecte pasive și tot atâtea obiecte active. Un exemplu de obiect real pasiv este arcul unui indian. Arcul “face ceva”, mai exact lansează săgeți, doar când indianul acționează asupra lui. Arcul nu lansează săgeți când are el chef. Un obiect real activ e ceasul de mână. El schimbă indicația orară tot timpul fără acțiunea unui alt obiect exterior. E adevărat că un obiect exterior îl poate regla dar asta nu-l face obiect pasiv. Dacă ar fi pasiv atunci el ar trebui reglat din exterior la trecerea fiecărei secunde pentru a arăta ora exactă. Nu aş vrea un asemenea ceas de mână.

De ce avem nevoie de obiecte active? În orice sistem software trebuie să existe cel

puțin un obiect activ, altfel sistemul în ansamblul său nu ar “face nimic”. Nu ar exista nimic în acel sistem care să manipuleze obiectele pasive ce-l compun pentru ca acestea să “facă ceva”. Dacă într-un sistem avem mai multe obiecte active este posibil ca mai multe obiecte din acel sistem, inclusiv pasive, să “facă ceva” în același timp, iar sistemul în ansamblul său va “face” mai multe lucruri simultan. Spunem în astfel de situații că avem un sistem sau program concurrent.



Un indian este evident un obiect real activ. Doi indieni pot acționa simultan două arcuri distincte și prin urmare aceste două obiecte reale pasive pot lansa săgeți în același timp!

11.2 Ce este un fir de execuție?

În secțiunea anterioară am vorbit despre obiecte active și pasive. Până acum noi am definit, creat și manipulat o sumedenie de obiecte în Java. Erau ele pasive sau active?

Toate obiectele definite de noi până acum erau pasive: pentru ca ele să “facă ceva” trebuia să acționăm asupra lor din exterior prin apelarea unei metode. După cum am spus la început, un obiect activ “face ceva” fără a se acționa din exteriorul său (tipărește mesaje pe ecran, apelează metode ale altor obiecte, etc.). În aceste condiții spunem că un obiect activ are propriul său *fir de execuție*. Noțiunea de fir de execuție își are originea în domeniul sistemelor de operare. În continuare vom discuta principal ce înseamnă un fir de execuție din punct de vedere fizic.

Important

Discuția ce urmează nu corespunde perfect realității dar e utilă pentru înțelegerea conceptelor.

Când un program Java e lansat în execuție mașina virtuală îi alocă o porțiune de memorie. Această porțiune e împărțită în trei zone: o zonă de date (heap), o zonă pentru stive și o zonă pentru instrucțiunile programului. În timp ce programul se execută, putem să ne imaginăm că instrucțiuni din zona de instrucțiuni “curg” înspre procesor, acesta executându-le succesiv. Acest flux de instrucțiuni (mai exact copii ale instrucțiunilor deoarece ele nu pleacă din memorie) ce “curge” dinspre zona de instrucțiuni spre procesor se numește *fir de execuție*. În cadrul unui program Java este posibil ca la un moment dat să avem două sau mai multe fluxuri de instrucțiuni ce “curg” dinspre aceeași zonă de instrucțiuni spre procesor. Spunem că acel program se execută în mai multe fire de execuție simultan. Efectul pe care-l percepe programatorul este că mai multe părți din același program (nu neapărat distincte din punct de vedere fizic) se execută în același timp.

Dar cum poate un singur procesor să primească și să execute în același timp două fluxuri de instrucțiuni? Răspunsul e că nu poate. Este însă posibil ca fluxurile de

instrucțiuni să “curgă” spre două procesoare diferite în cazul sistemelor de calcul cu mai multe procesoare. Dacă nu avem decât un singur procesor atunci mașina virtuală aplică un algoritm de *time-slicing*: o perioadă de timp sau o cuantă de timp procesorul execută instrucțiuni dintr-un flux, apoi o altă perioadă de timp din altul, apoi din altul, ș.a.m.d. Practic procesorul e dat de mașina virtuală o perioadă de timp fiecărui fir de execuție individual creînd astfel iluzia unei execuții simultane a mai multor părți din același program. Când unui fir i se ia procesorul, el va aștepta până îl va primi înapoi și doar atunci își va continua execuția. De unde? Exact de acolo de unde a lăsat-o când i s-a luat procesorul.

Legat de firele de execuție în care se execută la un moment dat un program, trebuie subliniate următoarele aspecte:

- toate firele de execuție văd același heap. Java alocă toate obiectele în heap, deci toate firele au acces la toate obiectele.
- fiecare fir are propria sa stivă. Variabilele locale și parametrii unei metode se alocă în stivă, deci fiecare fir are propriile valori pentru variabilele locale și pentru parametrii metodelor pe care le execută.
- corecta funcționare a unui program concurent nu are voie să se bazeze pe presupuneri legate de viteza relativă de execuție a firelor sale. Argumente de genul “știu eu că firul ăsta e mai rapid ca celălalt” sau “știu eu că obiectul ăsta nu e accesat niciodată simultan de două fire de execuție diferite” sunt interzise.

11.3 Utilizarea firelor de execuție în Java

11.3.1 Definirea, crearea și declanșarea firelor de execuție

În Java, definirea și crearea unui obiect activ se traduce prin definirea și crearea unui obiect fir de execuție căruia îi va corespunde un unic fir de execuție fizic din mașina virtuală. Pentru a se putea executa, un program Java are nevoie de cel puțin un fir de execuție în mașina virtuală (altfel instrucțiunile sale nu ar avea cum ajunge la procesor). Principial, am putea spune că în orice program Java aflat în execuție există un obiect fir de execuție: el nu e definit și nici creat explicit de programator, dar el există, fiind creat automat de mașina virtuală Java la pornirea programului avînd rolul de a apela metoda *main*.

Și un programator poate defini și crea obiecte fire de execuție și deci poate scrie programe ce se execută în mai multe fire de execuție fizice ale mașinii virtuale. Există două variante pe care le prezentăm mai jos. A doua porțiune de cod ne arată cum anume creăm astfel de obiecte (corespunzător celor două variante) și cum anume declanșăm funcționarea lor (mai exact cum dăm drumul firelor lor fizice de execuție).

```
//Varianta 1
class FirUnu extends Thread {
    public void run() {
        ...
    }
}

//Varianta 2
class FirDoi implements Runnable {
    public void run() {
        ...
    }
}
```

```
//Varianta 1
FirUnu o = new FirUnu();
o.start();

//Varianta 2
Thread o = new Thread(new FirDoi());
o.start();
```

În prima variantă definirea unui obiect fir de execuție se face printr-o clasă ce moștenește clasa predefinită *Thread*. Mai mult, se suprascrie metoda *run* moștenită de la clasa *Thread*. Ea va avea rolul unui fel de metodă *main*, în sensul că de acolo începe să se execute respectivul fir de execuție. Crearea unui obiect fir se reduce în această variantă la crearea unei instanțe a respectivei clase. Pentru declanșarea execuției firului trebuie să apelăm metoda sa *start*, moștenită de la clasa *Thread*.

În cea de-a doua variantă definirea unui obiect fir de execuție se face printr-o clasă ce implementează interfața *Runnable* și care implementează metoda *run* ce are același rol ca în cazul primei variante. Crearea unui obiect fir de execuție se va face în acest caz prin crearea unei instanțe a clasei *Thread* utilizând un constructor ce primește ca parametru un obiect *Runnable*. În cazul nostru el va primi ca parametru o instanță a clasei despre care am vorbit înainte. Pornirea firului se face utilizând tot metoda *start*.

11.3.2 Cooperarea firelor de execuție

O caracteristică importantă a firelor de execuție este că ele văd același heap. Acest lucru poate duce la multe probleme în cazul în care un obiect oarecare (care e o resursă comună pentru toate firele) este accesat din două fire de execuție diferite. Vom considera un exemplu ipotetic pentru a vedea ce se poate întâmpla.



Este evident că în momentul în care doi indieni vor să utilizeze **același** arc simultan vor apare ceva probleme.

```
class Patrat {  
  
    private int latime, inaltime;  
  
    public void set(int latura) {  
        /*1*/latime = latura;  
        /*2*/inaltime = latura;  
        /*3*/if(latime == inaltime) {  
            System.out.println("Patratul are latura " + latime);  
        }  
        /*4*/else System.out.println("ERROR");  
    }  
}
```

Să presupunem clasa de mai sus ce modelează un pătrat. Exemplul are doar un rol ilustrativ: cine ar folosi pentru a memora latura unui pătrat două câmpuri despre care știm că sunt egale tot timpul? Să presupunem însă că se întâmplă acest lucru și că respectiva clasă este utilizată într-un program concurent. Evident, în programele în care avem un singur fir de execuție nu apare nici o problemă: niciodată nu se va tipări pe ecran mesajul ERROR. La fel se întâmplă și într-un program concurent dacă nici un obiect pătrat nu e accesat simultan din mai multe fire de execuție distincte. Probleme apar când două (sau mai multe) fire de execuție acționează simultan asupra ACELUIAȘI obiect pătrat. De exemplu, un fir apelează metoda *set* cu parametrul actual 1 și un alt fir apelează metoda *set* cu parametrul actual 2 (dar pentru ACELAȘI obiect). Se poate întâmpla următorul scenariu:

- primul fir execută apelul și apoi linia 1 din program, dând valoarea 1 câmpului lățime.
- după ce se execută linia 1 mașina virtuală hotărăște că trebuie să dea procesorul altui fir, oprește pentru moment execuția primului fir și pornește execuția celui de-al doilea.
- al doilea fir execută apelul metodei *set* și apoi instrucțiunile 1 și 2 dând aceeași valoare (2) ambelor câmpuri; apoi execută testul, totul e ok, și iese din metodă.
- la un moment dat mașina virtuală dă iar procesorul primului fir și acesta reîncepe execuția cu instrucțiunea numărul 2. Cum am spus că fiecare fir are propria stivă, parametrul *latura* are valoarea 1 în acest fir; prin urmare înălțimea va lua valoarea

1, după care se execută instrucțiunea 3. SURPRIZĂ: laturile nu mai sunt egale și prin urmare se tipărește pe ecran ERROR.

Important

Același scenariu poate apare și dacă avem mai multe procesoare, nu numai când se folosește algoritmul de time-slicing.

Totuși, intenția programatorului a fost de a ține cele două laturi tot timpul egale. Unde a greșit? Problema se datorează faptului că programatorul, deși știa că asupra aceluiași obiect ar putea acționa mai multe fire de execuție, a presupus implicit că odată ce un fir începe să execute metoda *set* pentru un obiect, el o va termina înainte ca un alt fir să înceapă să execute și el metoda *set* pentru același obiect. Cu alte cuvinte, a neglijat faptul că pentru ca programul să meargă corect, nu e voie să se facă vreo presupunere legată de viteza relativă de execuție a mai multor fire.



În cazul exemplului de mai sus se spune că programatorul a introdus în sistem o *condiție de cursă*. Astfel de condiții nu au voie să apară în sistemele concurente. Menționăm că nu toate condițiile de cursă se vor elimina așa de ușor ca în exemplul nostru.

Exemplul de mai sus ne arată că atunci când avem mai multe fire de execuție pot apare probleme extrem de subtile. Pentru eliminarea lor este necesar ca firele de execuție să poată coopera cumva între ele. În Java există două mecanisme de cooperare: mecanismul de excludere mutuală și mecanismul de cooperare prin condiții.



Imaginați-vă o orchestră simfonică. Fiecare membru al orchestrei e un obiect activ care cântă partea sa din compoziția interpretată. Dacă fiecare ar cânta după bunul său plac ar rezulta un haos. Dar nu se întâmplă acest lucru pentru că aceste “fire de execuție” cooperează între ele într-o manieră impusă de dirijor. Din păcate în sistemele software aflate la execuție nu există un astfel de dirijor. Programatorul joacă rolul dirijorului în sensul că el utilizează anumite mecanisme de limbaj ce impun firelor un mod clar de cooperare.

Mecanismul de excludere mutuală

Excluderea mutuală se traduce informal prin faptul că la un moment dat un obiect poate fi manipulat de un singur fir de execuție și numai de unul. Cum specificăm acest lucru? O posibilitate constă în utilizarea modificadorului de acces la metode *synchronized*. Rolul său e simplu: în timpul în care un fir execută instrucțiunile unei metode *synchronized* pentru un obiect, nici un alt fir nu poate executa o metodă declarată *synchronized* pentru ACELAȘI obiect.

Important

Excluderea mutuală se poate obține și prin utilizarea *blocurilor de sincronizare*. Despre ele nu vom vorbi aici, dar principiul e foarte asemănător.

Cine impune această regulă? Principial vom spune că mașina virtuală Java. Ea are grijă singură, fără nici o intervenție exterioară, ca această regulă să fie respectată. Cum? Destul de simplu. În momentul în care un fir de execuție apelează o metodă sincronizată pentru un obiect se verifică dacă obiectul respectiv se află în starea “liber”. Dacă da, obiectul e trecut în starea “ocupat” și firul începe să execute metoda, iar când firul termină execuția metodei obiectul revine în starea “liber”. Dacă nu e “liber” când s-a efectuat apelul, înseamnă că există un alt fir ce execută o metodă sincronizată pentru același obiect (mai exact, un alt fir a trecut obiectul în starea “ocupat” apelând o metodă sincronizată sau utilizând un bloc de sincronizare). Într-o astfel de situație firul va aștepta până când obiectul trece în starea “liber”.



Dacă observați cu atenție, este același mod de cooperare pe care-l aplică oamenii în multe situații. Oamenii (firele de execuție) stau la coadă la un ghișeu și nu năvălesc toți deodată să vorbească cu unica persoană de la ghișeu (obiectul pe care toate firele vor să-l acceseze). Când un client discută cu persoana de la ghișeu aceasta e ocupată. Când clientul a terminat de discutat, el pleacă iar ghișeul devine liber și altcineva poate să-l ocupe.

Să reluăm acum exemplul din secțiunea anterioară declarând metoda *set* ca având un acces sincronizat.

```
class Patrat {  
  
    private int latime, inaltime;  
  
    public synchronized void set(int latura) {  
        /*1*/latime = latura;  
        /*2*/inaltime = latura;  
        /*3*/if(latime == inaltime) {  
            System.out.println("Patratul are latura " + latime);  
        }  
        /*4*/else System.out.println("ERROR");  
    }  
}
```

În această situație nu mai există posibilitatea ca pe ecran să se afișeze ERROR indiferent cât de multe fire de execuție accesează simultan un același obiect pătrat. Pentru a înțelege motivul să reluăm scenariul de execuție prezentat anterior: un fir apelează metoda *set* cu parametrul actual 1 și un alt fir apelează metoda *set* cu parametrul actual 2 (dar pentru ACELAȘI obiect).

- primul fir execută apelul și apoi execută linia 1 din program, dând valoarea 1

câmpului lățime.

- după ce se execută linia 1 mașina virtuală hotărăște că trebuie să dea procesorul altui fir, oprește pentru moment execuția primului fir și pornește execuția celui de-al doilea.
- al doilea fir ar vrea să execute apelul dar NU poate deoarece obiectul e “ocupat”, primul fir aflându-se în execuția metodei *set* pentru acel obiect. Ca urmare, firul așteaptă. Mașina virtuală își dă seama că firul nu poate continua și dă procesorul altui fir.
- la un moment dat mașina virtuală dă iar procesorul primului fir iar acesta reîncepe execuția cu instrucțiunea numărul 2. Cum am spus că fiecare fir are propria stivă, parametrul latura are valoarea 1 în acest fir; prin urmare înălțimea va lua valoarea 1, după care se execută instrucțiunea 3 fără nici o surpriză, laturile fiind egale. Firul termină apoi execuția metodei iar obiectul va fi trecut în starea “liber”. Când procesorul va ajunge iar la al doilea fir, acesta va putea continua (evident dacă obiectul nostru nu a trecut între timp în starea “ocupat” datorită unui apel la *set* din alt fir ce s-a executat înainte ca al doilea fir să primească procesorul).

Cooperarea prin condiții

Există situații în care excluderea mutuală este necesară pentru rezolvarea unei probleme de concurență, dar nu e și suficientă. Astfel de situații pot apare atunci când un fir execută o metodă sincronizată (deci obiectul apelat e “ocupat”) dar el nu poate să termine execuția metodei pentru că nu s-a îndeplinit o anumită condiție. Dacă acea condiție poate fi îndeplinită doar când un alt fir ar apela o metodă sincronizată a aceluiasi obiect, situația e fără ieșire: obiectul e ținut “ocupat” de firul ce așteaptă apariția condiției, iar firul ce vrea să apeleze o metodă sincronizată a aceluiasi obiect pentru îndeplinirea condiției, pur și simplu nu poate face acest lucru (obiectul fiind “ocupat” iar metoda e sincronizată). Aceasta e o situație de *impas* (*deadlock*). Astfel de situații nu au voie să apară în sisteme software concurente pentru că le blochează.



O astfel de cooperare aplică și oamenii în anumite situații. Să presupunem că mai multe persoane (fire de execuție) iau masa la un restaurant. Pe masă există o singură sticlă de sos de roșii. Evident e necesar un mecanism de excludere mutuală pentru utilizarea acelei sticle: doar o singură persoană o poate utiliza la un moment dat. Dar ce se întâmplă când o persoană constată că sticla e goală? Ea deține sticla la momentul respectiv, dar nu o poate utiliza. Ce se întâmplă? De obicei intervine un alt fir de execuție, mai exact chelnerul, care umple respectiva sticlă: o preia de la persoana care o deține, o umple, după care o dă înapoi respectivei persoane. Condiția fiind îndeplinită, mai exact sticla conține sos, persoana poate să o utilizeze. Și în cazuri reale ar putea apare o situație de impas: persoana ce deține sticla goală e atât de încăpățânată, încât nu vrea să dea sticla chelnerului pentru

a o umple.

Mecanismul de cooperare prin condiții se bazează pe o pereche de două metode definite în clasa *Object* (datorită acestui lucru toate obiectele dintr-un program au aceste metode). Un lucru important ce trebuie spus încă de la început este că un fir de execuție care apelează aceste metode pentru un obiect, trebuie să fie singurul fir care poate accesa acel obiect la momentul respectiv. Altfel spus, obiectul trebuie să fie “ocupat” și să fi trecut în această stare datorită execuției firului ce apelează aceste metode. Altfel se va produce o excepție. În cazurile studiate de noi în această lecție, un apel la aceste metode e sigur dacă el apare în metode sincronizate și are ca obiect țintă obiectul *this*. Situațiile mai complicate nu vor fi prezentate în această lecție. Motivul acestei constrângeri rezultă din tabelul ce explică semantica metodelor *wait()* și *notifyAll()*.

Prototip	Descriere
<code>wait()</code>	Când un fir de execuție apelează această metodă pentru un obiect, firul va fi pus în așteptare. Aceasta înseamnă că nu se revine din apel, că ceva ține firul în interiorul metodei <code>wait()</code> . Acest apel este utilizat pentru “blocarea” unui fir până la îndeplinirea condiției de continuare. În timp ce un fir așteaptă în metoda <code>wait()</code> apelată pentru un obiect, obiectul receptor va trece temporar în starea “liber”.
<code>notifyAll()</code>	Când un fir de execuție apelează această metodă pentru un obiect, TOATE firele de execuție ce sunt “blocate” în acel moment în metoda <code>wait()</code> a ACELUIAȘI OBIECT sunt deblocate și pot reveni din apelul respectivei metode. Acest apel este utilizat pentru a anunța TOATE firele care așteaptă îndeplinirea condiției de continuare că această condiție a fost satisfăcută.

Tabelul 11.1: METODE UTILIZATE ÎN MECANISMUL DE COOPERARE PRIN CONDIȚII.

Se consideră un program simplu compus din două fire de execuție. Un fir pune numere într-un obiect container, iar alt fir ia numere din ACELAȘI obiect container. Clasa *Container* va avea două metode: una de introducere a unui număr în container, iar alta de extragere a unui număr din container. Containerul poate conține la un moment dat maxim un număr. Se pun condițiile: nu putem pune un număr în container dacă containerul e plin și nu putem scoate un număr din container dacă containerul e gol. O primă variantă incorectă de definire a clasei *Container* e următoarea.

```
class Container {

    private int numar;
    private boolean existaNumar = false;
```

```
public synchronized void pune(int l) {
    while(existaNumar); //astept sa fie containerul gol
    numar = l;
    existaNumar = true; //am pus un numar deci nu mai
                        //e loc de altul
}

public synchronized int extrage() {
    while(!existaNumar); //astept sa fie containerul umplut
    existaNumar = false; //scot numarul, deci nu mai e nimic
                        //in container

    return numar;
}
}
```

De ce e această variantă incorectă? Pentru că poate apare o situație de impas (dead-lock). Primul fir poate pune un număr în containerul nostru. Prin urmare *existaNumar* devine *true*. Din păcate nu putem să garantăm că al doilea fir este suficient de rapid să vină și să extragă numărul din container. Prin urmare este posibil ca tot primul fir să mai vrea să pună alt număr în container, dar cum acesta e plin, el stă și așteaptă în ciclul *while* până se îndeplinește condiția de continuare: adică până al doilea fir ia numărul din container: mai exact va aștepta pe vecie. Motivul? Al doilea fir nu are cum extrage numărul pentru că metoda *extrage* e sincronizată!!! Așadar avem un impas. Și dacă programul ăsta controlează ceva de genul unei centrale nucleare, cu siguranță nu ar fi prea plăcută această situație.

O soluție naivă ar fi să spunem: nici o problemă, facem ca metoda *extrage* să nu fie sincronizată. Dacă am face așa ceva am da direct din lac în puț! Continuând situația ipotetică de mai sus s-ar putea ca firul doi să înceapă să execute metoda *extrage* și exact după ce pune *existaNumar* pe *false* mașina virtuală să-i ia procesorul și să-l dea primului fir. Grav! Primul fir pune alt număr în container! La acordarea procesorului celui de-al doilea fir se va continua de la *return* și iată cum s-a pierdut un număr! Și iarăși ar putea să iasă rău dacă programul controlează ceva dintr-o centrală nucleară!

Soluția pentru rezolvarea acestei probleme e utilizarea cooperării prin condiții. Codul corect e prezentat mai jos. Mai mult, codul funcționează bine și dacă există mai multe fire care pun numere în același container și mai multe fire ce extrag numere din același container. Vom prezenta acum și codul ce definește cele două fire de execuție.

```
class FirPuneNumere extends Thread {

    private Container c;
```

```
public FirPuneNumere(Container c) {
    this.c = c;
}

public void run() {
    int i = 0;
    while(true) { i++; c.pune(i % 1000);}
}
}

class FirExtrageNumere extends Thread {

    private Container c;

    public FirExtrageNumere(Container c) {
        this.c = c;
    }

    public void run() {
        while(true) { System.out.println(c.extrage());}
    }
}

class ProgramTest {

    public static void main(String[] argv) {
        Container c = new Container();
        new FirPuneNumere(c).start();
        new FirExtrageNumere(c).start();
    }
}

class Container {

    private int element;
    private boolean existaNumar = false;

    public synchronized void pune(int l) {
        while(existaNumar) {
            try { wait(); //astept indeplinirea conditiei "container gol"
            } catch(InterruptedException e) {...}
        }
        element = l;
        existaNumar = true; //am pus un numar deci nu mai
                           //e loc de altul
    }
}
```

```
        notifyAll(); //Anunta indeplinirea conditiei "container plin"
                        //Toate firele ce sunt blocate in acest moment in wait-
                        //ul din metoda extrage se pregatesc sa revina din el
    }

    public synchronized int extrage() {
        while(!existaNumar) {
            try { wait(); //astept indeplinirea conditiei "container plin"
            } catch (InterruptedException e) {...}
        }
        existaNumar = false; //scot numarul, deci nu mai e nimic
                        //in container
        notifyAll(); //Anunta indeplinirea conditiei "container gol"
                        //Toate firele ce asteapta in acest moment la wait-ul
                        //din metoda pune se pregatesc sa revina din el

        return element;
    }
}
```

Pentru a înțelege funcționarea metodelor *wait* și *notifyAll* să vedem mai întâi ce ne încurca în prima versiune a codului. Ne încurca faptul că primul fir stătea în ciclul *while* în așteptarea golirii containerului. Cum metoda *extrage* e sincronizată, el ținea obiectul container “ocupat” și nu-l lăsa pe al doilea fir să golească containerul, metoda *extrage* fiind și ea sincronizată. Prin urmare am avea nevoie de ceva asemănător ciclului *while* dar care pe lângă faptul că ține firul în așteptare ar trebui să pună temporar obiectul în starea “liber”! Exact acest lucru îl face metoda *wait*: ține firul în așteptare și pune obiectul apelat (la noi, *this*) în starea “liber”. Prin urmare nu e nici o problemă pentru al doilea fir să extragă numărul din container. După ce-l extrage, al doilea fir apelează *notifyAll* anunțând primul fir ce e “blocat” în wait-ul din metoda *pune* că poate să-și reia activitatea, condiția de golire a containerului fiind satisfăcută. Când are loc pornirea primului fir? Nu se poate spune exact pentru că depinde de mașina virtuală, dar el sigur va porni. Mai mult, mașina virtuală îl va porni în așa fel încât condiția de excludere mutuală impusă de clauzele *synchronized* să fie respectate.

Ar mai rămâne un singur aspect de discutat. De ce e nevoie de două perechi de *wait-notifyAll*. Simplu, cazul de deadlock expus de noi presupunea că se umple containerul și nu se mai poate goli, dar poate apărea și situația simetrică: containerul se golește și nu se mai poate umple. Tratarea situației este exact identică.

Important

Aceast exemplu reflectă un caz particular de problemă de tip producător-consumator. Producătorul e firul care pune numere în container, iar consumatorul e firul care ia numere din ACELAȘI container. Generalizarea problemei apare atunci când există mai mulți producători, mai mulți consumatori iar în container

se pot pune mai multe numere.



V-ați întrebat de ce *wait* e cuprins într-un ciclu *while*? Dacă am avea mai multe fire care pun valori în container, atunci se poate întâmpla ca mai multe fire să fie “blocate” în wait-ul din pune. Când al doilea fir extrage un număr, el apelează *notifyAll* deblocând toate firele ce așteaptă la *wait*-ul asociat. Dar după ce un astfel de fir pune un număr în container, restul de fire trebuie să se întoarcă înapoi în *wait* deoarece condiția de continuare, deși a fost pentru scurt timp adevărată, a devenit iarăși falsă. Prin urmare ciclul *while* se folosește pentru a retesta la fiecare deblocare condiția de continuare. Retestarea condiției de continuare e necesară și datorită faptului că *notifyAll* deblochează și eventualele alte fire ce așteaptă în *wait*-ul din *extrage*. Ele trebuie să-și verifice condiția de continuare pentru a vedea dacă apelul la *notifyAll* este sau nu destinat lor.

11.4 Exerciții

1. Într-o benzinărie există un singur rezervor de benzină de capacitate 1000 de litri. Acest rezervor este alimentat de mai multe cisterne de benzină de capacități diferite. Evident, dacă rezervorul nu poate prelua întreaga cantitate de benzină pe care o conține o cisternă, aceasta va aștepta eliberarea capacității necesare și numai atunci va depune în rezervor întreaga cantitate de benzină conținută. Golirea cisternelor nu ține cont de ordinea de sosire a cisternelor în benzinărie.

De la acest rezervor se alimentează autoturisme, un autoturism preluând cantități aleatoare de benzină din rezervor, dar unul preia tot timpul un singur litru (oare de ce?). Evident, dacă rezervorul nu conține cantitatea cerută de un autoturism, acesta va aștepta până când poate prelua integral întreaga cantitate cerută (care e tot timpul mai mică decât 1000). Alimentarea autoturismelor nu ține cont de sosirea în benzinărie a autoturismelor.

Să se implementeze un program care simulează situația descrisă mai sus utilizând fire de execuție. O cisternă va fi modelată printr-un obiect fir de execuție cu capacitatea specificată prin constructor și care depune la infinit benzină în rezervorul benzinăriei. Un autoturism va fi modelat tot printr-un obiect fir de execuție care preia la infinit cantități aleatoare de benzină din rezervorul benzinăriei. Se va scrie, de asemenea, o metodă *main* care creează 3 cisterne de capacități diferite (mai mici de 1000) și 5 autoturisme după care declanșează firele de execuție asociate acestora.

2. Într-un magazin de gresie există doi agenți de vânzări și o singură caserită. Odată ce un agent face o vânzare el depune într-o coadă de așteptare unică un bon ce conține numele agentului și suprafața de gresie vândută. Capacitatea cozii este de 20 de bonuri. Dacă nu mai există loc în coadă agentul așteaptă eliberarea unui loc pentru a depune bonul. Caserita preia bonurile din coadă în ordinea în care au

fost depuse și emite o chitanță de forma: Nume Agent - Suprafața Vândută - Preț. Prețul unui metru pătrat de gresie este de 25 de lei. Dacă nu există nici un bon în coada de așteptare casei așteaptă.

Să se implementeze problema de mai sus (producător - consumator) folosind trei obiecte fire de execuție: două pentru simularea agenților (producătorii) ce citesc suprafețele vândute din două fișiere, și unul pentru simularea casei ce emite chitanțe (consumatorul) scriind “chitanțele” într-un fișier. Implementarea trebuie să garanteze faptul că pentru fiecare bon se emite o chitanță și numai una.

3. Cinci filosofi se află așezați în jurul unei mese rotunde. Fiecare are în față o farfurie cu mâncare. De asemenea, între oricare doi filosofi există o singură furculiță. Masa fiind rotundă, rezultă că există cinci furculițe. Fiecare filosof repetă la infinit următoarea secvență de operații:

- gândește o perioadă aleatoare de timp, caz în care nu are nevoie de nici o furculiță (el trebuie să lase furculițele pe masă când gândește).
- i se face foame, caz în care încearcă să pună stăpânire pe cele două furculițe din vecinătatea sa.
- mănâncă o perioadă aleatoare de timp, dar numai după ce a obținut cele două furculițe.

Să se scrie un program Java care simulează activitățile acestor filosofi. Fiecare filosof va avea propriul fir de execuție care va realiza activitățile descrise mai sus (afișând mesaje corespunzătoare pe ecran). Programul trebuie astfel implementat încât să nu apară situații de impas.

NOTĂ Pentru a realiza cooperarea între filosofi, fiecărei furculițe trebuie să-i corespundă un obiect. Cel mai bine e să modelați furculițele printr-o clasă *Furculita*. Ce metode trebuie să aibe această clasă? Dacă alegeți pentru a obține excluderea mutuală a accesului la o furculiță blocurile de sincronizare, nu mai e necesară nici o metodă dedicată cooperării. Dacă nu folosiți blocuri de sincronizare atunci va fi necesară utilizarea și a cooperării prin condiții, iar clasa *Furculita* va avea o metodă prin care va trece un obiect furculiță în starea “furculiță utilizată” și o alta prin care va trece un obiect furculiță în starea “furculiță neutilizată” (Atenție: nu există nici o legătură între aceste stări și starea “ocupat”/“liber” a unui obiect din contextul metodelor sincronizate). Cooperarea prin condiții e utilizată în ideea că un filosof nu poate trece o furculiță în starea “utilizată” dacă ea e deja în această stare.



Problema filosofilor este o problemă celebră în contextul programelor concurente. Situația de impas poate apare dacă, de exemplu, fiecare filosof a luat furculița din dreapta sa și așteaptă să preia furculița din stânga sa. Într-o astfel de situație nici un filosof nu poate continua deoarece

furculița din stânga sa este deținută de filosoful din stânga lui care și el așteaptă să preia furculița din stânga sa pe care însă nu o poate lua deoarece e deținută de filosoful din stânga lui ș.a.m.d. Pe lângă situația de impas, în această problemă mai poate apare și o situație cunoscută în domeniul programării concurente ca situație de *înfometare* (*starvation*). Pe scurt, aceasta înseamnă că un filosof nu ajunge niciodată să mănânce, el așteptând să ia de exemplu furculița stângă dar filosoful din stânga sa o lasă jos după care imediat o ia înapoi (și asta tot timpul). În implementarea voastră nu trebuie să evitați și această situație.

Bibliografie

1. James Gosling, Bill Joy, Guy L. Steele Jr., Gilad Bracha, *Java Language Specification*, <http://java.sun.com/docs/books/jls>, 2005.

Lecția 12

Probleme frecvente în cod. Javadoc. Jar

12.1 Câteva probleme frecvente în cod

Martin Fowler prezintă în *Refactoring: Improving the Design of Existing Code* 22 de carențe de proiectare ce pot să apară în cadrul sistemelor orientate pe obiecte (cunoscute și sub numele de *bad smells in code*). În această secțiune vom prezenta câteva dintre ele.

12.1.1 Cod duplicat

Prin cod duplicat înțelegem existența într-o aplicație a mai multor porțiuni de cod identice. Duplicarea codului îngreunează într-o mare măsură dezvoltarea și modificarea ulterioară a aplicației.

Să presupunem că definim clasa *Masina* de mai jos. În cadrul metodei *porneste* a fost introdusă accidental o eroare deoarece în cadrul ei se depășesc limitele tabloului referit de *usi*.

```
class Masina {  
  
    protected Motor motor;  
    protected Usa[] usi;  
  
    public Masina(Motor motor, Usa[] usi) {  
        this.motor = motor;  
        this.usi = usi;  
    }  
  
    public void porneste() {
```

```
    for (int i=0; i<=usi.length; i++)  
        usi[i].blocheaza();  
    motor.demareaza();  
}  
    ...  
}
```

Să presupunem acum că înainte de a testa clasa *Masina* se scrie o altă clasă, *MasinaABS* iar în cadrul metodei suprascrise *porneste* se face o copiere a corpului metodei din clasa de bază. Evident, se copiază și eroarea!!!

```
class MasinaABS extends Masina {  
  
    public MasinaABS(Motor motor, Usa[] usi) {  
        super(motor, usi);  
    }  
  
    public void porneste() {  
        System.out.println("Masina ABS");  
        for (int i=0; i<=usi.length; i++)  
            usi[i].blocheaza();  
        motor.demareaza();  
    }  
    ...  
}
```

Este clar faptul că eliminarea erorii din clasa derivată ar putea să nu aibe loc în momentul în care ea e detectată și eliminată din clasa *Masina*, pentru că pur și simplu nu se mai știe de unde, ce, și cât cod a fost duplicat.



Duplicarea de cod dintr-un program mărește artificial dimensiunea codului, mărește efortul necesar eliminării erorilor (o eroare nefiind localizată într-un singur loc) precum și efortul necesar pentru introducerea de noi cerințe în cadrul aplicației. Mecanismul *copy-paste* rezolvă superficial probleme! În cadrul exemplului anterior, un simplu apel *super.porneste()* în metoda *porneste* din clasa *MasinaABS* elimină duplicarea.

12.1.2 Listă lungă de parametri

O listă lungă de parametri la o metodă este greu de înțeles și poate deveni inconsistentă și greu de utilizat când este subiectul unor frecvente modificări.

```
class Sertar {

    private int lungime, inaltime, adancime;

    public Sertar(int lungime, int inaltime, int adancime) {
        this.lungime = lungime;
        this.inaltime = inaltime;
        this.adancime = adancime;
    }

    public int getLungime() {
        return lungime;
    }

    public int getInaltime() {
        return inaltime;
    }

    public int getAdancime() {
        return adancime;
    }
}

class Etajera {

    private Sertar[] sertare = new Sertar[2];

    public Etajera(int lungime, int inaltime, int adancime) {
        sertare[0] = new Sertar(lungime, inaltime, adancime);
        sertare[1] = new Sertar(lungime, inaltime, adancime);
    }

    //Returneaza o singura lungime, sertarele fiind suprapuse
    public int getLungime() {
        return sertare[0].getLungime();
    }

    public int getInaltime() {
        return sertare[0].getInaltime() + sertare[1].getInaltime();
    }

    public int getAdancime() {
        return sertare[0].getAdancime();
    }

    public Sertar[] getSertare() {
        return sertare;
    }
}
```

Presupunem că o etajeră este formată din două sertare, ambele sertare având aceleași dimensiuni. Implementările pentru clasele *Sertar* și *Etajera* sunt prezentate mai sus. Dorim să definim o metodă statică care să verifice dacă o etajeră încapă într-un spațiu oarecare de dimensiuni $L \times A \times H$. Metoda definită, în loc să aibe patru parametri (dimensiunile $L \times A \times H$ precum și o referință spre un obiect etajeră), are 5 parametri, dimensiunile $L \times A \times H$ precum și două referințe spre sertarele incluse de etajeră. Au fost trimise două referințe spre obiecte de tip *Sertar* fiindcă, în fond, o etajeră este alcătuită din două sertare suprapuse.

```
class Verifica {  
  
    public static boolean incapeEtajera(int L, int A, int H,  
                                       Sertar s1, Sertar s2) {  
  
        int LS, AS, HS;  
  
        //Calculeaza dimensiunile etajerei  
  
        LS = s1.getLungime();  
        AS = s1.getAdancime();  
        HS = s1.getInaltime() + s2.getInaltime();  
  
        return ((L>LS) && (A>AS) && (H>HS));  
    }  
}
```

După cum se observă, în interiorul metodei *incapeEtajera* se calculează dimensiunile etajerei în care sunt dispuse cele două sertare referite de *s1* și *s2*. Acest fapt implică o cunoaștere a modului în care sunt dispuse cele două sertare, adică a unei particularități de realizare (implementare) a unei etajere.

Pe de o parte, schimbarea implementării etajerei (spre exemplu, dispunerea pe orizontală a celor două sertare) necesită și modificarea implementării metodei *incapeEtajera* pentru obținerea unei validări corecte. Din păcate, calcule asemănătoare cu cele din cadrul metodei *incapeEtajera* pot exista în mai multe locuri în cadrul aplicației și e posibil să uităm să efectuăm peste tot modificările necesare, în acest mod fiind facilitată introducerea de erori în aplicație.

Pe de altă parte, componența unei etajere ar putea fi modificată, în sensul că am putea avea etajere formate din trei sertare. Este evident faptul că trebuie modificată lista de parametri a metodei *incapeEtajera* precum și toate entitățile din cadrul aplicației care apelează metoda. Dacă însă am fi transmis metodei noastre o singură referință spre un obiect de tip *Etajera* iar în loc de a calcula dimensiunile etajerei, le-am fi obținut direct prin intermediul acestei referințe, metoda *incapeEtajera* precum și entitățile apelante nu ar fi trebuit să fie modificate la schimbarea structurii etajerei.



Trimiterea la apelul unei metode de referințe spre atributele unui obiect în loc de o referință spre obiectul cărui atributele este o practică defectuasă pentru aplicațiile orientate pe obiecte și se manifestă de multe ori prin liste lungi de parametri.

12.1.3 Instrucțiuni switch și if-else-if

Mai jos e definită excepția *TablouException* ce e generată în diferite situații anormale ce pot apare la apelarea metodelor clasei *Tablou*. Această clasă permite adăugarea de întregi într-o colecție de dimensiune limitată precum și obținerea unui element de pe o anumită poziție. Evident, parametrii metodelor clasei *Tablou* pot avea valori eronate pentru obiectul asupra căruia se cere efectuarea respectivelor servicii, acest fapt fiind semnalat prin emiterea unei excepții. După cum reiese din definiția clasei, există mai multe motive pentru care se poate emite excepția.

```
class TablouException extends Exception {  
  
    private String message;  
  
    public TablouException(String message) {  
        this.message = message;  
    }  
  
    public String toString() {  
        return message;  
    }  
}
```

```
class Tablou {  
  
    private int[] tablou;  
    private int nrElem = 0;  
  
    public Tablou(int dim) {  
        tablou = new int[dim];  
    }  
  
    public void addElement(int el) throws TablouException {  
        if(nrElem < tablou.length)  
            tablou[nrElem++] = el;  
        else  
            throw new TablouException("Tablou Plin");  
    }  
}
```

```
public int getElementAt(int pos) throws TablouException {
    if(pos >= tablou.length)
        throw new TablouException("Pozitie Invalida");
    if(nrElem <= pos)
        throw new TablouException("Prea Putine Elemente");
    return tablou[pos];
}
```

Totuși, uneori este necesară execuția anumitor instrucțiuni în funcție de motivul concret care a generat excepția. Evident, motivul concret al excepției poate fi aflat doar din mesajul ei. Acest fapt face ca întotdeauna când trebuie să fie luate decizii în sistem legate de acest aspect să apară o înșiruire de instrucțiuni *if-else-if*.

```
public static oMetoda(Tablou t) {
    int pos = 29;
    try {
        t.addElement(5);
        t.addElement(12);
        int el = t.getElementAt(pos);
        ...
    } catch(TablouException e) {
        if (e.toString().equals("Tablou Plin")) {
            System.err.println("Adaugarea in tablou nu s-a putut face.");
        } else if (e.toString().equals("Pozitie Invalida")) {
            System.err.println("Dimensiunea tabloului este mai mica.");
            pos = -1;
        } else {
            System.err.println("Nu exista atatea elemente in tablou.");
        }
    }
    ...
}
```

Atenție

În multe situații, în loc de lanțuri *if-else-if* poate apare o instrucțiune *switch*. O altă modelare a excepției de mai sus ar putea conduce la utilizarea unui *switch* pentru a discerne între diferitele situații anormale.

Utilizarea instrucțiunii *switch* precum și a lanțurilor *if-else-if* în scopul descris mai sus conduce însă la apariția duplicărilor de cod de fiecare dată când e nevoie să se afle cauza concretă ce a generat excepția. Mai mult, în viitor, metodele clasei *Tablou* ar putea emite excepția *TablouException* și datorită altor cauze. Prin urmare va fi necesar să căutăm prin tot programul locurile în care se testează motivul apariției excepției și să mai adăugăm o ramură *if-else* în acele locuri.

Soluția eliminării instrucțiunilor *if-else-if* este crearea de subclase ale clasei *TablouException* pentru fiecare motiv ce poate conduce la emiterea de excepții în interiorul metodelor clasei *Tablou*.



Într-un program orientat pe obiecte existența unor lanțuri *if-else-if* sau a unor instrucțiuni *switch* cu rolul de a discerne între diferite “feluri de obiecte este, de cele mai multe ori, un semn al lipsei utilizării polimorfismului!

12.1.4 Data Class

O clasă de tip Data Class este o clasă ce conține doar atribute precum și metode pentru setarea, respectiv returnarea valorilor atributelor conținute (aceste metode se numesc metode accesori). În multe situații, clienții unei astfel de clase, în loc să ceară efectuarea de servicii de la instanțele clasei, preiau datele de la obiecte în scopul efectuării “serviciilor” ce ar fi trebuit furnizate de clasa de date.

Imaginați-vă o clasă *Telefon* definită ca în exemplul de mai jos. Oare ce am putea face cu o instanță a acestei clase? Din păcate, o instanță a acestei clase nu oferă serviciile specifice unui telefon (formează număr, răspunde la apel, pornește/oprește telefon). Un client al unui obiect de acest tip, pentru a porni spre exemplu telefonul, ar trebui să-i ceară acestuia să-i furnizeze ecranul iar apoi să-i ceară ecranului să se pornească. Din păcate, în cele mai multe cazuri, pornirea unui telefon nu implică numai pornirea unui ecran. Pentru realizarea cu succes a operației de pornire a telefonului se impune probabil efectuarea mai multor operații similare. Astfel, un client al unui telefon ajunge să fie obligat să cunoască funcționarea în detaliu a unui astfel de aparat. În același timp, un obiect telefon nu are un singur client, ci mai mulți. Prin urmare, este ușor de înțeles că toți clienții telefonului trebuie să cunoască detalii de implementare ale telefonului pentru a-l utiliza. În acest mod complexitatea sistemului poate crește foarte mult.

```
class Telefon {  
  
    private Ecran ecran;  
    private Buton onOff;  
    .... //Alte componente  
  
    public void setEcran(Ecran ecran) {  
        this.ecran = ecran;  
    }  
  
    public Ecran getEcran() {  
        return ecran;  
    }  
}
```

```
public void setOnOff(Buton onOff) {  
    this.onOff = onOff;  
}  
public Buton getOnOff() {  
    return onOff;  
}  
...  
}
```



Imaginați-vă că o firmă constructoare de mașini ar furniza în loc de o mașină ce răspunde la anumite comenzi, componentele mașinii, cerându-le clienților să le folosească în mod direct. La fel se petrec lucrurile și în programarea orientată pe obiecte, clienții unui obiect fiind interesați de posibilele servicii furnizate de către un obiect și nicidecum de atributele sale!!! Data class-urile apar din cauza deficiențelor de modelare a obiectelor din program, în exemplul de mai sus a unui telefon. Eliminarea acestei probleme se realizează principal prin identificarea și introducerea în clasa de date a serviciilor necesare clienților clasei respective.

12.2 Despre javadoc și jar

12.2.1 Instrumentul software javadoc

În cadrul mai multor lecții au fost făcute trimiteri la documentația Java oferită de firma Sun la adresa <http://java.sun.com/j2se/1.5.0/docs/api>. Această documentație, în formatul HTML, a fost creată folosind instrumentul software *javadoc*, instrument obținut odată cu instalarea platformei Java.

În figura 12.1 e prezentată documentația aferentă clasei *Object*. După cum se vede, este precizat scopul clasei precum și al fiecărei metode existente în această clasă. Și noi putem genera, folosind *javadoc*, documentație aferentă claselor scrise de noi. Pentru a introduce în documentația generată informații privind scopul clasei, al metodelor, al parametrilor unei metode, etc. e necesară inserarea în cod a *comentariilor de documentare*. Acest tip de comentarii se inserează în cod între */*** și **/*.

Atenție

Comentariile de documentare sunt singurele comentarii recunoscute de *javadoc*. Dacă comentariile sunt de tip */** și **/* nu se vor produce afișările corespundente în documentația HTML generată de *javadoc*.

În interiorul comentariilor de documentare pot fi introduse tag-uri *javadoc*. Tag-urile *javadoc* se inserează după caracterul @ (caracter ce poate fi precedat doar de spații și, opțional, de *) și permit generarea unei documentații detaliate și uniforme.

Tag	Descriere
@author nume	Adaugă o intrare de tip Author în documentație.
@param nume descriere	Adaugă parametrul cu numele și descrierea specificată în secțiunea de parametri a unei metode sau a unui constructor.
@return descriere	Adaugă o intrare de tip Returns în documentația aferentă unei metode. Descrierea trebuie să cuprindă tipul returnat și, dacă este cazul, plaja de valori returnată.
@throws nume descriere	Adaugă o intrare de tip Throws în documentația specifică unei metode sau al unui constructor; nume este numele excepției ce poate fi emisă în interiorul metodei.
@version	Adaugă o intrare de tip Version în documentație.

Tabelul 12.1: CÂTEVA TAG-URI JAVADOC.

Fiecare comentariu de documentare trebuie să fie plasat înainte de entitatea comentată (clasă, interfață, constructor, metodă, atribut, etc.). Mai jos este documentată o parte din clasa *Tablou* definită în secțiunea 12.1.3.

```
/**
 * Aceasta clasa stocheaza elemente intregi intr-un tablou si permite
 * accesarea unui element prin intermediul unui index.
 * @author LooseResearchGroup
 */
class Tablou {

    /**
     * Returneaza elementul de pe pozitia indicata.
     * @param pos pozitia de pe care se returneaza elementul
     * @throws TablouException In cazul in care pos este parametru invalid
     */
    public int getElementAt(int pos) throws TablouException {
        if(pos>=tablou.length) throw new TablouException("Pozitie Invalida");
        if(nrElem<=pos) throw new TablouException("Prea Putine Elemente");
        return tablou[pos];
    }
}
```

Pentru obținerea documentației trebuie apelat

```
javadoc -sourcepath fisiere.java
```

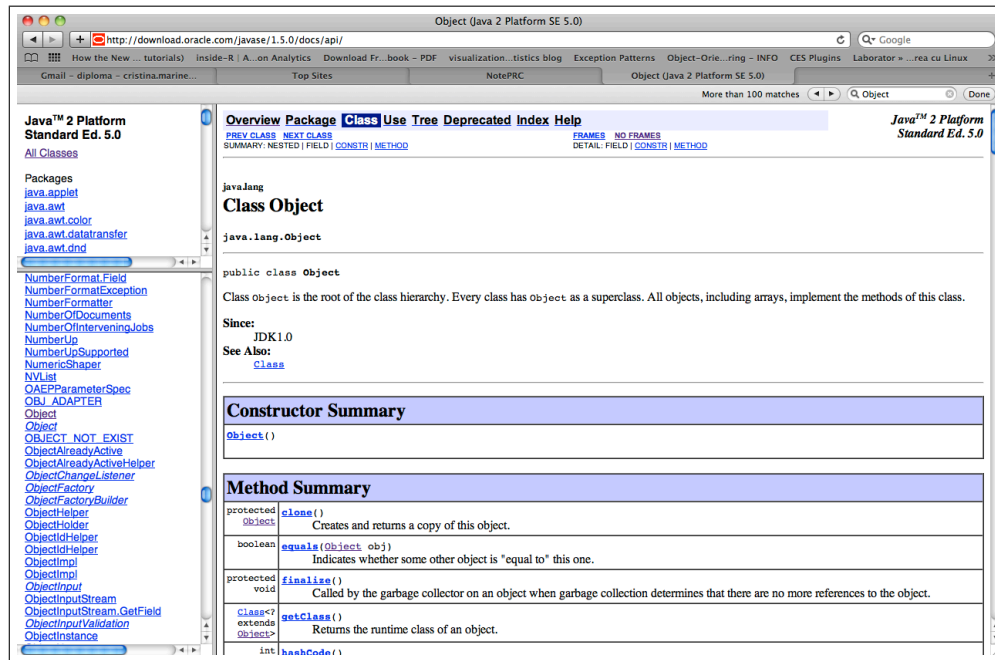


Figura 12.1: DOCUMENTAȚIA CLASEI OBJECT OFERITĂ DE JAVA API.

12.2.2 Instrumentul software jar

Până acum toate aplicațiile care au fost create constau din fișierele sursă respectiv din fișierele cu extensia *class* rezultate în urma compilării surselor. Atunci când o aplicație este furnizată unui client nu e indicat ca respectivul client să primească și fișierele sursă ale aplicației! Ca urmare, un client primește o mulțime de fișiere cu extensia *class*. Totuși, livrarea la un client a mai multor fișiere cu extensia *class* nu se recomandă și nu e prea profesionistă.

Se recomandă ca aplicațiile Java (fișierele *class* asociate ei) să se livreze clienților într-o arhivă *jar*. Pentru realizarea unei arhive folosind instrumentul software *jar*, obținut odată cu instalarea platformei Java, trebuie să executăm în linia de comandă:

```
jar cf app.jar fisier1 ... fisierN
```

Opțiunile *cf* indică faptul că se va crea o nouă arhivă a cărei denumire este *app.jar* iar fișierele incluse de ea sunt cele specificate de lista de fișiere.

Rularea unei aplicații împachetate într-o arhivă *jar* se face incluzând arhiva în argumentul *classpath* al mașinii virtuale Java și specificând numele complet al clasei ce

include metoda *main*. Totuși, acest lucru nu e prea convenabil pentru un client. O altă variantă de rulare a aplicației este prezentată mai jos.

```
java -jar app.jar
```

Pentru ca o aplicație să poată fi rulată în acest mod, este necesar ca arhiva să conțină un fișier *manifest* care să conțină numele clasei ce conține metoda *main*:

```
//Fișierul manifest.tmp
//Clasa este numele clasei ce contine metoda main
Main-Class: Clasa
Name: Clasa.class
```

Crearea arhivei ce conține fișierul *manifest.tmp* se va face prin execuția comenzii de mai jos:

```
jar cfm app.jar manifest.tmp fisier1 ... fisierN
```

12.3 Exerciții

1. Eliminați duplicarea de cod din porțiunea de cod de mai jos.

```
class Matrice {

    private Integer[][] elemente;

    //Se face o citire a elementelor matricei din fisier
    public Matrice(int n) {
        elemente = new int[n][n];
        FileInputStream file = new FileInputStream("matrice.txt");
        InputStreamReader is = new InputStreamReader(file);
        BufferedReader stream = new BufferedReader(is);
        for (int i=0; i<n; i++)
            for (int j=0; j<n; j++)
                elemente[i][j] = Integer.parseInt(stream.readLine());
        stream.close();
    }

    //Se face o citire a elementelor matricei din fisier
    //daca neinitializat este false, in caz contrar elementele vor fi 0
```

```
public Matrice(int n, boolean neinitializat) {
    if(neinitializat == true) elemente = new int[n][n];
    else {
        elemente = new int[n][n];
        FileInputStream file = new FileInputStream("matrice.txt");
        InputStreamReader is = new InputStreamReader(file);
        BufferedReader stream = new BufferedReader(is);
        for (int i=0; i<n; i++)
            for (int j=0; j<n; j++)
                elemente[i][j] = Integer.parseInt(stream.readLine());
        stream.close();
    }
}
```

2. Generați documentația specifică aplicației dezvoltate în cadrul primului exercițiu din Lecția 9.
3. Creați o arhivă jar pentru aplicația dezvoltată în cadrul primului exercițiu din Lecția 9.

Bibliografie

1. Harvey Deitel & Paul Deitel. *Java - How to program*. Prentice Hall, 1999, Appendix G, Creating HTML Documentation with javadoc.
2. David Flanagan, *Java In A Nutshell. A Desktop Quick Reference*, Third Edition, O'Reilly, 1999.
3. Martin Fowler. *Refactoring: Improving the Design of Existing Code*. Addison Wesley, 1999.
4. Sun Microsystems Inc., *The Java Tutorial*, <http://java.sun.com/docs/books/tutorial/jar>, 2005.

Lecția 13

Exerciții și probleme propuse spre rezolvare

1. Scrieți o clasă *CC* care furnizează o metodă pentru aflarea numărului de instanțe ale clasei *CC* care s-au creat.
2. Avem clasa *Exemplu*. În ce condiții putem scrie

```
AltExemplu ae = new Exemplu(); ?
```

Dați două variante de răspuns și explicați diferența/diferențele dintre ele.

3. Codul următor nu poate fi compilat.

```
interface A {  
    void m1();           // 1  
    public void m2();    // 2  
    protected void m3(); // 3  
    private void m4();   // 4  
}
```

Specificați linia/liniile care cauzează necompilarea codului și explicați pe scurt de ce se întâmplă acest fapt.

4. Un obiect poate avea mai multe tipuri?
5. Scrieți o clasă *GrupaPOO* ce are un constructor public cu un parametru de tip *int* ce reprezintă numărul studenților din grupă. Implementați clasa astfel încât aceasta să poată fi instanțiată doar dacă numărul studenților din grupă este cuprins în intervalul [12-15].

6. Presupunem că în clasa *C* avem metoda

```
public void oMetoda throws Exceptie2, Exceptie3 {}
```

și apelul ei

```
public static void main(String[] args) {  
    try {  
        C c = new C();  
        c.oMetoda();  
    }  
    catch(Exceptie1 e1) {  
        System.out.println("Exceptie1");  
    }  
}
```

Care e relația între clasele *Exceptie1*, *Exceptie2* și *Exceptie3* astfel încât codul de mai sus să poată fi compilat?

7. Clasele *E1* și *E2* sunt excepții. Ce trebuie să fie îndeplinit pentru ca următorul cod să fie corect (Atenție: nu se va modifica în niciun fel codul de mai jos !!!).

```
class SomeClass {  
    public void foo(boolean b1, boolean b2) {  
        if (b1 && b2)  
            throw new E1();  
        else  
            throw new E2();  
    }  
}
```

8. Specificați relația/relațiile de moștenire care există între clasele *Object*, *RuntimeException*, *Error*, *Throwable* și *Exception*.
9. Se dă secvența de cod de mai jos:

```
class A {  
    public static void main (String[] args) {  
        Error error = new Error();  
        Exception exception = new Exception();  
        System.out.print((exception instanceof Throwable) + ",");  
        System.out.print(error instanceof Throwable);  
    }  
}
```

- Care este rezultatul rulării acesteia?
 - Dați explicațiile pentru care se produce acest rezultat.
10. Presupunem că într-o excepție există o metodă a cărei nume va trebui setat de voi. Să se implementeze această excepție astfel încât să existe o altă metodă care ne poate spune de câte ori a fost apelată metoda a cărei nume a fost setat de voi.
 11. Scrieți un exemplu în care să creați o excepție verificată și una neverificată. Explicați diferențele dintre cele două tipuri de excepții.
 12. În clasa *C* avem metoda

```
public void oMetoda throws Exceptie1, Exceptie2, Exceptie3 {}
```

și apelul ei

```
public static void main(String[] args) {
    try {
        C c = new C();
        c.oMetoda();
    }
    catch(Exceptie1 e1) {
        System.out.println("Exceptie1");
    }
    catch(Exceptie2 e2) {
        System.out.println("Exceptie2");
    }
    catch(Exceptie3 e3) {
        System.out.println("Exceptie3");
    }
}
```

Care e relația dintre clasele *Exceptie1*, *Exceptie2* și *Exceptie3* dacă afirmația următoare e adevărată: indiferent de excepția aruncată de metoda *oMetoda* pe ecran, întotdeauna se afișează “Exceptie1”?

13. Presupunem că avem pachetul *unu* în care sunt definite clasele publice *U1*, *U2* și pachetul *unu.pp* în care sunt definite clasele publice *P1*, *P2*. În fișierul *F* din pachetul *poo* avem instrucțiunea *import unu.**. În aceste condiții, în fișierul *F* se poate instanția clasa *P1*?
14. De câte feluri pot fi specificatorii de acces ai unei clase în interiorul unui pachet? Dați exemplu de o clasă care poate fi folosită doar în interiorul pachetului în care a fost declarată.

15. Presupunem că avem pachetul *unu* în care sunt definite clasele publice *U1* și *U2* și pachetul *unu.pp* în care sunt definite clasele publice *P1* și *P2*, acestea neavând nici un constructor definit de programator. În fișierul *F* din pachetul *doi* avem instrucțiunea *import unu.**. În fișierul *F* avem o clasă *C*. Se poate scrie în interiorul unei metode a clasei *C* codul *P1 p = new P1()*?
16. Fie clasele de mai jos. Cum pot fi folosite instanțe noi ale clasei *PackageAClass* în metoda *foo()* din clasa *PackageBClass* fără a modifica clasa *PackageAClass* sau pachetul în care se află; pot fi adăugate clase noi.

```
//fișierul PackageAClass.java
package a;
public class PackageAClass {
    PackageAClass() { }
}

//fișierul PackageBClass.java
package b;
public class PackageBClass {
    PackageBClass() { }
    void foo() { }
}
```

17. Ce fel de clase pot fi folosite în afara pachetului în care au fost declarate? Ce fel de metode aparținând unei clase dintr-un pachet nu pot fi folosite în afara pachetului în care a fost declarată clasa?
18. Avem o clasă *Elemente* care încapsulează o colecție *ArrayList*. În această colecție dorim să adăugăm obiecte periferice de tipul *Tastatura*, *Ecran* și *Difuzor*. În acest scop am folosit următoarele metode pentru adăugare:

```
public void add(Tastatura t)
public void add(Ecran e)
public void add(Difuzor d)
```

În viitorul apropiat s-ar putea să dorim să adăugăm și alte tipuri de periferice cum ar fi un *Scanner* și atunci ar trebui să adăugăm în clasa *Elemente* metoda

```
public void add(Scanner t).
```

Propuneți o soluție pentru adăugarea de elemente în colecție astfel încât adăugarea unui element de tip *Scanner* să nu implice modificări asupra clasei *Elemente*.

19. Avem o clasă *Persoana* care are două atribute: *nume(String)* și *varsta(int)*. Dorim să reținem obiecte de acest fel într-o colecție de tip *SortedSet* ordonată crescător

după nume. În acest context implementați clasa *Persoana*, creați mai multe obiecte de acest tip, adăugați-le în colecție și precizați rezultatul afișării colecției.

20. Precizați utilitatea iteratorilor. Creați o colecție în care adăugați elemente de tip *Persoana* (o persoană are un nume, o funcție, un venit și o vârstă). Folosind un iterator măriți cu 1000 venitul persoanelor care au funcția “programator”.
21. În care din implementările următoare ale interfeței *List* accesul la un element din mijlocul listei prin intermediul unui index se face mai încet și de ce: *ArrayList* sau *LinkedList*?
22. Avem o clasă *Persoana* care are două atribute: *nume(String)* și *cod numeric personal(String)*. Fiecare persoană are un cod numeric personal unic. Să se implementeze o metodă care returnează intersecția a două colecții de tip *Set* care conțin doar obiecte de tip *Persoana*.
23. Explicați câteva diferențe dintre *Thread* și *Runnable*.
24. Să se implementeze o clasă *ThreadManager* în care se pot înregistra thread-uri după nume și se pot porni thread-uri folosind numele registrat:

```
ThreadManager m = new ThreadManager();  
//t1,t2 - referinte spre doua threaduri  
m.register(t1, "ThreadOne");  
m.register(t2, "ThreadTwo");  
m.startThread("ThreadTwo");
```

25. Modelați un fir de execuție care numără de la 1 la 99, afișând pe ecran fiecare număr. Instanțiați și porniți firul.
26. Dați un scurt exemplu în care să ilustrați conceptul de producător /consumator.
27. Avem o *Memorie* ce poate stoca cel mult un șir de caractere (*String*). Există mai multe procesoare (fire de execuție) care pot scrie în această memorie doar dacă elementul scris anterior de către un procesor a fost preluat de către un scriitor (fir de execuție) ce citește la infinit din memorie, eliberând-o și scriind conținutul ei pe un support fizic. Procesorul poate scrie în memorie doar dacă aceasta este goală, în caz contrar el trebuie să aștepte eliberarea ei, iar dacă memoria este plină și conținutul ei este egal cu cel dorit a fi scris de processor, acesta abandonează scrierea în memorie. Să se implementeze clasa *Memorie*, comentându-se fiecare linie a sa.

28. Care e rezultatul execuției codului de mai jos? Detaliați răspunsul dat.

```
class ThreadOne extends Thread {  
  
    public ThreadOne(Runnable r) {  
        super(r);  
    }  
  
    public void run() {  
        System.out.print("ThreadOne");  
    }  
}  
  
class ThreadTwo implements Runnable {  
    public void run() {  
        System.out.print("ThreadTwo");  
    }  
}  
  
class C {  
    public static void main(String[] args) {  
        new ThreadOne(new ThreadTwo()).start();  
    }  
}
```

29. Explicați de ce codul următor nu afișează pe ecran întotdeauna “LaboratorulPOO
LaboratorulPOO....LaboratorulPOO”.

```
class T extends Thread {  
  
    private void scrie() {  
        System.out.print("Laboratorul");  
        System.out.print("POO");  
    }  
  
    public void run() {  
        int i = 100;  
        while(i>0) {  
            scrie();  
            i--;  
        }  
    }  
}
```

```

class C {
    public static void main (String[] args) {
        new T().start();
        new T().start();
    }
}

```

Modificați codul astfel încât acest lucru să se întâmple, dar să rămână cele două metode pentru afișarea pe ecran.

30. Se cere să se definească un ansamblu de clase necesare unei aplicații de evidență a unui depozit de produse. Un produs este caracterizat prin cod, nume, preț de bază și cantitate. Produsele sunt de două feluri: compensate și necompensate. Prețul produselor compensate este obținut prin înmulțirea prețului de bază cu un coeficient subunitar ce se transmite la crearea unui produs compensat iar al celor necompensate este chiar prețul de bază.

Un client al depozitului trebuie să poată executa următoarele operații asupra depozitului:

- adăugarea unui produs în felul următor: dacă produsul există în depozit, se actualizează doar cantitatea iar în caz contrar se mărește numărul de produse existent cu o unitate (ATENȚIE: produsele se vor reține sub forma unui tablou iar în depozit se pot afla oricâte produse!).
- tipărirea inventarului de produse la un moment dat (pentru fiecare produs existent în depozit se va tipări codul, numele, prețul și cantitatea).

Se cer:

- diagrama UML pentru clasele necesare aplicației.
 - implementarea claselor necesare.
 - o metodă main în care se va crea un depozit, se vor adăuga produse și se va tipări inventarul depozitului creat.
 - ce s-ar întâmpla dacă în cadrul acestei aplicații nu s-ar folosi polimorfismul?
 - explicați mecanismul prin care la adăugarea unui produs se testează dacă produsul există sau nu în depozit.
31. Să se modeleze un sistem software simplificat orientat-pe-obiecte. Acesta este alcătuit din mai multe fișiere. Un fișier are un număr nelimitat de clase. O clasă are un anumit număr de linii de cod iar pe parcursul existenței sale poate fi supusă unor modificări datorită unor probleme(bug-uri). Detalii despre toate bug-urile dintr-o clasă sunt stocate de către aceasta sub forma unei colecții de Bug-uri, existând posibilitatea adăugării unui număr nelimitat de bug-uri unei clase. Fiecare fișier are asociat un autor. Atât clasa cât și fișierul au asociate câte o denumire.

Bug-urile sunt de mai multe feluri:

- NullBug - acest tip de bug a apărut datorită folosirii unei referințe ce nu a indicat spre nici un obiect. Afișarea acestui tip de bug pe ecran va determina mesajul *NullBug referinta*, unde referința este numele referinței utilizate eronat. Numele referinței va fi setat la instanțierea unui Bug de acest tip.
- CastBug - acest tip de bug apare datorită efectuării unei operații de conversie eronate dintre un tip și alt tip. Acest tip de Bug se va tipări sub forma *CastBug referinta: Tipul ei - Tipul spre care s-a încercat conversia*. Toate valorile atributelor menționate (trei) vor fi stocate sub formă de șiruri de caractere și setate la crearea acestui tip de bug.

Pe lângă caracteristicile enunțate deja, entitatea Clasa mai are două servicii, unul furnizează numărul de linii de cod și altul furnizează numărul total de bug-uri ce au apărut în cadrul clasei iar entitatea Fisier mai are o metodă ce furnizează numărul total de bug-uri existent în fișierul respectiv precum și o altă metodă ce furnizează numărul de clase conținute. Sistemul software are un număr nelimitat de fișiere. El, prin intermediul serviciilor, furnizează următoarele:

- numărul de fișiere conținute
- numărul de clase conținute
- numărul de autori distincți ce au dezvoltat sistemul (un autor poate dezvolta mai multe fișiere)
- un serviciu pentru tipărire. Rezultatul acestui serviciu va informa clienții clasei despre numărul de fișiere existente în sistem iar numele fiecărui autor distinct va fi furnizat împreună cu fișierele deținute de acesta. Atenție - numele unui autor apare o singură dată! Dacă un autor deține mai multe fișiere, denumirile acestora se vor tipări pe câte o linie distinctă.

Entitățile Clasa, Fisier și Sistem stochează sub formă de colecții de obiecte numărul nelimitat de membri conținuți (Clasa are Bug-uri, Fisierul are Clase, Sistemul are Fișiere) și fiecare entitate menționată are un serviciu pentru adăugarea de elemente în propria colecție.

Să se implementeze clasa sau, după caz, clasele ce modelează conform cerințelor de mai sus bug-urile existente precum și entitățile Clasa, Fisier și Sistem. Să se instanțieze într-o metodă main un sistem cu câteva fișiere și clase, unele clase având și bug-uri atașate.

Observație: Clasele, în afara serviciilor menționate pe care acestea trebuie să le pună la dispoziția utilizatorului, pot conține oricâte atribute/servicii considerați că sunt necesare.

32. Să se scrie o clasă ce modelează conceptul de coadă, elementul ce este folosit pentru stocarea elementelor fiind tabloul. Elementele ce aparțin cozii la un moment dat sunt niște carduri bancare, în cazul nostru Visa, Maestro și MasterCard. Fiecare tip de card are imprimat numele utilizatorului (tip String, maxim 15 caractere), un număr format din 16 cifre (tip char[]) iar cardurile Visa și MasterCard au un număr de control format din 3 cifre (tip char[]) inscripționat pe spate. Toate valorile acestor attribute sunt setate prin intermediul constructorilor acestor clase. Dimensiunea cozii este setată prin intermediul unui constructor ce primește ca parametru un int. Dacă parametrul primit este negativ atunci acest constructor va genera o excepție neverificată creată de voi. Clasa care modelează coada pune la dispoziția unui client:

- o metodă pentru introducerea unui element (card bancar) în coadă. Această metodă are ca unic parametru elementul ce se dorește a fi introdus. Dacă nu mai există loc în coadă pentru introducerea unui nou element, clientul (obiectul care a apelat metoda de adăugare pusă la dispoziție de coadă) va fi informat de faptul că momentan nu mai pot fi introduse elemente. Atenție - această informare nu va fi realizată prin tipărirea unui mesaj iar tipul returnat de metodă va fi void!
- o metodă pentru returnarea și ștergerea următorului element (dintre elementele existente primul introdus) din coadă. Dacă coada este goală, acest lucru va fi semnalat clientului exact în modul în care a fost semnalat și faptul că nu mai pot fi adăugate elemente.
- o metodă pentru afișarea conținutului cozii. Elementele vor fi afișate în ordinea PrimulIntrodus...UltimulIntrodus și pentru fiecare card se vor tipări valorile tuturor atributelor conținute însoțite de tipul cardului.
- o metodă public boolean esteInCoadă(Strategie strategie), unde Strategie e o interfață ce are metoda public boolean conditie(CardBancar card). Aceasta metodă (esteInCoadă) returnează true dacă există cel puțin un element în coadă care satisface condiția definită de strategie. Concret, parcurge toate elementele existente în coadă și dacă există cel puțin un element pentru care apelul metodei conditie a strategiei (evident, având ca parametru elementul curent (cardul curent)) returnează true, metoda esteInCoadă returnează true. Dacă un astfel de element nu este identificat, metoda va returna false.

Se cer:

- Să se implementeze conform cerințelor clasa Coadă împreună cu toate clasele și subclasele folosite de aceasta.
- Să se implementeze interfața Strategie de către două clase, astfel încât să existe:
 - O clasă ce primește printr-un constructor un String corespunzător unui nume. În acest caz metoda public boolean conditie(CardBancar card) returnează true dacă numele utilizatorului primit ca parametru prin constructor este egal cu cel stocat în cadrul obiectului tip CardBancar referit de către parametru.

- O clasă pentru care metoda public boolean conditie(CardBancar card) returnează true dacă numărul cardului conține de trei ori cifra 5.
 - Să se creeze într-o metodă main o coadă. Să se apeleze toate metodele puse la dispoziție de către obiectul creat.
 - Să se explice care sunt avantajele folosirii moștenirii de tip în cadrul acestei aplicații.
 - Descrieți modificările necesare astfel încât pe parcursul existenței programului clasa Coada să nu poată fi instanțiată decât o singură dată. Apoi modificați implementarea clasei astfel încât aceasta să nu poată avea mai mult de o instanță.
33. Fie o scenă grafică care conține diferite tipuri de figuri. Concret, putem avea figuri de tip Cerc, Patrat și Triunghi. Fiecare figură are o culoare, conține o metodă pentru calculul perimetrului și poate fi afișată sub următoarea formă: Tip Figura, Culoare, Perimetru (Culoare desemnează valoare culorii(ex: galben, roșu) iar Perimetru valoarea perimetrului). Se consideră că două figuri sunt egale dacă au același tip și același perimetru. Figurile de tip Cerc au o rază, cele de tip Pătrat au o latură iar cele de tip Triunghi au trei laturi. Toate figurile au metode pentru modificarea atributelor prezentate anterior.

Atunci când se instanțiază, fiecare figură se înregistrează la un observator. Un observator este o clasă ce are o singură instanță și conține următoarele metode:

- o metodă pentru adăugarea oricărui tip de figură (Cerc, Patrat sau Triunghi) într-o colecție de obiecte, colecția fiind un atribut al clasei.
- o metodă prin care se anunță observatorul că starea unei figuri conținute (culoarea și/sau raza/latura/laturile) s-a modificat. Apelul acestei metode e realizat de către toate tipurile de figuri atunci când se modifică caracteristica/caracteristicile acestora și determină afișarea tuturor figurilor monitorizate pe ecran.
- o metodă pentru afișarea tuturor figurilor ce sunt monitorizate.

Se cer:

- Să se implementeze conform cerințelor ierarhia de figuri precum și clasa ce modelează conceptul de observator.
 - Să se creeze într-o metodă main cel puțin o figură de fiecare tip, iar cel puțin una dintre ele să i se modifice un atribut. Se va exemplifica rezultatul rulării metodei main.
 - Să se explice care sunt avantajele folosirii polimorfismului în cadrul acestei aplicații.
34. Poliția comunitară acordă diferite tipuri de amenzi pentru următoarele categorii de fapte, după cum urmează:
- parcare ilegală a mașinii – amendă fixă 500 RON

- aruncatul gunoaielor pe jos – amendă 200 RON * factorZonal, unde factorZonal este un coeficient cuprins între 1..4, corespunzător zonei în care s-a produs fapta
- distrugerea bunurilor din parcuri – amendă 300 RON * valoarea bunului

Fiecare tip de amendă are cel puțin următoarele caracteristici:

- conține codul numeric personal (CNP) al cetățeanului căruia a fost aplicată
- are o metodă ce returnează valoarea amenzii
- poate fi afișată sub următoarea formă: CNP Tip Amenda, Valoare. Tip Amenda desemnează cauza pentru care amenda a fost acordată iar Valoare reprezintă suma care trebuie încasată de la cetățean.

În cadrul sistemului folosit există o clasă pentru gestiunea tuturor amenzilor acordate. Această clasă conține:

- o colecție de obiecte pentru stocarea tuturor amenzilor acordate
- o metodă pentru introducerea unei noi amenzi (de orice tip)
- o metoda public int valoare(Strategie strategie), unde Strategie e o interfață ce are metoda public boolean conditie(Amenda amenda). Această metodă (int valoare(Strategie strategie)) returnează suma amenzilor corespunzătoare elementelor din colecție ce satisfac proprietatea definită de strategie. Concret, parcurge toate elementele colecției și pentru fiecare element apelează metoda conditie a strategiei (evident, având ca parametru elementul curent (amenda curentă)) și dacă aceasta returnează true, adaugă valoarea amenzii la suma ce va fi returnată.

Se cer:

- Să se implementeze conform cerințelor ierarhia de amenzi precum și clasa ce gestionează amenzile acordate.
- Să se implementeze interfața Strategie de către diferite clase, astfel încât să existe:
 - o clasă ce primește printr-un constructor un String corespunzător unui cod numeric personal. În acest caz metoda public boolean conditie(Amenda amenda) returnează true dacă codul numeric personal primit ca parametru prin constructor este egal cu cel stocat în cadrul obiectului tip Amenda referit de către parametru.
 - o clasă pentru care metoda public boolean conditie(Amenda amenda) returnează true dacă amenda a fost generată datorită nerespectării semnului *Parcarea Interzisă*.
- Să se exemplifice printr-o metodă main funcționarea sistemului. Se vor crea diferite tipuri de amenzi (cel puțin câte una de fiecare tip) și se va returna suma amenzilor datorate de cetățeanul ce are codul numeric personal 1031274456709.

- Să se explice care sunt avantajele folosirii polimorfismului în cadrul acestei aplicații.
35. Într-un sistem simplificat de execuție de sarcini (nu are legătură cu firele de execuție) există mai multe feluri de task-uri. Indiferent de tipul său concret, pe un obiect task se poate apela din exterior:
- (a) o metodă denumită *execute* ce întoarce sub formă de întreg timpul de execuție al task-ului în milisecunde (presupunem că metoda și execută cumva fizic task-ul dar pentru simplitate pe noi ne interesează doar timpul de execuție)
 - (b) o metodă care întoarce reprezentarea sub formă de șir de caractere a obiectului task. Calculul timpului de execuție și reprezentarea sub formă de șir de caractere se realizează funcție de tipul concret al task-ului.

Tipurile concrete de task-uri sunt:

- SimpleTask - un astfel de obiect primește la creare (prin constructor) un întreg reprezentând timpul de execuție al task-ului. Pentru acest fel de task reprezentarea sub formă de șir de caractere este SimpleTask(timp_executie) unde timp_executie e întregul dat prin constructor. Valoarea acestui întreg poate fi modificată printr-o metodă changeTime (prin parametrul ei) definită în această clasă.
- ConditionalTask - un astfel de obiect primește prin constructor două obiecte task ce pot fi de orice fel concret. Timpul de execuție pentru un astfel de tip de task se calculează ca 5 milisecunde la care se adaugă aleator fie timpul de execuție al primului task fie timpul de execuție a celui de-al doilea (trebuie ținut cont că timpul de execuție al unui task condițional poate diferi de la o execuție la alta; pentru acest aleatorism puteți utiliza metoda statică random din clasa Math). Pentru acest fel de task reprezentarea sub formă de șir de caractere este ConditionalTask(r1,r2) unde ri(i=1..2) este reprezentarea sub formă de șir de caractere a task-ului i dat prin constructor.
- BlockTask - reprezintă o înșiruire de task-uri de orice fel (simple, condiționale, blocuri) într-un număr nelimitat. Timpul de execuție pentru un astfel de tip de task se calculează ca suma timpului de execuție pentru fiecare task conținut. Clasa mai pune la dispoziție o metodă insert prin intermediul căreia se poate adăuga la un task bloc (din exteriorul unui astfel de obiect) un task de orice fel (prin intermediul parametrului metodei). Reprezentarea sub formă de șir de caractere are forma Block(r1,r2,..) unde ri(i=1,2,..) este reprezentarea sub formă de șir de caractere a task-ului i conținut de task-ul block.

Se cer:

- Implementarea claselor descrise și a altora dacă sunt necesare.
- Să se construiască într-o metodă main (și ținând cont de implementarea claselor anterioare) un obiect task a cărui reprezentare sub formă de șir de caractere este ConditionalTask(SimpleTask(155), BlockTask(SimpleTask(5), SimpleTask(4))).

- Să se implementeze un mecanism prin care să putem afla în câte taskuri compuse (adică condiționale sau bloc) a fost inclus un obiect task. În acest scop se va introduce corespunzător în clasa / clasele de task-uri o metodă care să întoarcă această valoare.

Notă: Schimbarea timpului de execuție a unui task simplu, a unui task block (prin inserarea la ea de alte task-uri), etc., trebuie să fie observabilă în timpul de execuție a altor task-uri compuse ce includ task-urile anterioare.

36. Într-un program de gestiune a angajaților unei firme există mai multe feluri de angajați. Fiecare obiect angajat este caracterizat de numele său și are o metodă `calculSalar` care întoarce un `double` reprezentând salariul acelui angajat. Tipurile concrete de angajați și modul specific de calculare a salariilor (pe lângă alte informații) sunt următoarele:

- `AngajatCuSalarFix` - la crearea unui astfel de obiect se dă numele angajatului și un `double` reprezentând salariul său fix. Aceste date vor fi memorate în starea obiectului. În cazul unui astfel de angajat, metoda `calculSalar` întoarce valoarea salariului dată la crearea obiectului. Clasa mai definește o metodă denumită `schimbaSalarFix` prin intermediul parametrului căreia se poate schimba valoarea salariului fix specificat la crearea obiectului.
- `AngajatCuOra` - la crearea unui astfel de obiect se dă numele angajatului și un `double` reprezentând salariul pe oră primit de angajat. Aceste date vor fi memorate intern în starea obiectului. Clasa mai definește metoda `adaugaOre` care primește ca parametru un `double` reprezentând un număr de ore lucrate. Toate orele lucrate vor fi memorate intern într-un obiect de acest fel, într-un tablou de maxim 31 de intrări (se presupune că nu se adaugă mai mult de 31 de ore). Tot această clasă definește metoda `schimbaSalarPeOra` prin intermediul parametrului căreia se poate schimba salariul pe oră setat inițial (se poate da altă valoare prin intermediul parametrului metodei). În cazul unui astfel de angajat metoda `calculSalar` întoarce ca valoare numărul total de ore (suma orelor din tablou) înmulțită cu salariul pe oră.

Sistemul mai definește o clasă `Firma`. Aceasta definește o metodă `angajeaza` prin care un angajat de orice fel (dat ca parametru metodei) este memorat ca angajat al acelei firme. Angajații unei firme sunt memorați intern într-un tablou de maxim 1024 intrări. Dacă se încearcă angajarea a mai mult de 1024 de angajați, metoda întoarce valoarea -1. Mai mult, metoda întoarce valoarea -2 în cazul în care se încearcă adăugarea aceluiași angajat la aceeași firmă (un angajat nu poate fi adăugat de două ori la aceeași firmă). Se consideră că două obiecte angajat sunt egale atunci când numele angajaților sunt egale (reprezintă aceleași secvențe de caractere). În situația în care angajarea se realizează cu succes metoda `angajeaza` întoarce valoarea 0.

Clasa Firma mai definește o metodă `salariuMediu` care întoarce un double reprezentând salariul mediu calculat pentru toți angajații săi (media salariilor). Dacă obiectul de tip Firma apelat nu are nici un angajat, metoda întoarce valoarea 0.

Se cer:

- Implementarea claselor descrise mai sus și a altora necesare.
- O metodă `main` în care se instanțiază câte două obiecte angajat de fiecare fel și o firmă. Se adaugă obiectele angajați la firma creată și se calculează (și afișează) salariul mediu pentru acea firmă. Se va exemplifica apoi cazul în care se încearcă angajarea aceleiași persoane de mai multe ori la aceeași firmă. Se va schimba apoi salariul fix sau pe oră a unui angajat din cei creați anterior și se va reafișa salariul mediu pentru acea firmă.

Notă: Se pot adăuga și noi membri la clasele amintite mai sus dacă se consideră necesar. Schimbarea valorii salariului unui angajat trebuie să fie vizibilă dintr-un obiect Firma care conține obiectul angajat menționat anterior.